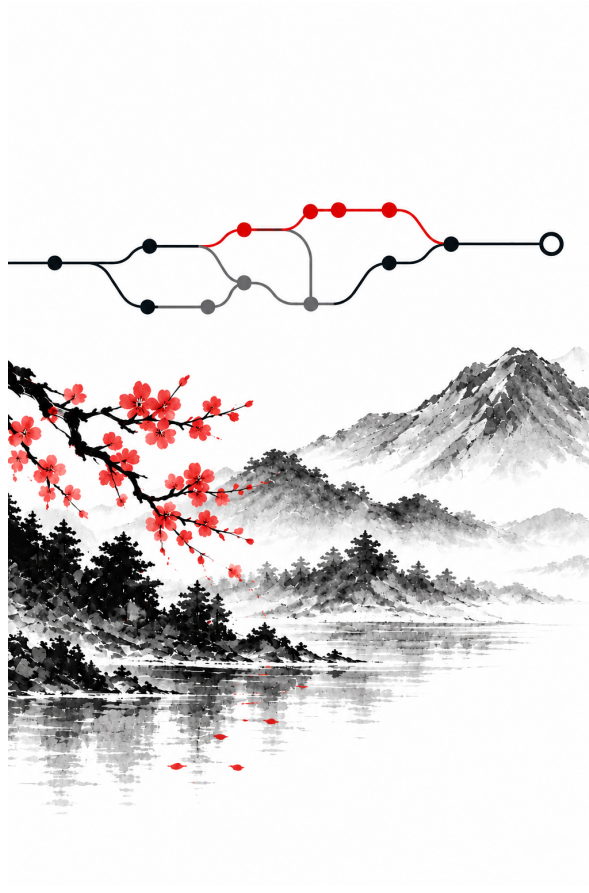


A GRAPH-FIRST FIELD BOOK

Jujutsu for Git Experts

Graph-First Version Control



Advanced conceptual guide, workflow manual, language reference, and Linux field book
Target release: jj 0.45.1 · Git 2.53.0 · build timestamp: 2026-09-17T12:37:37Z

ADVANCED EDITION · KINDLE SCRIBE PRINT PROFILE

This is a bespoke book created by someone for his own learning. It has not been created nor edited to be suitable for a more general audience. It is publically available just in case it might be useful to someone, somewhere, at sometime.

The creator thanks everyone involved in the creation of Jujutsu. It is fantastic.

Please report any issues you find with the content to <https://github.com/nevdelap/jj-book/issues>.

Contents

This contents page is generated by the PDF renderer from the same curated chapter headings used by the HTML navigation. Page numbers belong to this edition of the book; the linked HTML navigation remains available in the electronic source.

Preface and version policy	7
Part I — jj primer	8
1. Starting from any repository state	8
2. The core jj cycle	10
3. The everyday jj vocabulary	11
4. First graph operations	13
5. Publication, conflicts, and recovery	14
Part II — The jj mental model for Git experts	15
6. DAGs, snapshots, and identities	15
7. The working-copy commit and the default workspace	18
Part III — Rewriting history as ordinary work	33
9. Patch-stack surgery	33
10. Rebasing selection and destination algebra	35
Part IV — Revsets: a query language	58
12. Revsets as values	58
Evaluate operators on one explicit graph	61
Precedence worked from the parse tree	63
Field predicates: one field, several questions	66
Date patterns in detail	67
The regex dialect jj actually uses	68
13. Revset operators and public functions	69
14. Revset algebra cookbook	72
Revset semantics laboratory: evaluate an expression by hand	90
Revset public functions: signatures and operational uses	96
State predicates with graph evidence	98
Part V — Filesets	105
Fileset semantics laboratory: path sets, not path strings	116
Part VI — Bookmarks and publication refs	122
Part VII — Git backend and interoperability	131
Part VIII — GitHub workflows	141
GitHub field laboratory: one local graph, several PR views	146
Part IX — Gerrit	150
Gerrit stack lifecycle: three identities and two review iterations	159
Part X — Conflicts and merges	165
Conflict lifecycle laboratory: from graph result to resolved tree	169
What the materialised file looks like	170
The three stable marker styles and one experimental value	172
Part XI — Operation history, change evolution, undo, and recovery	177
Operation history: repository-wide transformations	177
Evolution log: following one change through time	179
Correlating operation history with change evolution	190
Recovery laboratory: named states, undo, and repair	190
Concurrency, retention, and cross-machine limits	196
Part XII — Workspaces and multiple machines	198
Workspace laboratory: one store, several materialisations	201
Part XIII — Configuration	206
Configuration laboratory: construct policy without losing the layers	220
Part XIV — Template language	226

Template language laboratory: types, evaluation, and stable output	245
Part XV — Exhaustive CLI reference for jj 0.45.1	253
Revision arguments: positional aliases versus real options	257
jj abandon	258
jj absorb	259
jj arrange	260
jj bisect and jj bisect run	261
jj bisect run	262
jj bookmark	262
jj bookmark advance	263
jj bookmark create	263
jj bookmark delete	264
jj bookmark forget	264
jj bookmark list	265
jj bookmark move	265
jj bookmark rename	266
jj bookmark set	266
jj bookmark track	266
jj bookmark untrack	267
jj commit	267
jj config	268
jj config edit	269
jj config gc	269
jj config get	269
jj config list	270
jj config path	270
jj config set	270
jj config unset	271
jj converge 0.45.1	271
jj describe	272
jj diff	273
jj diffedit	274
jj duplicate	274
jj edit	275
jj evolog	276
jj file	276
jj file annotate	277
jj file chmod	277
jj file list	277
jj file search	278
jj file show	278
jj file track	278
jj file untrack	279
jj fix	279
jj gerrit and jj gerrit upload	280
jj gerrit upload	281
jj git	282
jj git clone	282
jj git colocation	283
jj git colocation disable	283
jj git colocation enable	283
jj git colocation status	283
jj git export	284
jj git fetch	284
jj git import	284
jj git init	285
jj git push	286
jj git remote	287
jj git remote add	287
jj git remote list	287
jj git remote remove	287
jj git remote rename	288
jj git remote set-url	288
jj git root	288
jj help	288
jj interdiff	289
jj log	289

jj metaedit	290
jj new	291
jj next	291
jj operation / jj op	292
jj operation abandon alias: jj op abandon	293
jj operation diff alias: jj op diff	293
jj operation integrate alias: jj op integrate	293
jj operation log alias: jj op log	294
jj operation restore alias: jj op restore	294
jj operation revert alias: jj op revert	295
jj operation show alias: jj op show	295
jj parallelize	295
jj prev	296
jj rebase	296
jj redo	298
jj resolve	298
jj restore	299
jj revert	299
jj root	300
jj run	300
jj show	301
jj sign	301
jj simplify-parents	302
jj sparse	303
jj sparse edit	303
jj sparse list	303
jj sparse reset	303
jj sparse set	304
jj split	304
jj squash	305
jj status	306
jj tag	306
jj tag delete	306
jj tag list	307
jj tag set	307
jj tag track	307
jj tag untrack	307
jj undo	307
jj unsign	308
jj util	308
jj util backend and jj util backend name	309
jj util backend name	309
jj util completion	309
jj util config-schema	310
jj util exec	310
jj util gc	310
jj util install-man-pages	311
jj util markdown-help	311
jj util snapshot	311
jj version	312
jj workspace	312
jj workspace add	313
jj workspace forget	313
jj workspace list	313
jj workspace rename	314
jj workspace root	314
jj workspace update-stale	314
Command contracts: how to read an advanced jj command	322
Part XVI — Advanced mechanics	329
Part XVII — Workflow map and complete real workflows	344
Part XVIII — Practice, reference, and fieldbooks	364
The advanced field manual — practice and decision procedures	365
Book-wide case study: one change through its entire life	378
Language fieldbook — practice: deriving selections and output	386

Revset reference atlas — lookup	396
Configuration and template reference — lookup	407
Standalone references — lookup: keys and command paths	415
Command reference atlas — lookup: the remaining surface	430
Mechanics synthesis: why the graph changes	436
Synthesis: proof-driven jj practice	442
Advanced mechanics labbook — practice	457
Practice curriculum — twenty graph-first labs	464
Hosting protocol fieldbook — workflows	472
Appendix directory and quick lookups	479
Appendix A — Complete command tree	479
Appendix B — Option index	480
Appendix C — Complete revset operator/function reference	490
Appendix D — Complete fileset reference	497
Appendix E — Complete template types and methods	498
D.1 — Fileset grammar and consumer matrix	501
E.1 — Template type and method map	503
E.2 — Operators and formatting rules	504
F.1 — Configuration reference by task	505
Appendix F — Complete configuration families	507
Appendix G — Git-to-jj lookup	508
Appendix H — GitHub recipes fieldbook	511
Appendix I — Gerrit recipes fieldbook	515
Appendix J — Recovery recipes fieldbook	518
Appendix K — Linux shell integration fieldbook	520

Preface and version policy

This book assumes that you already know Git's object model, refs, branch-based development, detached HEAD, interactive rebase, reflog recovery, and the practical difference between a diff and a name. It is written for both the Git user who organises work around local branches and the advanced user who keeps private work detached and uses the reflog to recover earlier positions. Its subject is the different set of invariants that `jj` makes easy to preserve.

This edition targets `jj 0.45.1`. Commands, options, revset functions, filesets, templates, and configuration claims are stated for that release. Later releases may add commands or alter defaults, so check `jj version` and the installed `jj help` before relying on a version-sensitive detail after upgrading.

Version policy. The prose and examples in this edition describe `jj 0.45.1`. Current-main documentation is useful for tracking future direction, but is not silently treated as 0.45.1 behaviour. After upgrading, compare the release notes and the relevant `jj help` pages before adapting a command or configuration.

Print profile. The HTML is authored for PDF printing to a Kindle Scribe: portrait `7.5 × 10` in pages preserve the device's 3:4 aspect ratio, use a generous inner margin for handwriting, suppress screen navigation, and reduce table density only in print. Graph diagrams intentionally use a PDF-safe ASCII glyph set (`-`, `|`, `+`, `\`, and `>`) so the bundled Type 1 PDF fonts cannot substitute missing Unicode box characters. In a Chromium/Firefox print dialog choose *Save to PDF*, scale 100%, background graphics on, headers/footers off, and portrait orientation. The book has no network dependency at render time; its cover illustration is bundled with the source.

Official material consulted includes the Jujutsu documentation, the Git comparison, revsets, filesets, templates, configuration, GitHub, Gerrit, conflicts, operation log, workspaces, and the release notes. These links may move; the URL/title/version ledger is authoritative for this edition.

Licensing is treated conservatively. The book is original prose and diagrams, uses short command examples, includes no third-party images or logos, and does not reproduce long documentation passages. The distribution includes the required notices for the software and image assets used to produce it.

Part I — jj primer

This part supplies the jj knowledge that the rest of the book assumes. It assumes Git expertise, but it assumes nothing about your jj experience and nothing about the state of the directory in which you begin. You may be opening an existing jj repository, adding jj to an existing Git repository, cloning a remote repository, or simply reading the commands before trying them.

The goal is not to make you memorise a Git-to-jj translation table. It is to establish a usable first vocabulary: how jj identifies the current working copy, how filesystem edits become graph nodes, how a change is evolved or separated, how a name is created for publication, and how a local mistake is recovered. The later parts replace these working descriptions with precise graph, identity, language, and operation semantics.

1. Starting from any repository state

Begin by identifying what you actually have. Do not assume that the directory is empty, that Git's `HEAD` is the work you want, or that a local bookmark represents the intended publication point:

```
jj version
jj root
jj status
jj log
jj workspace root
jj git root
```

If these commands work, you are already inside a jj workspace. The unqualified symbol `@` selects its working-copy commit. `jj status` tells you whether the filesystem has edits or conflicts; `jj log` shows the visible commit graph and

names; `jj workspace root` identifies the directory context; and `jj git root` identifies the backing Git repository when the repository uses the Git backend. These inspections are useful even when you do not intend to change anything.

If the directory is an ordinary Git repository, initialise `jj` in place with `jj git init`. In `jj` 0.45.1, colocation is the default unless `git.colocate=false` is configured; `--no-colocate` explicitly chooses a non-colocated layout. `jj` imports the existing Git history and refs into its view. If you want a new clone, use `jj git clone URL DESTINATION`, then enter the destination and inspect it. If you are genuinely starting with no repository, `jj git init DIRECTORY` is the setup path, but that is only one starting state, not the assumed one for this book.

In a colocated repository, native Git commands may also be useful for inspecting or transporting Git refs and objects:

```
git status
git log --all --decorate --oneline
jj git import
jj status
```

The Git view and the `jj` view are related but not identical. Git's `HEAD` is not the definition of `jj`'s `@`, and a Git ref is not the definition of a `jj` change. Run native Git deliberately, then import Git-side ref changes before reasoning from `jj`'s graph. Part VII develops this boundary in detail.

Starting state	First useful action	What not to assume
Existing <code>jj</code> repository	<code>jj status</code> , <code>jj log</code> , then inspect <code>@</code> .	That the current change is clean, described, or published.
Existing Git repository	<code>jj git init</code> , then inspect imported history.	That Git <code>HEAD</code> and <code>jj @</code> are the same pointer.
Remote repository	<code>jj git clone URL DESTINATION</code> , then fetch/inspect.	That a remote branch is a local bookmark or that remote state is current forever.
Empty directory	<code>jj git init</code> , then describe or create work.	That every reader starts with an empty root.

2. The core jj cycle

Once the starting state is known, the daily cycle is small:

1. Inspect the current working-copy commit and its parent.
2. Edit files with normal Linux tools.
3. Run a `jj` command that snapshots the workspace, or explicitly choose how snapshotting should work.
4. Inspect the resulting diff.
5. Describe the current change when its purpose is clear.
6. Use `jj new` when the next piece of work should be a new child change.

For an existing graph, the transition can be pictured without pretending that the base is a newly created root:

```
before:

main@origin -> A -> B -> @
                        current unfinished change

edit src/parser.rs
jj status
jj diff
jj describe -m "Handle quoted parser input"

after snapshot and description:

main@origin -> A -> B -> @'
                        successor of the old @
                        same evolving change, new commit object
```

There is no `git add` stage in this cycle. The filesystem is a working input to `jj`'s snapshot mechanism. A normal command generally snapshots the current workspace at its beginning, so `jj status`, `jj diff`, or `jj describe` may first record edits that were made by the editor. The precise snapshot boundary, including `--ignore-working-copy`, is explained in Part II; the practical rule is to inspect after a command and do not confuse live files with the last recorded tree.

```
printf 'new parser case\n' >> tests/parser.txt
jj status
jj diff
jj describe -m "Handle quoted parser input"
jj show -r @
jj new -m "Document the parser case"
jj log
```

`jj show -r @` displays the current commit and its diff. `jj new` creates a child and makes it the current workspace commit; the preceding change remains in the graph without needing a local branch name. If the current change is not ready to separate, continue editing it instead. If you need to work on an existing commit, use `jj edit R`; that moves the workspace’s current commit and updates the directory to its tree, rather than creating a new branch.

The two choices—evolve the current change or create a child—are the central everyday decision:

Intent	Command family	Graph consequence
Continue the current change	Edit files, snapshot, describe	Successor version of the current logical change.
Start dependent work	new	New child change; current @ moves to it.
Return to an existing change	edit R	Workspace pointer moves; the selected commit is not rewritten merely by editing it.
Inspect without updating files	--ignore-working-copy	Reads the recorded workspace state, which may be older than the filesystem.

3. The everyday jj vocabulary

The rest of this book uses these terms precisely. They are not interchangeable versions of “the current branch”:

Term	Working meaning	Important distinction
Commit ID	Identity of one stored commit object.	Changes when tree, parents, or metadata change.
Change ID	Identity of an evolving logical change.	Normally survives rewrites of that change; not a Git commit ID or Gerrit Change-Id.
Working-copy commit	The commit representing this workspace's recorded files; selected by @.	Not a Git index or an uncommitted overlay.
Bookmark	A movable local name pointing at a commit.	Usually a publication name, not the mandatory container for local work.
Remote bookmark	Imported observation such as <code>main@origin</code> .	Not a live server lookup and not a local bookmark that follows @.
Workspace	The association between a directory and its working-copy commit.	Not a bookmark, clone, or branch; detailed use comes in Part XII.
Operation	A repository state transition recorded in the operation log.	Not a commit in the user-visible commit DAG.
Revset	Expression selecting commits.	Different language from filesets and templates.
Fileset	Expression selecting paths.	Different language from commit selection.
Template	Expression rendering repository values.	Does not select or rewrite commits by itself.

Several states are independent. A change may be mutable or immutable; private or published; clean or conflicted; visible or hidden. A bookmark may point at an older commit while @ advances. A fetched remote bookmark may be immutable in the local rewrite policy while new local descendants remain mutable. These combinations are normal and are why the later chapters use separate names for separate state dimensions.

4. First graph operations

The basic editing operations are graph transformations. You do not need their full algebra yet, but you need to recognise their intent:

```
before:

root -> A -> B -> @
      API   tests

jj squash --into A

after:

root -> A' -> @
      API + tests

A' is an evolved successor of A
```

`jj squash` combines selected content into a destination. `jj split` divides one change into two according to selected paths or interactive editing. `jj restore` restores selected file contents; it is not a reset of a ref. `jj revert` creates a new change that undoes the changes introduced by an earlier commit. `jj duplicate` copies a commit as a new logical change. `jj abandon` removes selected commits from the visible graph according to `jj`'s rewrite rules. `jj rebase` changes parent relationships or moves selected sources to selected destinations. In all of these, inspect the graph afterward because descendants can be rewritten automatically.

When a rewrite cannot apply a change's diff cleanly, `jj` records a conflict in the affected commit. Use `jj status`, `jj log`, and `jj resolve --list` to find it. Resolve the materialised conflict with your editor or a configured tool, then use `jj resolve` as appropriate. There is commonly no separate Git-style `rebase --continue`; resolving the conflict is another change to the stored commit, and descendants can continue to exist while the conflict is present.

For a first safe experiment, create a temporary bookmark before a broad rewrite:

```
jj bookmark create before-experiment -r @
jj log -r 'before-experiment::@'
```

```
jj squash
jj log
jj undo
```

The bookmark gives the graph a readable rescue name. The operation log records the transformation independently, and `jj undo` restores the prior repository operation when the mistake is local. A bookmark is not a substitute for understanding the operation log; it is simply a convenient additional reference.

5. Publication, conflicts, and recovery

Local `jj` work does not need a bookmark until another system needs a Git ref. The basic publication cycle is:

```
jj bookmark create parser-review -r @
jj git push --bookmark parser-review --remote origin
jj bookmark list --all-remotes
```

The local bookmark names the selected commit. The push exports that name and reachable Git objects. Later edits to `@` do not automatically move `parser-review`; move it explicitly, inspect the proposed change, and push again. Fetching updates observations such as `main@origin` but does not automatically rebase unnamed work. `jj bookmark track parser-review --remote origin` records a relationship between local and remote names; it does not make the bookmark follow the workspace.

On GitHub, a pushed bookmark can be the branch behind a pull request. On Gerrit, the same local stack may be uploaded through a review ref and may also carry a Gerrit commit-message `Change-Id`. The `jj` change ID, Git commit ID, bookmark, remote bookmark, and hosting-service identity are separate. Parts VI–IX develop publication and hosting without collapsing those identities.

When something goes wrong, inspect before performing a second repair:

```
jj status
jj log
jj op log
jj op show OPERATION_ID
jj undo
```

`jj op log` is the beginning of recovery, not an analogue to a branch list. It records repository transformations, including snapshots, rewrites, ref movement, and imports. It cannot undo an external server side effect such as an already accepted push, but it can often recover the local graph or ref state. Part XI turns this short sequence into a recovery laboratory.

Checkpoint. Before continuing, you should be able to enter an existing or newly initialised `jj` repository, identify the workspace-local `@`, inspect and describe a change, create a child with `jj new`, return with `jj edit`, recognise a bookmark versus a remote bookmark, publish a selected commit, and locate the operation that preceded a harmless mistake. You do not need to know the full `revset`, `fileset`, `template`, or `rebase` languages yet; those are the subjects of the following parts.

Part II — The `jj` mental model for Git experts

6. DAGs, snapshots, and identities

Git users arrive here from two common habits. A branch-oriented user gives the current line a local name and moves that name as work develops; a detached-HEAD-oriented user leaves private tips unnamed and uses commit IDs and the reflog to recover earlier positions. `jj` can accommodate either starting point, but its own model is different: unnamed mutable changes are normal, while bookmarks are publication names rather than the organising unit of local work. The detached-HEAD analogy is therefore useful for one audience, not a prerequisite for the other.

The primary object is a commit in a directed acyclic graph. A commit has a tree, parents, metadata, a *commit ID*, and a stable *change ID*. Rewriting a change generally creates a new commit ID while preserving the change ID. The change ID is the identity of the evolving change; the commit ID is the identity of one particular version of its content, parents, and metadata.

```
main@origin
|
A immutable imported commit
|
B mutable change: qpv... commit: 91ab...
|
C mutable change: wmx... commit: 42de...
|
@ working-copy commit; the filesystem is its next snapshot
```

Commit IDs therefore move after amend-like work. Change IDs normally do not. The evolution log records predecessors and successors, and a divergent change ID means two visible successors of the same logical change exist because independent operations rewrote it. A numeric offset such as `xyz/0` and `xyz/1` can select the current and prior visible evolution when the release supports that notation.

Mutable, immutable, hidden, and visible

Mutable commits may be rewritten. Immutable commits are protected by the configured immutable set; remote-tracked commits and other configured heads commonly form that boundary. `--ignore-immutable` bypasses the protection check, not the meaning of `immutable_heads()`. Hidden commits are not in ordinary visible searches but remain addressable by IDs, evolution, or explicit operation queries. Visible heads are the tips of the visible graph, not “all branches”.

The root commit has no parent. It is an ordinary graph anchor for expressions such as `root()::`; it is not an editable working change. Empty commits are still commits: “empty” describes the tree delta relative to parents, not absence from the graph.

Snapshot timing is a semantic boundary

Filesystem state and the last recorded tree are different states. By default `jj` snapshots the working copy at the beginning of every command, then updates the filesystem at the end if the working-copy commit was changed. Thus a command can observe edits that no previous command recorded. `--ignore-working-copy` deliberately reads a possibly stale commit and does not update it. `--at-operation` implies that mode because the historical repository view cannot safely snapshot today's filesystem.

Commands that inspect or mutate the repository normally trigger a snapshot. This includes familiar read commands such as `jj status` and `jj log`; use `--ignore-working-copy` for automation that needs a stable read or when another process owns snapshotting. The snapshot itself is an operation, and a mutating command may also create descendant rebases and working-copy updates within the same operation.

What the Git analogy gets right—and wrong

Concept	Useful Git analogy	Where it breaks
commit ID	Git commit object ID	May be replaced routinely by a rewrite; do not use it as the long-lived change identity.
change ID	Partly like a logical change identity or Gerrit review identity	It is jj identity; the Git backend may serialise it in a non-standard header, but it is not a standard Git object identity or a server-side review key.
working-copy commit	A detached HEAD commit	It is continuously snapshotted and may be rewritten; @ is workspace-local.
bookmark	A local branch ref when publishing	It is a movable name, not the organising unit of local development.
remote bookmark	Remote-tracking branch	Tracking is an explicit relationship; it does not make a local bookmark follow @.
operation log	Partly like a reflog	It records repository transformations as an operation DAG, with undo and operation restore.
conflict	Unmerged index state	It is a stored value in a commit and can propagate through descendants.

7. The working-copy commit and the default workspace

The default workspace, for now

For the mental model in this part, commands are assumed to run in jj's default workspace. A workspace is the association between a working directory and a working-copy commit; the unqualified symbol @ means the working-copy commit for the workspace from which the command is run. That is the only workspace fact needed to follow the snapshot and rewrite examples here.

Do not read “workspace” as a synonym for bookmark, branch, or clone. The full model—additional directories, workspace names, shared repository state, stale

workspaces, and selective filesystem materialisation—is deferred to Part XII. Until then, treat the default workspace as the ordinary local context in which `@` is resolved.

`@` names the current workspace's working-copy commit. A filesystem edit changes the next snapshot of `@`; it does not directly mutate an arbitrary ancestor. `jj new` creates a new empty child and normally edits it, leaving the old working copy as a parent. `jj edit R` moves the workspace's working-copy pointer to one existing commit and updates files to that tree.

Each workspace has its own working-copy commit and workspace symbol, although all workspaces share the repository's commit store and operation history. `other@` refers to another workspace's working-copy commit. A workspace is therefore a second working directory plus a local point of view, not a separate clone and not a bookmark.

```
shared commit DAG:      workspace A      workspace B
                        A@ --- child      B@ --- review
                          \              /
                          same immutable ancestors
```

Think in two dimensions: graph state (commits and edges) and materialised filesystem state (one checkout per workspace). Many confusing situations disappear when you ask which one a command is changing.

8. A repository is more than the commit graph

For difficult `jj` problems, model the repository as several coupled state machines. The commit DAG is the object users publish. The working-copy state is one local materialisation of one commit. Bookmarks and tags are names attached to commits. Remote bookmarks are imported observations and tracking relationships. The operation DAG records transitions among these states. A useful diagnostic asks which component changed instead of asking whether “the repository changed”.

State component	Local scope	Moves when	What it cannot tell you
Commit store/DAG	Repository	Commit creation, rewrite, import, abandon, restore, operation recovery.	Whether a file is currently materialised or published.
Working-copy commit	Workspace	Filesystem snapshot, edit/new/movement, rewrite affecting its target.	Whether a bookmark points at it or a remote has it.
Filesystem	Workspace directory	Editor/build tools, checkout/update, and jj command completion.	Whether content has been recorded in a commit.
Local bookmark/tag	Repository ref state	Explicit create/set/move/delete/rename and imported ref changes.	Whether its target is current @ or accepted by a server.
Remote bookmark/tag	Local observation of remote	Fetch/import and local forget/prune operations.	Current server state when the last fetch is old.
Operation DAG	Clone/repository	Any integrated repository operation.	Uncommitted external files or another clone's history.

These components are coupled without being identical. A filesystem edit is not a new commit until snapshotting. A commit rewrite can update the filesystem if a workspace points at the rewritten path. A bookmark move does not rewrite a tree. A push can change a server while leaving the local commit DAG untouched. A fetch can create remote bookmarks without changing local mutable commits. A workspace operation can make files appear or disappear while leaving every commit tree unchanged.

Snapshot boundary experiment

Run this in a disposable repository and record both the commit ID shown by `jj log -r @` and the filesystem checksum:

```
printf 'one\n' > file.txt
sha256sum file.txt
jj log -r @ -T 'commit_id ++ "\n"'
jj status
jj log -r @ -T 'commit_id ++ "\n"'
printf 'two\n' > file.txt
```

```
jj log --ignore-working-copy -r @ -T 'description ++ " " ++ commit_id ++ "\n"'
jj status
jj log -r @ -T 'description ++ " " ++ commit_id ++ "\n"
```

The first log can cause the edit to be snapshotted depending on the repository's current state and command boundary. The explicitly ignored log must not record the second edit; status without the ignore option then observes/snapshots it. The exact operation count is not the conceptual point: filesystem state can be newer than the recorded tree, and normal commands reconcile that boundary.

Concurrency rule. Do not use `--ignore-working-copy` as a universal “read-only” flag while another process owns a workspace and then act on the result as if it were current. It gives a stable repository view at the cost of possibly stale filesystem information.

Commit ID, change ID, and evolution

A commit ID identifies one immutable description of tree, parents, metadata, and conflict contents. A change ID names the logical change whose successive commit versions are linked by evolution. If A is rewritten into A' , then:

```
before:          root -- A -- B -- @
                  change A = qqg...
after describe A: root -- A' -- B' -- @'
                  change A = qqg...
                  commit A' ≠ commit A
```

B and $@$ can acquire new commit IDs even if their file content appears unchanged, because their parent commit ID changed. The successor relation is therefore a graph/evolution fact, not a hash relationship a user should recreate manually. An author timestamp or signature rewrite has the same property: tree identity can remain equal while commit identity changes.

When two independent operations rewrite the same logical change, both successors may remain visible:

```

      A_1 (change qqg..., commit 111...)
    /
root --- A
    \
      A_2 (change qqg..., commit 222...)

```

This is divergence, not corruption. The change ID alone no longer selects one commit; an offset or explicit commit ID is needed. Inspect `jj evolog` before choosing which successor to retain, merge, duplicate, or abandon. A template can display `divergent()` and `change_offset()`; a revset can select `divergent()`.

Visible heads and immutable heads

A head is a graph boundary calculated from a set. `heads(S)` removes selected ancestors that have a selected descendant. `visible_heads()` is a repository-level notion of visible tips, useful for understanding why a log's graph has a boundary that is not named by a bookmark. An immutable head is a safety boundary used to reject rewrites, commonly including imported trunk and other configured public points.

These are different questions:

```

jj log -r 'heads(all())'           # all graph tips in the chosen domain
jj log -r 'visible_heads()'        # visible repository tips
jj log -r 'immutable_heads()'      # rewrite-protection boundaries
jj log -r 'heads mutable() ~ ::remote_bookmarks()'
```

Do not equate a visible head with a branch, or an immutable head with a server branch. A bookmark can point to a non-head commit in the middle of a stack, and a remote bookmark can point to a visible head without making the whole ancestry immutable unless the configured immutable set says so.

Git branches and detached HEAD as starting points, not the whole model

If you normally work on a Git branch, treat that branch as a publication or coordination name that happens to be useful locally; you do not need to reproduce it as the organising centre of every jj stack. If you normally work detached, jj will feel familiar because a local tip can remain unnamed, but do not reduce @ to a detached Git HEAD. In both cases jj extends the familiar workflow in four directions: the working-copy commit is explicit and snapshotted; mutable ancestors can be edited directly; descendants are rebased as a consequence; and publication names are optional until a remote needs them. In jj, @ is a named workspace state and its evolution is part of the repository's normal operation history.

A practical consequence is that “where should I create a branch?” becomes “which graph boundary needs a stable publication name?” If the answer is “none yet”, keep the stack unnamed. If a GitHub PR or Gerrit review needs a ref, create a bookmark at that boundary and treat it as an interface object. Local graph reasoning remains independent of the publication naming convention.

Part II field manual: observing state without collapsing it

An advanced jj session becomes predictable when you stop asking for “the current state” as though there were one. Ask instead which projection you need: the filesystem projection, the working-copy commit, the visible commit graph, the named-ref projection, or the operation history. The commands overlap in output, but each is an instrument for one projection.

Projection	Primary question	Instrument	Typical stale source
Filesystem	What bytes and modes are on disk?	Shell tools, <code>jj status</code> , fileset-aware file commands.	Unrecorded edits or generated files.
Working-copy commit	What snapshot does jj currently associate with this workspace?	<code>jj show @</code> , <code>jj diff</code> , templates.	Disk changed after last snapshot.

Projection	Primary question	Instrument	Typical stale source
Commit DAG	Which commits and parent edges exist?	<code>jj log</code> , <code>revsets</code> , <code>jj show</code> .	Hidden commits or a historical operation view.
Ref graph	Which names publish which commits?	<code>jj bookmark list</code> , <code>tags</code> , <code>remote-bookmark revsets</code> .	Remote state not fetched or native Git refs not imported.
Operation DAG	Which repository transformation produced this view?	<code>jj op log</code> , <code>op show</code> , <code>op diff</code> .	Unintegrated/concurrent operation or later recovery operation.

Experiment: an edit that is not yet a commit rewrite

Use a disposable repository and observe the sequence with the same full identifiers. First create a non-empty change:

```
mkdir -p /tmp/jj-observe/src
cd /tmp/jj-observe
jj git init --colocate
printf 'one\n' > src/value.txt
jj status
jj show @
jj op log -n 5
```

At this point a normal command may have snapshotted the file, depending on the command and configuration. The important observation is not the exact number of operations; it is that a filesystem edit and the stored working-copy commit are different moments. Make a second edit and use a repository-only query:

```
printf 'two\n' >> src/value.txt
jj --ignore-working-copy show @
jj show @
jj status
```

The first `show` deliberately asks the stored graph not to incorporate current disk changes. The second `show` may snapshot according to normal command behaviour. Compare its operation log entry and commit ID. This experiment is the antidote to the phrase “jj commits automatically”: jj snapshots at defined command boundaries, and the filesystem can be ahead of the stored snapshot.

Experiment: the same logical change through three commit IDs

```
jj describe -m 'Parser: add a limit'
first=$(jj log -r @ --no-graph -T 'commit_id ++ "\n"')
change=$(jj log -r @ --no-graph -T 'change_id ++ "\n"')
printf 'limit = 42\n' >> src/value.txt
jj describe -m 'Parser: add a bounded limit'
second=$(jj log -r @ --no-graph -T 'commit_id ++ "\n"')
printf '%s\n' "$first" "$second" "$change"
jj evolog -r @ -T 'commit_id ++ " " ++ change_id ++ "\n"'
```

The commit ID changes because the tree and description changed. The change ID is the handle for the evolving logical change. The evolution log can expose the predecessor even after the old commit becomes hidden from ordinary output. If a script stored only `first`, it now has an exact historical object; if it stored `change`, it can resolve the current evolution, subject to visibility and divergence.

Experiment: descendants are rewritten because parent identity is input

```
jj new -m 'Tests: exercise the limit'
printf 'test limit\n' > src/value-test.txt
jj status
jj describe -m 'Tests: exercise the limit'
jj log -r '@:: & mutable()' -T 'commit_id.short() ++ " " ++ change_id.short() ++ " " ++ description'
```

Now edit the parent change explicitly. A normal workflow can locate it by its change ID rather than by an unstable commit prefix:

```
parent=$(jj log -r 'description(substring:"Parser: add")' --no-graph -T 'change_id ++ "\n"')
jj edit "$parent"
printf 'limit = 43\n' >> src/value.txt
jj describe -m 'Parser: add a bounded limit'
jj log -r '::-@' -T 'commit_id.short() ++ " " ++ change_id.short() ++ " " ++ description.first_line'
jj evolog -r 'change_id("$parent")'
```

The child's parent commit ID changed even when the child's own diff text did not. `jj` must represent the child relative to the new parent object, so it creates an evolved successor or a rebased descendant. This is why a review stack can acquire new Git commit IDs without each change being independently edited.

Four meanings of “current”

Phrase	Precise interpretation	Use a different phrase when you mean
Current working copy	The workspace-local commit resolved by @.	A bookmark target or the integrated operation.
Current graph	The repository view at the integrated operation head.	A graph at <code>--at-operation</code> .
Current trunk	The configured trunk revset, often a bookmark/remote bookmark.	The server’s latest branch if no fetch has happened.
Current change	The logical change selected by a change ID or workspace target.	The exact commit object that was previously checked out.
Current remote	The locally observed remote ref state.	The server’s live state before a fetch.

Hidden commits, divergent changes, and visible heads

jj’s visible graph is a useful working projection, not a complete object inventory. A rewrite can create a successor while hiding the predecessor. An imported ref can make a previously hidden commit reachable again. A divergent change can have multiple visible evolutions sharing one change ID. These states are meaningful because they preserve the evidence needed to reason about history evolution.

```
predecessor P_0 (hidden after rewrite)
  | evolution
successor P_1-----+
                   |-- child C_1
divergent P_2-----+
same logical change ID; different visible commit IDs
```

`visible_heads()` is a topology query over the visible working graph. It is not “all bookmarks”, not “all remote tips”, and not “the commits a server considers heads”. A commit can be a visible head without a bookmark, and a bookmark can point to a non-head commit because another child is intentionally local. Use `heads(X)` over a carefully defined set when the question is about a particular graph region.

When diagnosing divergence, begin with:

```
jj log -r 'divergent()'
jj evolog -r 'divergent()'
jj log -r 'change_id("PREFIX")' --no-graph -T \
  'commit_id ++ " " ++ change_id ++
  " " ++ description.first_line() ++ "\n"'
jj op log -n 20
```

Compare tree diffs, descriptions, authorship, and parent edges. A convergence decision may choose one evolution, merge content, or preserve both as independent changes. Do not use `jj abandon` on one evolution merely to make the warning disappear; that is a semantic decision that may rewrite descendants and publication refs.

Immutable heads as a moving boundary

Immutable heads are a policy set derived from configuration and graph state. Their descendants may remain mutable. This lets a repository protect imported trunk while allowing a local stack above it to evolve. The boundary can move after fetch, tag import, or configuration changes, so “I edited this yesterday” is not a stable safety argument.

```
immutable head T
|
|-- mutable A
|   \-- mutable B(@)
|-- another imported line
```

Query the boundary before a broad rewrite:

```
jj log -r 'immutable_heads()'
jj log -r 'mutable() & ::@'
jj log -r 'mutable() ~ ::main@origin'
```

If a source set includes an immutable commit, the command should fail rather than silently change public history. `--ignore-immutable` is an escape hatch for deliberate maintenance, not a normal solution to a mistaken revset. Record the

operation ID, the exact immutable set, and the reason for crossing it.

When Git branch and detached-HEAD analogies help—and fail

The branch analogy helps at the publication boundary: a local bookmark is a movable name that can be exported as a Git ref, and a remote bookmark is an imported observation of such a ref. The detached-HEAD analogy helps with private local work: a stack can remain unnamed while the graph and change identities are inspected directly. Neither analogy defines the jj workflow. The workspace points at a working-copy commit, and the user can move it with `edit` or create children with `new` without making a bookmark the organising pointer.

Both analogies fail in five important ways. First, jj's `@` is a workspace symbol, not a Git branch ref or detached HEAD file. Second, jj's working-copy commit is a first-class mutable graph node, not an uncommitted index/worktree overlay. Third, descendants are automatically rebased when an ancestor evolves. Fourth, conflicts are values stored in commits and can propagate without an interrupted rebase session. Fifth, operation history records repository transformations beyond ref movements. Use the branch or detached-HEAD analogy only as a bridge, then replace it with the jj state model.

Notation and proof habits

Throughout the book, a capital letter names a commit only within a diagram. A bookmark is written `name`; an imported remote bookmark is written `name@remote`; a logical change is written `X(change)`; a workspace-qualified working copy is written `workspace-name@` when disambiguation is needed; an operation is written `On`. A diagram edge is a parent relationship, not necessarily a bookmark movement.

For every graph-changing example, prove three things: the selected source set, the resulting parent edges, and the identity/evolution mapping. For every publication example, prove the local bookmark target and the observed remote target. For every recovery example, prove which operation created the restored view. These

proof habits scale better than memorising command folklore.

Graph-first diagnosis: the repository-state tuple

When a `jj` result surprises you, do not collapse the explanation into “the branch moved”. At minimum, describe the state as a tuple:

```
R = (C, W, F, B, T, Q, O)
```

C commit DAG and trees
W workspace records and each workspace's @
F materialised filesystem per workspace
B local bookmarks and their targets
T tags and remote observations/tracking relationships
Q configuration, visibility, and immutability policy
O operation DAG/current operation view

A command usually changes one component and may derive changes in others. Filesystem editing changes `F`; the next snapshot updates `C` and `O`. `jj edit` changes `W` and `F`, and may snapshot the old `F` first. `jj rebase` changes `C` and `O`, may update `W/F` when a workspace target is affected, and can move `B` when a bookmark points into the rewritten graph. `jj git fetch` changes `C`'s imported objects and `T`'s remote observations but does not change an unnamed local child. `jj git push` changes the external server and may update local tracking state, but it is not a commit rewrite.

Observation	Possible explanations	Disambiguating command
File exists on disk but is absent from <code>jj diff</code> .	It is untracked/ignored, the snapshot is stale, or the diff commit is not @.	<code>jj status</code> , then <code>jj log -r @</code> ; compare with <code>--ignore-working-copy</code> .
Commit ID changed but description did not.	Parent, tree, timestamp, author, signing, or export metadata changed.	<code>jj evolog -r CHANGE</code> and <code>jj op show -p</code> .
Bookmark is behind the current working copy.	Bookmarks do not follow @ automatically.	<code>jj bookmark list</code> ; move it explicitly if publication is intended.
Remote bookmark is present but push is rejected.	Local remote observation is stale, server ref advanced, or ancestry is not acceptable.	Fetch, compare both targets and the graph, then decide.

Observation	Possible explanations	Disambiguating command
Another workspace shows a new commit.	Shared graph rewrite or shared ref movement; its own @ is still local.	<code>jj workspace list</code> and logs from both directories.
<code>--at-operation</code> shows a different tree than disk.	Historical repository view deliberately ignores today's filesystem.	Return to the current view and inspect the working-copy snapshot separately.

This tuple prevents a common category error: interpreting an operation-level change as a commit-level change, a ref observation as a server action, or a filesystem absence as a deletion from history. Write the tuple components in an incident note before trying a repair.

Snapshotting as a state machine

```

filesystem edit
  |
  ▼
F_new differs from recorded tree C(@)
  | normal jj command
  ▼
snapshot operation records C(@)' with F_new
  | command mutation
  ▼
rewrite/create descendants and update workspace filesystem
  |
  ▼
new operation view O_next

```

Snapshot timing is a semantic boundary because a command can both observe and cause the transition. A normal `jj log` may first record an editor's changes, then render the new working-copy commit. A log with `--ignore-working-copy` reads the recorded tree without reconciling disk. A command at `--at-operation OP` cannot safely snapshot current files into a historical operation, so it implies the same non-snapshot mode.

Use these paired experiments when debugging an editor or build process:

```

printf 'unrecorded\n' > note.txt
jj log --ignore-working-copy -r @ -T 'commit_id ++ "\\n\\n"'
jj log -r @ -T 'commit_id ++ "\\n\\n" ++ description.first_line() ++ "\\n\\n"'

```

```

jj op log -n 3
printf 'second\n' >> note.txt
jj status
jj log -r @ -T 'commit_id ++ \" \" ++ description.first_line() ++ \"\\n\\n\"'

```

The exact number of operations depends on whether the first recorded tree already contained the file and on configuration. The invariant to test is not a fixed operation count; it is that ignored inspection cannot be used to claim current filesystem state, while normal snapshotting reconciles the working copy at the command boundary.

Identity through a rewrite

```

before: base--A--B--C(@)
      A id a1 / ch A
      B id b1 / ch B
      C id c1 / ch C

edit A: base--A'--B'--C'(@)
      A' id a2 / ch A
      B' id b2 / ch B
      C' id c2 / ch C

```

Identity	Before	After	Interpretation
Commit ID of A	a1	a2	New exact object because content/metadata changed.
jj change ID of A	ch A	ch A	Same logical evolution under ordinary rewrite.
Commit ID of B/C	b1/c1	b2/c2	Parent identity changed, so descendants are successors.
Bookmark at C	points c1	normally points c2	Local ref follows the rewritten graph, subject to conflict/divergence rules.
Remote bookmark	observes server object	still observes old object until publication/fetch	Remote observation is not local bookmark auto-follow.
Operation	O_before	O_after	Repository transition records the rewrite and collateral effects.

The table explains why a review workflow can retain a logical change identity while sending a new commit object to a Git host. It also explains why a descendant's commit ID is not evidence that its own files were edited: the parent ID alone is sufficient to change its object ID.

Visibility and divergence diagnosis

Visibility controls ordinary search, not object existence. An abandoned predecessor can remain in the store and in an evolution chain. A divergent change has more than one visible evolution for one jj change ID, usually because two operations independently rewrote the same logical change. A hidden predecessor and a divergent successor can coexist. Use `all()`, `hidden()`, `divergent()`, and `evolog` as distinct probes rather than one broad “show everything” command.

```
jj log -r 'divergent()'
jj log -r 'hidden()'
jj evolog -r 'change_id(CHANGE)'
jj op log
jj log -r 'heads(mutable() ~ ::remote_bookmarks())'
```

A visible head is a maximal node in a selected graph domain. It need not have a bookmark. A bookmark can point to an internal node with a child that is intentionally local. A remote bookmark can be a visible head without making all its ancestors immutable if the policy says otherwise. These relationships are reasons to state the revset domain whenever using a word such as “head”, “public”, or “unpublished”.

The Git branch and detached-HEAD analogy boundary test

Git intuition	jj result	Where the intuition fails
A branch names the current line of local work.	A bookmark names a selected commit.	A bookmark can be absent, point into the middle of a stack, or exist only to publish a review boundary.
There is one unnamed tip.	@ names this workspace's working-copy commit.	Other workspaces have other @ values and the graph can have many unnamed heads.
Editing a file dirties the checkout.	Editing dirties the next snapshot of @.	Normal jj commands may record that state; the boundary is not <code>git commit</code> .
Amend replaces HEAD.	Rewriting makes a successor with a normally stable change ID.	Descendants are automatically rebased and conflicts are values.
Branch is needed to publish.	Bookmark is created at publication.	Bookmarks are not the organising unit of local editing.
Reflog recovers ref movement.	Operation DAG and evolution log recover different questions.	Operation IDs are clone-local and not Git transport objects.

The analogies are therefore bridges, not the destination. Use the branch analogy to understand publication names and the detached-HEAD analogy to understand unnamed private work, then deliberately switch to the tuple, graph, identity, and operation vocabulary for anything involving rewriting, publication, or recovery.

Part III — Rewriting history as ordinary work

9. Patch-stack surgery

The default question is not “which branch do I rebase?” but “which commits should be rewritten, and where should their parent edges point?” Every rewrite produces successor commit IDs and usually preserves change IDs. Descendants are automatically rebased when a rewritten commit changes their parent or diff context.

Editing and metadata

Command	Graph/content effect	Use
<code>jj edit R</code>	Moves the workspace pointer; does not rewrite R.	Inspect or continue work on an existing change.
<code>jj describe R</code>	Rewrites description and selected metadata.	Rename a change while preserving its change ID.
<code>jj metaedit R</code>	Rewrites metadata without changing file content.	Author, committer, timestamp, or signing metadata as supported.
<code>jj diffedit R</code>	Replaces selected content using a diff editor.	Touch up a diff in place.
<code>jj squash</code>	Moves selected content into a destination, commonly its parent.	Fold fixups or selected files.
<code>jj split</code>	Replaces one change with two successors.	Separate logical concerns interactively or by fileset.
<code>jj absorb</code>	Moves individual diff hunks from a change into suitable mutable ancestors.	Distribute fixups across a stack.
<code>jj duplicate R</code>	Creates a new change with R's content.	Copy a change's content to another location; identity is new.
<code>jj abandon R</code>	Removes selected commits from visible graph and reconnects descendants.	Discard a change; recover through operation history if needed.
<code>jj restore</code>	Restores selected paths in a destination commit to content from a source tree; without explicit operands, restores @ from its parent.	Content undo or replacement without moving parent edges or selecting a new commit location.
<code>jj revert R</code>	Creates a new change applying the inverse of R.	Publish an explicit reversal; it is not abandonment.

For a stack `A-B-C-@`, editing A may produce `A'-B'-C'-@'`. B, C, and @ have new commit IDs because their parent trees changed; their change IDs identify the evolving logical changes. A conflict in B' is not a failed transaction to be continued: it is a value in B', and later operations can rewrite or resolve it.

Content movement example

```
before:      A -- B -- C -- @
             fixup in C

jj absorb:
after:       A' -- B' -- C' -- @'
             an individual diff hunk is applied to the earliest compatible mutable change

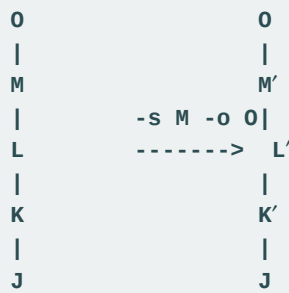
jj squash --from C --into B:
after:       A' -- B'' -- C'' -- @''
             C'' is generally empty or contains its remaining diff
```

Use `jj restore --from R --into S -- fileset` to undo or replace selected content in `S`: the selected paths in `S` are restored to the content they had in source commit `R`. In the common case, `jj restore` restores the working-copy commit from its parent, undoing the current commit's content changes while preserving its description and other metadata. Use `jj restore --changes-in R` when the intent is specifically to undo the diff introduced by `R`. This is different from `jj duplicate R`, which creates a new change with copied content, and from `jj revert R`, which creates a new inverse change. These are three different intentions that Git users often collapse into “cherry-pick”.

10. Rebasing selection and destination algebra

0.45.1 baseline. The installed help defines source selectors `-s/--source`, `-b/--branch`, and `-r/--revisions`, with `-b @` as the default. Destinations are `-o/--onto` (aliases `-d/--destination`), `-A/--insert-after` (alias `--after`), and `-B/--insert-before` (alias `--before`). The long forms are preferable in durable documentation.

`-s X` selects `X` and all descendants; multiple source roots become direct children of the destination. `-r X` selects exactly `X` (or the set supplied), preserving dependencies inside the set and filling holes by rebasing descendants. `-b X` selects the branch relative to the destination: in the common single-root case, `(destination..X)::`. None of these selectors means “a Git branch”.



`-o O` moves selected commits onto O and leaves O's existing descendants alone.

`-A O` inserts the selected commits after O and rebases O's existing descendants on top of them. `-B O` inserts before O by using O's parents, then rebases O and its descendants. Repeat `-o` to create a merge commit:

```
jj rebase -s L -o K -o M
```

That command gives L' parents K and M. It is often clearer than manually constructing a merge. `-A` and `-B` can be combined for merge surgery.

Rebase is graph transformation plus preservation of the selected diff. If the destination contains incompatible content, jj records the conflict in the rebased successor. A subsequent `jj resolve`, `jj diffedit`, or ordinary edit changes that value; there is no Git-style “rebase session” state that requires `--continue`.

11. Rewrite experiments: preserve a diff, change its context

Every history rewrite is easier to reason about when the diff and its parent context are written separately. A commit's effective diff is the difference between its tree and the automatic merge of its parents. Rebase changes parent context and asks jj to preserve that diff. Squash changes which commit owns content. Split partitions one diff. Restore changes selected file contents directly. Metaedit changes metadata while leaving the tree alone.

Operation	Primary input	Primary result	Typical identity result
<code>describe/metaedit</code>	metadata	same tree, new metadata	new commit ID; same change ID unless requested otherwise.
<code>diffedit</code>	one commit's diff	edited diff	successor of same change.
<code>squash</code>	source content and destination	content ownership moves	source/destination evolutions; descriptions need review.
<code>split</code>	one diff/fileset/individual diff hunks	two sequential changes	two resulting change identities; inspect evollog.
<code>absorb</code>	individual diff hunks plus compatible ancestors	inference-based distribution	several changes can evolve.
<code>restore</code>	source/destination commits or <code>changes-in</code>	Restores selected destination paths from a source tree, or undoes a commit's diff.	Destination successor; descendants may be rebased.
<code>revert</code>	selected commit diff	new inverse change	new change identity; original remains.
<code>duplicate</code>	selected commit tree	new copy at new parents	new change identity.
<code>abandon</code>	selected graph nodes	hidden nodes and reconnecting children	successors for affected descendants.

Amending an early change

```

before:
root -- A -- B -- C -- @
    |   |   |
    API tests docs

jj edit A
# edit API
jj describe -m 'API: corrected quota invariant'

after:
root -- A' -- B' -- C' -- @'
    |   |   |
    API tests docs
A' is the successor of A; B', C', and @' preserve descendants over new context.

```

The operation is not “change A and leave B/C hash identities intact”. B's parent ID changed, so B's commit ID necessarily changes even if its diff is textually identical. This is why commit IDs should not be used as stable review identity in local automation. A bookmark pointing to C follows the rewritten successor chain, while an already-pushed remote bookmark remains a server state to be updated.

Partial squash and diff ownership

```
before:                                after jj squash --from C --into B -- src/**
root - A - B - C - @                  root - A' - B' - C' - @'
      B: API                          B': API + src/ fix
      C: docs + fix                   C': docs
```

The result is not necessarily an empty C: the selected files moved, unselected content remained. If the fileset selects only part of a file, the diff editor's individual-hunk selection determines the actual partition. A child depending on the old tree is rebased automatically. Use `jj diff -r C'` and `jj show -r B'`, not only descriptions, to verify ownership.

Splitting a change without losing descendants

```
before:                                split A by tests
root -- A -- B                        root -- A_1 -- A_2 -- B'
      API+tests                       API      tests
```

B is not simply left pointing at A_1. Its diff must be reapplied over A_2, so B' is a successor and can conflict if B depended on the combined original tree. A publication bookmark at B follows B' locally. If A was already uploaded to a review system, review identity and commit identity must be considered separately; for Gerrit, the footer in each resulting description is a server concern.

Abandoning an ancestor

```
before:                                abandon B
A -- B -- C -- D                      A ----- C' -- D'
  \                                     C' preserves C's diff over A where possible.
   useful content                      With --restore-descendants, content preservation
                                       rather than diff preservation changes the trade-off.
```

Abandon is therefore not equivalent to deleting a node from a drawn graph. It changes the parent context of descendants. The default tries to preserve descendant diffs; `--restore-descendants` instead preserves their content, which can change their effective diff. Use the option only when that distinction is the desired one.

Rebase source selection as a set transformation

For a non-linear graph, write out the source set before choosing a destination:

```
jj log -G -r 'roots(main@origin..@)'  
jj log -G -r 'main@origin..@'  
jj rebase -r 'roots(main@origin..@)' -o main@origin
```

`-r` operates on the explicitly selected commits and preserves dependencies inside that set. `-s` selects a commit and all descendants, which may include unrelated side branches. `-b` is a branch-oriented convenience selection relative to the destination and is useful for the usual stack but should be inspected in a fork. If the selected set has multiple roots, each root can become a child of the destination; this is a feature, not an implicit merge.

Insert-after and insert-before

before:	<pre> B C \ / A</pre>	after -A A:	<pre> B C \ / X A</pre>
before:	<pre>A -- B -- C</pre>	after -B B with X:	<pre>A -- X -- B' -- C'</pre>

Insert-after creates a new intermediate change after selected commits and rebases their existing children on it. Insert-before places selected commits before the target and rebases the target and its descendants. These operations alter edges beyond the selected commit, so bookmark and descendant consequences are larger than a simple “move X”. Use the long option names in scripts and inspect the resulting

graph.

Conflicts are results, not interrupted sessions

When a diff cannot be reapplied cleanly, `jj` stores a conflict in the successor commit. The rewrite operation can still finish, and descendants can be rebased over that conflicted value. This means the next command can be unrelated work, `jj resolve`, another rebase, or a deliberate inspection. The absence of a `--continue` flag is a consequence of storing the intermediate state, not a promise that conflicts are less real.

Rewrite safety protocol

1. Save the current operation ID and log view for a complicated rewrite.
2. Inspect the source set and destination set separately.
3. Run a dry/read command showing the expected paths and commits.
4. Perform the rewrite with an explicit long-form option.
5. Inspect `jj log -r '::-@'`, changed IDs, bookmarks, and `conflicts()`.
6. Use `jj evolog` for change continuity and `jj op show` for repository-wide effects.
7. Only then move or push publication bookmarks.

Worked model: one filesystem, three identities, two histories

It is worth slowing down over a state that Git users often describe with a single word—“the current commit”—because `jj` deliberately exposes several different objects. In the following laboratory, use `0` for the operation head, `C` for the commit currently checked out in the workspace, `W` for the workspace record, and `R` for a remote bookmark. These are not alternative spellings for one pointer.

```
operation view 0_0:
  root--A--B--C(@)
      ▲
      \-- local workspace W/dev records C

Git publication view:
```



```
main@origin -- B
(no local bookmark)

working directory:
files on disk = snapshot(C), plus an unrecorded edit to src/parser.rs
```

After editing `src/parser.rs`, the file has changed but the commit graph has not. A command that needs a current working-copy view normally snapshots the filesystem into a new working-copy commit. The result is not “the file is staged”; there is no index-shaped intermediate object in the jj model. The graph becomes:

```
root--A--B--C--D(@)
      ▲
      \-- D is the new working-copy commit

main@origin -- B
filesystem = snapshot(D)
```

The change ID normally remains stable as C evolves into D because the evolution is a rewrite of one logical change. The commit ID changes because the commit object contains a new tree and therefore has new content. If C had a descendant E, jj may rebase E automatically onto D. That rebase can produce a new commit ID for E even though no user explicitly selected E. The operation log records the whole transaction, including both the direct snapshot and descendant update.

Now create a local publication bookmark:

```
jj bookmark create parser-review -r @
jj git push --bookmark parser-review --remote origin
```

The state now has a local publication name and a remote observation. Editing @ again creates E, but `parser-review` still points to D until moved. This is the key separation:

```
root--A--B--C--D--E(@)
      ▲
parser-review ----+? no: it still points to D

main@origin -- B
```

In a graph display the bookmark glyph may make the relationship look visually close to @, but it is a named ref with its own target. A publication operation is therefore explicit state transition: move the bookmark, then push it. This separation lets either a branch-based or a detached-HEAD-style local workflow continue after publication without turning every local movement into a remote branch update.

Snapshot timing is part of the command contract

“The working copy is a commit” does not mean every shell write immediately creates a repository operation. The filesystem can be ahead of the last recorded snapshot. Commands that inspect or mutate repository state may first update the working-copy commit; global options can alter that behaviour. A reproducible script should choose deliberately between a live view and a repository-only view.

Question	Live default	Stable/explicit alternative
Should disk edits be represented?	Allow the command to snapshot the workspace.	<code>--ignore-working-copy</code> for a stored-graph query.
Should the command use the current integrated operation?	Use the repository's normal operation head.	<code>--at-operation OP</code> for historical inspection.
Should a rewrite cross a policy boundary?	Respect immutable heads.	<code>--ignore-immutable</code> only as a deliberate override.
Should an automation operation become the caller's current operation?	Integrate normally.	<code>--no-integrate-operation</code> , followed by explicit inspection/integration.

These switches are not interchangeable. Ignoring the working copy changes what is read from disk; selecting an earlier operation changes the entire repository view; ignoring immutability changes which rewrite is permitted; suppressing operation integration changes the operation DAG's current head. Treat them as four independent axes in a script's design.

Commit identity under rewrite

A commit ID identifies one immutable commit object in the backend. A change ID identifies an evolving logical change and is carried through its sequence of rewritten commit objects. The distinction becomes visible in a three-step amendment:

```
logical change X:  
X_0 = commit id 1a..., change id k1..  
X_1 = commit id 4f..., change id k1... (description amended)  
X_2 = commit id 92..., change id k1... (parent rebased)  
  
predecessor relation:  
X_0 → X_1 → X_2
```

When a rewrite is copied instead of evolved—`jj duplicate`, for example—the new commit receives a new change identity. A later operation that edits the duplicate evolves the duplicate’s change, not the source. This is why a review workflow should decide whether feedback means “new patch set for the same logical change” or “a separate change derived from this one”. The command choice encodes that decision.

Commit IDs are suitable for immutable evidence: a build record, a fetched object, a server-side patch set, or a signed object. Change IDs are suitable for local evolution and for identifying a logical change while its commit ID changes. Neither should be used as a replacement for a publication bookmark: a bookmark names a movable graph position, while an ID names a commit or logical evolution.

Mutable and immutable are graph predicates

A useful mistake to avoid is treating mutable as a synonym for private. Mutability is a property of whether `jj`’s rewrite policy permits changing a commit; publication is a property of whether a ref has been sent to another repository. A mutable commit can be pushed, and a user may intentionally amend it before a host-side or native-Git force update. Conversely, a private commit can be made immutable by configuration. Use revsets to ask the precise question you need:

```
jj log -r 'mutable() & ::@'
jj log -r 'immutable_heads()'
jj log -r 'hidden()'
jj log -r 'divergent()'
```

Visibility is another independent predicate. Hidden predecessors are omitted from ordinary output and selection, but they remain part of the commit store and can be reached through evolution-oriented commands or explicit historical views.

Divergence means that multiple visible commits share a change identity; it is not merely “two commits with similar text”. A convergence operation must decide how to combine those evolutions, and that decision deserves an operation-log checkpoint.

Workspaces make @ plural without making the graph plural

Each workspace has a workspace-local working-copy commit. The symbol @ resolves relative to the workspace from which the command is run. The commit store and operation history are shared by workspaces in one repository, so a rewrite in one workspace can update or stale another workspace’s checkout.

```
shared commit DAG:
  trunk--A--B
           |-- dev@ C
           \-- review@ D

workspace records:
dev    → C
review → D
```

`jj edit C` in the review workspace changes review’s @; it does not move dev’s @. A graph rewrite can nevertheless affect both records if the commits they name are rewritten. The operational rule is to use explicit commits in scripts and use @ only where workspace-local intent is genuinely wanted. For dashboards, build jobs, and review commands, print both the workspace name and the resolved commit ID.

Rewrite laboratory: diff ownership, context, and descendants

History editing in jj is best understood as changing diff ownership in a graph. The following stack has a base, an API change, a test change, and a working copy:

```
main@origin
|
A  (base)
|
B  (API)
|
C  (tests)
|
@  (documentation edits)
```

Suppose a review comment says that one individual diff hunk in C belongs in B. The operation is not “stage that diff hunk, amend B, and hope C can be replayed”. First choose the logical operation: move content from a descendant into an ancestor while preserving the rest of both changes. One safe sequence is:

```
jj edit C
jj squash --into B --interactive
jj edit B
jj describe -m "API: expose parser diagnostics"
jj log -r 'B:: & mutable()'
```

The interactive squash changes B’s tree and leaves only the residual diff in C. When B evolves, C and @ are descendants and are automatically rebased. If their diffs apply cleanly, their logical changes continue with new commit IDs. If the context no longer exists, the rebased descendant records a conflict rather than forcing the user into an interrupted rebase session.

```
before:
A--B(API)--C(tests)--@ docs

after moving one diff hunk from C to B:
A--B'(API + selected change)--C'(remaining tests)--@' docs

possible conflict:
A--B'--C'[conflict]--@[conflict propagation]
```

The important invariant is that the command's source and destination are explicit. "Squash" is not a universal synonym for "combine whatever is nearby": it has a source commit, a destination commit, optional file selection, and rules for descriptions and empty commits. The same discipline applies to `restore`: restore content from a source tree into a destination commit; do not describe it as a reset unless you have specified which tree and which commit are changing.

Choosing a rewrite primitive

Intent	Primitive	Identity consequence	Graph consequence
Make the current change's description or metadata correct	<code>describe</code> , <code>metaedit</code>	Evolves the same change.	Usually no parent/child topology change.
Change which files or diff hunks belong to a change	<code>diffedit</code> , <code>squash</code> , <code>split</code> , <code>absorb</code>	Source/destination changes evolve; a split creates new logical changes as appropriate.	Diff ownership changes; descendants may rebase.
Move an existing logical change to another parent	<code>rebase</code>	Change identity is intended to persist.	Parent edge changes; descendants follow.
Create another logical copy	<code>duplicate</code>	New change identity.	New node and descendants only if selected later.
Remove a node's contribution while retaining descendants	<code>abandon</code>	Source disappears from visible graph; descendants are rebased.	Topology is simplified, possibly with conflicts.
Apply an inverse diff	<code>revert</code>	New change; source remains.	New child or selected destination receives inverse content.

Rebase source modes

The three source selectors answer different graph questions. `--branch REV` selects a branch-like region from the specified commit to its descendants under the command's source rules. `--source REVSET` selects the commits in the set and their relevant descendants according to rebase semantics. `--revisions REVSET` selects exactly the commits to rewrite. In a linear stack these can appear equivalent; in a fork they are not.

```
      B--C
     /
A-----+
     \ D--E

--revisions 'C'      selects C
--source 'B'         selects the source region rooted at B
--branch 'B'         selects the branch beginning at B
```

Do not infer selection from the visual nearest node. Evaluate the revset first with `jj log -r`, then run the rebase with an explicit destination. A destination is a graph insertion instruction: `--destination/--onto` places the selected commits after a parent; `--insert-after` and `--insert-before` place them relative to a commit while preserving a different relationship. Repeating a destination can intentionally create a merge-shaped result. If the destination is a set, the result can have multiple parents; this is a graph operation, not a textual “move branch” shortcut.

When automatic descendant rebasing surprises you

Automatic rebasing is a consequence of maintaining a coherent graph after a rewrite. It is surprising when the user thinks only of the selected node and forgets that descendants refer to its old commit ID. Consider a child that touched the same lines as its rewritten parent:

```
old: A--B(line 10 = old)--C(line 10 = test)--D
new: A--B'(line 10 = new)--C'[conflict]--D'[conflict]
```

The conflict belongs to C's relationship to B'; D' may display conflict state because its parent is conflicted or because its own diff cannot be applied. Resolve at the earliest commit whose conflict is conceptually understood, then inspect descendants. Resolving only the working copy can leave the earlier stack member conflicted and cause the next rewrite to re-materialise the problem.

Rewrite exercises with invariants

The safest way to learn a rewrite command is to write an invariant before executing it. An invariant is a statement about what must remain true; it is stronger than "the command should probably work". Examples include "the API change remains one logical change", "the test change remains a descendant of the API change", "the release bookmark does not move", and "files outside the selected fileset are byte-for-byte unchanged". After the operation, test the invariant with a `revset`, `diff`, `template`, or external test.

Exercise: amend an ancestor without losing the stack

```
before:
main@origin--A--B--C--D--@
                API tests docs current
```

```
invariant:
B's change ID remains the API review
C and D remain descendants
main@origin is unchanged
```

```
jj bookmark create safety/ancestor-rewrite -r @
api=$(jj log -r 'subject(substring:"API")' --no-graph -T 'change_id ++ "\n"')
jj edit 'subject(substring:"API")'
# edit implementation
jj describe -m 'API: enforce the documented invariant'
jj log -r 'safety/ancestor-rewrite::@'
jj log -r 'main@origin'
jj log -r '::-@ & conflicts()'
```

Compare the old and new API commit IDs through the safety bookmark and `evolog`. The change ID should identify the evolution, while C and D may have successor commit IDs. If the safety bookmark is deleted before inspection, the

graph can still be recovered through operations, but the user's intended boundary is no longer named.

Exercise: split by path, then repair the boundary

```
X:
src/api.rs
src/cli.rs
docs/api.md
tests/api.rs

desired:
X_1 = src/api.rs + tests/api.rs
X_2 = src/cli.rs + docs/api.md
```

A path-only split cannot express the first grouping because it is not a directory boundary. Start with an interactive split or use a temporary split followed by squash. The key is to preserve a testable first commit:

```
jj split -r X --interactive
jj log -r 'X::@'
jj diff -r X
jj diff -r 'X+'
# If tests belong in the implementation change:
jj squash --from 'X+' --into X -- 'root-file:tests/api.rs'
```

The resulting descriptions matter. A commit containing `src/api.rs` but no test can be acceptable if the project's test suite supports it; otherwise the split is structurally neat but semantically poor. The operation log can reverse the whole experiment, which is preferable to trying to reconstruct diff ownership by hand.

Exercise: duplicate versus revert

```
A--B(feature)--C
\-----D(other line)

duplicate B onto D:
A--B--C
\----D--B'

revert B after C:
A--B--C--R(inverse of B)
```

Use `duplicate` when the feature should be applied as a new logical change in another context. Use `revert` when the original feature remains part of history but its effect should be counteracted. The two can produce similar files and radically different review semantics. A duplicate should receive a new change ID; a revert is also a new change whose description should explain which prior effect it negates.

Exercise: abandon an ancestor with dependent descendants

```
jj bookmark create safety/abandon -r @
jj abandon B
jj log -r 'safety/abandon::@'
jj log -r '::~@ & conflicts()'
jj op show
```

Abandoning B removes B's contribution and reconnects descendants according to jj's rules. If C depended on B's files, C may become conflicted or change content. This is not an operation to use for "cleaning an ugly local history" without reviewing descendants. If B was a publication boundary, inspect every bookmark and remote bookmark that reached it.

Exercise: arrange a fork deliberately

```
A--B--C
 \
  D--E
```

With an arrangement operation, select only the intended graph region and retain a safety bookmark at the head. Afterward use parent output rather than visual indentation to prove whether the result is a fork, a linear reordering, or a merge. For a scripted result, express the same intent with `new`, `rebase`, or `parallelize`, which make the source and destination visible in a code review.

Patch-stack review as a graph invariant

A stack is useful when each change has a review-sized purpose and its parent relationship reflects a dependency. A stack is not automatically a linear sequence: a merge, fork, or independent sibling can be part of a larger review graph. Before publication, classify each edge:

Edge type	Meaning	Review implication
Dependency edge	Child diff assumes parent's API/content.	Parent rewrite can rebase/conflict the child.
Ordering-only edge	Child happens later but could stand alone.	May be parallelised or published separately.
Integration edge	Merge combines independent lines.	Parent order and conflict resolution are semantic.
Publication edge	Bookmark/ref names a tip.	Moving it changes server-visible reachability, not logical dependency.

This classification guides rebase and publication. A GitHub stacked PR generally wants explicit linear dependency edges. Gerrit can represent a linear patch series and topics, but server policy still determines how it displays independent parents. A local graph can be richer than either host's review UI; publication should select the projection that the host understands.

Metadata surgery and identity policy

Descriptions are part of commit identity and review protocol. A description may carry a human subject, body, trailers, Gerrit Change-Id, issue references, release notes, and automation markers. Changing it evolves the commit ID and can rebase descendants even if the tree is unchanged. Decide whether a trailer belongs to the logical change before using `metaedit --update-change-id`.

Author metadata describes who authored the change; committer metadata describes the operation that created the current commit object. An amended change can retain its author while changing its committer timestamp. Rewriting an

imported commit can therefore be visible in both the object ID and metadata even when the diff content is identical. Queries using author and committer predicates answer different audit questions.

```
jj log -r 'author_email(exact:"author@example.org")'  
jj log -r 'committer_email(exact:"rewriter@example.org")'  
jj log -r 'author_date(after:"2026-01-01") & committer_date(after:"2026-09-01")'
```

Use explicit timestamps in reproducible release work. A “recent” revset based on the local clock is useful interactively but is not a stable build input. The operation log adds a local record of when a rewrite happened, while the commit’s author/committer fields are part of the Git object exported to a remote.

Rewrite recovery checklist

1. Stop mutating the repository.
2. Save the current operation ID and workspace name.
3. Save a graph listing with full commit and change IDs.
4. Identify whether the mistake is a tree edit, parent rewrite, ref movement, or operation-view problem.
5. Use `op diff` and `evolog` to identify the intended predecessor/successor.
6. Prefer one operation-level undo/restore over manual reconstruction.
7. Re-run the invariant checks and tests.
8. Move or push bookmarks only after the local graph is correct.

The checklist is intentionally conservative around public refs. An operation can be undone locally after a push, but the remote ref remains changed until a new push updates it. Treat local recovery and publication rollback as two separate incidents with separate evidence.

Rewrite laboratory: diff ownership and graph surgery

History rewriting is ordinary jj work, but “ordinary” does not mean consequence-free. A rewrite changes an exact commit object and can change the parent identity supplied to every descendant. The useful unit of planning is the transformation of a graph region: identify source nodes, choose new parent edges, predict successor relationships, and inspect the resulting diff stack.

Select the operation by intent

Intent	Primary command	Why not the tempting equivalent?
Change only the message or metadata.	<code>describe/metaedit</code>	<code>restore</code> changes file contents; <code>revert</code> adds inverse content.
Touch up selected diff hunks in one change.	<code>diffedit</code>	<code>squash</code> moves content across a boundary.
Move selected content into another change.	<code>squash</code>	<code>duplicate</code> creates another logical change.
Divide one diff into reviewable pieces.	<code>split</code>	<code>new</code> creates a boundary but does not redistribute an existing diff.
Move a whole change/stack to a new base.	<code>rebase</code>	<code>edit</code> changes the workspace pointer, not parent edges.
Copy a change's diff to another line.	<code>duplicate</code>	<code>rebase</code> follows the same logical change; copy needs a new identity.
Publish an inverse without rewriting the original.	<code>revert</code>	<code>restore</code> changes a destination tree and can erase ownership evidence.
Declare a change no longer wanted.	<code>abandon</code>	Deleting a bookmark only removes a name.

The commands can be composed, but the order communicates ownership. Split before squash when one mixed diff should become two review changes; squash before rebase when the destination context should be the context in which content is reviewed; rebase before editing an early change when the conflict should be resolved against the new base.

Rebase source matrix

```
base--A--B--C(@)
```

```
  \
   X--Y
```

```
-r B      select exactly B
-s B      select B and every descendant in the visible graph
-b B      select B's branch relative to the destination
-r B|X    select two roots/regions; inspect whether they are connected
-r B|C    select a set with a hole; do not infer a stack from labels
```

`-r` is the precise choice when the selected set itself is the policy. `-s` is the stack choice when a change and all descendants should move together. `-b` is a branch-relative convenience and depends on the destination. A source set can contain several roots and a destination can be repeated; the resulting operation may create several children or a merge shape. Never replace a source selector with another because it is shorter without evaluating the set.

```
jj log -G -r 'B'
jj log -G -r 'B::'
jj log -G -r 'roots(B::)'
jj log -G -r 'heads(B::)'
jj rebase -r B -o X
jj rebase -s B -o X
jj rebase -b B -o X
```

Destination matrix

Destination	Graph operation	Failure/edge case
<code>-o Y/-d Y</code>	Place selected source on Y.	Y inside the source closure would create a cycle and is rejected.
Repeated <code>-o Y -o Z</code>	Give the selected change multiple parents.	Creates a deliberate merge; review both parent diffs.
<code>-A Y</code>	Insert selected commits after Y.	Y's existing descendants are rebased over the insertion.
<code>-B Y</code>	Insert selected commits before Y.	Y and its descendants may receive successors.

```
before: base--A--B--C
          \
          X--Y
```

```
-r B -o X:
base--A--B'--C'
          \
          X
```

```
-r B -A X:
base--A--X--B'--C'
```

```
-r B -B X:
base--A--B'--X'--Y'
```

The apparent similarity between `-o`, `-A`, and `-B` hides which existing edge is replaced. `-o` chooses parent destinations for the selected source. Insert-after and insert-before alter an existing line and consequently move a larger graph region. Use a temporary bookmark at the intended tip and compare `jj op show -p` when testing a new pattern.

Ancestor rewrites and descendants

Suppose C is a child of B and its commit object records B's commit ID as a parent. If B becomes B', C's old object cannot remain the current representation of the same logical change on that edge: its parent field would be wrong. `jj` creates a successor C' whose diff is applied to B'. If the diff applies cleanly, C' is clean. If it does not, C' stores a conflict. If C's description or tree was independently rewritten at the same time, its change evolution must still be interpreted separately.

```
before: main--B--C--D(@)
rewrite B:
after:  main--B'--C'--D'(@)
        B/B' same change IDs
        C/C' same change IDs
        D/D' same change IDs
        all commit IDs may differ
```

A bookmark targeting D normally follows the rewritten graph target, while a remote bookmark remains the last fetched server observation. If a local bookmark and remote bookmark now identify different descendants, that is exactly the state to

inspect before a push. The operation log captures the bulk rewrite; each evolution log captures one logical change's continuity.

Partial squash and diff ownership

```
A--B(@)
B diff:
  src/parser.rs
  docs/parser.md
  tests/parser.rs

squash docs into A:
A'--B'(@)
A' owns docs/parser.md
B' owns src/parser.rs + tests/parser.rs
```

A fileset narrows the content transfer, but individual diff-hunk boundaries can still matter within a file. Use an interactive diff editor when the policy is “some portions of this file”. After a partial squash, inspect both source and destination diffs. An empty source may remain as an intentional message boundary or may be abandoned; do not infer the desired cleanup from its emptiness.

```
jj diff -r @
jj squash --into @- -- 'root-file:"docs/parser.md"'
jj diff -r @-
jj diff -r @
jj log -G -r '::@'
```

Split with a descendant

```
before: A--B--C(@)
        B = API + docs
        C = tests depending on API

split B:
after:  A--B1--B2--C'(@)
        API docs tests rebased
```

Splitting introduces a new parent edge between B1 and B2. C is automatically rebased over the resulting sequence. If C's diff was written against the combined B tree, the rebase can be clean or conflicted depending on context. Verify both diffs and the change IDs of B1/B2/C'. A split creates a new logical change for one side; it

does not preserve one change ID across both output commits.

Duplicate, revert, abandon: three histories

duplicate:	main--A--B	
	\	
	Bcopy	new change identity
revert:	main--A--B--R	R applies inverse of B
abandon:	main--A----C	B removed/hidden, C reconnected as policy permits

Duplicate copies a tree/diff to a new location and keeps the original. Revert adds an explicit inverse commit, preserving the original's history and review evidence. Abandon removes the original from ordinary work and asks jj to update dependants; it is appropriate only when the change itself is no longer part of the intended graph. If a remote review exists, remote cleanup remains separate.

Rewrite safety protocol

1. Write the graph question in words and select a revset.
2. Render the source set and destination set with no graph and a template.
3. Check immutability and publication refs.
4. Create a temporary bookmark at a useful tip.
5. Run the smallest operation that expresses the intent.
6. Inspect commit/change IDs, descendants, bookmarks, conflicts, and operation detail.
7. Run tests against the new workspace tree.
8. Publish only after the local graph and remote comparison are understood.

This is not ceremony borrowed from Git. Mutable history makes small edits cheap, so the scarce resource becomes accurate graph intent. An operation checkpoint gives recovery; a change ID gives continuity; a bookmark gives a publication handle; and a revset gives a repeatable selection.

Part IV — Revsets: a query language

12. Revsets as values

A revset is a value denoting a set of commits. Commands impose cardinality constraints: `jj edit`, `jj describe`, and many destination arguments require exactly one; `jj log -r`, `jj abandon`, `jj rebase -r`, and `jj diff` can accept sets. A syntactically valid expression can therefore still be operationally invalid.

Symbols and resolution

`@` is the current workspace commit; `workspace@` selects another workspace. Full or unique-prefix commit IDs resolve to one commit. Full or unique-prefix change IDs resolve to a visible commit, except ambiguous/divergent IDs require an offset such as `xyz/0`. Tags, bookmarks, and remote bookmarks are symbols. In 0.45.1 the documented precedence is tag name, bookmark name, then commit/change ID; use `commit_id(X)`, `change_id(X)`, `bookmarks(exact:"X")`, or `tags(exact:"X")` when scripts must disambiguate.

Shell quoting is part of the language boundary. Quote expressions containing `|`, `~`, parentheses, or spaces. A literal symbol that looks like an operator can require nested quoting, for example `jj log -r '"x-"'`.

Revset operators and precedence

The table gives operator precedence, not an evaluation order over commits. First the parser groups the expression according to this table; then `jj` resolves each operand to a set and applies the operator to those sets. The distinction matters because `x.y` and `x:y` are graph operations, while `~`, `&`, and `|` are set operations.

For the set-notation column, let `U` mean the current revset search domain, `Anc(S)` mean `S` plus all ancestors of commits in `S`, `Desc(S)` mean `S` plus all descendants of commits in `S`, `Par(S)` mean the immediate parents of members of `S`, and `Ch(S)`

mean their immediate children. These definitions make clear which expressions include their operand and which do not.

Precedence	Form	Plain English	Pure set notation	Boundary and common mistake
1 — strongest	<code>f(x)</code>	Call function <code>f</code> with the supplied revset or pattern arguments.	<code>f :</code> <code>(Set/Pattern)^n</code> <code>→ Set</code>	Function arguments bind before surrounding operators. A valid expression can still fail because the function requires one commit, a pattern, or a particular argument count.
2	<code>x-</code>	All immediate parents of commits in <code>x</code> .	<code>Par(x)</code>	Only one edge is followed. A root contributes nothing; a merge can contribute several parents.
2	<code>x+</code>	All immediate children of commits in <code>x</code> .	<code>Ch(x)</code>	Only one edge is followed. A fork can contribute several children; use <code>x::</code> for all descendants.
3	<code>p:x</code>	Interpret <code>x</code> using pattern kind or alias <code>p</code> .	<code>parse_p(x)</code>	This changes how a predicate argument is interpreted; it does not traverse the commit graph. Pattern kinds include <code>exact:</code> , <code>glob:</code> , and <code>regex:</code> .
4	<code>x::</code>	<code>x</code> and every descendant of every commit in <code>x</code> .	<code>Desc(x)</code>	It includes <code>x</code> , follows every child edge, and can contain forks, merges, and several tips. It is not a first-parent chain.

Precedence	Form	Plain English	Pure set notation	Boundary and common mistake
4	$x..$	Every commit in the current domain that is neither x nor an ancestor of a commit in x .	$U \setminus \text{Anc}(x)$	It is not “ x and its descendants”. Because a commit is its own ancestor, x is excluded. It can include unrelated branches and descendants of x .
4	$::x$	x and every ancestor of every commit in x .	$\text{Anc}(x)$	It includes the root when reachable and follows every parent of a merge, not only the first parent.
4	$..x$	Every ancestor of x , including x , except the root commit.	$\text{Anc}(x) \setminus \{\text{root}\}$	Do not confuse it with $x..$: the former is an ancestor set with the root removed; the latter is a complement of an ancestor set.
4	$x::y$	Commits that are descendants of x and ancestors of y . It is the corridor from x towards y .	$\text{Desc}(x) \cap \text{Anc}(y)$	Exactly $x::$ & $::y$. It includes either endpoint only when that endpoint satisfies the graph relation.
4	$x..y$	Commits reachable backwards from y that are not also reachable backwards from x .	$\text{Anc}(y) \setminus \text{Anc}(x)$	It can include side history brought in through a merge; it is not necessarily one first-parent path.
4	$::$	All visible commits in the current repository view.	U	It normally excludes hidden commits unless the expression explicitly introduces them.
4	$..$	All visible commits except the root commit.	$U \setminus \{\text{root}\}$	Use $::$ when the root belongs in the domain.

Precedence	Form	Plain English	Pure set notation	Boundary and common mistake
5	$\sim x$	Everything in the current domain except the commits in x .	$U \setminus x$	This is not "ancestors not in x ". To exclude an ancestry cone, write $U \sim :: x$ or an explicitly bounded equivalent.
6	$x \& y$	Only commits that belong to both x and y .	$x \cap y$	It filters sets; it does not create a merge commit.
6	$x \sim y$	Commits in x , after removing every commit that is also in y .	$x \setminus y$	It removes the selected set, not automatically its ancestors or descendants. Equal precedence means $x \sim y \& z$ is $(x \sim y) \cap z$.
7 — weakest	$x y$	Every commit that is in x , in y , or in both.	$x \cup y$	Union is weaker than intersection and difference, so $x y \& z$ means $x \cup (y \cap z)$.

These are the jj 0.45.1 operators. Equal-strength infix operators associate left-to-right. The identities worth memorising are $x :: y = x :: \& :: y$, $x..y = ::y \sim ::x$, $..x = ::x \sim \text{root}()$, and, in the ordinary visible domain, $x.. = \sim ::x$. The last identity is domain-sensitive when hidden commits have been explicitly introduced. Notice the precise meaning of $x..$: because $x \in \text{Anc}(x)$, it excludes x itself. The unusual identity is the left-hand range union: $(A | B).. = A.. \& B..$, not $A.. | B..$.

Evaluate operators on one explicit graph

Use this small merge graph for the examples below. Every set is written explicitly; braces mean set membership, not log display order. The arrows point from parent

to child, so D has two parents, B and C.



Parents: $D = \{B, C\}$, $B = \{A\}$, $C = \{A\}$, $A = \{\text{root}\}$, $\text{root} = \{\}$
 Children: $\text{root} = \{A\}$, $A = \{B, C\}$, $B = \{D\}$, $C = \{D\}$, $D = \{\}$

Expression	Expansion	Exact result	What the result demonstrates
D^-	Immediate parents of D	$\{B, C\}$	A merge has two immediate parents.
A^+	Immediate children of A	$\{B, C\}$	A child query can return a fork.
$B::$	Descendants of B, including B	$\{B, D\}$	It does not include sibling C or their common ancestor A.
$::B$	Ancestors of B, including B	$\{\text{root}, A, B\}$	It follows B's parent edge back to the root.
$B..$	All visible commits minus $::B = \{\text{root}, A, B\}$	$\{C, D\}$	The result includes the sibling C and merge D; it is not B's descendants.
$..B$	$::B$ minus root	$\{A, B\}$	The result is an ancestor prefix with the root boundary removed.
$B::D$	$B:: \cap ::D = \{B, D\} \cap \{\text{root}, A, B, C, D\}$	$\{B, D\}$	The corridor keeps B and D but excludes sibling C.
$B..D$	$::D \setminus ::B = \{\text{root}, A, B, C, D\} \setminus \{\text{root}, A, B\}$	$\{C, D\}$	The range excludes B's ancestry and includes the side parent C.
$(B C)::$	Descendants of both fork members	$\{B, C, D\}$	Traversal distributes over a union of starting points.

Expression	Expansion	Exact result	What the result demonstrates
<code>(B C)..</code>	$B.. \cap C.. = \{C,D\} \cap \{B,D\}$	<code>{D}</code>	Left-side range with a union is an intersection of complements.

Run the same calculations in a fixture with stable descriptions and suppress the graph renderer when checking membership:

```
jj log --no-graph -r 'D-' -T 'description.first_line() ++ "\n"'
# expected set: B, C
jj log --no-graph -r 'B::D' -T 'description.first_line() ++ "\n"'
# expected set: B, D
jj log --no-graph -r 'B..D' -T 'description.first_line() ++ "\n"'
# expected set: C, D
jj log --no-graph -r '(B|C).. ' -T 'description.first_line() ++ "\n"'
# expected set: D
```

The order printed by `jj log` is a topological presentation chosen by the command; it is not part of the set's meaning. To assert a result in automation, render a stable identifier or description, sort it in the shell if order is irrelevant, and compare the resulting membership. To understand a surprising result, first print each operand separately, then print the parenthesised expression that combines them.

Precedence worked from the parse tree

These expressions look similar but ask different questions:

Written expression	Parser grouping	Meaning
<code>A B & C</code>	<code>A (B & C)</code>	Take all of A, then add only the commits common to B and C.
<code>(A B) & C</code>	Explicit grouping	Take the union first, then retain only members also in C.
<code>A ~ B & C</code>	<code>(A ~ B) & C</code>	Remove B from A, then intersect with C; it is not <code>A ~ (B & C)</code> .
<code>mutable() & @:: bookmarks()</code>	<code>(mutable() & @::) bookmarks()</code>	Show the current mutable descendant domain plus every bookmark target.

Written expression	Parser grouping	Meaning
<code>mutable() & (@:: bookmarks())</code>	Explicit grouping	Show only mutable commits that are either current descendants or bookmark targets.
<code>::A B</code>	<code>(::A) B</code>	Ancestors of A, plus B; it is not ancestors of the union.
<code>::(A B)</code>	Explicit grouping	Ancestors of both A and B, including ancestors introduced only through B.

For a policy expression, write the intended set in words before writing punctuation. For example, “mutable descendants of @, plus publication tips” is `(@:: & mutable()) | bookmarks()`. “Mutable commits in either the current descendant set or the bookmark targets” is `mutable() & (@:: | bookmarks())`. Those are different policies, even if a simple repository happens to make their results equal.

Patterns, dates, and aliases

String predicates accept pattern kinds such as `exact:`, `substring:`, `glob:`, and `regex:`, with quoting for spaces and punctuation. Date predicates use one of two explicitly named patterns: `after:"DATE"` is inclusive, while `before:"DATE"` is exclusive. A bounded interval is their intersection, not a `..` expression inside the date argument:

```
author_date(after:"2026-08-01") & author_date(before:"2026-09-01")
```

The date string may be an ISO date such as `2024-02-01`, an ISO date-time such as `2024-02-01T12:00:00` or `2024-02-01T12:00:00-08:00`, a space-separated date-time such as `2024-02-01 12:00:00`, or a supported relative form such as `2 days ago`, `5 minutes ago`, `yesterday`, `yesterday 5pm`, or `yesterday 10:30`. Quote the complete revset for the shell and quote the date string inside the pattern. Prefer an absolute timestamp for reproducible reports; relative dates are evaluated when the query runs.

The important rule is that a predicate does not receive an untyped “search string”. It receives a value in the revset pattern language. If the kind is omitted, jj 0.45.1 treats

a string as a glob pattern. Every string predicate below therefore has two questions: which commit field is being inspected, and which pattern semantics are applied to that field.

Pattern	Matching rule	Positive example	Contrast and composition
<code>exact:</code>	The complete field must equal the supplied string.	<code>subject(exact:"Fix parser")</code> matches exactly that subject.	<code>subject(exact:"parser")</code> does not match <code>Fix parser</code> ; combine with <code>&</code> <code>author_email(...)</code> to disambiguate a duplicate subject.
<code>substring:</code>	The field contains the supplied literal substring. Metacharacters are ordinary text.	<code>description(substring:"CVdescription(substring:"a.b")</code>	does not mean "a, any character, b"; use <code>regex:</code> for that.
<code>glob:</code>	Unix-style wildcard matching, using the globset syntax linked by jj's help.	<code>subject(glob:"fix: *")</code>	<code>*</code> is a wildcard, not a regex quantifier. Use <code>glob-i:</code> when case should not matter.
<code>regex:</code>	Regular-expression substring matching. The expression need not consume the whole field.	<code>description(regex:"(?!CV)EOL")</code>	Use <code>^</code> and <code>\$</code> for boundaries. <code>regex-i:</code> is the case-insensitive form.
<code>exact-i:</code> , <code>substring-i:</code> , <code>glob-i:</code> , <code>regex-i:</code>	The corresponding matcher, case-insensitive.	<code>author_email(exact-i:"DEVON")</code>	<code>DEVON-i</code> belongs to the pattern kind; it is not a shell option and must remain inside the quoted revset.

These are deliberately different from shell globs. The shell must pass one revset string to jj; jj then parses the pattern inside it. For example, the single quotes protect `*`, `!`, parentheses, spaces, and `$` from the shell, while the double quotes inside the revset delimit the pattern value. If a pattern contains a literal double quote, escape it for the revset parser as well as choosing suitable shell quoting.

```
# The shell passes one argument; jj applies the four different matchers.
jj log -r 'subject(exact:"API: quota")'
```

```

jj log -r 'subject(substring:"quota")'
jj log -r 'subject(glob:"API: *")'
jj log -r 'subject(regex:"^API: (quota|rate limit)$")'

# Case-insensitive forms are pattern kinds, not command flags.
jj log -r 'author_email(exact-i:"DEV@EXAMPLE.ORG")'
jj log -r 'description(regex-i:"security|authentication")'

```

Field predicates: one field, several questions

The following examples are intentionally paired. The first query asks for a field; the second narrows the same field query with graph or metadata state. This makes the result useful in a real repository rather than merely demonstrating that a function parses.

Predicate	Field it tests	Basic query	Useful composed query
<code>description(P)</code>	Full commit message.	<code>description(substring:"mutable() &</code>	<code>description(regex-i:"^(fix revert</code>
<code>subject(P)</code>	Subject/first line of the message.	<code>subject(glob:"API: *")</code>	<code>subject(exact:"API: quota") &</code> <code>author_email(exact:"dev@example.o</code>
<code>author(P)</code>	Author signature as a name/email value.	<code>author(substring:"Example</code>	<code>author(substring:"Example")</code> <code>& :main@origin</code>
<code>author_name(P)</code>	Author name only.	<code>author_name(exact:"A. Engineer")</code>	<code>author_name(substring:"Engineer")</code> <code>&</code> <code>author_date(after:"2026-01-01")</code>
<code>author_email(P)</code>	Author email only.	<code>author_email(exact:"dev@example</code>	<code>author_email(glob:"*@example.org"</code> <code>& mutable()</code>
<code>committer(P)</code>	Committer signature.	<code>committer(substring:"CI")</code>	<code>committer(substring:"CI")</code> <code>&</code> <code>~author(substring:"CI")</code>
<code>committer_name(P)</code>	Committer name only.	<code>committer_name(exact:"Release Bot")</code>	<code>committer_name(exact:"Release Bot") & tags()</code>
<code>committer_email(P)</code>	Committer email only.	<code>committer_email(exact:"ci@example</code>	<code>committer_email(exact:"ci@example</code> <code>&</code> <code>author_email(exact:"dev@example.o</code>

Predicate	Field it tests	Basic query	Useful composed query
<code>mine()</code>	Commits whose author email matches the configured user email.	<code>mine()</code>	<code>mine() & mutable() & ::@</code>

An author and committer may differ after a rebase, import, automated rewrite, or bot-mediated publication. A query written against `author_email()` is therefore not interchangeable with one written against `committer_email()`. Likewise, `description()` can match a trailer or body line while `subject()` restricts the question to the first line.

Date patterns in detail

In jj 0.45.1, date predicates accept the two boundary kinds `after:` and `before:`. `after:` includes a commit whose timestamp is exactly the boundary; `before:` excludes a commit exactly at the boundary. The date parser documents these forms:

Form	Example	Meaning and reproducibility
Calendar date	<code>after:"2024-02-01"</code>	The date at the parser's date boundary; convenient for day-based reports.
ISO local date-time	<code>after:"2024-02-01T12:00:00"</code>	Date and time without an explicit offset. Use only when the environment's interpretation is understood.
ISO date-time with offset	<code>before:"2024-02-01T12:00:00-08:00"</code>	Best for a portable audit because the instant is explicit.
Space-separated date-time	<code>after:"2024-02-01 12:00:00"</code>	Equivalent style to the local date-time form; retain the inner quotes.
Relative duration	<code>after:"2 days ago", after:"5 minutes ago"</code>	Evaluated when jj parses the query, so the same command changes meaning with time.

Form	Example	Meaning and reproducibility
Relative named day/time	<code>after:"yesterday",</code> <code>after:"yesterday 5pm",</code> <code>after:"yesterday 10:30",</code> <code>after:"yesterday 15:30"</code>	Useful interactively; do not use as the sole definition of a reproducible release report.

The official 0.45.1 help says “including” for the date forms, so the table records the documented portable surface rather than claiming that the parser can never accept another human-readable spelling. Test any additional spelling in the target `jj` binary. This is separate from commit creation timestamps: date predicates compare the selected author or committer field, and a command that creates or rewrites a commit may set those fields according to different rules.

```
# Inclusive lower bound and exclusive upper bound.
jj log -r 'author_date(after:"2026-08-01T00:00:00+00:00") & author_date(before:"2026-09-01T00:00:00+00:00")'

# Author and committer can answer different questions.
jj log -r 'author_date(after:"2026-01-01") & ~committer_date(after:"2026-01-01")'

# Interactive query; the boundary moves whenever it is run.
jj log -r 'committer_date(after:"2 days ago")'
```

The regex dialect `jj` actually uses

The 0.45.1 revset help links `regex:` to the Rust `regex` crate syntax. That means this is a Rust-regex-style engine, not PCRE, Perl, or a shell-regex implementation. The stable useful subset includes literal characters, character classes such as `[0-9]`, groups, alternation, repetitions such as `*`, `+`, `?`, and `{m,n}`, anchors such as `^` and `$`, and inline flags such as `(?i)`. The exact crate version is an implementation dependency of `jj` 0.45.1; the user-facing contract is the syntax documented by `jj`'s linked Rust-regex reference, not a promise that a particular crate semver remains installed forever.

Rust's `regex` syntax deliberately does not provide look-around or backreferences. `jj` consequently rejects expressions such as `(?=foo)bar` and `(foo)\1` as invalid string patterns. Rewrite those queries using consuming groups, alternation, character classes, or a second revset predicate. Regexes match substrings unless

anchored, so `description(regex:"fix")` can match a body line as well as a subject; `subject(regex:"^fix:")` is a more precise question.

```
# Supported Rust-regex features.
jj log -r 'description(regex:"(?i)^(fix|revert):")'
jj log -r 'description(regex:"CVE-[0-9]{4}-[0-9]{4,}")'
jj log -r 'author_email(regex:"^[^@]+@example\\.org$")'

# These fail in jj 0.45.1: look-around and backreferences are not supported.
jj log -r 'description(regex:"(?=security)security")'
jj log -r 'description(regex:"(security)\\1")'
```

There are two escaping layers in the last examples. Single quotes protect the whole revset from the POSIX shell; `\\.` and `\\1` then reach the revset string parser as regex escapes. If the pattern is generated by a script, log the exact argument or use a shell array so word splitting cannot silently alter it.

```
[revset-aliases]
mine = 'author(exact:"A. Engineer")'
stack(x) = 'x:: & mutable()'
reviewable(x) = 'x:: & ~immutable_heads() & ~empty()'
```

stack(x) is not a builtin jj function. It becomes available only after this alias has been placed in an effective `[revset-aliases]` configuration table. Standalone commands in this book therefore use the equivalent explicit expression `x:: & mutable()` unless the surrounding example is deliberately teaching the alias. Aliases are parsed as revsets and parameterised aliases substitute expressions, not shell text. Keep aliases small, name them after the set they denote, and test them against a fixture graph. A repository alias can shadow a user alias; inspect effective configuration with `jj config list`.

13. Revset operators and public functions

The following reference groups the public 0.45.1 functions by the property they expose. Function availability and signatures follow the installed release's `jj help -k revsets`; that help remains the release-specific authority when a later version differs.

Family	Functions and signatures	Notes
Universe/topology	<code>all()</code> , <code>none()</code> , <code>root()</code> , <code>heads(x)</code> , <code>roots(x)</code> , <code>ancestors(x, [depth])</code> , <code>descendants(x, [depth])</code> , <code>parents(x, [depth])</code> , <code>children(x, [depth])</code> , <code>first_parent(x, [depth])</code> , <code>first_ancestors(x, [depth])</code> , <code>reachable(srcs, domain)</code> , <code>connected(x)</code>	Heads and roots are relative to the supplied set; traversal depth is optional.
Graph boundaries	<code>visible_heads()</code> , <code>immutable_heads()</code> , <code>mutable()</code> , <code>immutable()</code> , <code>visible()</code> , <code>hidden()</code> , <code>conflicts()</code> , <code>empty()</code> , <code>merges()</code> , <code>forks()</code>	These are predicates/sets over repository state, not Git branch lists.
Graph selection	<code>latest(x, [count])</code> , <code>fork_point(x)</code> , <code>merge_point(x)</code> , <code>bisect(x)</code> , <code>exactly(x, count)</code> , <code>coalesce(revsets...)</code>	Useful for reducing a set to recent, boundary, balanced, or exact-cardinality results.
Identity	<code>commit_id(prefix)</code> , <code>change_id(prefix)</code> , <code>description(pattern)</code> , <code>subject(pattern)</code> , <code>author(pattern)</code> , <code>author_name(pattern)</code> , <code>author_email(pattern)</code> , <code>committer(pattern)</code> , <code>committer_name(pattern)</code> , <code>committer_email(pattern)</code> , <code>mine()</code>	Pattern arguments are typed string patterns; mine uses the configured current user email.
Time/signing	<code>author_date(pattern)</code> , <code>committer_date(pattern)</code> , <code>signed()</code>	Use date patterns rather than relying on locale-formatted output.
Ref state	<code>bookmarks([pattern])</code> , <code>remote_bookmarks([name_pattern], [remote_tag_predicate])</code> , <code>tracked_remote_bookmarks(...)</code> , <code>untracked_remote_bookmarks(...)</code> , <code>tags([pattern])</code> , <code>remote_tags(...)</code>	jj 0.45.1 provides tracked/untracked <code>remote_tag_predicate</code> ref predicates; use explicit ref predicates when symbols may be ambiguous. Native Git refs are inspected through Git/backend commands, not a revset function.

Family	Functions and signatures	Notes
Remote state	<code>remote_bookmarks(pattern, remote),</code> <code>tracked_remote_bookmarks(),</code> <code>present(x)</code>	Remote bookmarks are imported refs; tracking is a relationship.
Workspaces	<code>working_copies()</code>	Selects the commits currently serving as working copies across all workspaces; workspace names themselves are managed by <code>jj</code> workspace.
Files/content	<code>files(expression),</code> <code>diff_lines(text, [files]),</code> <code>diff_lines_added(text, [files]),</code> <code>diff_lines_removed(text, [files])</code>	These query trees or diffs; the nested fileset expression is independent of revset syntax.
Evolution/operations	<code>divergent(),</code> <code>working_copies(),</code> <code>at_operation(op, x)</code>	Useful for recovering rewritten changes and inspecting historical repository views.

Function names are stated for `jj` 0.45.1 and have deliberately not been inferred from Git. If automation depends on a function, run `jj log -r 'function(...)'` with the target release during CI.

Composition and cardinality

```
immutable_heads().. & mutable() & descendants(@-)
bookmarks() ~ remote_bookmarks()
author(substring:"Engineer") & committer_date(after:"2026-01-01")
roots((main..@):: & conflicts())
```

First derive the set, then add a boundary, then add a predicate. Use `roots()` to turn a set of stack commits into its minimal roots before passing it to a command that should operate once per stack. Conversely, `heads()` is useful for publication candidates. Always inspect with `jj log -r 'EXPR'` before mutating.

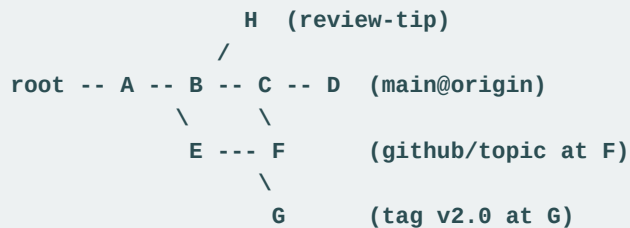
14. Revset algebra cookbook

Question	Revset	Reasoning
My unpublished mutable work	<code>mutable() ~ ::remote_bookmarks()</code>	Mutable changes excluding anything reachable from an imported remote ref.
Ancestors of @ not on a remote	<code>::@ ~ ::remote_bookmarks()</code>	Reachable from @, subtract remote reachability.
Mutable descendants of R	<code>R:: & mutable()</code>	Descendant closure intersected with mutability.
Conflicted commits	<code>conflicts()</code>	Stored conflict value, not merely filesystem markers.
Empty commits	<code>empty()</code>	Tree equals parent merge result as defined by jj.
Merges	<code>merges()</code>	Multiple-parent commits.
Not reachable from publication bookmarks	<code>all() ~ ::bookmarks()</code>	Use a narrower visible set when hidden commits are not intended.
Heads without bookmarks	<code>heads(all()) ~ bookmarks()</code>	Graph tips that have no local bookmark name.
Recent authored work	<code>author(exact:"A. Engineer") & committer_date(after:"2026-08-01")</code>	Use explicit identity/date patterns.
Release to trunk	<code>v2.4.0..main</code>	Ancestors of main not already reachable from tag.
Divergent changes	<code>divergent()</code>	Find concurrent successors of one change ID.
Gerrit stack rooted at R	<code>roots(R:: & mutable()):: & mutable()</code>	Find mutable stack roots then its mutable descendants.
GitHub PR candidates	<code>heads(mutable() ~ ::main@origin)</code>	Candidate tips not already reachable from imported trunk.

These are recipes, not universal policy. For example, “remote” may include an unrelated fork, and a publication bookmark may intentionally point to mutable work. The algebra tells you exactly what you asked; naming conventions and review policy decide whether that set is useful.

Revset laboratory: evaluate the set, then the command

A revset becomes reliable when it can be evaluated by hand. Use the following graph as a small algebraic universe. Letters are commits; names on the right are bookmarks or tags. Arrows point from parent to child.



I is a second successor of B's change ID, so B/0 and B/1 are divergent.
J is a conflicted descendant of C.
K is an empty mutable child of D.

This graph is intentionally not a single stack. It contains an ancestry fork, a merge-like region, a publication bookmark, a tag, a divergent change, a conflict, an empty commit, and a remote boundary. The exact parent edges should be recorded in the fixture rather than inferred from the drawing; the drawing's purpose is to make the set calculations visible.

Primitive closures

Expression	Set operation	Question it answers
<code>x</code>	Resolve a symbol/set.	Which exact commit(s) does this name denote?
<code>x-</code>	Immediate parents.	What are the direct inputs to x?
<code>x+</code>	Immediate children.	Which commits directly continue x?
<code>::x</code>	Ancestors including x.	What can reach x?
<code>x::</code>	Descendants including x.	What can x reach?
<code>x::y</code>	<code>x::</code> & <code>::y</code> .	Which descendants of x also lie on ancestry to y?
<code>x..y</code>	<code>::y ~ ::x</code> .	What is reachable from y but not already reachable from x?
<code>roots(S)</code>	Minimal members of S.	Where do selected stack components begin?
<code>heads(S)</code>	Maximal members of S.	Which selected tips need publication or review?

Suppose `A` is an ancestor of `D`. Then `A..D` contains commits reachable from `D` after subtracting the ancestry of `A`; it is not a list of only the first-parent path in a graph with merges. `A::D` includes `A` and the descendants that are also ancestors of `D`. The distinction is inclusion versus difference, not two spellings for “between”.

```
jj log -G -r '::D'
jj log -G -r 'A..D'
jj log -G -r 'A::D'
jj log -G -r 'heads(A::D)'
jj log -G -r 'roots(A::D)'
```

Why range expressions surprise Git experts

Git's commit notation is commonly used in commands whose own semantics impose a first-parent or symmetric interpretation. `jj revsets` are values passed to many different consumers. The same set can be rendered by `log`, `diffed`, `abandoned`, `rebased`, or `counted`. Therefore start with membership, then ask whether the consumer accepts a set and whether its operation is meaningful for a disconnected set.

```
# These are different questions:
jj log -r 'A..D'                # render selected commits
jj diff -r 'A..D'              # total connected diff, if valid
jj rebase -r 'A..D' -o main@origin # rewrite each selected commit
jj abandon 'A..D'              # hide all selected commits
```

The last three commands do not have interchangeable consequences. A `diff` rejects gaps that `log` can display. `Rebase` preserves/rewrites `diffs` and can produce conflicts. `Abandon` changes visibility and reconnects children. `Revset` syntax alone cannot tell you whether the operation is safe.

Precedence experiments

Operators bind in this order: function call; postfix parent/child; pattern prefix; ancestry/range; complement; intersection/difference; union. Write parentheses around policy. Compare these expressions in `query -lab`:

```
jj log -r 'mutable() & ::@ | bookmarks()'
jj log -r '(mutable() & ::@) | bookmarks()'
jj log -r 'mutable() & (::@ | bookmarks())'
jj log -r 'heads(mutable() ~ ::remote_bookmarks())'
```

The first two are equivalent under the documented precedence; the third is different because the union is inside the intersection. This matters for “unpublished work”: `mutable() & (::@ ~ ::remote_bookmarks())` excludes remote ancestry before intersecting, while `(mutable() & ::@) ~ ::remote_bookmarks()` subtracts it afterward. They often coincide in a simple graph but diverge when the remote reaches a mutable commit outside `::@`.

Visibility and the search domain

`all()`, `visible()`, and `hidden()` are not interchangeable views of one infinite universe. Ordinary symbol resolution and many commands operate on visible commits. Hidden commits remain in the store and can be found through explicit IDs, evolution, operation views, or a revset whose domain includes them. When a recipe says `all()`, explain whether it is intentionally forensic or merely using a broad visible set.

```
jj log -r 'visible_heads()'
jj log -r 'hidden()'
jj log -r 'all() & hidden()'
jj log -r 'present(commit_id("prefix"))'
```

A hidden commit can be a predecessor of a visible successor. It may be valuable recovery evidence. Do not treat hidden as equivalent to unreachable or garbage, and do not run GC while investigating a missing change.

Function families with operational reasoning

Family	Functions	How to reason about them
Universe	<code>all()</code> , <code>none()</code> , <code>root()</code>	Start/end sets. <code>none()</code> is useful in aliases and conditional construction.
Traversal	<code>parents</code> , <code>children</code> , <code>ancestors</code> , <code>descendants</code>	Optional depth limits immediate breadth; no depth means transitive closure.
First-parent	<code>first_parent</code> , <code>first_ancestors</code>	Use only when merge parent ordering is a deliberate policy.
Reachability	<code>reachable(srcs, domain)</code> , <code>connected</code>	Separate roots/sources from the domain to avoid accidentally traversing all refs.
Boundaries	<code>roots</code> , <code>heads</code> , <code>visible_heads</code>	Reduce a set to minimal/maximal members; useful before rebase or publication.
Graph points	<code>fork_point</code> , <code>merge_point</code> , <code>latest</code> , <code>bisect</code>	These choose graph-derived points; verify cardinality on forks/merges.
Refs	<code>bookmarks</code> , <code>remote_bookmarks</code> , <code>tracked/untracked forms</code> , <code>tags</code>	Names are sets of target commits; tracking changes which refs are included. Native Git ref inspection uses <code>jj git</code> or Git tooling.
Identity	<code>commit_id</code> , <code>change_id</code> , <code>present</code> , <code>divergent</code>	Commit identity and change identity answer different questions.
Metadata	<code>description</code> , <code>subject</code> , <code>author/committer forms</code> , <code>mine</code> , <code>date forms</code> , <code>signed</code>	Pattern kind and author/committer choice are part of the query.
Tree/conflict	<code>files</code> , <code>diff-line functions</code> , <code>empty</code> , <code>conflicts</code> , <code>merges</code> , <code>forks</code>	Query stored tree/diff state; conflict markers alone are not the predicate.
State/evolution	<code>mutable</code> , <code>immutable</code> , <code>hidden</code> , <code>visible</code> , <code>at_operation</code> , <code>working_copies</code>	Bind the query to local repository view, operation, or workspace state.

For each function, test the empty set, a singleton, a merge, a fork, a hidden commit, and a divergent change where the function's domain permits it. This is more valuable than testing only the happy-path linear stack.

Patterns and dates

String predicates accept explicit kinds including `exact:`, `substring:`, `glob:`, and `regex:`. A bare pattern's default is not a safe assumption for shared configuration. Use `subject(exact:"API: quota")` when one exact subject matters, `description(substring:"quota")` for a search, and `description(regex:"^API:")` for a policy prefix.

```
jj log -r 'subject(exact:"API: quota")'
jj log -r 'author_email(exact:"a@example.test")'
jj log -r 'subject(regex:"^(fix|revert):")'
jj log -r 'description(glob:"*security*")'
```

Date patterns are the `after:"DATE"` and `before:"DATE"` forms described above. `after:` includes a commit whose selected timestamp is exactly the boundary; `before:` excludes a commit exactly at the boundary. Compare author and committer dates deliberately: an imported commit can have a historical author date but a newer committer date after a rewrite. Use an explicit offset or UTC when communicating a reproducible cutoff.

```
jj log -r 'author_date(after:"2026-01-01")'
jj log -r 'committer_date(before:"2026-09-01")'
jj log -r 'author_date(after:"2026-08-01") & author_date(before:"2026-09-01")'
jj log -r 'author_date(after:"yesterday 10:30")'
```

Symbol resolution and cardinality

A symbol can name a tag, local bookmark, remote bookmark, workspace, commit ID prefix, or change ID prefix. Ambiguity is not a cosmetic concern: `jj edit` may reject a set where `jj log` displays it. Use explicit functions or full IDs when a script crosses a publication boundary.

Consumer class	Typical cardinality	Safe practice
Display: <code>log</code> , <code>diff</code> , <code>abandon</code> , <code>sign</code>	Many commits accepted, with command-specific connectedness.	Inspect set; use <code>--count</code> or <code>log</code> before mutation.
Target: <code>edit</code> , <code>describe</code> , <code>metaedit</code>	Exactly one or an intentionally supported set.	Use <code>jj log --count -r 'EXPR'</code> and require one.

Consumer class	Typical cardinality	Safe practice
Destination: <code>rebase</code> , <code>restore</code> , <code>duplicate</code>	Often one; repeated options may intentionally mean multiple parents.	Distinguish one set-valued option from repeated destination options.
Ref: bookmark/tag operations	Name patterns may match many; target commit often one.	Qualify remote and inspect tracking state.

Aliases as named algebra

```
[revset-aliases]
stack(x) = 'x:: & mutable()'
unpublished() = 'mutable() ~ ::remote_bookmarks()'
reviewable(x) = 'x:: & ~immutable_heads() & ~empty()'
publication_heads() = 'heads(mutable() ~ ::remote_bookmarks())'
'author:x' = 'author(substring:x)'
```

Alias parameters substitute revset expressions, not shell strings. A parameterised alias should have a mathematical description in its documentation: `stack(x)` means the mutable descendant closure of `x`, not “the branch”. Test nested aliases, empty parameters if the grammar permits them, and names that might collide with builtin functions or user/repository layers.

Derivation exercises

1. Start with “all my mutable changes”: `mine()` & `mutable()`.
2. Restrict to the current stack: intersect with `::@` or use a named stack alias.
3. Remove anything already reachable from publication: subtract `::bookmarks()` or a deliberately selected remote set.
4. Reduce to candidate tips: apply `heads()`.
5. Require a reviewable change: subtract `empty()`, `conflicts()`, and immutable heads according to policy.

```
heads(mine() & mutable() & ::@ ~ ::remote_bookmarks())
~ empty()
~ conflicts()
~ immutable_heads()
```

Each line is a policy decision. A change can be empty but intentionally a merge marker; a conflicted commit may be useful in a WIP review; and a remote bookmark

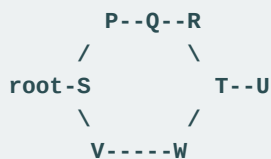
can be an unrelated fork. Treat the final revset as a policy expression and name it accordingly rather than hiding it behind a generic alias such as `work()`.

Appendix C is the compact public-function and operator lookup. This chapter supplies the evaluation method and the counterexamples that a table cannot provide. Before using a non-trivial expression for a mutating operation, evaluate it against a representative graph and inspect the selected commits.

Revset reasoning: a small algebra, not a bag of aliases

A revset expression denotes a set of commits in a particular repository view. The view matters: visibility, the selected operation, workspace state, and imported refs all affect symbol resolution and the universe against which predicates are evaluated. Read an expression in two stages. First determine the set denoted by each subexpression. Then determine how the consuming command interprets the set: as a source set, a destination set, a range, a singleton, or a set of independent targets.

Let G be the directed acyclic commit graph, with edges from a child to its parents. Let $Anc(x)$ include x and every ancestor reachable by repeatedly following parent edges; let $Desc(x)$ include x and every descendant reachable in the opposite direction. In the notation used by jj, $::x$ denotes $Anc(x)$ and $x::$ denotes $Desc(x)$. The operators are set operators layered over these graph traversals, so a complex query can be calculated on paper before it is given to the CLI.



$Anc(U) = \{U, T, R, Q, P, S, root\}$ $Desc(S) = \text{every node below } S$
 $Anc(U) \ \& \ Desc(S) = \{S, P, Q, R, T, U\}$
 $U- = \{T\}$ $U+ = \{ \}$ in this graph

The diagram is intentionally non-linear. A range expression that works by accident on a linear stack can be wrong here. For example, `S..U` is defined in terms of ancestry: it selects ancestors of `U` excluding ancestors of `S`. It is not “the commits

printed between two lines”, and it does not mean “all nodes after S in a chosen first-parent walk”. If a command needs the first-parent corridor, express that corridor explicitly with `first_ancestors` or a corresponding supported function.

Precedence and parentheses

Parse from the inside out. Function calls and grouping bind most tightly; postfix parent/child operators bind more tightly than infix set operators. Intersection and difference bind more tightly than union. A useful safe style is to parenthesise every union used as an operand of a traversal or difference:

```
jj log -r '(heads.mutable() & ::@) | main@origin'
jj log -r '::-(@ | main@origin)'
jj log -r '::-@) ~ (:::main@origin)'
```

The first query is the union of two already computed sets. The second computes the ancestor closure of a union. Those are not generally equal in a graph with two tips. The third is “ancestors of @ excluding ancestors of main@origin”; it is a precise unpublished-ancestry query only when the remote bookmark is the correct publication boundary.

Expression	Reading	Common mistaken reading
<code>A B</code>	All commits in either set.	A path from A to B.
<code>A & B</code>	Commits in both sets.	A merge of A and B.
<code>A ~ B</code>	A with commits in B removed.	A minus only the tip named by B.
<code>A::</code>	A and all descendants.	Only the first child chain.
<code>::A</code>	A and all ancestors.	Only first-parent history.
<code>A..B</code>	Ancestors of B excluding ancestors of A.	Every commit after A by date.
<code>A::B</code>	Descendants of A that are ancestors of B.	All commits reachable from either endpoint.

Symbols, identity, and ambiguity

Symbol resolution is a semantic operation, not shell variable expansion. A name can resolve to a local bookmark, a remote bookmark qualified by a remote, a tag, a workspace symbol, a change ID, or a commit ID prefix. If a short prefix is ambiguous, `jj` reports the ambiguity rather than silently selecting one. In scripts use a full enough prefix or disambiguating syntax, and prefer a bookmark or an alias for a stable human role.

```
jj log -r 'main@origin'
jj log -r 'review@origin'
jj log -r 'review@'
jj log -r 'change_id("qxy...")'
jj log -r 'commit_id("3f0...")'
```

Use `change_id()` when the intent is “the current visible commit for this evolving logical change”. Use `commit_id()` when the intent is “this exact immutable object”. This difference is crucial in scripts that run after a rewrite. A build verifier should normally pin a commit ID; a command that asks to edit a review change should normally resolve its change ID or bookmark, then verify the selected singleton.

Cardinality is a property of the consumer, not of the syntax alone. `jj log` accepts a set. `jj rebase --revisions` accepts a source set. `jj edit` requires one commit. Some commands accept zero or one commit and default to `@`. Before composing a script, test a query with an output that includes both IDs:

```
jj log -r 'my_query' -T 'commit_id.short() ++ " " ++ change_id.short() ++ "\n"
```

If the command requires one result, make the cardinality visible and fail early in the shell:

```
count=$(jj log -r 'my_query' --no-graph -T 'commit_id ++ "\n"' | wc -l)
test "$count" -eq 1 || { printf 'expected one commit, got %s\n' "$count" >&2; exit 2; }
jj edit 'my_query'
```

This is more robust than adding `latest()` merely to silence an error. `latest()` is a selection policy: it chooses by committer timestamp from a set and can choose a

different commit after a clock correction or import. Use it only when “latest” is actually the desired semantics.

Topology functions in detail

`parents(X)` and `children(X)` expose immediate graph relationships; in jj 0.45.1 they also accept an optional depth, so `parents(X, 3)` is equivalent to `X---` and `children(X, 3)` to `X+++`. Postfix `X-` and `X+` are convenient immediate-neighbour operations. `ancestors(X)` and `descendants(X)` perform transitive traversal; the prefix/suffix forms are compact equivalents for common cases. `roots(X)` and `heads(X)` reduce a set to nodes with no selected parent or no selected child respectively. These functions operate on the supplied set where their signatures say so, which is why `heads(::@)` differs from `heads(all())`.

```
jj log -r 'parents(@)'  
jj log -r 'children(main@origin)'  
jj log -r 'headsmutable() & ::@'  
jj log -r 'roots(::@ ~ ::main@origin)'  
jj log -r 'merges() & ::@'  
jj log -r 'forks() & ::@'
```

`merges()` and `forks()` are zero-argument structural predicates; restrict either result with set intersection when you need a domain, for example `merges() & ::@`. A merge is a commit with multiple parents; a fork identifies commits whose selected topology branches. They do not identify “large” changes or review stacks.

`reachable(S, T)` is useful when you need to ask whether the graph can reach one set from another, while `connected(X)` groups connected components where that is the function’s documented interpretation. When a query’s result seems too broad, replace it with a sequence of named subqueries and inspect each one.

Predicate composition

Most useful revsets are an intersection of a graph boundary with a semantic predicate. Start with the universe, narrow by reachability, then narrow by state:

```
(mutable() & ::@) ~ ::main@origin  
(mutable() & descendants(main@origin)) & conflicts()  
(::@ & description(regex:"(?i)security|auth")) ~ hidden()  
heads(::@) & ~bookmarks()  
(::@ & mine()) & ~immutable()
```

In the first expression, “mutable” alone would include unrelated mutable work and “ancestors of @” alone would include imported immutable history. The intersection establishes both. The difference removes the published ancestry only if `main@origin` is the relevant publication boundary; a topic bookmark or a second remote may require subtracting a union of several refs instead.

Metadata predicates such as `author()`, `committer()`, `description()`, `subject()`, and date predicates can be combined with topology. Conflict and emptiness predicates describe commit state, not filesystem state at this instant. A commit may be empty relative to its parent but still carry a meaningful description or review marker; a conflicted commit may have a materialised conflict in the working copy or may be inspected without materialising it.

Pattern and date arguments

Text predicates accept explicit pattern kinds. Treat patterns as data and quote the whole revset so the shell does not consume punctuation:

```
jj log -r 'subject(exact:"Fix parser")'  
jj log -r 'description(substring:"parser")'  
jj log -r 'description(glob:"Fix * parser")'  
jj log -r 'description(regex:"(?i)CVE-[0-9]+")'  
jj log -r 'author_email(exact:"dev@example.org")'
```

An exact pattern is appropriate for a machine-generated subject; a substring is appropriate for an exploratory query; a glob is useful for a controlled family; a regex is powerful but should be tested against descriptions containing punctuation and newlines. Pattern syntax belongs inside the revset grammar, so escaping has two layers: the shell quote and the pattern’s own quoting/escaping rules.

Date patterns should likewise be explicit. The grammar is `after:"DATE"` or `before:"DATE"`; combine them with `&` for an interval. A relative phrase such as `2 weeks ago` is convenient interactively but can make an audit non-reproducible. For a reproducible report, use an absolute date-time with an explicit offset or a checked-in alias whose meaning is documented. Date predicates compare the relevant author or committer timestamp; do not assume they are interchangeable, especially after rebases and imports.

```
jj log -r 'author_date(after:"2026-08-01T00:00:00+00:00") & author_date(before:"2026-09-01T00:00:00+00:00")'
jj log -r 'committer_date(after:"2 days ago")'
```

Algebraic derivations

Here are derivations rather than recipes. Let `P` be the union of publication tips and `T` be the local trunk tip. “Ancestors of `@` that are not reachable from any publication tip” is:

```
(::@) ~ (P::)
```

where `P` itself may be `bookmarks()` | `remote_bookmarks()` or a deliberately restricted set such as `main@origin` | `release@origin`. “The patch stack above trunk” is not always `trunk..@`; for a stack with a merge or a topic base, use the ancestry corridor that reflects the publication boundary and then take heads or roots as needed:

```
stack = ::@ ~ ::main@origin
stack_heads = heads(stack)
stack_roots = roots(stack)
```

“Heads without bookmarks” is a two-part question. First compute graph heads, then remove commits carrying names:

```
heads(::@) ~ bookmarks()
```

This may still include a head with a remote bookmark if only local bookmarks are removed. If the policy is “not published anywhere”, subtract the union of local and remote bookmark targets, and include tags if tags count as publication in your environment. The correct expression depends on the policy; revset algebra exposes that policy instead of hiding it in a command alias.

Empirical evaluation protocol

For a non-trivial expression, record the fixture graph, the expression, the expected set by symbolic name, and the actual output containing commit and change IDs. Then test the expression after a rewrite. Queries based on a commit ID should intentionally stop matching the old object; queries based on a change ID or bookmark should follow the intended identity or ref. This protocol catches the most expensive class of revset bug: a query that succeeds while selecting the wrong commit.

```
# human-readable graph
jj log -r 'all()' --no-pager

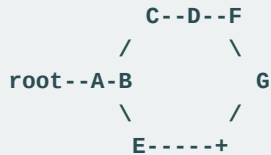
# stable evidence
jj log -r 'my_query' --no-graph \
  -T 'commit_id ++ "\t" ++ change_id ++ "\t" ++ description.first_line() ++ "\n"'

# historical view when an operation is under investigation
jj --at-operation OP log -r 'my_query' --no-graph
```

Keep a fixture alias such as `lab()` in a disposable config rather than adding broad aliases to a user’s global configuration. A representative non-linear graph lets you check selected IDs directly; use that evidence to support the explanation rather than relying on a pretty rendering.

Revset workbook: calculate before executing

Use this workbook with a disposable repository. Give each node a description matching its letter, then replace letters with full IDs in the output. The expected sets below are mathematical results; the command's presentation order is a separate concern.



```
parents(F) = {D}
ancestors(F) = {F,D,C,B,A,root}
descendants(B) = {B,C,D,E,F,G}
merges() & ::G = {G}
forks() & ::G = {B}
```

Now evaluate a corridor:

```
C::F = descendants(C) & ancestors(F)
    = {C,D,F}

B..F = ancestors(F) ~ ancestors(B)
    = {C,D,F} (assuming B is the boundary and E is not an ancestor of F)

heads(B::G) = {G}
roots(B::G) = {B}
```

The arithmetic is deliberately explicit. In a graph with a merge, a child can be reachable through more than one route, but set membership remains idempotent. If a query unexpectedly includes E, inspect the actual parent edges rather than relying on the diagram's spacing.

Exercise: published versus unpublished

```
main@origin--A--B
                \
                C--D--@
release@origin-----B
pr/api-----C
```

Define the policy “ancestors of @ not reachable from either remote publication”:

```
(::@) ~ (remote_bookmarks()::)
```

But if `remote_bookmarks()` includes an unrelated remote head, it may remove too much. Narrow it:

```
(::@) ~ (remote_bookmarks(substring:"main"))
```

Or use explicit names:

```
(::@) ~ (main@origin:: | release@origin:: | pr/api::)
```

These are not interchangeable when a remote contains multiple bookmarks. A cookbook recipe is correct only after its publication universe is specified.

Exercise: hidden and immutable state

```
jj log -r 'all()'
jj log -r 'hidden()'
jj log -r 'immutable_heads()'
jj log -r 'mutable() & ::@'
```

Compare the result of `all()` with the ordinary default log. The ordinary visible graph may omit hidden predecessors; `all()` is still evaluated in a view with its own visibility rules and explicit ancestors. Use `present(X)` when a symbol may not exist in the current view, and use `at_operation(OP, X)` when the question is explicitly historical.

Exercise: files and diff predicates

```
jj log -r 'files("root-glob:src/**/*.rs")'
jj log -r 'diff_lines_added("TODO", "root-glob:src/**")'
jj log -r 'diff_lines_removed("deprecated", "root-glob:src/**")'
```

These predicates select commits by their change/diff, not every commit whose resulting tree happens to contain a matching file. A rename, copy, merge, binary file, or conflict can make the intuitive reading wrong. Pair the query with `jj diff`

`-r` on the resulting commits and verify the path scope.

Exercise: reachability with a domain

`reachable(srcs, domain)` is the function to reach through a deliberately bounded domain. Let `E` be a descendant of `A` and let `A..` exclude `A`:

```
reachable(E, A..)
```

The domain prevents traversal outside the selected set. This is safer than using a broad descendant closure when the repository has multiple roots, but it also means a source not in the domain is ignored. Test the four cases: source in domain, source outside domain, path that exits domain, and disconnected source. A query that returns an empty set can be correct because the boundary excludes the source.

Exercise: dates and authors

```
jj log -r 'author_email(exact:"ada@example.org")'  
jj log -r 'author_date(after:"2026-01-01") & committer_date(before:"2026-09-01")'  
jj log -r 'mine() & mutable() & ::@'
```

Use exact patterns for an identity controlled by policy, substring patterns for interactive search, and regex only when the input convention warrants it. An imported commit can have an author date from months ago and a recent committer date; the intersection deliberately finds that shape. Do not call the result “recent work” without saying which timestamp was used.

Exercise: cardinality as a proof obligation

```
jj log -r 'exactly(subject(exact:"API: quota"), 1)'  
jj edit 'exactly(subject(exact:"API: quota"), 1)'
```

The first command proves that a subject is unique in the current view. The second uses the same proof as a singleton command. If a later import adds another matching commit, the command fails closed instead of editing an arbitrary commit. When the desired policy is “the newest matching review”, use `latest` explicitly and document why that is safe.

Alias design exercises

Aliases should package a domain boundary, not hide an accidental query. Start with a parameterised alias that accepts a stack tip and a base:

```
[revset-aliases]
'patches(base, tip)' = 'roots(base..tip)::'
'unpublished(tip)' = '::tip ~ remote_bookmarks()::'
'conflicted-stack(tip)' = 'tip:: & mutable() & conflicts()'
```

Before installing these globally, test the parameters with a fork and a merge. The alias `patches` can return multiple roots; a command that requires one root must apply `roots` or enforce cardinality. The alias `unpublished` encodes a broad publication policy and should be narrowed in a multi-remote repository. Alias design is policy design; review it like code.

Revset semantics laboratory: evaluate an expression by hand

A revset is not a textual shortcut for a branch name. It is an expression whose value is a set of commits in the repository view selected for the command. The same expression can produce a different set after a fetch, a rewrite, a visibility change, or a configuration change. That is a feature: a revset describes a question, while a commit prefix describes an object found at one moment.

The reliable way to learn an unfamiliar expression is to evaluate it in three passes. First resolve every symbol to a set. Next evaluate graph operators over those sets. Finally apply predicates and boundary functions. When an expression is used by a mutating command, add a fourth pass: determine whether the command accepts the resulting cardinality and whether its source/destination interpretation is what you intended.

```
A--B--C--D
  \
   E--F--G
     |--H
```

For the examples below, the important parent edges are:

```
parents(B) = {A}; parents(C) = {B}; parents(D) = {C}
parents(E) = {B}; parents(F) = {E}; parents(G) = {F}
parents(H) = {F}
```

The drawing is intentionally explicit about parent edges. It is easy to read a diagonal line as decoration and accidentally reason about a different graph. Names in a diagram are not automatically bookmarks. In an actual fixture, obtain the edges instead of trusting the visual layout:

```
jj log -G -r 'all()' -T 'description.first_line() ++ " parents=" ++ parents.map(|p| p.description
```

Set values and intermediate sets

Let $S = B :: G$. The descendant operator includes B and follows child edges, restricted by the right endpoint when one is present. In the graph above, $S = \{B, E, F, G\}$; C and D are on the other fork and H lies beyond G's endpoint. Applying `heads(S)` removes each selected commit that is an ancestor of another selected commit. F is an ancestor of G, so F is removed and the result is $\{G\}$. A merge does not make each parent a head merely because it has two incoming edges.

There are two common mistakes here. The first is to confuse a member with a head: a node can be a selected member and still be an ancestor of a later selected member. The second is to apply `heads()` before selecting the corridor. `heads(all())` asks for heads of the entire visible domain; `heads(B :: G)` asks for heads of the smaller set. A release script that uses the former may publish an unrelated local experiment.

Expression	Evaluation question	Typical use
$B \mid E$	Which commits are named by either operand?	Construct a review set with explicit alternatives.
$B \ \& \ :: G$	Is B both selected and an ancestor of G?	Check that a named change lies on a stack.
$B :: G$	Which descendants of B are ancestors of G?	One connected corridor through a stack or merge.

Expression	Evaluation question	Typical use
<code>G::</code>	Which descendants of G are reachable?	Find work added after a review tip.
<code>::G</code>	Which ancestors of G, including G, are reachable?	Inspect the complete history supporting a tip.
<code>heads(B::G)</code>	Which selected nodes have no selected descendant?	Reduce a corridor to publication candidates.
<code>roots(B::G)</code>	Which selected nodes have no selected parent?	Find the first changes in a selected stack.

Use a no-graph template in experiments. A graph log is excellent for human topology, but its indentation is not a machine-readable answer and can conceal that two selected nodes have the same description.

Range algebra and boundaries

Range expressions are where Git experts most often import an unhelpful analogy.

In jj, `x..y` is an ancestry difference: the ancestors of `y`, excluding the ancestors of `x`. It is not simply “commits after `x` in one named branch”. If `x` is not an ancestor of `y`, the result can contain several regions or all of `y`'s ancestry that is not also reachable from `x`. That is often exactly what a graph query should say, but it is not a linear interval.

```

      A--B--C--D
       \
        E--F

::D      = {root,A,B,C,D}
::F      = {root,A,B,E,F}
B..D     = {C,D}
B..F     = {E,F}
C..F     = {E,F}
B::F     = {B,E,F}

```

The last line is a corridor, not a textual interval. If `F` is changed into a merge with both `C` and `E` as parents, then `C..F` includes `F` and its `C/E`-supported ancestry according to reachability. Always inspect parent edges in a merge. A good mental

model is “subtract one reachable ancestor cone from another”, not “subtract line positions”.

The prefix and infix forms are compositional. `roots(main@origin..@)::` first computes changes not reachable from the imported trunk, reduces them to their roots, and then expands each root through descendants. That is useful when local work contains a connected stack. If it contains two independent roots, the expression intentionally selects two stacks. If the consuming command requires one source, cardinality must be checked rather than silently choosing one.

Boundaries can be placed on either side of a predicate. Compare:

```
heads.mutable() & ::@  
heads.mutable() & ::@  
mutable() & heads(::@)
```

The first finds heads after restricting to mutable ancestors of `@`. The second finds mutable repository heads that are also ancestors of `@`. The third finds mutable ancestors of the heads of the current ancestry. In a linear stack these may coincide; in a fork they need not. Parentheses make the policy visible and protect it from a future grammar change.

Precedence as a proof obligation

Operator precedence is part of the language contract. Function calls and postfix navigation bind tightly; graph ranges and set operators bind according to the installed grammar. Do not rely on memory when an expression combines topology and set algebra. Write the intended grouping first, then compare it with `jj`'s parser by adding parentheses until the expression reads like the proof.

```
# Candidate local work whose ancestry includes @, excluding public boundaries  
heads((mutable() & ::@) ~ ::remote_bookmarks())  
  
# A different question: public tips which are ancestors of @  
heads(mutable()) & ::@ & ::remote_bookmarks()  
  
# Make the review domain explicit before reducing it  
heads((main@origin..@) ~ empty() ~ conflicts())
```

Difference is not a boolean “not”. `X ~ Y` removes commits in Y from X; it does not necessarily remove descendants of Y. To exclude a whole ancestry cone use `X ~ ::Y`; to exclude descendants use `X ~ Y::`. “Not already published” usually means “not reachable from any published remote tip”, so the exclusion should be `~ ::remote_bookmarks()`, not merely `~ remote_bookmarks()`.

Intersections distribute over unions in the usual set sense, but graph functions such as `heads()` are reductions and do not commute with arbitrary set operations. In general, `heads(X | Y)` is not equivalent to `heads(X) | heads(Y)`. The former removes a node if a descendant exists in either operand; the latter reduces each operand independently and can retain an ancestor of a head from the other operand. Write the expression from the operational question outward.

Symbols, prefixes, and cardinality

Resolution turns text into a set before topology is evaluated. The special symbol `@` is the current workspace's working-copy commit; `@-` and related offsets navigate relative to that workspace. A full commit ID identifies one commit. A change ID identifies an evolution and can resolve to one or more visible commit versions when divergence exists. Local bookmarks, qualified remote bookmarks, tags, and workspace symbols carry different qualification rules.

Short IDs are a user-interface convenience. An unambiguous prefix can become ambiguous after fetch or rewrite. A script that stores a short prefix as an external key has a latent failure. Use a full commit ID for object interchange, a change ID when the task is to follow a logical change, and a bookmark or tag when the task is to name a publication boundary.

Consumer	Safe input shape	What to do with a set
<code>jj log -r</code>	Any revset, including many commits.	Render or inspect the whole set.
<code>jj diff -r</code>	One connected commit set for a diff view.	Check connectivity and intended endpoint.
<code>jj edit</code>	Exactly one commit.	Use <code>exactly(X, 1)</code> or an explicit ID.

Consumer	Safe input shape	What to do with a set
<code>jj describe -r</code>	One mutable commit in ordinary use.	Do not assume a matching subject is unique.
<code>jj rebase -r</code>	Many commits are allowed.	Understand roots, holes, and descendant rewriting.
<code>jj bookmark set -r</code>	One destination commit.	Prove cardinality before moving a publication name.

A command may accept a many-valued revset but still impose a graph constraint. A rebase can reject a selection with a destination inside its own source set, a cycle, or an immutable source. Cardinality is the first precondition, not the only one.

Predicates describe stored commit state

Predicates such as `empty()`, `conflicts()`, `mutable()`, `immutable()`, `visible()`, and `hidden()` inspect commits in the repository view. They do not inspect unrecorded contents of today's filesystem. A dirty file becomes queryable only after a snapshot makes it part of the working-copy commit tree.

The same distinction applies to file predicates. `files(X)` selects commits whose changes involve matching paths; it does not mean that every selected commit's resulting tree contains those paths. A file may be deleted, renamed, or changed in a merge. Pair a file predicate with `jj diff` and a path-limited fileset when the question is “what changed in this directory”, not “which commits have a file with this name”.

```
jj log -r 'conflicts() & ::@'
jj log -r 'empty() & mutable()'
jj log -r 'files("root-glob:src/**/*rs") & ::@'
jj log -r 'author_email(exact:"a@example.org") & committer_date(after:"2026-01-01")'
```

Metadata patterns are typed. `author_email(exact:"...")` asks for the author email; `author(exact:"...")` matches the author signature according to the pattern form accepted by the release. There is no generic `email:` pattern kind in 0.45.1. Use `exact`, `substring`, `glob`, or `regex` explicitly and quote the whole revset for the

shell.

Revset public functions: signatures and operational uses

The following entries are contracts rather than name lists. R means a revset value and P means a typed string pattern. Unless stated otherwise, a function can return zero, one, or many commits. Inspect a result with `jj log --no-graph` before passing it to a singleton command.

Topology functions

Function	Result and edge case	Operational use
<code>root()</code>	The repository root singleton; it has no parent.	<code>root()::</code> anchors a complete graph query.
<code>all()</code>	All commits in the current search domain.	Use only with a deliberate exclusion/boundary.
<code>none()</code>	The empty set.	A safe false branch for an alias or script.
<code>parents(R)</code>	Immediate parents; duplicates collapse.	Inspect merge edges or derive roots.
<code>children(R)</code>	Immediate children; a fork can return many.	Never pass to <code>edit</code> without cardinality proof.
<code>ancestors(R)</code>	R plus every reachable ancestor.	Compare ancestry cones and public bases.
<code>descendants(R)</code>	R plus every reachable descendant.	Bound a stack with <code>&::TIP</code> .
<code>first_parent(R)</code>	The first parent relation where present.	Report integration history without following every side branch.
<code>first_ancestors(R)</code>	First-parent ancestry closure.	Contrast a release line with full reachability.
<code>roots(R)</code>	Selected nodes with no selected parent.	Turn a source set into rebase roots.
<code>heads(R)</code>	Selected nodes with no selected descendant.	Find maximal publication tips; an ancestor of a selected merge is not a head.
<code>reachable(S,D)</code>	Sources reachable through an explicit domain.	Prevent traversal crossing an unrelated fork; empty can be correct.
<code>connected(R)</code>	Tests/selects connected graph regions according to the target grammar.	Precondition a diff or rewrite policy.

Optional depth parameters, where exposed by 0.45.1 help, limit traversal. They do not turn ancestry into a first-parent query. Verify overloads before putting them in a shared alias.

State and boundary functions

Function	Meaning	Important non-equivalence
<code>visible_heads()</code>	Visible graph tips.	Not all bookmarks.
<code>immutable_heads()</code>	Effective rewrite-protection boundary.	Not necessarily every pushed commit.
<code>builtin_immutable_heads()</code>	Built-in protection expression.	Can differ after configuration override.
<code>mutable()/immutable()</code>	Rewrite policy membership.	Neither means unpublished/published.
<code>visible()/hidden()</code>	Visibility in the current view.	Hidden is retained data, not erased data.
<code>conflicts()</code>	Stored conflict values.	Does not mean the current filesystem has markers.
<code>empty()</code>	No tree change relative to parent merge result.	Empty commits can be intentional.
<code>merges()</code>	Multiple-parent commits.	A conflict is not necessarily a merge.
<code>forks()</code>	Commits participating in a fork.	A fork is a graph relation, not a pair of refs.
<code>divergent()</code>	Several visible commits for one change identity.	Similar diffs with different change IDs are not divergent.

State predicates with graph evidence

State predicates become less ambiguous when the query is paired with a graph boundary and rendered with an identity. The examples below assume a local mutable stack above `main@origin`, at least one immutable imported commit, and that conflicts or divergence have been created deliberately. They are query patterns, not claims that every repository contains every state.

Predicate	What it selects	Standalone example	Useful composition
<code>mutable()</code>	Commits permitted by the current immutability policy to be rewritten.	<code>jj log -r 'mutable()'</code>	<code>mutable() & ::@ ~ ::main@origin</code> — local mutable ancestry outside imported trunk.

Predicate	What it selects	Standalone example	Useful composition
<code>immutable()</code>	Commits protected from ordinary rewriting.	<code>jj log -r 'immutable()'</code>	<code>immutable() & ::@</code> — protected commits reachable from this workspace.
<code>visible()</code>	Commits in the current repository view.	<code>jj log -r 'visible()'</code>	<code>visible() ~ ::@</code> — visible history outside the current ancestry cone.
<code>hidden()</code>	Retained commits omitted from the normal visible view.	<code>jj log -r 'hidden()'</code>	<code>hidden() & divergent()</code> — hidden divergent history worth investigating.
<code>visible_heads()</code>	Visible graph tips according to jj's view.	<code>jj log -r 'visible_heads()'</code>	<code>visible_heads() ~ bookmarks()</code> — visible tips without a local bookmark.
<code>immutable_heads()</code>	The effective rewrite-protection boundary.	<code>jj log -r 'immutable_heads()'</code>	<code>immutable_heads() & ::@</code> — relevant protection boundaries.
<code>builtin_immutable_heads()</code>	The built-in protection expression before policy overrides.	<code>jj log -r 'builtin_immutable_heads()'</code>	<code>immutable_heads() ~ builtin_immutable_heads()</code> — policy-added protection.
<code>conflicts()</code>	Commits containing a stored conflict value.	<code>jj log -r 'conflicts()'</code>	<code>conflicts() & ::@</code> — conflicts in the current ancestry.
<code>empty()</code>	Commits whose tree is empty relative to their parent merge result.	<code>jj log -r 'empty()'</code>	<code>empty() & mutable() & ::@</code> — local empty commits to inspect.
<code>merges()</code>	Commits with more than one parent.	<code>jj log -r 'merges()'</code>	<code>merges() & ::@</code> — integration commits reachable from the workspace.
<code>forks()</code>	Commits at the structural fork boundary defined by the function.	<code>jj log -r 'forks()'</code>	<code>forks() & ::@</code> — fork structure affecting current work.

Predicate	What it selects	Standalone example	Useful composition
<code>divergent()</code>	Visible successor commits sharing one change ID.	<code>jj log -r 'divergent()'</code>	<code>divergent()</code> & <code>mutable()</code> — divergence that may need reconciliation.

Several near-misses matter. `immutable()` is not “published”, because local policy can protect unpublished work. `empty()` is not “unimportant”, because an empty commit can be a deliberate marker. `conflicts()` is not “the working directory has markers”, because the value is stored in the commit graph. `divergent()` is about one change identity with multiple successors; two independently authored commits with identical text are not necessarily divergent.

Refs, identity, and metadata

`bookmarks()` selects commits named by local bookmarks. `remote_bookmarks()` selects commits named by remote bookmarks, with optional name/remote patterns. Tracking predicates distinguish relationships, not movement. `tags()` and `remote-tag` functions select release names; `jj 0.45.1` provides tracked/untracked `remote-tag` predicates. Native Git refs are inspected through `jj git` or Git tooling rather than a `revset` function. These functions return target commits; `template CommitRef` values are where names, conflict targets, and tracking counts are rendered.

```
jj log -r 'bookmarks()'
jj log -r 'remote_bookmarks("main", "origin")'
jj log -r 'remote_bookmarks(remote="origin")'
jj log -r 'bookmarks() & ~remote_bookmarks()'
jj log -r 'tags() | remote_tags()'
git for-each-ref --format='%(refname) %(objectname)'
```

Ref predicate	Basic example	What to verify in a real repository
<code>bookmarks(P)</code>	<code>bookmarks(glob:"release/*")</code>	It returns targets of local bookmarks matching the name pattern, not every commit below those bookmarks.

Ref predicate	Basic example	What to verify in a real repository
<code>remote_bookmarks(NAME, REMOTE)</code>	<code>remote_bookmarks(glob:"feature/*", exact:"origin")</code>	The first argument filters bookmark names and the second filters remote names. Named form <code>remote_bookmarks(remote="origin")</code> avoids positional confusion.
<code>tracked_remote_bookmarks()</code>	<code>tracked_remote_bookmarks() & remote_bookmarks(remote="origin")</code>	Tracking is a local relationship to a remote bookmark; it does not make a local bookmark follow the working copy.
<code>untracked_remote_bookmarks()</code>	<code>untracked_remote_bookmarks() & remote_bookmarks(remote="origin")</code>	Useful for finding imported remote names that have no local tracking relationship.
<code>tags(P)</code>	<code>tags(glob:"v2.*")</code>	Tags are release names, distinct from bookmarks; combine with <code>immutable()</code> only if policy makes that implication true.
<code>remote_tags(P, REMOTE)</code>	<code>remote_tags(glob:"v2.*", exact:"origin")</code>	Qualify the remote when GitHub and Gerrit remotes both publish tags.
<code>tracked_remote_tags()</code>	<code>tracked_remote_tags() & remote_tags(remote="origin")</code>	Shows remote tags with the local tracking relationship.
<code>untracked_remote_tags()</code>	<code>untracked_remote_tags() & remote_tags(remote="origin")</code>	Useful when explaining why a remote tag contributes to immutability.

Identity predicates are typed. `commit_id(P)` and `change_id(P)` match IDs; `description(P)` and `subject(P)` match message text; `author` and `committer` functions match metadata; `mine()` uses configured identity; date functions select timestamp ranges. Use explicit pattern kinds:

```
commit_id(prefix:"abc")
change_id(prefix:"xyz")
subject(exact:"API: introduce quota")
description(substring:"security")
author_email(exact:"a@example.org")
committer_date(after:"2026-01-01")
```

Do not invent `email:` as a pattern kind. An exact subject can match multiple duplicates; a prefix can become ambiguous after fetch; `author` and `committer` dates

can differ after import. These are language and data-model edge cases, not reasons to select the first line of output in a shell pipeline.

Identity/time predicate	Positive query	Contrast or operational use
<code>commit_id(P)</code>	<code>commit_id(prefix:"abc123")</code>	Matches an object identity; it stops following the object after a rewrite. Use a full ID when a script must name one commit.
<code>change_id(P)</code>	<code>change_id(prefix:"kmpv")</code>	Matches an evolution identity; it can return multiple commits when the change diverged. Pair with <code>heads()</code> only when you want current tips.
<code>description(P)</code>	<code>description(regex-i:"Reviewed-by: *")</code>	Searches the complete message, including body/trailers. Use <code>subject()</code> when a trailer must not count.
<code>subject(P)</code>	<code>subject(glob:"fix: *")</code>	Restricts the question to the subject line; <code>subject(regex:"^fix:")</code> makes the prefix explicit.
<code>author(P)</code>	<code>author(substring:"Example")</code>	Matches the combined author signature; use the name/email predicates when one component must be exact.
<code>author_name(P)</code>	<code>author_name(exact:"A. Engineer")</code>	Distinguishes a name change from an email change.
<code>author_email(P)</code>	<code>author_email(exact:"dev@example.org")</code>	Useful for ownership reports: <code>author_email(glob:"*@example.org") & ::main@origin.</code>
<code>committer(P)</code>	<code>committer(substring:"Release Bot")</code>	Finds who recorded the commit, which can differ from the author.
<code>committer_name(P)</code>	<code>committer_name(exact:"Release Bot")</code>	Use with <code>tags()</code> to inspect bot-created release targets.
<code>committer_email(P)</code>	<code>committer_email(exact:"ci@example.org")</code>	Finds imported or automated commits by recording identity.
<code>mine()</code>	<code>mine()</code>	Uses configured <code>user.email</code> ; inspect config before using it in a multi-identity report.

Identity/time predicate	Positive query	Contrast or operational use
<code>author_date(P)</code>	<code>author_date(after:"2026-01-01")</code>	Measures when the author says the work was authored; use an interval by intersecting <code>after</code> and <code>before</code> .
<code>committer_date(P)</code>	<code>committer_date(before:"2026-09-01")</code>	Measures recording time; it often changes when a stack is rewritten or imported.
<code>signed()</code>	<code>signed()</code>	Finds commits with a recognised signature; combine with <code>immutable()</code> when auditing protected history.

File and content predicates

File predicates accept a `fileset/pattern` argument and return commits whose changes satisfy it. They do not return paths and they do not necessarily mean that a path exists in the resulting tree. Renames, deletions, merge diffs, binary entries, and conflicts make the distinction important.

```
jj log -r 'files("root-prefix-glob:src") & ::@'
jj log -r 'diff_lines_added(substring:"TODO") & mutable()'
jj diff -r 'files("root-prefix-glob:src")'
```

File/content predicate	Basic example	Composition and boundary
<code>files(EXPRESSION)</code>	<code>files("root-prefix-glob:src")</code>	<code>files("root-glob:**/*.rs") & ::@</code> finds current ancestry commits modifying Rust paths. It does not assert that every selected tree still contains the path.
<code>diff_lines(TEXT, FILES)</code>	<code>diff_lines(substring:"TODO", "root-prefix-glob:src")</code>	Matches changed lines on either side of the diff. Add <code>& mutable()</code> to restrict an audit to rewritable history.
<code>diff_lines_added(TEXT, FILES)</code>	<code>diff_lines_added(substring:"TODO", "root-prefix-glob:src")</code>	Finds additions, not merely a commit whose final file contains TODO. Add <code>& ~empty()</code> to exclude marker commits.

File/content predicate	Basic example	Composition and boundary
<code>diff_lines_removed(TEXT, FILES)</code>	<code>diff_lines_removed(regex:"deprecat", root-glob:"**/*.rs")</code>	<code>roots @ ~ vals</code> ; pair with <code>author_email()</code> or a release range to audit a particular change set.

The first argument of the diff-line functions is itself a string pattern, so the same `exact`, `substring`, `glob`, `regex`, and case-insensitive forms apply. The optional second argument is a fileset expression, not a revset. The command-line path arguments of a surrounding command are not a substitute for this nested fileset argument; keep the path scope inside the function when the query must be portable and explicit.

Inspect the exact target-version signature before using a content predicate in policy. Use `jj file list` for final-tree membership and `jj diff` for diff membership. A revset selects commits; a fileset selects paths inside the selected trees or diffs.

Composition patterns and proof order

```
# Reviewable heads in current mutable ancestry:
heads(mine() & mutable() & ::@ ~ empty() ~ conflicts())

# First changes of each unpublished local stack:
roots((mutable() & ::@) ~ ::remote_bookmarks())

# Source changes after imported trunk:
files("root-prefix-glob:src") & main@origin..@

# Safe formatter domain:
mutable() & ::@ & ~immutable_heads() & ~conflicts()
```

Each expression follows the same proof order: choose a domain, constrain ancestry/publication, apply state/content predicates, then reduce to roots or heads if a consumer needs boundaries. `heads(X | Y)` is generally not equivalent to `heads(X) | heads(Y)`, because the first reduction sees descendants from both operands. Likewise, `X ~ Y` excludes selected nodes while `X ~ ::Y` excludes the whole ancestry cone of Y.

Failure cases

Symptom	Cause	Repair
Invalid pattern kind	<code>email:</code> or unsupported prefix.	Use exact, substring, glob, or regex.
Unknown function	Wrong release track.	Compare local help and gate the alias.
Empty bounded result	Source is outside the domain or not visible.	Print each operand separately.
Many results at singleton consumer	Duplicate subject, divergent change, or ambiguous prefix.	Use <code>exactly</code> , an explicit reduction, or a bookmark.
Unexpected head	Reduction applied before a set exclusion.	Print intermediate sets with no graph.
Unexpected remote result	Remote omitted or fetch changed observations.	Qualify remote and display both sides.

A function reference is useful only when it describes the wrong results as well as the right ones. The examples here establish how to reason about composition and command preconditions, while the installed help settles any release-specific spelling.

Part V — Filesets

`jj` has three distinct languages: **revset** selects commits; **fileset** selects paths; **template** renders values. A path expression is not a commit expression with slashes in it.

Filesets include literal paths, recursive paths, globs, and set operators/functions as documented by the installed `jj help -k filesets`. Quote them so the shell does not expand them first. Typical forms are `glob:src/**`, `glob:*.rs`, and a literal path such as `src/lib.rs`.

```
jj diff --from @- --to @ -- 'glob:src/**'
jj restore --from main@origin --to @ -- 'glob:docs/**'
jj squash --into @- -- 'glob:src/**'
jj split --from @ -- 'glob:tests/**'
```

Set operations let a command operate on “all generated files except this directory” or combine multiple patterns. Fileset aliases live in configuration and should be named like path sets, for example `source-files = 'glob:src/** | glob:include/**'`. In 0.45.1 the function surface is deliberately small; the complete supported forms are listed in Appendix D.

Validation rule. First run `jj file list -- fileset` or a no-op `jj diff -- fileset` and inspect the result. A shell glob, a fileset glob, and a Git pathspec have different quoting and recursion rules.

Filesets as a path language

The fileset language comes from the same family of path-set languages as Mercurial's filesets. It evaluates to a set of repository paths. It is neither a Git pathspec nor a shell glob. The most important consequence is that three parsers may see one command line: the shell first, jj's fileset parser second, and the command's path consumer third.

```
shell text → jj fileset AST → selected repository paths
                (prefix/glob/set algebra)  (diff, tree, snapshot, or rewrite)
```

Working-directory and repository-root coordinates

The unqualified form `"path"` is a cwd-relative prefix glob. It matches the path itself and files below a directory. `cwd:` makes that coordinate explicit. `root:` and its variants use the workspace/repository root instead. This distinction matters when a script runs from `src/` but an alias was written relative to the repository root.

Pattern	Coordinate	Match
<code>"path", cwd:"path"</code>	current directory	prefix: path and descendants.
<code>file:"path", cwd-file:"path"</code>	current directory	exact file path, not directory contents.
<code>glob:"*.c", cwd-glob:"*.c"</code>	current directory	cwd-relative wildcard; <code>*.c</code> is non-recursive.

Pattern	Coordinate	Match
<code>prefix-glob:"*.d"</code>	current directory	glob plus matching path prefixes and descendants.
<code>root:"src"</code>	workspace root	root-relative prefix and descendants.
<code>root-file:"Cargo.toml"</code>	workspace root	exact root-relative file.
<code>root-glob:"**/*.rs"</code>	workspace root	root-relative Unix glob.
<code>root-prefix-glob:"src/*.rs"</code>	workspace root	root-relative glob plus descendants of matched prefixes.
<code>glob-i:"*.TXT",</code> <code>cwd-glob-i:"*.TXT"</code>	current directory	case-insensitive cwd-relative glob.
<code>root-glob-i:"*.TXT"</code>	workspace root	case-insensitive root-relative glob.

Glob syntax is Unix-style. A star does not cross a slash in the ordinary glob form; `**` is the recursive form. `prefix-glob:` is a different operation: it takes the glob result and also includes everything below any directory whose path matches the same pattern. In algebraic terms, `prefix-glob:P` behaves like `glob:P | glob:P/**`. It is useful when the pattern names either files or directories and the directory case should select the directory's entire subtree.

```
tree:
src/main.c
src/legacy/old.c
src/legacy/README
src/other.txt

from the repository root:
glob:"src/*"           → src/main.c, src/legacy, src/other.txt
prefix-glob:"src/*"    → src/main.c, src/legacy, src/legacy/old.c,
                        src/legacy/README, src/other.txt
glob:"src/**/*.c"      → src/main.c, src/legacy/old.c
```

The middle result is the important one: `src/legacy` itself matches `src/*` as a directory, so `prefix-glob:"src/*"` follows that matched prefix and includes its descendants, regardless of their names or extensions. It does not recursively search every directory merely because one exists; only directories matched by the prefix pattern are expanded. If the requirement is “all C files at arbitrary depth”, use

`glob:"src/**/*.c"` instead. If the requirement is “the selected directories, plus everything below them”, use `prefix-glob:`.

Fileset operators and precedence

Precedence	Expression	Meaning
1	<code>f(x)</code>	Function call.
2	<code>p:x</code>	Pattern or fileset alias.
3	<code>~x</code>	All paths except x.
4	<code>x & y, x ~ y</code>	Intersection; difference.
5	<code>x y</code>	Union.

Operators at the same level associate left-to-right. Thus `x | y & z` means `x | (y & z)`, while `x ~ y & z` means `(x ~ y) & z`. Parenthesise policy expressions even when precedence is known:

```
jj file list -- 'root:"src" ~ root-glob:"**/*.generated."'
jj diff -- '(glob:"**/*.rs" | glob:"**/*.toml") ~ glob:"target/**"'
jj split -- 'tests & ~glob:"**/fixtures/**'"
```

The function inventory is intentionally small in the target release: `all()` selects every path and `none()` selects no paths. Most practical composition is written with patterns and operators. Fileset aliases can add domain vocabulary:

```
[fileset-aliases]
LOCK = '**/Cargo.lock | **/package-lock.json | **/uv.lock'
SOURCE = 'root-prefix-glob:"src/**" | root-prefix-glob:"include/**"'
'not:x' = '~x'
```

An alias can be a symbol, a function, or a pattern alias. Alias functions can be overloaded by parameter count, but a user alias with a builtin name shadows the builtin. Description tables with `definition` and `doc` make aliases visible to completion.

Quoting: three concrete failures

Command	What happens	Correct form
<code>jj diff ~Cargo.lock</code>	The shell may interpret <code>~</code> as a home-directory expansion or pass an unintended token.	<code>jj diff '~Cargo.lock'</code>
<code>jj diff '~"Foo Bar"'</code>	The inner quotes are fileset-language quotes because <code>~</code> is an operator. The outer quotes are shell quotes that preserve the whole expression.	For a simple path, either <code>jj diff 'Foo Bar'</code> or <code>jj diff Foo\ Bar</code> is sufficient; with the operator, use <code>jj diff '~"Foo Bar"'</code> .
<code>jj diff "Foo(1)"</code>	The shell removes its double quotes, so <code>jj</code> receives <code>Foo(1)</code> and may parse the parentheses as fileset syntax.	<code>jj diff -- 'file:"Foo(1)"':</code> outer quotes are for Bash; inner quotes are for <code>jj</code> 's fileset parser.
<code>jj diff ~glob:**/*.rs</code>	Shell metacharacters and <code>jj</code> operators are exposed simultaneously.	<code>jj diff '~glob:"**/*.rs"'</code>

There are two different quoting layers. In `jj diff -- 'file:"Foo(1)"'`, the outer single quotes are removed by Bash and preserve the complete fileset argument; the inner double quotes reach `jj` and tell its fileset parser that `Foo(1)` is a literal path, not an expression containing parentheses. For a simple path containing spaces, the fileset parser does not require inner quotes: the shell can either quote the argument, as in `jj diff 'Foo Bar'`, or escape the space, as in `jj diff Foo\ Bar`. Once the fileset contains an operator or function call, use inner fileset quotes for the literal path, as in `jj diff '~"Foo Bar"'`. Do not confuse shell escaping with fileset escaping: a backslash inside an outer single-quoted fileset is passed to `jj` rather than removed by Bash. The safest durable rule is to quote the complete fileset and quote inner literal strings when the fileset parser needs to distinguish a path from an operator.

Evaluate before rewriting

There is no separate fileset evaluator command, so use a read-only consumer. `jj file list` displays matching paths in a tree; `jj diff --name-only` shows changed paths; and `jj file search --name-only` tests a content-related path restriction.

```

jj file list -r @ -- 'root-prefix-glob:"src/**"'
jj diff -r @ --name-only -- 'root:"src" ~ root-glob:"**/*.rs"'
jj file list -r main@origin -- LOCK

```

When evaluating a fileset for a rewrite, first establish the selected tree and then the selected change. A fileset cannot select a path that is absent from the source tree. The separate question of which paths are materialised in the current workspace is covered with workspace management in Part XII.

Consumer contracts

Consumer	Fileset role	Important interaction
<code>diff</code>	Restricts displayed paths.	Does not change commit range; use <code>-r/--from/--to</code> separately.
<code>restore</code>	Restricts copied/undone content.	Commit options choose source/destination; fileset is not a commit selector.
<code>squash</code>	Restricts diff ownership movement.	Partial squash can leave a non-empty source and rewrite descendants.
<code>split</code>	Selects content for one side.	Unselected paths remain in the other resulting change; inspect both.
<code>fix</code>	Restricts fixer paths.	Commit domain is selected by <code>-s</code> ; fixer configuration also filters files.
<code>commit</code>	Selects paths included before child creation.	Unselected working-copy edits remain in the new child.
<code>file list/show/search</code>	Tree/search path selection.	Commit option controls the tree; file search pattern controls content.
<code>resolve</code>	Restricts conflicted paths.	Use <code>resolve --list</code> to discover valid paths first.

The fileset language selects paths for a command; it does not decide the general materialisation policy of a workspace. That second concern is intentionally deferred to Part XII.

Fileset review checklist

1. Which coordinate system does the expression use: cwd or root?
2. Is the intent exact file, prefix, non-recursive glob, recursive glob, or prefix-glob?
3. Did the shell preserve the expression?
4. What does the expression select in `jj file list`?
5. Is the consumer selecting output, file contents, diff ownership, or workspace materialisation?
6. Does a partial operation leave a source remainder or rewrite descendants?
7. Would a fileset alias shadow a builtin or depend on a repository-specific config layer?

The 0.45.1 grammar, patterns, precedence, aliases, and functions in this chapter follow `jj help -k filesets`. Appendix D repeats them as a compact reference; the chapter supplies the reasoning and command interactions.

Fileset laboratory: evaluate paths against a real tree

A fileset is evaluated against a tree, not against a commit graph. Begin with a repository tree like this:

```
.
|-- Cargo.toml
|-- src/
|   |-- main.rs
|   |-- parser.rs
|   \-- parser/
|       |-- lexer.rs
|       \-- tests.rs
|-- tests/
|   |-- cli.rs
|   \-- fixtures/
|       |-- basic.txt
|       \-- malformed.txt
|-- docs/
|   |-- design.md
|   \-- api/
|       \-- parser.md
\-- .github/workflows/check.yml
```

The same path can be selected in several ways, with different scope. A literal file names one path. A glob pattern is interpreted according to the fileset grammar and

its matching scope. A root-qualified expression makes the repository root explicit; a cwd-qualified expression makes the command's current directory explicit. This distinction matters when a command is run from `src/parser` or from a script whose working directory is not stable.

```
jj file list 'file:src/parser.rs'
jj file list 'root-file:src/parser.rs'
jj file list 'glob:src/**/*.rs'
jj file list 'root-glob:src/**/*.rs'
jj file list 'cwd-file:parser.rs'
jj file list 'cwd-glob:/**/*.rs'
```

Run these from the repository root and from `src/parser` to see why explicit root forms are safer in documentation and automation. A shell glob such as `src/**/*.rs` can be expanded by the shell before `jj` sees it, depending on shell options. Quote the entire fileset. The `glob:` prefix is part of `jj`'s language and is not a request for the shell to expand anything.

Set algebra over paths

Fileset union, intersection, and difference operate over the set of paths matched in the selected tree. A typical review selection can be expressed as:

```
'glob:src/**/*.rs | glob:tests/**/*.rs'
'glob:/**/*.rs ~ glob:*/tests.rs'
'glob:src/**/*.rs & glob:*/parser/**'
```

Read these left to right only after applying the documented precedence.

Parenthesise mixed expressions. The first selects Rust source and Rust tests. The second removes files named `tests.rs` from an otherwise recursive Rust selection. The third selects parser paths that are also Rust paths. `all()` and `none()` are useful as identity and empty values in aliases, but an unconstrained `all()` passed to a rewriting command is a powerful operation: inspect it first.

```
[fileset-aliases]
rust = 'glob:/**/*.rs'
parser-code = 'glob:src/parser/**/*.rs | root-file:src/parser.rs'
non-tests = 'rust ~ glob:*/tests.rs'
```


Keep fileset aliases semantically narrow. An alias named `source` should document whether it includes generated code, tests, examples, and hidden directories. Aliases are configuration, so a repository-local alias can alter the meaning of a command copied from another repository. When portability matters, expand the alias in a review script or print the active config with `jj config list`.

Filesets and commit selection are orthogonal

Many commands accept both a commit selection and a fileset. The commit answers “which tree or diff is being operated on?”; the fileset answers “which paths within that tree or diff?”. Confusing these layers is a common source of destructive surprises.

Command family	Commit role	Fileset role
<code>diff</code>	Compare a commit or range of trees.	Limit displayed paths.
<code>restore</code>	Choose source and destination commits.	Copy only selected paths from source to destination.
<code>squash</code>	Choose source and destination changes.	Move selected diff paths into the destination.
<code>split</code>	Choose the commit to divide.	First commit owns selected paths; remainder stays in the other commit.
<code>fix</code>	Choose commits whose files are passed through fixers.	Some fixers or command forms restrict paths.
<code>file list/show/search</code>	Choose the tree or commit to inspect.	Filter the displayed or searched paths.

For a partial squash, inspect both the source and destination before and after:

```
jj diff --from C --to C- 'glob:src/**/*.rs'
jj squash --from C --into B 'glob:src/parser/**/*.rs'
jj show C
jj show B
```

The exact option spellings and whether a command accepts a positional fileset or an option are version-sensitive; use the 0.45.1 entries in Part XV. The semantic test

remains stable: a partial operation should leave an inspectable residual diff or a changed destination, not an unexplained disappearance.

Quoting and pattern failures

There are three different failure classes. First, the shell expands an unquoted wildcard and `jj` receives several path arguments or none. Second, `jj` parses a valid fileset but its scope is relative to a different working directory than intended. Third, the fileset is valid but selects an empty set because glob syntax was mistaken for regular-expression syntax. Diagnose in that order:

```
# Show what the shell would pass without running jj.
printf '%s\n' 'glob:src/**/*.*rs'

# Ask jj to enumerate the selection.
jj file list 'root-glob:src/**/*.*rs'

# Compare a literal and a glob.
jj file list 'root-file:src/parser.rs'
jj file list 'root-glob:src/**/parser.rs'
```

Do not use a regex mental model for glob wildcards. A glob's `*`, `**`, directory separators, and prefix rules have language-specific semantics. Case-insensitive forms are explicit; they should not be assumed from the filesystem's case behaviour. A path containing spaces or brackets must be quoted for both the shell and, where applicable, the fileset parser.

Filesets in patch-stack surgery

Filesets are most valuable when a change is conceptually mixed. Suppose B contains an API implementation and an unrelated documentation edit, while C contains tests for both. Rather than splitting by individual diff hunk interactively without a stated invariant, select the path family first:

```
B: src/parser.rs + docs/api/parser.md
C: tests/cli.rs + tests/fixtures/*
```

```
intent:
  move only docs/api/parser.md from B into a documentation change
  keep src/parser.rs in B
  preserve all C tests
```

Use `diff` to confirm the diff ownership, then `squash` or `split` with an explicit fileset. If a descendant's tests become conflicted after the parent's tree changes, that is evidence that the test diff depended on the old context; it is not evidence that the fileset was ignored. Inspect the evolved descendant and resolve or adjust it.

With `restore`, the direction matters. `--from` selects the source tree and `--into` selects the destination commit. A fileset limits copied content; it does not select which commit is edited. This makes a useful “take one generated file from a known commit” operation, but it is not a branch reset and should not be described as one.

Filesets as an API boundary

For shell automation, accept a fileset expression as one argument and pass it as one argument. Avoid concatenating user text into a larger unquoted revset/fileset. Print selected paths before mutation, and use a temporary operation bookmark if the operation is broad. A robust pattern is:

```
set -eu
paths='root-glob:src/**/*.rs ~ root-glob:src/**/*.generated/**'
jj file list "$paths"
jj diff -r @ "$paths"
# Only after review:
jj fix -s 'mutable() & ::@' -- "$paths"
```

If a command does not consume a fileset in the installed release, do not emulate it by passing paths after a revset. Some commands treat extra arguments as commits, options, or errors. The reference entry should be consulted for the command's grammar, while this chapter supplies the language model needed to reason about selections.

Fileset semantics laboratory: path sets, not path strings

A fileset evaluates against a repository tree and returns a set of paths. The returned set has no commit identity and no template type. This separation is more than terminology: a command can select one commit and a fileset can select many paths within that commit's tree. If a command accepts both, the two selectors are evaluated independently and combined at the command's semantic boundary.

shell token	jj parser	consumer
"src ~ glob:\"**/*.gen\""	→ path-set expression operators/functions	→ {src/**} minus generated files applied to a tree or diff

There are three parsers to keep apart. The shell performs quote removal and, unless prevented, pathname expansion. jj then parses the remaining argument as a fileset. Finally the command decides whether the set applies to a tree, a diff, a snapshot policy, or a file metadata operation. A correct fileset can still produce an unexpected result if it is passed to a command whose default commit is not the one the reader had in mind.

Pattern families

In 0.45.1, an unqualified path is a cwd-relative prefix-glob. It matches the named file and paths below a named directory. The explicit forms make the intended scope reviewable:

Form	Meaning	Example and consequence
<code>cwd:"path"</code>	Cwd-relative path prefix.	<code>cwd:src</code> means <code>src</code> below the invocation directory.
<code>file:"path"</code>	One exact cwd-relative file.	Does not recursively include siblings.
<code>glob:"*.rs"</code>	Cwd-relative Unix glob.	Matches Rust files at that directory level, not below every subdirectory.
<code>prefix-glob:"*.rs"</code>	Glob plus matching descendants of a matched prefix.	Equivalent in shape to <code>glob:"*.rs"</code> <code>glob:"*.rs/**"</code> .
<code>root:"docs"</code>	Workspace-root-relative prefix.	Stable when a script changes cwd.

Form	Meaning	Example and consequence
<code>root-file:"LICENSE"</code>	One exact root-relative file.	Useful for a root policy file without matching a directory.
<code>root-glob:"docs/**/* .md"</code>	Root-relative recursive glob.	Matches Markdown files below docs.
<code>root-prefix-glob:"src"</code>	Root-relative prefix glob.	Includes the src directory and its descendants.
<code>glob-i:"*.TXT"</code>	Case-insensitive glob.	Explicitly controls case; do not infer it from the filesystem.

“Recursive” belongs to the pattern form. `glob:"*.c"` is not the same as a shell command that expands `**/* .c`, and a plain `src` is not the same as `file:"src"`. When reproducibility matters, prefer an explicit root-prefixed pattern and state whether it is exact, glob, or prefix-glob.

Fileset consumer precedence and operators

The fileset operators are set algebra over paths. Function calls and pattern aliases bind most tightly. Prefix complement `~x` selects every path except `x`; intersection `x & y` selects paths in both; difference `x ~ y` selects `x` minus `y`; union `x | y` selects paths in either. In the target grammar, intersection and difference bind more strongly than union, and equal-strength infix operations associate from the left.

```
src | docs & ~glob:"**/*.generated"
(src | docs) & ~glob:"**/*.generated"
root-prefix-glob:"src" ~ root-glob:"src/**/generated/**"
all() ~ root-file:"Cargo.lock"
```

The first expression means `src | (docs & ~generated)`, not “everything under `src` or `docs`, then remove generated files”. The second expresses the latter. A path expression that looks obvious in prose should be parenthesised when it combines a positive union with an exclusion. This also makes diffs and review scripts resilient if another person later adds a third path family.

Fileset complement is bounded by the relevant tree. It does not select a path that is not present in the tree merely because that path would exist in the filesystem.

Fileset evaluation for a historical tree is consequently independent of whether that path happens to be present in the current working directory.

Quoting and shell failure modes

Quoting has two jobs: protect the argument from the shell, and, when required, quote a path literal inside the fileset grammar. A bare simple path can omit the inner quotes, but whitespace, parentheses, operators, and punctuation require them. Glob characters do not necessarily require inner quotes for `jj`, but shell quotes are still needed to stop the shell expanding them.

Command	What the shell may do	Robust spelling
<code>jj diff src/*.rs</code>	Expands files before <code>jj</code> sees one fileset; result depends on cwd contents.	<code>jj diff 'glob:src/*.rs'</code>
<code>jj diff ~foo</code>	Unquoted operator can be interpreted or split by shell policy.	<code>jj diff '~"foo"'</code>
<code>jj diff "Foo Bar"</code>	Shell protects whitespace; <code>jj</code> can parse a literal path.	<code>jj diff '"Foo Bar"'</code> when combining with operators.
<code>jj diff src docs</code>	The shell creates a pipeline.	<code>jj diff 'src docs'</code>
<code>jj diff 'glob:**/*.rs'</code>	Safe shell token, recursive <code>jj</code> glob.	Same; document the recursive intent explicitly.

In a shell function, do not place untrusted path text inside a larger unquoted revset/fileset. Pass paths as separate arguments when the command supports literal file arguments, or construct one carefully quoted fileset with a language-aware interface. A path beginning with a dash belongs after `--` where the consumer supports it.

Evaluate a real tree

Use this tree for experiments:

```
Cargo.toml
Cargo.lock
src/main.rs
```

```
src/generated/schema.rs
src/parser/mod.rs
src/parser/tests.rs
tests/cli/basic.rs
tests/fixtures/input.json
docs/guide.md
docs/generated/api.md
scripts/check.sh
```

Evaluate each expression with `jj file list` before passing it to a rewrite:

```
jj file list -- 'root-prefix-glob:src'
jj file list -- 'root-glob:src/**/*.rs'
jj file list -- 'root-glob:src/**/*.rs ~ root-glob:src/**/generated/**'
jj file list -- 'root-prefix-glob:src | root-prefix-glob:tests'
jj file list -- 'root-prefix-glob:docs & ~root-glob:docs/generated/**'
jj file list -- 'root-file:"Cargo.toml" | root-file:"Cargo.lock"'
```

The expected selected sets are, respectively, all `src` paths; all Rust files below `src`; all non-generated Rust files below `src`; `src` and `tests`; non-generated `docs`; and the two root lock/manifest files. Record actual output in a fixture rather than embedding guessed order in a policy script: path ordering is a presentation concern unless the command defines it as a contract.

To test `cwd` semantics, run once from the repository root and once from `src/`. A plain `parser` means different things in those locations because it is `cwd`-relative. The root-prefixed spelling means the same repository path in both locations. This is one of the most useful reasons to write aliases in root coordinates.

Consumer contracts

The same fileset language is consumed by commands with different effects:

Consumer	Commit side	Fileset effect	Risk boundary
<code>diff</code>	Endpoint or selected commit.	Narrows rendered diff paths.	It does not narrow the commit set itself.
<code>restore</code>	Source and destination/change.	Copies or reverses selected paths.	Wrong direction can overwrite intended content.

Consumer	Commit side	Fileset effect	Risk boundary
squash	Source/current and destination parent.	Moves selected diff hunks/files into another change.	Descendants may be rebased and conflict.
split	One change, usually current.	Separates selected paths or diff hunks into a new change.	Unselected content remains at the original boundary.
fix	Configured commit set.	Restricts fixer input and changed files.	Formatter output can rewrite many changes.
file list/show/search	Selected tree/commit.	Lists or reads matching entries.	Inspection does not mean filesystem materialisation.
file track/untrack	Working-copy snapshot policy.	Changes what future snapshots consider.	It is not deletion or publication.

For a diff operation, first inspect the source diff and destination tree. Then use the command's documented read-only form—`jj diff` for content inspection or `--dry-run` on a command that provides it—save the operation ID, and only then apply the fileset. The path selector answers “which paths”; it does not answer “which side wins”, “which change owns the result”, or “whether a descendant should be restored”. Those are command semantics.

Fileset aliases

Fileset aliases provide names for path policy. They live under `[fileset-aliases]` and may be a symbol, a parameterised function, or a table with definition and completion documentation:

```
[fileset-aliases]
source = 'root-prefix-glob:"src" | root-prefix-glob:"include"'
generated = 'root-glob:"**/generated/**" | root-glob:"**/*.lock"'
reviewable(x) = 'x ~ generated'

docs.definition = 'root-prefix-glob:"docs"'
docs.doc = 'Documentation files'
```

Use a descriptive alias only when its coordinate and exclusions are stable. An alias named `source` should not silently include generated code one day and exclude it

the next. Parameterised aliases are language expressions, not shell interpolation; callers pass a fileset expression as one argument:

```
jj diff -- 'reviewable(source)'
jj file list -- 'docs ~ root-glob:"docs/generated/**"'
```

Test aliases from the repository root and a subdirectory, against complete repository trees, and against a tree with spaces or punctuation in a path. Keep fileset aliases separate from revset aliases with similar names; a command will not convert one language into the other.

Case study: split a mixed change without changing its review story

Suppose the current change contains a parser implementation, generated schema output, and a documentation example. The intended review story is “implementation first, generated output and docs second”. A fileset can make that ownership explicit, but it cannot decide whether generated output should be regenerated or copied.

```
before:
  A--B(@)
    B = src/parser/mod.rs
        src/generated/schema.rs
        docs/guide.md

desired:
  A--B--C(@)
    B = src/parser/mod.rs
    C = src/generated/schema.rs + docs/guide.md
```

```
jj diff -r @ -- 'root-file:"src/parser/mod.rs"'
jj diff -r @ -- 'root-glob:"src/generated/**" | root-file:"docs/guide.md"'
jj split -- 'root-glob:"src/generated/**" | root-file:"docs/guide.md"'
jj log -r '::~@'
jj diff -r @-
jj diff -r @
```

The exact interactive result depends on the split mode and fileset supplied, so verify the resulting parent edge and diff ownership rather than inferring it from success. If a descendant existed, it would be automatically rebased over the new boundary and could receive a new commit ID or a stored conflict. Path selection controls content

ownership; the DAG records the consequences.

Fileset review checklist

1. Which parser sees the token first: shell, jj fileset, or the consumer?
2. Is the path coordinate cwd-relative or workspace-root-relative?
3. Does the pattern match one file, a directory prefix, a glob, or a recursive glob?
4. Are operators parenthesised so exclusion applies to the intended union?
5. Does the target command use the fileset for a diff, a tree copy, a snapshot policy, or metadata?
6. What is the exact commit set on which the fileset is being applied?
7. Could the selected path be absent from the current filesystem even though it exists in the repository tree?
8. What commit, descendant, bookmark, and operation changes will follow?
9. Has the selection been printed with `jj file list` or a read-only diff?

The reference card in Appendix D is intentionally dense, but this reasoning process prevents an apparently harmless path argument from becoming a broad rewrite.

Part VI — Bookmarks and publication refs

A bookmark is a name attached to a commit. It is a publication handle, not the unit of local work. A local bookmark can be created, moved, deleted, forgotten, renamed, listed, tracked, or untracked. The 0.45.1 command family is: `jj bookmark advance`, `create`, `delete`, `forget`, `list`, `move`, `rename`, `set`, `track`, and `untrack`.

```
jj bookmark create review -r @
jj bookmark set review -r @
jj bookmark move review --to @
jj bookmark track review --remote origin
jj bookmark untrack review --remote origin
jj bookmark forget review
jj bookmark delete review
```

`set` chooses or creates a local name at a commit; `move` changes an existing name; `delete` asks the remote publication machinery to delete when pushed; `forget`

removes local knowledge of the bookmark without necessarily expressing a remote deletion. Inspect help for exact confirmation and force flags in your release.

Tracking is a relationship

```
local bookmark review <----- tracking relationship -----> review@origin
                        points to @                               points to server tip

@ can move without moving review.
review can be moved without changing @.
Tracking does not mean "follow the working copy".
```

Remote bookmarks are imported names such as `main@origin`. If two remotes have a bookmark named `main`, the remote qualifies it: `main@origin` and `main@gerrit`. A local bookmark may track one corresponding remote bookmark; tracking controls expected push/pull relationship and safety checks, not automatic movement.

Tags are separate immutable-ish names supplied by Git-compatible repositories. Use `jj tag list`, `set`, `delete`, `track`, and `untrack` as the installed command permits; do not use tags as a substitute for a moving review bookmark.

Bookmark and tag state laboratory

A bookmark has a local target, optional remote targets, and possibly a tracking relationship. Treat those as fields in a state record:

```
local ref record:
  name = review
  local target = C
  tracked remote = review@origin
  observed remote target = B
  status = ahead by one / unsynchronised

remote ref record:
  name = review
  remote = origin
  target = B
  tracking local = review
```

The local bookmark does not “point to the working copy” unless its target happens to equal @. Moving @ by new, edit, or ordinary snapshotting does not move review. Moving review does not change the tree of C. A push turns the local ref record into a server request; a fetch turns a server ref into a remote-bookmark observation.

Create, set, move, advance

```
jj bookmark create review -r @
jj bookmark set review -r @
jj bookmark move review --to @
jj bookmark advance review --to @
```

Create requires that the name not already exist. Set is a convenient create-or-update operation and can allow a backwards/sideways move with --allow-backwards. Move targets existing names and can select names by their current commits. Advance is intended to move forward to a descendant or a configured target; it is a useful batch operation when several publication refs should follow a known graph boundary.

Operation	Changes	Does not change
create	adds a local name at a target	working copy, commit trees, remote server
set	moves/creates names at a target	commit graph; remote until push
move	moves existing names	content and change identities
advance	moves eligible names forward	unrelated refs and commit content

Delete versus forget

Delete expresses that a local bookmark should cease to exist and, when tracked and pushed, can express remote deletion. Forget removes local knowledge. These are intentionally different because a user may have merged a GitHub PR and want to remove the local name without deleting a server branch, or may want to delete a stale server branch.

```
# local cleanup, preserve server state
jj bookmark forget review
jj bookmark forget --include-remotes review

# request server deletion
jj bookmark delete review
jj git push --remote origin --deleted
```

`--include-remotes` forgets remote observations too; a future fetch recreates a remote bookmark that still exists at the server. It is not a server deletion. Keep the distinction in cleanup scripts.

Tracking is a relationship, not movement

```
jj bookmark track review --remote origin
jj bookmark untrack review --remote origin
jj bookmark track review@origin
```

Tracking associates a local bookmark with a corresponding remote bookmark. It makes ahead/behind/synced state meaningful and gives push safety a known expected remote target. It does not move the local bookmark when `@` moves, does not rebase a stack, and does not pull changes automatically.

With two remotes, the relationship is three-dimensional:

```
review (local) --tracks-- review@github
|
\----- review@gerrit is a different remote bookmark

main@github and main@gerrit can point to different commits.
```

Use `--remote` or an exact `name@remote` to avoid tracking the wrong remote. A bookmark may be tracked to one or more corresponding remote states according to the backend's model; the command's list output is authoritative for the actual repository.

Normal, deleted, and conflicted remote state

Fetching a remote that deleted or moved a ref changes the imported remote-bookmark state. A local bookmark can be ahead, behind, diverged, or absent. A conflicted bookmark/tag has multiple added or removed targets rather than one normal target. Before pushing a conflicted ref, inspect all targets and resolve the naming/ref policy; do not assume the last fetched value is the desired one.

```
jj bookmark list --all-remotes
jj bookmark list --remote origin --tracked
jj log -r 'remote_bookmarks("review", "origin")'
jj log -r 'bookmarks("review")'
```

The exact string-pattern arguments to revset ref functions are release-sensitive; use the local help form when filtering names. The conceptual distinction remains stable: a local name is an explicit local ref, a remote name is an observation, and tracking is metadata linking them.

Tags and release policy

Tags are useful at release points because their meaning should not change as a mutable change stack evolves. A local tag can be set to @, listed, tracked to a remote tag, and pushed. Moving an existing tag needs `--allow-move` in the target CLI and should require a release-policy review.

```
jj tag set v2.4.0 -r @
jj tag list --all-remotes
jj tag track v2.4.0@origin
jj git push --remote origin --tag v2.4.0
```

Do not use a tag as a review branch just because both are names. A moving review boundary wants a bookmark; a release identity wants a tag. jj 0.45.1 provides tracked/untracked remote-tag predicates; build release queries from the supported tag and ref functions documented in Appendix C.

Publication invariant

Before publishing, be able to answer four questions:

1. Which exact local bookmark is being pushed?
2. Which remote and URL will receive it?
3. Which remote bookmark was fetched as the expected old target?
4. Which commit/change is the intended tip, and is it conflicted, empty, divergent, or immutable?

```
jj git remote list
jj bookmark list --remote origin --all-remotes
jj log -r 'review | review@origin'
jj diff --from review@origin --to review
jj git push --dry-run --remote origin --bookmark review
```

If any answer is unclear, stop at inspection. A publication ref is a small interface with potentially large external consequences; the local graph-first workflow does not remove that boundary.

Git backend: two object worlds and an import/export boundary

In a Git-backed repository, jj and Git share enough storage to interoperate, but they do not share the same semantic model. Git stores commit objects, trees, blobs, and refs. jj stores change metadata, operation history, workspace state, conflict values, and its view of bookmarks/tags. Some jj state is represented in Git-compatible objects; not all of it is meaningful to Git.

```
jj commit/change/operation/workspace model
|
|- exportable commit/tree/blob → Git object database
|- local bookmark/tag → Git ref when exported
\ - change evolution/op/workspace/conflict metadata → jj semantics

Git refs/objects ← fetch/import → jj remote-bookmark/ref view
```

Colocated repository discipline

Colocation is valuable when GitHub/Gerrit tooling, hooks, Git inspection, and jj need one working directory. It also creates two writers and two sets of expectations. A Git command can update refs without jj seeing them until import. A Git command that updates the worktree can race with jj's snapshot. A jj rewrite can make Git's current branch/ref appear surprising until export.

```
jj git colocation status
jj git export
git -C "$(jj root)" show-ref
git -C "$(jj root)" log --all --oneline
git -C "$(jj root)" fetch origin
jj git import
jj bookmark list --all-remotes
```

Use one writer for the working directory. Git inspection commands that do not touch the worktree are usually safe; Git checkout/reset/restore commands require a snapshot boundary and explicit coordination. After Git-side ref changes, import before interpreting jj's remote state. After jj bookmark changes, export before asking a Git-native tool to inspect refs.

Clone and init choices

Choice	Where metadata lives	Use when
<code>jj git clone</code> (0.45.1 default)	jj plus a top-level Git checkout, unless <code>git.colocate=false</code>	Normal choice when Git/gh/Gerrit tools are part of daily work.
<code>jj git clone --no-colocate</code>	jj repository with its backing Git data hidden under <code>.jj</code>	Use when jj should own the workspace and Git is primarily transport.
<code>jj git clone --colocate</code>	Explicitly request the normal colocated layout	Useful as documentary/script intent even though it is normally the default.
<code>jj git init</code> (0.45.1 default)	Initialise a colocated Git-backed jj repository unless <code>git.colocate=false</code>	Start a repository that ordinary Git tools can also open.
<code>jj git init --no-colocate</code>	Initialise with the backing Git repository hidden below <code>.jj</code>	Explicit non-colocated layout.
<code>jj git init --git-repo PATH</code>	Use an existing Git repository as the backing repository; mutually exclusive with <code>--colocate</code>	If PATH is the destination itself and contains <code>.git</code> , jj creates a colocated workspace. If PATH names a different Git directory, jj uses a non-colocated layout.

After a clone, identify the trunk bookmark and remote name. The default remote is often `origin`, but a GitHub fork/upstream or Gerrit repository may use different names. Do not write scripts that assume the remote URL from a directory name.

Fetch, import, export, push

```
jj git fetch --remote origin
jj git import
# graph work happens here
jj git export
jj git push --remote origin --bookmark review
```

Fetch is network object/ref transfer plus local remote-bookmark update. Import is local Git-ref observation. Export is local jj-to-Git-ref update. Push is network publication. A normal jj command can perform some import/snapshot work at its boundary, but explicit commands make automation and diagnosis clearer.

Operation	Network?	Primary direction	Changes local commit DAG?
fetch	yes	remote Git → local objects/remote refs	usually no mutable rewrite
import	no	local Git refs → jj view	no, but ref state changes
export	no	jj bookmarks/tags → Git refs	no
push	yes	local Git refs/objects → remote	no, but remote state changes

Native Git commands that are safe, conditional, or hazardous

Command class	Assessment	Required boundary
<code>git show</code> , <code>git log</code> , <code>git cat-file</code>	Useful inspection of Git objects.	Remember Git cannot show jj change/evolution semantics.
<code>git show-ref</code> , <code>git for-each-ref</code>	Useful ref inspection.	Import after ref changes.
<code>git fetch</code>	Can update Git refs.	Run <code>jj git import</code> before relying on them.
<code>git push</code>	External publication.	Explicit ref/remote; jj will not make a pushed mistake local-undoable.
<code>git checkout</code> , <code>git reset</code> , <code>git restore</code>	Worktree/ref writers.	Do not race jj snapshot; import refs afterward.
<code>gh pr checkout</code>	Git-oriented hosting workflow.	Inspect worktree and import refs before jj work.

Credentials, SSH, and signing

jj's Git transport commonly uses Git's URL, credential helper, SSH agent, and known-host configuration. Gerrit and GitHub may have different hosts, keys, or permissions. Keep transport authentication separate from commit signing: an SSH key used to connect to a remote is not automatically a commit-signing key, and a good local signature does not grant push permission.

```
jj git remote list
ssh-add -l
```

```
git config --get-regexp 'credential|url|core.ssh'
jj sign -r @
jj log -r @ -T 'if(signature, signature.status(), "unsigned") ++ "\n"
```

Do not put secrets in `jj` aliases or repository config. Test authentication with a harmless fetch, inspect the destination URL, and use a controlled key/agent environment in CI. Signing a stack rewrites IDs; publish the resulting bookmark only after checking the entire descendant chain.

Private commits and Git compatibility

Some `jj`/Git policies allow commits to be marked private so they are not pushed by ordinary publication. Private is a publication policy, not a different change identity or a hidden filesystem state. The exact private-commit configuration and push flags are release-sensitive; inspect the 0.45.1 help/schema before depending on them.

```
jj log -r 'mutable() ~ ::remote_bookmarks()'
jj git push --remote origin --tracked
jj git push --remote origin --all
```

Use explicit bookmark selection for review work. Broad `--all` is appropriate only when all local names/tags are intended for that remote. A private mutable change can still be a descendant of a public bookmark and can still affect a pushed descendant if the graph boundary is not selected carefully.

Part VII — Git backend and interoperability

With the Git backend, `jj` stores its own metadata under `.jj` and uses Git objects/refs for interoperability. A colocated repository has both `.git` and `jj` metadata at one worktree; a non-colocated repository keeps `jj`'s view separate. Colocation is convenient for an advanced Git user because Git-native tools, `gh`, hooks, and editors can inspect the same object database. It also means two programs can race to update refs or the worktree.

```
jj git init --colocate
jj git clone https://example.test/project.git project
```

```
jj git fetch --remote origin
jj git import
jj git export
jj git push --bookmark review --remote origin
jj git remote list
jj git root
```

jj notices Git-side ref changes during import/snapshot boundaries; after running native Git, use `jj git import` or a normal `jj` command that imports refs. Native Git does not understand change IDs, operation history, conflicts as `jj` values, or workspace pointers. It can inspect and transport Git objects, but its rewrites may make `jj` bookmarks/remote state stale until imported.

For colocated scripts, `GIT_DIR=$(jj git root)/.git` is sometimes useful, but prefer `git -C "$(jj root)" ...` and let `jj` report its root. Never use Git to reset the worktree while a `jj` process is snapshotting it. Git credentials, SSH agents, and signing configuration are generally consumed through the Git transport or configured `jj` signing/authentication facilities; test the exact credential helper in the target environment.

Push safety

Push bookmarks, not arbitrary working-copy commits, unless you deliberately use `jj 0.45.1`'s change-based push mode and understand the bookmark it creates. A tracked bookmark gives `jj` a lease-like expectation about the remote value. An exceptional host-side force update is safe only when the changed remote tip is the one you intended to replace; review `jj bookmark list`, `jj log -r 'review|review@origin'`, and `jj's --dry-run` preview first.

Git interoperability laboratory

The Git backend has two related but distinct storage views. `jj` stores its commit and operation metadata in the repository's `jj` backend; the Git backend can also read and write Git objects and refs. A colocated repository adds a Git view to an existing checkout so that native Git commands and `jj` commands operate on nearby data. Colocation is convenient precisely because it removes a transport boundary; it is also risky because two tools can mutate related state without sharing `jj`'s operation semantics.

```
working tree
|
|-- jj view: commit DAG, change IDs, operation DAG, workspace
|
\-- Git view: commits/trees/blobs, refs, index, remote-tracking refs
```

A Git commit object is still a Git object, but `jj` gives it additional meaning. In the `jj 0.45.1` Git backend, `jj`-created commits can contain a non-standard `change-id` commit header when `git.write-change-id-header` is enabled. The value is a reverse-hex encoding of `jj`'s change identity, as described in the Git compatibility documentation. This header is object data, not a standard Git identity field: ordinary `git log` and `git show` do not normally print it, `git cat-file -p` can, and native Git operations are not required to preserve it. Independently, `.jj` records the change's evolution, operations, and workspace view. Thus “the change ID is in the `jj` view” is the semantic statement; “the Git backend serialises a change-id header” is the `0.45.1` storage detail.

Laboratory: `jj @`, Git `HEAD`, and object reachability

0.45.1 verified boundary. Run this experiment in a disposable colocated repository. It is intentionally about pointers and reachability, not about converting a `jj` commit into Git at publication time.

Start with one ordinary Git commit, then let `jj` snapshot an edit without creating or moving a publication bookmark:

```
git -c init.defaultBranch=main init demo
cd demo
git config user.name "Example User"
git config user.email user@example.invalid
printf '%s\n' base > note
git add note && git commit -m base
jj git init --colocate
printf '%s\n' jj-snapshot >> note
jj status
```

Now ask both tools what they consider current. Use the full ID printed by the first command in the later commands:

```
jj log --no-graph -r @ -T 'commit_id ++ "\n"'
git status --short
git symbolic-ref HEAD
git log --oneline --decorate -3
git log --all --oneline --decorate -6
git show-ref
git cat-file -t JJ_COMMIT_ID
git cat-file -p JJ_COMMIT_ID | sed -n '1,14p'
```

In a colocated repository, `jj log` reports a new working-copy commit and `git cat-file -t` reports commit. The object was created locally when `jj` made the snapshot; `jj git push` is not the conversion step. At the same time, `git symbolic-ref HEAD` may remain `refs/heads/main`, with that ref still pointing at the earlier Git commit. Therefore `git log`, whose default starting point is Git's HEAD, need not show the `jj` working-copy commit. `git log --all` may show more because it walks every ordinary and implementation ref that Git considers reachable, including a temporary `refs/jj/keep/*` retention ref in this release; do not treat that implementation ref as a publication interface. `git show-ref` makes the distinction visible. An object can exist in the object database without being reachable from HEAD, a local branch, or any other ref that a particular Git command walks.

The two status commands answer different questions. `jj status` describes the filesystem relative to `jj`'s working-copy commit and snapshots the filesystem at the command boundary. Git status compares the filesystem with the Git index and HEAD. In a colocated repository, a `jj` snapshot can therefore include `note` in the `jj`

commit while Git still reports the path as modified relative to its index. The Git index is not `jj`'s staging area, and a Git status letter is not evidence that `jj` failed to record the edit.

Finally create an explicit publication boundary:

```
jj bookmark create review -r @
jj git export
git show-ref refs/heads/review
git symbolic-ref HEAD
```

The local `review` ref now reaches the same commit object, while Git `HEAD` can still name `refs/heads/main`. In a colocated workspace, creating or moving a bookmark may export its ref as part of the normal `jj` operation, so the explicit `jj git export` is a useful idempotent boundary for scripts. If the bookmark named by `HEAD` is deliberately moved or checked out through a coordinated Git operation, the target of `HEAD` can change; that is a different event from `jj` advancing its workspace-local `@`. A push sends the selected exported ref and reachable objects to a remote; it does not promise to move local `HEAD`.

Event	jj state	Git state	What may move
Filesystem edit	Working tree differs from the recorded snapshot.	Index/worktree comparison changes.	No object or ref need exist yet.
<code>jj</code> snapshot or rewrite	<code>@</code> , commit DAG, operation view, and possibly descendants change.	Git-backend commit/tree/blob objects are created; implementation retention refs may appear.	Not necessarily <code>HEAD</code> or <code>refs/heads/*</code> .
Bookmark create/set and export	A named <code>jj</code> bookmark points at a selected commit.	Its corresponding local Git ref is updated by export/normal colocated synchronisation.	The bookmark ref; <code>HEAD</code> only if it names that ref and it is moved.
<code>jj git fetch</code> then import	Remote-bookmark observations enter <code>jj</code> 's view.	Objects and remote-tracking refs are fetched, then read by <code>jj</code> .	Remote-tracking refs; not local <code>@</code> by itself.

Event	jj state	Git state	What may move
<code>jj git push</code>	Local graph is selected through a bookmark/tag/ref.	Remote Git ref and reachable remote objects are updated.	Remote state only, apart from local transport bookkeeping.
Native Git checkout/reset/commit	May leave jj's view state until import/snapshot.	Git <code>HEAD</code> , refs, index, worktree, or objects can change.	Whatever that Git command explicitly writes.

This model also explains the directory boundary. In a colocated repository, top-level `.git` remains the Git object database, refs, remotes, and native-tool layer; `.jj` remains the jj view, operation log, workspace metadata, and jj backend state. In a non-colocated repository, the backing Git repository is hidden below `.jj`; ordinary Git commands run in the worktree do not find a top-level `.git`. Use `jj git root` and explicit Git directory arguments only when you have deliberately chosen that integration.

Classify native Git commands before using them in a colocated workflow:

Class	Examples	Boundary
Read-only inspection	<code>git log</code> , <code>git show</code> , <code>git cat-file</code> , <code>git show-ref</code>	Safe for state, but Git cannot display jj evolution or workspace semantics.
Git-ref mutation	<code>git update-ref</code> , <code>git branch -f</code> , <code>git fetch</code>	Import before relying on the new ref view; record the exact ref.
Worktree/index mutation	<code>git checkout</code> , <code>git reset</code> , <code>git restore</code> , <code>git add</code> , <code>git commit</code>	Serialise with jj; snapshot or import before interpreting the result. Git's index is separate.
Network publication	<code>git push</code> , <code>gh pr create</code>	Explicitly select remote and ref; local jj operation recovery cannot undo server state.

Reference: colocated versus non-colocated layout

Question	Colocated	Non-colocated
Native Git commands in the project directory?	Yes, subject to safe coordination.	Use a separate Git checkout or explicit Git directory.
Git refs visible to jj?	Imported/exported through the colocated backend.	Managed through jj's Git backend and transport.
Index meaning	Git tools may update it; jj snapshots filesystem state, not Git staging intent.	There is no native Git index in the normal working directory.
Concurrent writers?	More ways to race; serialise Git/jj mutations.	Still serialise jj operations, but fewer accidental ref mutations.
Best use	GitHub/Gerrit tooling, hooks, IDEs, gradual adoption.	Clean jj-only automation and isolated experiments.

In a colocated repository, a native `git switch` or `git reset` changes Git's checkout/index/ref state. `jj` may not immediately incorporate all such changes into its integrated view. Run the appropriate `jj git import` command or a normal `jj` command that refreshes the Git view before relying on a Git-side ref movement. Conversely, a `jj bookmark push/export` can update Git refs that a native Git process sees later. Treat the directory as a shared mutable resource.

Clone, init, fetch, import, export

```
jj git clone https://example.org/project.git project
jj git init --colocate
jj git remote list
jj git fetch --remote origin
jj git import
jj git export
jj git root
```

`jj git clone` creates a `jj` repository backed by Git objects and configures a remote; in 0.45.1 it is colocated by default unless `git.colocate=false` or `--no-colocate` is used. `jj git init` has the same default, while `--no-colocate` explicitly hides the backing Git repository below `.jj`. `--colocate` is worth spelling in scripts when the layout is a deliberate precondition. `fetch`

transfers remote objects and updates remote-bookmark observations; `import` refreshes `jj`'s view from Git refs where needed; `export` writes `jj` bookmark state into Git refs. The distinction between `fetch` and `import` matters when native Git has changed refs or when a script needs to separate network activity from local ref reconciliation.

Use `jj git root` to obtain the Git root when a tool expects `GIT_DIR` or a repository path:

```
GIT_DIR=$(jj git root) git -C "$(jj root)" show HEAD
git --git-dir="$(jj git root)" for-each-ref
```

Check the command's output shape before embedding it in a path. A `root` command returns a directory; it is not automatically the value of `GIT_DIR` in every layout. Prefer `git -C` when operating on the working tree, and use `--git-dir` only when the command's path semantics require it.

Fetch and remote bookmarks

`Fetch` updates the local observation of remote refs. It does not move the workspace's `@` and does not rebase local work automatically merely because the remote trunk advanced. After fetching, inspect:

```
jj git fetch --remote origin
jj log -r 'main@origin'
jj log -r 'main@origin..@'
jj log -r 'remote_bookmarks()'
```

A remote bookmark is not a local bookmark with a decorative suffix. It is a name imported from a remote namespace. A local bookmark can track it, which gives `jj` a relationship for ahead/behind and push safety. Tracking does not make the local bookmark follow a working-copy commit; the local target still moves only through an explicit bookmark operation or a graph rewrite that preserves its target relationship.

Push safety and ref publication

Push sends selected local bookmarks or tags to a remote. In `jj 0.45.1`, the safest scripted form names the remote and bookmark and performs the available

`--dry-run` preview before the real push:

```
jj log -r 'bookmarks(exact:"pr/parser")' -T 'commit_id ++ "\n"'
jj git push --remote origin --bookmark pr/parser --dry-run
jj git push --remote origin --bookmark pr/parser
```

A non-fast-forward update is not automatically evidence that force is required. Compare the local bookmark target with the remote bookmark target and determine whether the local target descends from the remote target, whether another actor moved the remote, and whether server policy permits replacement. The tracking relationship supplies safety context; it is not a lease substitute for all server-side races. In `jj 0.45.1`, `jj git push` has no `--force`, `--force-with-lease`, or generic allow-rewrite option: its own safety check accepts a backwards update only when the remote still matches the state `jj` last fetched. Treat an exceptional host-side replacement as a separate, explicitly audited native-Git or hosting operation.

`--tracked`, `--all`, `--deleted`, and `tag` options widen the publication set. They are convenient interactively and dangerous in generic scripts. A script that publishes every tracked bookmark can unexpectedly update a second review or a stale local publication name. Prefer one explicit bookmark per publication action.

Credentials, SSH, and signing

`jj` Git transport normally uses Git's configured remote URLs and credential mechanisms. SSH keys, agents, known-host verification, credential helpers, and HTTPS tokens remain deployment concerns; do not put secrets in `jj config` committed to a repository. Test authentication with a read-only fetch before a push. A remote URL can be inspected with `jj git remote list` or native Git configuration, but redact credentials from logs.

Signing is a property of commits or transport policy, not a synonym for a `jj` change ID. `jj 0.45.1` provides `jj sign` and `jj unsign`; configure the signing key and inspect the resulting signature fields through templates or Git verification. A rewrite after signing produces a new commit and generally requires signing the successor again. Gerrit and GitHub may apply additional server-side verification; local signing does not guarantee acceptance.

Native Git tooling: a safe contract

Use Git for tools that have no `jj` equivalent—`gh`, hooks, server-specific commands, object inspection, and specialised diff tools—but establish a boundary:

1. Before a Git-side mutation, inspect `jj`'s current operation and workspace state.
2. Make one class of mutation at a time: files, Git refs, or `jj` graph.
3. After changing refs with Git, run the `jj import/refresh` path and inspect remote/local bookmarks.
4. After changing graph state with `jj`, export/import as required and check what Git sees.
5. Do not run two writers against the same working directory concurrently.

For GitHub, `gh pr view` and `gh pr merge` operate on server state and may fetch or inspect refs; they do not understand `jj`'s operation DAG. For Gerrit, server review commands and `refs/for/...` transport semantics are likewise outside `jj`'s local graph model. Keep the local bookmark that a host consumes explicit and verify it after every host-side action.

Part VIII — GitHub workflows

Branchless feature and pull request

```
main@origin
|
A
|
B
|
C
|
@
```

Fetch main, create or edit a child, and work without a local publication bookmark. Your stack is visible by graph and revset. When the change is ready, create a bookmark only for publication:

```
jj git fetch --remote origin
jj new main@origin
# edit files; let jj snapshot
jj describe -m 'Add bounded cache'
jj bookmark create cache-pr --revision @
jj git push --bookmark cache-pr --remote origin
```

GitHub sees a branch ref because Git transport requires one; locally, the ref is disposable. After the pull request merges, fetch, move/edit onto updated main as appropriate, and `jj bookmark forget cache-pr` or delete it after confirming the server state.

Stacks and review

For one PR use one change. For stacked PRs, publish one bookmark per review boundary, each pointing at a stack tip:

```
jj bookmark create pr-base -r A
jj bookmark create pr-feature -r B
jj git push --bookmark pr-base --bookmark pr-feature --remote origin
jj rebase -b @ -o main@origin
```

Rewriting A automatically rebases B and its descendants. Move the publication bookmarks after the rewrite and push through jj's fetched-state safety check; jj 0.45.1 has no force flag on `jj git push`. An exceptional host-side replacement is a separate, explicitly reviewed operation. GitHub's UI merge operation is a server-side ref/graph action; jj learns its result on fetch.

`gh pr create --head cache-pr`, `gh pr view`, and `gh pr checkout` can coexist with jj. Treat `gh pr checkout` as a Git-oriented worktree/ref operation and inspect the result with `jj status` and `jj git import`. Fork remotes need explicit names: `origin` may be your fork while `upstream` is the source; qualify remote bookmarks in revsets.

GitHub PR lifecycle: local graph versus publication ref

GitHub consumes Git refs. jj lets the developer organise mutable work without local bookmarks and creates a bookmark only when a server-facing boundary is needed. The lifecycle has two independent timelines:

```
local jj timeline:  edit → rewrite → rebase → inspect → rewrite
GitHub timeline:    publish ref → review → update ref → merge
```

A local rewrite can happen many times between two GitHub ref updates.
The remote bookmark is an interface snapshot, not the local working state.

Single-change PR

```
main@origin -- base -- @
                        unnamed mutable change

publish:
main@origin -- base -- P
                        ▲
                        pr/cache
                        pr/cache@origin
```

```
jj git fetch --remote origin
jj new main@origin -m 'Cache: bound memory use'
# edit files
jj status
```

```
jj diff
jj bookmark create pr/cache -r @
jj git push --remote origin --bookmark pr/cache
gh pr create --head pr/cache --title 'Bound cache memory'
```

The local bookmark is now a publication name. The working-copy commit can still be amended with `jj describe`, `jj squash`, or ordinary filesystem edits. After the next snapshot, the bookmark still points at the old commit until it is moved:

```
jj describe -m 'Cache: bound memory use and expose metrics'
jj bookmark set pr/cache -r @
jj git push --remote origin --bookmark pr/cache
```

Whether the push is a fast-forward depends on the server's previous ref and the rewrite. `jj`'s push safety checks and remote tracking expose the expected old target; do not solve a rejection with an unreviewed native-Git replacement. In `jj 0.45.1`, `jj git push` has no force flag. Inspect:

```
jj bookmark list --remote origin --tracked
jj log -r 'pr/cache | pr/cache@origin'
jj diff --from 'pr/cache@origin' --to pr/cache
```

Stacked PRs

```
main@origin -- A -- B -- C
                ▲   ▲
                pr/api pr/metrics
```

Use one publication bookmark per server review boundary. The base PR's bookmark points to B and the dependent PR's bookmark points to C; the dependent PR should be created with the base PR as its target branch according to GitHub's UI policy.

```
jj bookmark create pr/api -r B
jj bookmark create pr/metrics -r C
jj git push --remote origin --bookmark 'pr/*'
gh pr create --head pr/api --base main
gh pr create --head pr/metrics --base pr/api
```

When A or B is rewritten, descendants are rebased locally and both bookmark targets may receive new commit IDs. Update bookmarks deliberately, then push both. A common mistake is moving only the top bookmark, leaving the base PR pointing to a predecessor that no longer represents the intended boundary.

```
jj edit B
# modify the early API change
jj describe -m 'API: correct quota invariant'
jj log -r 'pr/api | pr/metrics | ::@'
jj bookmark set pr/api -r B
jj bookmark set pr/metrics -r C
jj git push --remote origin --bookmark pr/api --bookmark pr/metrics
```

Main moves during review

GitHub's main branch can move while local PR refs are under review. Fetch first, then rebase the mutable stack onto the new remote bookmark. In a stack, select the stack roots or use the branch selection that matches the actual graph:

```
jj git fetch --remote origin
jj log -r 'main@origin'
jj rebase -b 'roots(main@origin..pr/metrics)' -o main@origin
jj log -r 'pr/metrics:: & mutable() & conflicts()'
jj resolve
jj bookmark set pr/api -r 'subject(exact:"API: correct quota")'
jj bookmark set pr/metrics -r @
jj git push --remote origin --bookmark 'pr/*'
```

The exact stack-root expression should be verified in the repository; the important sequence is fetch, inspect, select, rebase, resolve, move publication refs, push. Do not rebase a remote bookmark itself. Remote bookmarks are observations of server refs and are normally immutable boundaries for local mutable work.

Merge, closure, replacement, and cleanup

After GitHub merges, the local repository does not learn the server graph until fetch. Fetch the updated main and decide whether local changes are now ancestors of it:

```
jj git fetch --remote origin
jj log -r 'main@origin | pr/cache | ::pr/cache'
```



```
jj log -r 'pr/cache ~ ::main@origin'  
jj bookmark forget pr/cache  
jj bookmark forget --include-remotes 'pr/cache'
```

Use `forget` when the local publication name should disappear without asking GitHub to delete the branch. Use `bookmark delete` followed by an explicit push when the server branch should be removed. If a PR is closed and replaced, keep the old bookmark long enough to compare/refute server state, then forget it and create a new publication name or move the existing one intentionally.

Workflow: fork/upstream topology

```
jj git remote add github git@github.com:me/project.git  
jj git remote add upstream git@github.com:org/project.git  
jj git fetch --remote upstream  
jj new main@upstream  
jj bookmark create me/cache -r @  
jj git push --remote github --bookmark me/cache
```

Here `main@upstream` is the imported source boundary and the local bookmark is pushed to the fork. A local `origin` default can be unsafe if it changes meaning between clones. Use explicit remote arguments in aliases and qualify both remote bookmarks in revsets.

Native gh/Git interaction

`gh pr view` and `gh pr create` can operate against a colocated repository. Commands such as `gh pr checkout` are Git-oriented and may create/move refs or alter the worktree in ways `jj` must import. After any command that changes Git refs:

```
jj git import  
jj status  
jj bookmark list --all-remotes
```

Do not allow `gh`, `Git`, and `jj` to write the same worktree concurrently. The host/UI can merge or close a PR without local filesystem changes; `fetch` is the observation boundary.

Push rejection decision tree

1. Fetch the named remote and inspect local/remote bookmark targets.
2. Determine whether the server changed independently or the local bookmark was rewritten.
3. Compare the intended tree and ancestry with `jj diff` and `jj log`.
4. If the ref should advance normally, rebase or move it to the expected descendant.
5. If replacement is intentional, use the hosting system's documented replacement mechanism and record why; `jj git push` itself has no force flag in `jj 0.45.1`.
6. If the wrong ref was pushed, repair the server ref separately; `jj undo` only repairs local operation state.

GitHub field laboratory: one local graph, several PR views

A pull request is a server-side interpretation of a Git ref. `jj` has no obligation to organise local work around that ref. The local graph can therefore be optimised for editing while a publication bookmark exists only at the boundary GitHub needs to see. The cost is that the bookmark is an interface object whose target must be maintained explicitly.

A single PR over time

```
time 0:      main@origin--A--B(@)      no local bookmark
publish:     main@origin--A--B
              ▲
              pr/cache
              pr/cache@origin
after review: main@origin--A--B'(@)
              ▲
              pr/cache      (old until moved)
              pr/cache@origin (server's last copy)
```

The distinction between `pr/cache` and `pr/cache@origin` is the useful state. The local bookmark can move to `B'` before the push; the remote bookmark remains the last fetched observation. A log that prints only the local bookmark hides work waiting to be published, while one that prints only the remote bookmark hides the

server's lag.

```
jj git fetch --remote origin
jj new main@origin -m 'Cache: bound memory use'
# edit, test, and inspect the working-copy commit
jj diff -r @
jj bookmark create pr/cache -r @
jj git push --remote origin --bookmark pr/cache
```

GitHub-specific commands can now create the pull request. The server may choose a merge base, report checks, and expose a review URL; those are server facts, not `jj` graph facts. The local invariant is that the bookmark named in the Git push points to the tree and ancestry intended for review.

When review requests a change, edit the logical change rather than creating a fixup branch:

```
jj edit 'change_id(PR_CHANGE)'
# edit files
jj status
jj diff
jj describe -m 'Cache: bound memory use and expose metrics'
jj bookmark set pr/cache -r @
jj diff --from pr/cache@origin --to pr/cache
jj git push --remote origin --bookmark pr/cache
```

A non-fast-forward refusal is a safety signal. It can mean that the remote advanced, the tracking observation is stale, the local target is not a descendant, or server policy/authentication failed. Fetch and compare before considering any replacement. In `jj 0.45.1` there is no force option on `jj git push`; if a host permits an exceptional replacement through another interface, treat that as a separate publication-policy decision, not a repair for an unclear graph.

Stacked PRs and review boundaries

```
main@origin--A--B--C--D(@)
      ▲      ▲
    pr/api  pr/metrics
```

One bookmark at D can publish one PR containing the stack. Two bookmarks can publish dependent review boundaries: the base bookmark at B and the dependent bookmark at D. The dependent PR's base is a GitHub setting; jj's responsibility is to publish the two intended refs and preserve their ancestry.

Intent	Local publication	Server boundary
One review containing a stack	One bookmark at the tip; push the range it makes reachable.	How GitHub displays individual commits and chooses its merge base.
Two dependent reviews	One bookmark per boundary; update both after rewrites.	Set the dependent PR's base to the first PR.
Independent reviews	Separate roots or bookmarks from one base.	Each PR targets trunk rather than another review ref.

When an early member changes, descendants may receive new commit IDs because their parent IDs changed. Their jj change IDs ordinarily remain stable because they are successors, not duplicates. Move every affected publication bookmark deliberately:

```
jj edit 'change_id(API_CHANGE)'  
# repair the early change  
jj describe -m 'API: reject negative quota'  
jj log -G -r 'roots(main@origin..@):.'  
jj diff --from pr/api@origin --to pr/api  
jj diff --from pr/metrics@origin --to pr/metrics  
jj bookmark set pr/api -r 'change_id(API_CHANGE)'  
jj bookmark set pr/metrics -r 'change_id(METRICS_CHANGE)'  
jj git push --remote origin --bookmark pr/api --bookmark pr/metrics
```

Do not select a current review target by an old commit ID after a rewrite. Use the current bookmark or change ID when the intention is to follow the logical change, and use `evolog` to confirm the successor.

Trunk movement, rejection, and cleanup

Upstream main can advance while a PR is under review. A safe update is fetch, inspect, rebase only local mutable roots, resolve stored conflicts, move publication boundaries, and push:

```

jj git fetch --remote origin
jj log -G -r 'main@origin | pr/api | pr/metrics | ::pr/metrics'
jj rebase -b 'roots(main@origin..pr/metrics)' -o main@origin
jj log -r 'conflicts() & ::pr/metrics'
# resolve, then update each boundary and publish
jj bookmark set pr/api -r 'change_id(API_CHANGE)'
jj bookmark set pr/metrics -r 'change_id(METRICS_CHANGE)'
jj git push --remote origin --bookmark pr/api --bookmark pr/metrics

```

If another machine published a successor of the same logical change, fetching may expose divergence rather than a simple rejection. Compare the evolutions before choosing a successor, creating a new bookmark, or using the 0.45.1 `converge` command. A host-side or native-Git force update that merely suppresses the warning discards evidence and possibly another contributor's review.

Forks, gh, and the Git boundary

```

jj git remote add github git@github.com:me/project.git
jj git remote add upstream git@github.com:org/project.git
jj git fetch --remote upstream
jj new main@upstream -m 'Fix parser'
jj bookmark create me/fix -r @
jj git push --remote github --bookmark me/fix

```

Here the fork is the publication destination and upstream is the source boundary. Qualify remote bookmarks and use explicit `--remote` in aliases. `gh pr view`, `gh pr create`, and `gh pr merge` operate on server state. `gh pr checkout` is designed around Git refs and worktrees; after any Git-side ref operation, serialise writers, run `jj git import` if necessary, and inspect status/bookmarks. Do not let `gh` and `jj` mutate one worktree concurrently.

GitHub lifecycle checklist

1. Fetch the intended source remote.
2. Develop unnamed mutable work until a review boundary exists.
3. Use change IDs for local continuity and bookmarks for server interfaces.
4. Print local and remote bookmark targets before every update.
5. Update every stack boundary after an early rewrite.
6. Fetch and compare before force-updating a rejected ref.

7. After merge or closure, fetch, compare, and clean names deliberately.

Part IX — Gerrit

Gerrit reviews commits as patch sets and uses a message footer such as `Change-Id: I...` to associate new commit IDs with an existing review. A `jj change` ID is different: it identifies a `jj` evolution and is not a Gerrit `Change-Id`. Preserve Gerrit's footer in descriptions when the server/plugin requires it, and do not infer review identity from the `jj change` ID.

`jj 0.45.1` includes the `gerrit` command family and `jj gerrit upload`. Gerrit upload support is specialised and should be checked against the installed help and server setup. Git transport still matters: a traditional upload ref is commonly `refs/for/main`, with topics and push options supplied according to Gerrit policy. The server-specific protocol and UI depend on the Gerrit installation; local `jj` testing can establish only the client-side transport and selection behaviour.

Realistic mutable-stack workflow

```
jj git clone ssh://gerrit.example/project repo
cd repo
jj git fetch --remote origin
jj new main@origin -m 'API: introduce quota'
# edit; snapshot
jj new -m 'API: wire quota'
# edit; snapshot
jj log -r '::~@ & mutable()'
jj gerrit upload --remote origin --remote-branch main
```

In `jj 0.45.1`, the `Gerrit` command accepts the options shown in this chapter; server-side acceptance remains a Gerrit policy question. The example expresses the workflow, not a universal server configuration. If using `Git push` directly, the equivalent transport is typically:

```
git push origin HEAD:refs/for/main
```

When review requests changes to the first change, `jj edit A`, edit or `jj squash`, then allow automatic descendant rebasing to update B. Re-upload the series. Gerrit associates each patch set by its Change-Id footer, while `jj` retains change IDs through rewrites. When a review lands, fetch and rebase the remaining mutable stack onto `main@origin`; abandon obsolete local changes only after inspecting the operation log.

GitHub and Gerrit in one repository

```
jj git remote add github git@github.com:me/project.git
jj git remote add gerrit ssh://gerrit.example/project
jj git fetch --remote github
jj git fetch --remote gerrit
jj log -r 'main@github | main@gerrit'
```

Use separate publication bookmarks such as `github/topic` and `gerrit/topic` where your naming policy permits, or at least use clear prefixes. Conditional configuration can select different push defaults by repository path. Never assume a remote named `origin` is safe: inspect the full remote URL before pushing.

Gerrit patch-stack lifecycle

Gerrit reviews a sequence of Git commits as patch sets. A new Git commit ID can still update an existing Gerrit review when its message contains the same Gerrit Change-Id footer and the server accepts it. `jj`'s change ID is generated and stored by `jj` for local evolution; it is not the Gerrit footer, is not a server lookup key, and must not be substituted for one.

```
jj identity:      change qq... → qq... → qq...
Git identity:     commit 111... → 222... → 333...
Gerrit identity:  Change-Id Iabc... (same review, new patch sets)
```

A stack:

```
main@origin -- A -- B -- C
              |   |   |
            I-A I-B I-C
```


A direct Git transport alternative is commonly `git push Gerrit HEAD:refs/for/main`, with server-specific push options in the ref suffix or options. That form bypasses jj's Gerrit-aware selection and should be used only when the repository's Gerrit policy is understood.

Review changes an early change

```
uploaded:
main -- A_1(I-A) -- B_1(I-B) -- C_1(I-C)

edit A locally:
main -- A_2(I-A) -- B_2(I-B) -- C_2(I-C)
      new Git IDs; same jj changes and Gerrit footers
```

[illegible]

The local descendants are automatically rebased over the new A. Their Git commit IDs change because their parent changed, even if their diffs are unchanged. Gerrit can recognise each new commit as a new patch set only if the footer/server rules match. jj's change ID is useful for selecting and following the local change; Gerrit's footer is the review association.

Rebase after trunk movement

```
jj git fetch --remote gerrit
jj log -r 'main@gerrit'
jj rebase -b 'roots(main@gerrit..@)' -o main@gerrit
jj log -r '::-@ & conflicts()'
jj resolve --list
jj gerrit upload --remote gerrit --remote-branch main --revision 'roots(main@gerrit..@)::'
```

If a change has already landed, the fetched trunk may make one stack member an ancestor of `main@gerrit`. Remove that landed change from the mutable stack by rebasing the remaining descendants onto the new trunk, not by blindly uploading the old series again. Inspect the graph and review IDs first.

Partial landing and remaining stack

```
before server submit:  main -- A -- B -- C
after A lands/fetch:   main'-- A(remote)
                        \
                        B' -- C'
```

The precise local shape depends on the server's submitted commit and fetch mapping. The invariant is that the remaining changes must be rebased onto the fetched trunk that contains the landed work. Their `jj` change IDs may persist through the local rebase; their Git commit IDs will change; Gerrit Change-Ids remain the server association if preserved.

Conflict in a reviewed stack

A trunk rebase can produce a stored conflict in `B'` while `C'` remains a descendant. Resolve the conflict in the local change, inspect every descendant, and re-upload the series. If the server rejects a patch set, separate the local graph repair from Gerrit-side review cleanup; changing a local description cannot delete a server review.

```
jj log -r '::-@ & conflicts()'
jj resolve --tool meld
jj diff -r 'roots(main@gerrit..@)::'
jj gerrit upload --dry-run --remote gerrit --remote-branch main
jj gerrit upload --remote gerrit --remote-branch main
```

Topics, reviewers, and options

Review metadata can be attached on upload with reviewer, CC, label, topic, hashtag, WIP/ready, notification, and custom option flags in the target CLI. Make these choices explicit in a publication alias or command; do not assume a topic is encoded by a local bookmark name. A local bookmark is optional for jj's stack, while Gerrit upload needs a selected commit set and remote branch.

Gerrit and GitHub from one repository

```
jj git remote add github git@github.com:me/project.git
jj git remote add gerrit ssh://gerrit.example/project
jj git fetch --remote github
jj git fetch --remote gerrit
jj log -r 'main@github | main@gerrit'
jj bookmark create github/pr-cache -r @
jj git push --remote github --bookmark github/pr-cache
jj gerrit upload --remote gerrit --remote-branch main --revision 'roots(main@gerrit..@)::'
```

The same local mutable stack can be published through two distinct mechanisms, but publication boundaries should be explicit. A bookmark intended for GitHub does not control Gerrit upload, and a Gerrit Change-Id footer does not make a GitHub branch a review. Use remote-qualified revsets and explicit remote options.

Local and server boundaries

Behaviour	Where it is determined
jj change IDs survive ordinary local rewrites	jj's local evolution model.
Commit IDs change when parents, content, or metadata change	Git object identity and jj's rewrite operation.
Git remote/ref fetch and push mechanics	jj's Git backend and the Git server.
Gerrit footer remains in a rewritten description if the user preserves it	Local commit-message content; footer acceptance is server policy.
Gerrit review association, patch-set UI, permissions, submit, and topics	The target Gerrit installation and its project policy.

This boundary matters: a bare Git remote can transport commits and refs, but it does not implement Gerrit's review database.

Gerrit protocol map: local commits and review refs

Gerrit workflows combine three identities: the Git commit ID uploaded over Git transport, jj's change ID used for local evolution, and Gerrit's `Change-Id` footer used by the server to associate patch sets. A fourth name may be involved: the local bookmark or remote bookmark used to reach the commit. Keep all four visible when diagnosing a review.

Identity	Where it lives	What survives a normal rewrite?	What it is for
Commit ID	Git object graph and server patch set	No; new tree/parent/metadata gives a new ID.	Exact object, build, fetch, server evidence.
jj change ID	jj change evolution metadata/rendering	Normally yes, as a successor relation.	Local logical change across amendments.
Gerrit Change-Id	Commit message footer	Yes if description is preserved.	Server review identity and patch-set association.
Bookmark/ref	Local/remote Git ref namespaces	Only if explicitly moved/exported.	Reachability and publication.

A review command should print all relevant fields before upload:

```
jj log -r '@:: & mutable()' -T '  
change_id.short() ++ "\t" ++ commit_id ++ "\t" ++  
description.first_line() ++ "\n"  
'  
git log -n 1 --format='%H%n%B' # inspect Gerrit footer through Git when needed
```

Use the native Git command only as a cross-check of the exported commit object. The jj description is the source of the commit message in the jj model; a colocated Git view may lag until export/import or normal backend synchronisation occurs.

Upload a stack deliberately

A stack upload should define its base and its upper boundary. The explicit query `@:: & mutable()` selects the mutable descendant closure of the current workspace commit, but can include a temporary child or omit a sibling review. For a repeatable review, select the exact stack by bookmark, remote trunk, or a named revset alias and print the result.

```
jj log -r 'roots(main@gerrit..@)::' --no-graph \
-T 'commit_id.short() ++ " " ++ description.first_line() ++ "\n"'
jj gerrit upload --dry-run --remote gerrit \
--remote-branch main -r 'roots(main@gerrit..@)::'
```

Do not assume that a range's root is the review base in a graph containing merges. If the stack forks, take the roots and heads explicitly and decide whether Gerrit should receive one linear series or several independent changes. Gerrit's server interpretation of multiple commits and dependencies is authoritative; `jj` can prepare the graph but cannot change server review policy.

Review feedback on an ancestor

When an early change is amended, all descendants are candidates for automatic rebasing. This is precisely the operation that makes mutable stacks attractive, but it also means every later review commit may receive a new Git ID. Preserve each descendant's description and Gerrit footer, inspect conflicts, run tests on the final head, and re-upload the series. If only one review is intended to change, do not accidentally upload an unrelated sibling.

```
jj edit 'change_id(qxy...)'
# edit and test
jj describe -m 'Core: reject invalid limits'

Change-Id: I1234567890abcdef'
jj log -r 'change_id(qxy...)::'
jj log -r 'change_id(qxy...):: & conflicts()'
jj gerrit upload --remote gerrit --remote-branch main -r 'change_id(qxy...)::'
```

If a descendant conflict is not yet understood, stop before upload. The conflict is stored and inspectable, but server-side review cannot generally represent an

unresolved `jj` conflict as a meaningful diff. Resolve at the lowest commit that establishes the intended API, then retest descendants.

Topics, options, and server state

Gerrit topics and upload options are server metadata. A topic can group a series without changing `jj` graph topology. Reviewers, CCs, labels, hashtags, notification policy, WIP/ready/private state, and custom push options should be treated as part of the upload request, not as commit identity. Record them in a script or checked-in review note if the review must be reproducible.

In 0.45.1, `gerrit.default-remote`, `gerrit.default-remote-branch`, and `gerrit.review-url` can provide repository-specific defaults. The portable form passes `remote` and `branch` explicitly. This book does not claim that a particular Gerrit server enables every upload option; server plugins, permissions, and version determine acceptance.

After a change lands

Landing changes on Gerrit changes the remote branch, not the local operation DAG. Fetch, inspect the new remote bookmark, identify the corresponding local logical change, and rebase remaining changes. If the landed commit is not equivalent to the local one—for example, the server applied a submit strategy that changed the resulting commit—use diff/content and message evidence rather than assuming commit IDs match.

```
jj git fetch --remote gerrit
jj log -r 'main@gerrit'
jj log -r 'remote_bookmarks() & description(substring:"Core")'
jj rebase -s 'roots(remaining_stack)' -d main@gerrit
```

Once a review is abandoned or replaced, keep a temporary bookmark until the replacement is verified. Then delete local publication names that no longer represent active server refs and use `jj git push --deleted` only when the deletion is intentional and permitted.

Gerrit stack lifecycle: three identities and two review iterations

Gerrit is unusually well matched to `jj`'s mutable history, but only if the identities are kept separate. A local change ID says which `jj` evolution is being edited. A commit ID says which exact tree, parents, and metadata are being uploaded. A Gerrit Change-Id: I... footer says which server review should receive a new patch set. The local bookmark or remote bookmark answers how the object is reached. None of these four identifiers is a substitute for another.

<code>jj graph</code>	<code>exported commit</code>	<code>Gerrit review</code>
A (base)	1a2...	-
B (API)	2b3... + Change-Id: Iapi	API, patch set 1
C (tests)	3c4... + Change-Id: Itest	tests, patch set 1
D (metrics)	4d5... + Change-Id: Imet	metrics, patch set 1

`jj` change IDs remain stable through ordinary rewrites;
commit IDs change; Gerrit associates the new object by footer.

An existing Change-Id footer remains the server-facing review identity: upload uses it to update the corresponding Gerrit change with a new patch set. If the selected commit has no footer, `jj 0.45.1` generates one from the `jj` change ID for the commit object it uploads. That generated trailer is added to the uploaded object only; it is not added to the local commit in the working repository. The uploaded commit ID can therefore differ from the local commit ID, and fetching the result can expose divergence. If you want the footer and resulting commit ID to exist locally before upload, configure `templates.commit_trailers` with `format_gerrit_change_id_trailer(self)`. This still does not make a `jj` change ID equal to a Gerrit review identity: it derives a footer for transport. Server association, permissions, and patch-set policy remain Gerrit-dependent.


```
description += "\n"
```

A stack may be uploaded through the specialised command:

```
jj gerrit upload --dry-run --remote gerrit \
  --remote-branch main --revision 'roots(main@gerrit..@)::'
jj gerrit upload --remote gerrit \
  --remote-branch main --revision 'roots(main@gerrit..@)::'
```

The dry-run boundary is valuable even when the server does not provide a full preview. It confirms the selected local commits and exposes command-level assumptions before an external side effect. The server decides whether the ref is `refs/for/main`, which commit messages are accepted, and how reviewers, topics, labels, WIP/ready, notifications, and validation options are interpreted. jj's local responsibility is selecting and exporting the commit set.

Review changes an early change

```
before review: main--B--C--D(@)
                B, C, D uploaded

review says:    B needs a stronger invariant
local edit:    main--B'--C'--D'(@)
                same jj changes, new commit IDs,
                same Gerrit footers if preserved
```

Edit the logical change by change ID, not by an old commit prefix:

[illegible]

Because C and D have B as an ancestor, automatic descendant rebasing uses B' as their new parent. Their diffs are replayed in the new context. If the replay is clean, C' and D' have new commit IDs but keep their jj change IDs; if context no

longer applies, one or more descendants become conflicted commits. The correct next action is not a Git-style rebase continuation. Inspect the stored conflict set, work on the affected change, resolve it, and then inspect the whole stack.

```
jj log -r '::-@ & conflicts()'  
jj diff -r 'roots(main@gerrit..@)::'  
jj resolve  
jj log -G -r 'roots(main@gerrit..@)::'
```

After the local stack is coherent, re-upload the same commit set. Gerrit should associate each commit with its footer when the server's rules permit it, generating new patch sets rather than unrelated reviews. That association is a Gerrit server fact; local validation can only prove that the footer survived and that the exported commit message contains the expected bytes.

A lower change lands while the upper stack remains

Suppose B lands first, perhaps after review and submit, while C and D remain under review:

```
server main:  main--B  
local before: main@gerrit--B--C--D(@)  
after fetch:  main@gerrit--B'    # server's landed object may differ  
local work:   C--D still based on old imported boundary
```

Fetch to import the server's new main and determine whether the landed change is identical, an amended equivalent, or a different object:

```
jj git fetch --remote gerrit  
jj log -G -r 'main@gerrit | roots(main@gerrit..@)::'  
jj diff --from main@gerrit --to 'roots(main@gerrit..@)::'
```

Rebase only the remaining mutable changes onto the new remote boundary. Do not rebase the remote bookmark itself and do not assume a commit ID equality is required for semantic equivalence:

```
jj rebase -b 'roots(main@gerrit..@)' -o main@gerrit  
jj log -r '::-@ & conflicts()'
```

```
jj gerrit upload --remote gerrit --remote-branch main \
--revision 'roots(main@gerrit..@)::'
```

The remaining commits may receive new IDs again. Their Gerrit footers should remain constant if descriptions are preserved, so the server can update the existing reviews. If the server's submit strategy created a merge or changed the effective base, inspect the resulting ancestry and choose the new remote bookmark as the authoritative base.

Topics, reviewers, and upload options

Gerrit upload options are a second channel of review metadata. A topic is not a bookmark name. A reviewer flag is not a commit author. A WIP/ready flag is not a jj mutability predicate. Keep these channels explicit:

Channel	Local jj meaning	Gerrit meaning
jj change ID	Evolution identity used to find a mutable change.	No direct server interpretation.
Commit message footer	Text in the commit description.	Review association when server policy recognises <code>Change-Id</code> .
Bookmark	Optional local publication name.	Git ref/name used to transfer a commit set.
<code>--topic</code>	Upload option.	Requests server-side topic grouping; Gerrit permissions and server policy determine whether the request is accepted.
<code>--reviewer/--cc</code>	Upload option.	Server review metadata, permissions permitting.
<code>--label/--hashtag</code>	Upload option.	Server label/hashtag request, subject to validation.

Pass explicit remote and branch arguments in portable scripts. The jj 0.45.1 configuration surface documents Gerrit defaults such as `gerrit.default-remote`, `gerrit.default-remote-branch`, and `gerrit.review-url`. A default reduces typing but increases the importance of inspecting effective configuration before a

public upload.

Conflict, abandonment, and replacement

If a conflict appears after trunk movement, resolve the local graph first and re-upload only after the stack's diffs are understood. If a review is abandoned on the server, that does not abandon the local `jj` change. The local change may be retained, renamed, duplicated into a new review with a new footer, or abandoned locally after an operation checkpoint. Choosing among these is a review-lifecycle decision.

```
# Preserve evidence before replacing a review
jj op log -n 10
jj evolog -r 'change_id(API_CHANGE)'
jj bookmark create rescue-gerrit -r 'change_id(API_CHANGE)'
# either amend the same footer and re-upload, or intentionally create a
# new change/message and new publication identity
jj duplicate -r 'change_id(API_CHANGE)' -o main@gerrit
```

Do not use `jj undo` to claim that an already uploaded patch set disappeared. Undo changes local repository state; it cannot retract a Gerrit review. Use Gerrit's abandon/delete/restore controls according to server policy and then fetch to observe the resulting refs. Local and server cleanup are separate operations.

Evidence boundary for Gerrit

Claim	Authority
<code>jj</code> change ID and commit ID have distinct roles.	<code>jj</code> 's local change and commit model.
Automatic descendant rebasing changes descendant commit IDs.	<code>jj</code> 's rewrite semantics.
Commit message footer survives if the user preserves it.	Local commit-message content.
Git refs and fetch/push mechanics.	<code>jj</code> 's Git backend and the Git server.
<code>refs/for/main</code> review creation and patch-set association.	Gerrit's upload protocol and server documentation.
Reviewer/label/topic permissions and submit strategy.	The target Gerrit installation and project policy.

Part X — Conflicts and merges

A `jj` conflict is a first-class, serialisable value in a commit. A rewrite can produce a conflicted successor; descendants can carry that conflict forward. The conflict may be materialised as marker files in the working copy, but the markers are a rendering of stored sides, not the authoritative state.

```
base -- A -- C
 \       \
  \- B -- merge/rebase result with sides A and B

jj log -r 'conflicts()'
jj resolve --tool meld
jj diff
jj describe -m 'resolved: quota conflict'
```

Use `jj resolve` with the configured merge editor/tool, or edit the materialised markers and let the next snapshot record the result. Multi-sided conflicts can have more than two input terms; do not assume a two-column Git index. Configure marker style and merge tools explicitly, and check the result with `jj diff` plus `jj log -r 'conflicts()'`.

Because the conflict is stored in the commit, you can continue unrelated work, resolve later, rebase a conflicted stack, split a conflict, or restore selected paths. A merge commit is an intentional multiple-parent commit; it is not the same as a conflict value. Use repeated rebase destinations or `jj new A B` where the command's syntax creates a merge, then resolve its content.

Conflict lifecycle laboratory

A conflict is a structured value in a commit tree. It has a base and one or more sides for a path; the working copy renders that value as conflict markers so a human or tool can produce a resolved file. The marker file is not the complete representation: a multi-sided conflict can encode more alternatives than a two-way marker layout suggests.

Creation by rebase

```
base -- A: change line 10 to "red"
\
  B: change line 10 to "blue"

before moving B onto A:
base -- A
\
  B

after jj rebase -r B -o A:
base -- A -- B'(conflict)
      |
      \- B's change is preserved as a stored conflict value
```

The command has completed: B' exists and can be selected by its successor/evolution relationship. The conflict is visible to `conflicts()`, and a current workspace at B' materialises markers. A descendant C is rebased onto B' and may itself be conflicted or may carry the conflict value through untouched paths.

```
jj rebase -r B -o A
jj log -r '::-@ & conflicts()'
jj resolve --list
jj diff
jj resolve --tool meld
jj log -r '::-@ & conflicts()'
```

Resolution rewrites B' into B''. C receives a successor C' because its parent changed from B' to B'', even if C's own file edits were not involved. The operation log contains both the conflict-producing rewrite and the resolving rewrite. If resolution was wrong, undoing the resolution returns to the stored conflict; restoring the earlier operation returns to the prior graph.

Manual markers versus a merge editor

There are two common resolution paths. `jj resolve --tool NAME` invokes the configured merge tool with the stored sides. Alternatively, edit the materialised file, remove markers, and let `jj` snapshot the working-copy commit. The latter is convenient for a simple file; the former is preferable when the conflict has binary files, directory/file changes, or a tool-specific merge policy.

```
# Discover paths before selecting a subset
jj resolve --list -r @
jj resolve --tool :ours -r @ -- 'root-file:"src/quota.rs"'
jj resolve --tool :theirs -r @ -- 'root-file:"docs/design.md"'
```

The built-in `:ours` and `:theirs` choices refer to applicable conflict sides, not universally to “my branch” and “their branch” in a Git branch sense. In a multi-sided conflict, inspect the materialisation and command help before choosing a side. A side choice is a semantic content decision, not a conflict acknowledgement.

Conflict propagation through a stack

```
A -- B -- C -- D
    ▲
    B' conflicted after B is rebased

A -- B'(conflict) -- C'(conflict?) -- D'
                        ▲
                    descendants preserve/reapply diffs over B'
```

Propagation is path-sensitive. `C'` may not be marked conflicted if its diff does not overlap the unresolved path, but its parent tree still contains a conflict value. A stack can therefore contain a conflicted ancestor even when a descendant's own `conflict()` is false. Query the ancestry as well as the individual commit:

```
jj log -r 'conflicts()'
jj log -r 'conflicts()::'
jj log -r '::-@ & conflicts()'
jj diff -r 'roots::-@ & conflicts())..@'
```

Do not use the presence of a marker file as the repository-level conflict test. The filesystem may be incomplete, the current workspace may point elsewhere, or markers may have been edited without a snapshot. The stored commit and `conflicts()` are the authoritative query.

Merge commits and conflict values

A merge commit has multiple parents; a conflict value is a tree entry whose content has multiple terms. A merge can be conflict-free, and a single-parent rebase can produce a conflicted commit. Conversely, resolving a conflict does not remove a merge commit's multiple-parent topology. Use `merges()` and `conflicts()` as separate predicates.

```
jj new A B -m 'Merge API and documentation'
jj log -r 'merges()'
jj log -r 'merges() & conflicts()'
```

The merge's diff is calculated against the automatic merge of its parents. If the parents disagree, resolving the conflicting file contents does not make the graph linear. Preserve the merge when it expresses integration history; use `rebase/parallelize` when the graph should express a stack instead.

Rewriting while conflicted

Because the conflict is stored, ordinary graph operations remain available. You can rebase a conflicted stack onto a new trunk, split the conflicted commit, restore one path from a known-good commit, or continue unrelated work in another workspace. Each operation may create a new conflict state; inspect the path-level result.

```
jj edit 'conflicts() & heads(:,:,@)'
jj restore --from main@origin --into @ -- 'root-file:"Cargo.toml"'
jj split --interactive -r @
jj rebase -s @ -o main@origin
jj log -r '::-@ & conflicts()'
```

There is no mandatory conflict queue. The useful discipline is to make conflict state visible in the daily log, keep the operation that produced it, and resolve at a

deliberate graph boundary rather than accidentally committing marker text as ordinary content.

Conflict checklist

1. Identify the commit and paths with `resolve --list` and `conflicts()`.
2. Determine whether the conflict is in an ancestor and therefore affects the whole stack.
3. Inspect the graph and the automatic merge base; do not infer sides from bookmark names.
4. Choose tool, side, or manual edit; record the policy when a review is involved.
5. Snapshot/resolve, then inspect diff, conflict predicate, descendants, and operation.
6. Only publish after the selected publication bookmark points to the resolved successor.

Conflict lifecycle laboratory: from graph result to resolved tree

A `jj` conflict is a value in a commit, not a paused rebase session. That difference changes the work sequence. A rewrite can finish by producing a conflicted successor; descendants can be rebased over that value; unrelated changes can be inspected or even edited; and resolution can happen later. There is no hidden index state that must be cleared before the next `jj` command can make progress.

```
base--A--B--C(@)          clean stack
  \      B changes same line
rewrite: base--A'--B'--C'(@) B' carries a conflict
                               C' may carry propagated conflict
```

The conflict has graph consequences and tree consequences. The graph stores the conflict as part of the commit's tree value. A working copy materialises conflict sides using the configured marker style so an editor can repair files. The materialised files are only the workspace view; the resolved content becomes a new snapshot and therefore a new successor commit. A conflict marker in a file is not, by itself, proof that the commit has a conflict object, and a conflicted commit need not currently be

checked out.

Creation and propagation

Conflicts arise when a diff cannot be applied cleanly to its new parent or when multiple parent trees cannot be merged without choosing content. Common producers are rebase, squash, restore, split, and an explicit merge. The producer operation can complete even when the result is conflicted. Automatic descendant rebasing then attempts to apply each descendant diff to the changed context. A descendant that does not touch the affected region may remain clean; one that does may carry a conflict of its own.

```
jj log -G -r '::-@'  
jj log -r 'conflicts() & ::@'  
jj log -r 'conflicts() & @:: & mutable()'  
jj show -r 'conflicts() & ::@' --summary
```

Query the graph before opening an editor. If B' is conflicted and C' is clean, editing C' first can still be useful for reviewing its independent diff, but a later repair to B' may rebase C' again. If the conflict is at an ancestor, resolving it can change every descendant commit ID even when their content remains semantically identical. Use change IDs to follow the logical changes and operation/evolution logs to account for object churn.

Materialisation and sides

For a two-sided conflict, the workspace contains a marker representation with a base and the competing sides according to the configured conflict marker style. A multi-sided conflict can contain more than two alternatives because repeated rebases and merges preserve more than one side. Treat markers as an input format to a merge editor, not a syntax to resolve by deleting lines at random.

What the materialised file looks like

In jj 0.45.1 the default marker style is `diff`. The following is an illustrative two-sided file; the abbreviated IDs and descriptions are labels, not part of the conflict algebra:

```
before:
apple
grape
orange

after materialising the conflict:
<<<<<< conflict 1 of 1
%%%%%%%% diff from: BASE1234 "merge base"
\\\\\\\\\\         to: SIDEA123 "change A"
apple
-grape
+grapefruit
orange
+++++++ SIDEB456 "change B"
APPLE
GRAPE
ORANGE
>>>>>> conflict 1 of 1 ends
```

Read this as an instruction for constructing a result, not as two independent files separated by a line of equals signs. The `++++++` section is a complete snapshot of one side. The `%%%%%%/\\\\\\\\` section describes a diff from the base to another side: context lines begin with a space, removed lines with `-`, and added lines with `+`. Applying that diff to the snapshot produces the other side. In this example the useful resolution is to combine the intent—`APPLE`, `GRAPEFRUIT`, `ORANGE`—and remove the markers before `jj` snapshots the result.

In that example, the one-space rule applies only to the diff section between %%%%%% and the next side marker: the literal leading spaces on `apple` and `orange` are context prefixes. The uppercase `APPLE`, `GRAPE`, and `ORANGE` lines belong to the complete snapshot section and intentionally have no diff prefix.

A multi-sided conflict uses the same structure but has one snapshot section and more than one diff section. Conceptually, for a base B and sides S1, S2, and S3, the file has the shape:

```
<<<<<< conflict 1 of 1
+++++++ S1 "one snapshot side"
...complete contents of S1...
%%%%%%%%% diff from: B "common base"
\\\\\\\\\\      to: S2 "second side"
...diff B → S2...
```

```

%%%%%%%% diff from: B "common base"
\\\\\\\\ to: S3 "third side"
...diff B → S3...
>>>>>> conflict 1 of 1 ends

```

The exact commit labels identify the stored terms; the marker count identifies the conflict region within the file. If a file contains two independent conflict regions, their headers are `conflict 1 of 2` and `conflict 2 of 2`. A conflict can therefore have both multiple regions and multiple sides. Do not infer the number of parents from the number of visible marker blocks: the conflict value records the terms, while the file is only its textual projection.

The three stable marker styles and one experimental value

Style	File representation	Multi-sided behaviour	Use it when
<code>diff</code> (default)	One complete snapshot plus diffs from the base to the other sides.	Supports arbitrarily many sides; additional sides appear as additional diff sections.	You want a compact, reviewable representation that makes each side's change explicit.
<code>snapshot</code>	A complete snapshot section for each side and the base.	Supports multiple sides directly, at the cost of repeated unchanged content.	You need to inspect the full contents of every term or use a tool that expects snapshot blocks.
<code>git</code>	Git diff3-style <code><<<<<<<</code> , <code> </code> , <code>=====</code> , and <code>>>>>>></code> sections.	Only supports two-sided conflicts; jj falls back to a snapshot-like representation for more than two sides.	A tool requires Git-style diff3 markers and the conflict is genuinely two-sided.
<code>diff-experimental</code>	Experimental alternative to the default diff-based materialisation.	Accepted by the jj 0.45.1 configuration schema and capable of materialising conflicts, but its textual layout is not treated as a stable book-level interface.	Only when deliberately testing the target release and willing to track changes in later jj versions.

These are alternatives; set exactly one value in an effective `[ui]` configuration table:

```
[ui]
# Choose one. jj's default: one snapshot plus one or more diffs
conflict-marker-style = "diff"

# Or: complete snapshots for base and every side
conflict-marker-style = "snapshot"

# Or: Git diff3 compatibility; not a general multi-sided format
conflict-marker-style = "git"
```

The 0.45.1 schema also accepts `conflict-marker-style = "diff-experimental"`. It is intentionally not given a fixed marker example here: the value is experimental, and its rendering is subject to change. The stable choices for a book, merge-tool configuration, or interchange script are the three styles described above. Inspect the target with `jj util config-schema` and run a disposable conflict fixture before relying on the experimental style.

For comparison, the two-sided part of snapshot style is represented as complete sections rather than a diff:

```
<<<<<< conflict 1 of 1
+++++++ SIDEA123 "change A"
apple
grapefruit
orange
----- BASE1234 "merge base"
apple
grape
orange
+++++++ SIDEB456 "change B"
APPLE
GRAPE
ORANGE
>>>>>> conflict 1 of 1 ends
```

A three-sided snapshot contains one `+++++++` section for each side plus the `-----` base section. It is easy to read but can repeat a large amount of unchanged text; the default diff style represents the same alternatives more compactly.

In Git style, the conceptual two-sided form is:

```

<<<<<< SIDEA123 "change A"
apple
grapefruit
orange
||||||| BASE1234 "merge base"
apple
grape
orange
=====
APPLE
GRAPE
ORANGE
>>>>>> SIDEB456 "change B"

```

Changing `ui.conflict-marker-style` changes only this materialised representation. It does not change the stored conflict value or make a conflict two-sided. Marker lines are normally seven characters long, but `jj` can lengthen them when file content could otherwise be mistaken for a marker. Files without a terminating newline receive special treatment so that every marker begins on its own line; the closing marker may consequently have no terminating newline. These details matter to scripts and parsers, so use `jj`'s configured merge tooling rather than matching marker text with an ad-hoc regular expression. The layouts and fallback behaviour in this section follow the official `jj` conflict-marker documentation; the abbreviated examples are original and use illustrative labels.

```

jj log -r @ -T 'conflict ++ \' sides=\' ++ conflicted_files.map(|f| f.conflict_side_count()).join
jj resolve --list
jj resolve --tool :builtin

```

In `jj 0.45.1`, `jj resolve --list` lists the conflicts, `jj resolve --tool NAME` selects a resolver, and the builtin tools include `:ours` and `:theirs`. The stable invariant is that a resolver reads the stored conflict and writes a chosen tree. If the resolver leaves a conflict, the resulting commit remains conflicted. Check with `jj log -r @ -T 'conflict'` and inspect the files rather than assuming the tool's exit status means all conflict objects disappeared.

Manual resolution is a snapshot boundary

A safe manual repair has four checkpoints:

1. Save the current operation ID and the conflicted change's change ID.
2. Inspect the conflict sides and the intended semantic result.
3. Edit files, run focused tests, and snapshot deliberately.
4. Verify that the change is no longer in `conflicts()` and review every descendant.

```
jj op log -n 5
jj log -r '@' -T 'change_id ++ \" \" ++ commit_id ++ \" \" ++ conflict ++ \"\\n\\n\"'
# edit files and test
jj status
jj log -r '@' -T 'change_id ++ \" \" ++ commit_id ++ \" \" ++ conflict ++ \"\\n\\n\"'
jj log -r '::~@ & conflicts()'
jj diff -r @
```

If a test or editor changes files after the initial snapshot, the next `jj` command may record them. If reproducibility matters, use an explicit snapshot command or run the command that owns the snapshot boundary, then record the operation. Avoid `--ignore-working-copy` for the verification that is supposed to observe the repair unless you first understand which recorded tree it will read.

Rewrite, squash, split, restore

When squashing into a conflicted or conflict-adjacent change, ask whether the incoming diff is meant to resolve the conflict or merely to add more content. A fileset can restrict the paths moved, but it cannot choose among conflict sides. When splitting a conflicted commit, the conflict may be divided between the resulting changes or remain in both if each diff depends on the unresolved context. Afterward inspect each commit's conflict predicate.

Restore can be used to replace a conflicted path with a known tree version, but it is not a conflict resolver in the semantic sense. It copies content from a source or reverses a selected change. A subsequent snapshot may clear the conflict if all conflict objects are represented by the edited tree; otherwise the conflict remains. Keep a temporary bookmark before a broad restore and use `jj diff --from/--to` to confirm the chosen side.

```
jj bookmark create before-resolve -r @
jj restore --from main@origin --into @ -- 'root-prefix-glob:"src"'
jj log -r '::~@ & conflicts()'
jj diff --from before-resolve --to @ -- 'root-prefix-glob:"src"'
```

Merge commits versus conflict results

A merge commit has multiple parents. It can be clean or conflicted. A conflicted single-parent successor created by a rebase is not automatically a merge commit; it records an unresolved diff application against one new parent. Conversely, an explicit `jj new P Q` creates a merge shape even if the trees happen to agree. Use parent inspection, not conflict status, to distinguish these cases.

```
jj new A B -m 'Merge independent lines'
jj log -r @ -T 'parents.map(|p| p.commit_id()).join("\n ") ++ "\n\n"'
jj log -r 'merges()'
jj log -r 'conflicts()'
```

A merge can be a deliberate review or integration boundary. Keep it only when the project wants that ancestry; otherwise rebase or simplify parents may produce a more linear diff story. A conflict may be resolved in a merge commit by editing the working copy, but the resulting tree must be reviewed as a merge result against both parents.

Validation matrix

State	Graph check	Filesystem check	Next safe action
Stored conflict, not checked out	<code>conflicts()</code> & REV selects it.	No marker is necessarily in current files.	<code>jj edit REV</code> or inspect with <code>show</code> .
Stored conflict, checked out	<code>conflict</code> template is true.	Markers/materialised sides are present.	Resolve or create a new child and leave it for later.
Clean descendant over conflicted ancestor	Descendant absent from <code>conflicts()</code> .	Depends on current workspace.	Review whether its diff is still semantically valid.
Resolution snapshot	Target removed from conflict set.	Resolved file is materialised.	Run focused tests and inspect descendants/evolution.
Unresolved multi-sided value	Conflict side count remains greater than one.	Marker style may show several blocks.	Use a merge tool or resolve sides explicitly.

The most dangerous conflict mistake is treating a clean status as a semantic proof. A descendant can be syntactically clean while its assumptions were invalidated by the ancestor rewrite. Review the diff and tests, not just the marker predicate.

Part XI — Operation history, change evolution, undo, and recovery

Operation history: repository-wide transformations

`jj` has two graphs:

```
commit DAG:    root - A - B - C
                \ rewrite
operation DAG:  op0 - op1 - op2 - op3
                \ concurrent op
```

The commit DAG describes source history. The operation DAG describes repository transformations: snapshots, rewrites, imports, ref moves, and recovery. This is richer than a Git reflog, which is primarily a per-ref movement record.

```
jj op log
jj op show OPERATION
jj op diff --from OP1 --to OP2
jj op restore OPERATION
jj op abandon OPERATION
jj op revert OPERATION
jj undo
jj redo
```

`jj undo` reverses the latest operation; `redo` reapplies the most recently undone operation when available. `op restore` loads a repository state from the operation DAG. `--at-operation OP` inspects an old state without snapshotting the current filesystem. Operations created with `--no-integrate-operation` are not merged into the current operation head until explicitly integrated.

Views and IDs

An operation ID identifies a repository transformation in this clone. Loading `jj --at-operation OP log` asks `jj` to evaluate the commit graph and refs as they were at that operation, while ignoring today's working-copy snapshot. It does not roll back the filesystem. A historical view is therefore an observation tool, not a checkout command.

```
jj op log -n 20
jj op show OP
jj op diff --from OP1 --to OP2
jj --at-operation OP log -r 'all()'
jj --at-operation OP status
```

Record operation IDs before a risky rewrite, not only after it. The ID printed by `op log` may be a short unique prefix; retain the full ID in an incident note when the repository is important. A later operation does not necessarily erase the earlier record, but garbage collection or operation abandonment can reduce recoverability.

Undo, restore, revert, and abandon are different

Operation	Target	Result	Use
<code>jj undo</code>	Most recent integrated operation.	Creates a reverse operation.	Immediate, unambiguous mistake.
<code>jj redo</code>	Available undone operation.	Reapplies it when redo state exists.	Recover from an over-hasty undo.
<code>jj op restore OP</code>	Named historical repository state.	Creates a new state matching selected operation data.	Return to a known graph/ref state.
<code>jj op revert OP</code>	Effect of a selected operation.	Creates a reversing operation while remaining in operation history.	Reverse a named operation after later history exists.
<code>jj op abandon OP</code>	Operation record.	Removes/abandons an operation according to local semantics.	Maintenance after recovery, not commit deletion.
<code>jj abandon REV</code>	Selected commit.	Hides/abandons a change and rebases or updates dependants.	Declare graph work no longer wanted.

In particular, `jj op revert` is not `jj revert`. The former reverses repository-state effects; the latter creates an inverse content change. Likewise, `jj op restore` does not promise to restore an external push or today's files. If the mistake crossed a network boundary, repair that server separately and then fetch to make the local observation agree.

Evolution log: following one change through time

`jj evolog` is the command to use when the question is “what happened to this logical change?” It follows one `jj` change ID through the successive commit objects that represented it. That is a different question from “what did the repository-wide operation do?” (`jj op show`) and from “where did a bookmark point?” (`jj bookmark list`). A Git commit ID identifies one object; an evolution entry connects that object to its predecessor versions of the same logical change.

logical change X

X0 --describe/rewrite-->	X1 --ancestor rebase-->	X2 --resolve-->	X3
commit c0	commit c1	commit c2	
operation o0	operation o1	operation o2	

The arrows are not ordinary parent edges in the commit DAG. A successor may have the same parent, a new parent, a different tree, or different metadata; its change ID normally remains the logical identity. Descendants can receive their own successors because their parent commit IDs changed. A bookmark normally follows the rewritten local graph target, while an already fetched remote bookmark remains the local observation of the server until the next import.

Evolution is therefore not a second commit DAG and not a ref log. It is a change-centric history layered over commit objects. The operation DAG supplies the wider causal context: it can show that one operation rewrote X, its descendants, a bookmark, and a workspace together. Evolution tells you which of those commit objects belong to X.

A reusable fixture: record identities before reasoning

Use a disposable repository with a named logical change, several amendments, and a descendant. The important discipline is to record full IDs, not the short graph labels used in diagrams:

```
mkdir -p /tmp/jj-evolog-lab
cd /tmp/jj-evolog-lab
jj git init --colocate
printf 'one\n' > api.txt
jj describe -m 'API: initial contract'
change=$(jj log -r @ --no-graph -T 'change_id ++ "\n"')
c0=$(jj log -r @ --no-graph -T 'commit_id ++ "\n"')

printf 'two\n' > api.txt
jj describe -m 'API: two contract'
c1=$(jj log -r @ --no-graph -T 'commit_id ++ "\n"')

printf 'three\n' > api.txt
jj describe -m 'API: three contract'
c2=$(jj log -r @ --no-graph -T 'commit_id ++ "\n"')
```

```

jj new -m 'Tests: cover API'
printf 'test\n' > test.txt
jj status
child=$(jj log -r @ --no-graph -T 'change_id ++ "\n"')

printf 'logical change: %s\n' "$change"
printf 'initial/current/descendant: %s %s %s\n' "$c0" "$c2" "$child"
jj evolog -r "$change" -G -T \
'commit.commit_id() ++ "\t" ++
commit.change_id() ++ "\t" ++
operation.id() ++ "\t" ++
commit.description().first_line() ++ "\n"'

```

The last command is deliberately a tabular evidence export. Its template runs in the `CommitEvolutionEntry` context, so the commit and operation are accessed as `commit.commit_id()` and `operation.id()`, not as the ordinary log keywords `commit_id` and `change_id`. Store the output when correlating a review or incident. Short IDs are useful for a screen; the full commit ID, change ID, and operation ID are the durable local evidence.

Stage	Commit-DAG state	Evolution evidence	Ref/operation consequence
Initial	root -> X0	c0 and change are recorded before the first rewrite.	The working-copy commit points at X0; no publication bookmark is required.
Amendments	root -> X1, then root -> X2	c1 and c2 differ, while change is normally unchanged.	Each <code>describe</code> creates an operation and hides the preceding commit from ordinary visible output.
Descendant	root -> X2 -> T	T has its own change ID and can be followed independently.	A bookmark at T names a publication boundary; it does not replace X's evolution history.
Ancestor rewrite	root -> X3 -> T'	evolog -r "\$change" finds X0...X3; evolog -r "\$child" can find T and T'.	T' may have a new commit ID even when its own intended diff is unchanged, because its parent changed.

This table separates four identifiers that are often collapsed in review conversations:

- **change ID**: the logical identity followed by `evolog`;
- **commit ID**: one exact tree/parent/metadata object, suitable for matching a CI result or exported Git object;
- **bookmark**: a local publication/navigation name that may move to a successor;
- **operation ID**: the repository-wide transformation that created or rewrote the commit.

Reading order, selection, and output modes

The basic command follows the change selected by the revision set. With no `-r`, the default is `@`; in scripts, select the logical change explicitly and save the full value first:

```
change=$(jj log -r @ --no-graph -T 'change_id ++ "\n"')
jj evolog -r "$change"
jj evolog -r "$change" --reversed
jj evolog -r "$change" -n 4
jj evolog -r "$change" -G
```

Normal output presents the current version first and walks backwards through predecessors. `--reversed` presents the oldest entry first and the current version last, which is usually better for a narrative of a review. `-n/--limit` limits the entries after `jj` has selected and topologically ordered the evolution; with `--reversed`, the limit therefore describes the beginning of the reversed display, not necessarily the newest versions. `-G/--no-graph` removes graph decoration but does not change which entries are selected.

Use a template when the question is identity or correlation rather than prose:

```
jj evolog -G -r "$change" -T \
'commit.commit_id().short(12) ++ " " ++
commit.change_id().short(12) ++ " " ++
operation.id().short(12) ++ " " ++
commit.description().first_line() ++ "\n"
```

The template context is an evolution entry. The important methods are `commit()`, `operation()`, `predecessors()`, and `inter_diff()`. Use the corresponding

commit methods when the report needs signatures and timestamps. Keep an unlabelled, newline-delimited form for scripts; terminal labels and graph glyphs are presentation, not evidence.

Comparing successive versions with `-p`

`jj evolog -p` adds the inter-diff between an evolution entry and its previous version. It answers “what changed in this logical change between these two versions?”, which is usually the question a reviewer means by “what changed in patch set 4?”:

```
jj evolog -r "$change" -p
jj evolog -r "$change" -p --stat
jj evolog -r "$change" -p --name-only
jj evolog -r "$change" --reversed -p
```

The first entry has no earlier version of the same change, so it has no meaningful predecessor diff. Later entries show the change between successive versions, not the complete diff from the repository root and not necessarily the diff between two arbitrary branch tips. Use `jj diff --from OLD --to NEW` when the endpoints are arbitrary commits rather than adjacent versions of one change.

Parent-rebase caveat. If the previous version has different parents, `jj` temporarily rebases that previous version to the new version's parents before computing the inter-diff. This prevents an unrelated ancestor change from appearing as though it were part of the logical change's revision. The displayed patch is therefore an evolution comparison, not necessarily the raw tree diff between the two stored commit objects. Do not reconstruct it by blindly running `jj diff --from OLD --to NEW` when their parent contexts differ.

For a machine-readable review record, combine the identity template with a separate patch command. Save the current and predecessor full IDs from the evolution output, then run the explicit diff only when you intentionally want object-to-object comparison:

```

jj evolog -G -r "$change" -T \
'commit.commit_id() ++ " " ++ operation.id() ++
" " ++ commit.description().first_line() ++ "\n"'
# After choosing adjacent entries OLD and NEW deliberately:
jj diff --from OLD --to NEW --git

```

Ancestor rewrites: one operation, several evolutions

Rewrite the named change after creating a descendant, then inspect both evolutions:

```

jj edit "$change"
printf 'four\n' > api.txt
jj describe -m 'API: four contract'
jj evolog -r "$change" --reversed -G
jj evolog -r "$child" --reversed -G
jj op log -n 10
jj op show

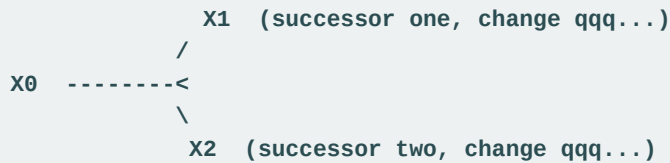
```

The change-centric result shows X's versions. The child-centric result shows that T received a successor when its parent changed. The operation-centric result shows the one command that caused the related graph rewrites. This is why a review stack can report several new Git commit IDs after the author changed one early patch: each descendant's parent field contains a different commit ID, so jj creates a successor even when the descendant's own intended content is unchanged.

Question	Use first	What it proves	What it does not prove
How did logical change X evolve?	<code>jj evolog -r X</code>	Successive commit versions and their operation links.	Why unrelated bookmarks or descendants also moved.
What did the rewrite operation affect?	<code>jj op show OP</code> or <code>jj op diff</code>	Repository-wide graph/ref/workspace effects.	That every affected commit belongs to one logical change.
What changed between two X versions?	<code>jj evolog -p -r X</code>	Successive-version inter-diff with parent-context handling.	A comparison between arbitrary non-successor commits.
Where was X published?	<code>jj bookmark list</code> , remote bookmarks, Git refs	Names and locally observed publication targets.	The complete local evolution history or server review history.

Divergence: two successors of one logical change

Two independent operations can start from one operation head and rewrite the same change differently. After both operations are integrated, the visible graph can contain two successors with one change ID:



This is not two ordinary commits that happen to have similar text. It is one logical change with multiple visible evolutions. A bare change-ID symbol is consequently ambiguous; inspect the evolution and select a specific offset or full commit ID only after comparing the alternatives:

```
jj log -r 'divergent()'
jj evolog -r 'divergent()'
jj log -r 'change_id("PREFIX")' --no-graph -T \
'commit_id ++ " " ++ change_id ++
" " ++ description.first_line() ++ "\n"'
jj op log -n 20
```

Possible decisions are different operations, not spelling variants: keep one successor and abandon the other; duplicate one as a new independent change; edit one and converge the visible evolutions with `jj converge`; or preserve both if they represent genuinely different work. Compare descriptions, parents, diffs, conflicts, bookmarks, and review identifiers before choosing. A convergence operation combines the selected divergent evolutions into one successor for that logical change; it is not a squash merely because several commits are involved.

When a change has diverged, `change_id("PREFIX")` selects all matching visible successors. To select one successor, append the offset to the bare full change ID or unique change-ID prefix; the offset is not appended to the `change_id()` function call:

```

jj show -r 'FULL_CHANGE_ID/0'
jj show -r 'FULL_CHANGE_ID/1'
jj diff --from 'FULL_CHANGE_ID/0' --to 'FULL_CHANGE_ID/1'
jj converge -r 'divergent()'
jj evolog -r 'change_id("PREFIX")'
jj op show

```

The offset is a selection of an evolution, not a permanent name. After another rewrite or convergence, use the full commit ID, bookmark, or current change-ID resolution shown by the latest `evolog`. If a command requires one commit, verify cardinality before passing a divergent query to it.

Recovery: evolog beside operation history

For a Git expert, the safest replacement for the single-word question “reflog?” is to choose the evidence layer:

Question	jj evidence	Git evidence	Gerrit evidence
How did this logical change evolve locally?	<code>jj evolog</code>	No direct equivalent; a reflog follows refs, not jj change identity.	Not a server-side Gerrit question.
Which repository-wide operation caused it?	<code>jj op log/show/diff</code>	Ref/reflog and command-specific logs, with less jj graph context.	Server audit where available.
Which exported object was reviewed?	Commit ID plus publication bookmark and fetched remote bookmark.	Commit ID and ref history.	Patch set number plus Gerrit <code>Change-Id</code> .
What changed between two versions?	<code>jj evolog -p</code> for successors, or explicit <code>jj diff</code> .	Commit/range diff.	Patch-set comparison.

A practical recovery sequence is:

```

# First preserve both kinds of evidence.
jj op log -n 20
jj evolog -r "$change" --reversed -G

# Inspect, without changing the live repository.
jj op show OP
jj op diff --from OP_BEFORE --to OP_AFTER

```

```

jj --at-operation OP_BEFORE log -r 'all()'

# Choose the repair only after comparing the evidence.
jj undo                # only for the latest integrated operation
jj op restore OP_BEFORE # restore a named repository state
jj op revert OP         # reverse one operation while retaining later history

```

`evolog` cannot restore a repository state. `op restore` cannot tell you which successor is the intended version of a review change. `undo` is appropriate only when the unwanted operation is the latest integrated operation; otherwise use a named operation or perform a deliberate graph repair. Neither local tool can undo a GitHub/Gerrit push: fetch the server state, identify the exported commit/ref, and repair the remote publication separately.

Publication, Git transfer, and review-system identity

A local evolution entry becomes externally meaningful only when a particular commit object is exported. Git transport transfers commit objects and refs; it does not transfer `jj`'s local operation DAG or the complete evolution ledger. A remote bookmark such as `review@origin` tells this clone which remote ref was observed at a fetch. It is not a server-side copy of `evolog`. If a later push replaces the publication bookmark, the server sees a new Git commit ID or ref target; the local change ID is not part of the Git protocol.

GitHub normally treats the pushed bookmark as the PR branch. Gerrit normally treats each commit pushed to `refs/for/<branch>` as a review change, matching later patch sets by the Gerrit `Change-Id` trailer. These are three different identity layers:

Layer	Stable across a local rewrite?	Where it is observed
jj change ID	Normally yes, while the logical change evolves	jj <code>evolog</code> , log output, local jj metadata
Git commit ID	No: a changed tree, parent, or metadata creates another object	Git refs, CI records, GitHub commits, Gerrit patch-set commit
Gerrit Change-Id	Usually yes when the trailer is preserved	Commit message and Gerrit review page

Layer	Stable across a local rewrite?	Where it is observed
GitHub PR / bookmark	The PR name may remain while its branch target moves	Publication ref and hosting service

For an upload incident, record all four where available. “The change is still the same” may be true in jj's evolution model and in Gerrit's review model while false for the Git commit ID used by CI. Conversely, a commit with a familiar Git message is not enough evidence that it is the intended successor: compare its full ID, parent, diff, bookmark, and operation.

```
# Export one exact evolution entry, then correlate it with transport state.
jj evolog -G -r "$change" -T \
'commit.commit_id() ++ " " ++ commit.change_id() ++
  " " ++ operation.id() ++ " " ++
  commit.description().first_line() ++ "\n"'
jj bookmark list --all-remotes
jj git fetch --remote origin
jj log -r 'review | review@origin' -T 'commit_id ++ " " ++ description.first_line() ++ "\n"'
jj op log -n 20
```

The exact inspection command depends on the publication path. Do not infer a remote's current state from a stale remote bookmark: fetch first. Do not infer an operation from a Gerrit patch-set number: use the local evolution and operation records, then match the exported commit ID and message trailer. The operation ID is local evidence and may have no counterpart on another machine.

Incident exercise: CI commit 2, Gerrit patch set 4, local commit 5

Use the following exercise to practise a realistic review investigation. The numbers are deliberately from different namespaces:

- **CI commit 2** is the Git commit ID recorded by an earlier CI run.
- **Gerrit patch set 4** is the fourth uploaded patch set for one Gerrit review, identified by its `Change-Id`.
- **local commit 5** is the current jj commit object for the same logical change, after a later local rewrite.

```

jj evolution:      X2 --> X3 --> X4 --> X5
                  |       |       |       |
CI:               c2       c3       c4       c5
Gerrit:           ps2      ps3      ps4      (not uploaded)
operations:       o2       o3       o4       o5

```

Do not answer “patch set 4 is commit 5” merely because both are described as the latest review. Perform the correlation:

1. Fetch the Gerrit remote and record the commit ID currently attached to the review ref or fetched bookmark.
2. Run `jj evolog -r CHANGE --reversed -G` and export full commit IDs, change IDs, operation IDs, and descriptions.
3. Find the entry whose full commit ID matches the commit Gerrit used for patch set 4. Verify its Change-Id trailer and parent(s).
4. Use `jj evolog -p -r CHANGE` to inspect what changed from the preceding local version; use an explicit `jj diff` only when comparing deliberately chosen stored objects.
5. Use `jj op show o5` and `jj op diff` to establish why local commit 5 exists. If an ancestor was rewritten, inspect the descendant's evolution as well.
6. Decide whether commit 5 is a new patch set to upload, an unintentional local successor, or a divergent alternative. Do not push until the bookmark and review identity have been checked.

Evidence	Question answered	Question not answered
Gerrit patch set 4 and its commit ID	Which Git object the server reviewed	Which local operation produced it
<code>jj evolog</code> entry for that commit	Which logical change version it was locally	Whether the server accepted a later push
<code>jj evolog -p</code>	What changed between adjacent local versions	What an arbitrary server-side comparison displayed
<code>jj op show/op diff</code>	What local rewrite or publication operation affected the graph	Whether a remote review was merged or abandoned
Fetched remote bookmark and server UI	Which object is currently published and reviewed	What was present only in an unfetched local clone

This is the practical purpose of `evolog`: it lets a GitHub or Gerrit user follow the logical change while separately accounting for the exact Git objects and the local operations that produced them. It complements, rather than replaces, the operation log, remote bookmarks, server review history, and Git commit inspection.

Correlating operation history with change evolution

The two histories answer different questions and become most useful together. `jj evolog` identifies the successive commit objects that represent one logical change; `jj op show` identifies the repository-wide transformation that created or rewrote it. Correlate them when an ancestor rewrite produces several successors, when a review upload must be matched to a local version, or when recovery needs both the intended change and the surrounding ref/workspace effects.

```
# Select one logical change and record both evidence layers.
change=$(jj log -r @ --no-graph -T 'change_id ++ "\n"')
jj evolog -r "$change" -G
jj op log -n 10
jj op show OPERATION
jj op diff --from OP_BEFORE --to OP_AFTER
```

An evolution entry is not an operation and an operation is not a commit. The compact rule is: use evolution history to choose *which version of the change* you mean, then use operation history to explain *what else changed at the same time*. The recovery laboratory below applies that distinction to named exercises; the later concurrency section covers operation branches, retention, and the limits of local history across clones.

Recovery laboratory: named states, undo, and repair

Recovery has two different targets. If the mistake is “the logical change is no longer where I intended”, use the commit graph, bookmarks, and evolution log. If the mistake is “the repository as a whole was transformed incorrectly”, use the operation DAG. A commit ID remembered from before a rewrite can locate a predecessor, but it does not by itself tell you which operation should be restored.

Question	First tool	Why
What did the last command do?	<code>jj op show</code>	Shows repository changes made by one operation.
Where did my change go?	<code>jj evolog -r CHANGE</code>	Follows one change's predecessors/successors.
What did the repository look like earlier?	<code>jj --at-op OP log -r all()</code>	Loads a historical repository view without snapshotting current files.
Undo the immediate mistake?	<code>jj undo</code>	Creates a reverse operation for the latest integrated operation.
Restore a named state?	<code>jj op restore OP</code>	Moves local repository state to an operation, with selectable portions.
Compare two candidate states?	<code>jj op diff --from A --to B</code>	Shows repository changes without guessing from current IDs.
Recover after a server push?	Server/ref inspection plus local operation tools	Local undo cannot undo an external remote mutation.

Exercise 1: rewrite the wrong change

```
jj log -r '@:: & mutable()'
jj op log -n 3
# mistakenly select an ancestor:
jj edit A
jj describe -m 'WRONG: temporary experiment'
jj op log -n 3
```

Do not immediately perform another edit to “put things back”. Record the newest operation ID and inspect it:

```
jj op show
jj evolog -r A
jj --at-op OP_BEFORE log -r '@:'
jj undo
```

`undo` reverses the operation that integrated the mistaken description/edit state. The working-copy filesystem may need a deliberate update, and any external files changed by an editor are outside the operation's protection. Verify the current `@`, the target change's description, and descendants before continuing.

Exercise 2: abandon useful work, then recover it

```
jj bookmark create rescue -r @
jj abandon -r 'change_id(important...)'
jj log -r 'visible()'
jj op log
jj evolog -r 'change_id(important...)'
```

The temporary bookmark is a publication/name safety net, but evolution and operation state are still useful. If the bookmark was deleted by the command, the commit may remain as a predecessor/hidden commit. Inspect the operation immediately and restore or undo before GC. A recovery bookmark can be created at a predecessor only after the exact commit has been selected.

Exercise 3: a bookmark moved to the wrong tip

```
jj bookmark list --all-remotes
jj bookmark set review -r wrong-tip
jj op show
jj log -r 'review | review@origin'
jj undo
```

Undoing the local bookmark operation does not undo a push that already happened. If the wrong tip was pushed, inspect the server's current ref and use an explicit, policy-approved update. If the remote changed independently, tracking state may make the expected old target visible; do not force simply because a local undo makes the local name look right.

Exercise 4: several operations happened before discovery

Undo is intentionally convenient but may be the wrong granularity after later work. Use the operation graph:

```
jj op log
jj op show OP_BAD
jj op diff --from OP_BEFORE --to OP_BAD
jj --at-op OP_BEFORE log -r 'all()'
jj op restore --what repo OP_BEFORE
jj op log
```


The restore creates a new operation; it does not delete the old operation node. This gives the repository an auditable “restored to earlier view” event. If remote-tracking state is restored along with repository state, later pushes may compare against old observations; the experimental `--what repo` form can be preferable when the goal is to repair local graph/bookmarks while retaining current remote observations.

Exercise 5: evolution divergence

```
jj evolog -r 'change_id(abc...)'
jj log -r 'divergent() & change_id(abc...)'
jj show -r 'FULL_CHANGE_ID/0'
jj show -r 'FULL_CHANGE_ID/1'
```

Choose the successor by content and graph context, not by offset number. One successor may contain the reviewed change and another a local experiment. You can edit one, rebase it, duplicate it to a new change, or abandon the unwanted successor. Then inspect descendants and bookmarks: a publication name can keep a commit visible even when the logical change has divergent forms.

Exercise 6: operation views are local

Clone A and clone B share commits only after fetch/push. Their operation IDs may both begin with a root operation but are not a shared timeline. A commit recovered through `jj op restore` on A does not become a recoverable operation on B. To transfer a useful state, create a temporary bookmark, push it, fetch it, and create a new local working-copy commit on B.

```
# A
jj bookmark create handoff -r @
jj git push --remote origin --bookmark handoff

# B
jj git fetch --remote origin
jj log -r 'handoff@origin'
jj new 'handoff@origin'
jj bookmark forget 'handoff'
```

Delete the temporary remote bookmark only after verifying the receiving clone. The filesystem's uncommitted edits and local operation history are not transferred by this

procedure; commit them deliberately or copy the files through a separate channel.

Recovery stopping rules

1. Stop mutating the repository when the mistake is noticed.
2. Record operation ID, current workspace, bookmarks, and relevant commit/change IDs.
3. Use `--at-op` for inspection and avoid snapshotting into historical views.
4. Choose undo for the latest operation; choose restore/diff for a named state.
5. Do not run GC, abandon recovery evidence, or push until the local graph is understood.
6. Separate local repair from server repair and record both.

Conflict laboratory: a conflict is stored in the graph

Construct two changes from the same base that edit the same lines, then merge or rebase them. `jj` can represent the unresolved result as a commit with a conflict value. The commit is still a graph node: it has an ID, a change ID, parents, descendants, and an operation that created it. “Unresolved” describes one value in its tree, not an interrupted global transaction.

```
      B: timeout = 30
      /
A-----+
      \ C: timeout = 60

merge M has parents B and C:
A--B-----+
  \      |--M[conflict]
   C----+
```

When a conflict is materialised in a working copy, `jj` writes conflict markers or the configured marker representation into files. Materialisation is a projection of the stored conflict into the filesystem; editing the markers and running `jj resolve` records a resolved tree in the relevant commit. The conflict value itself can have multiple sides, including nested or repeated conflict terms after successive rewrites. Do not assume every conflict is a two-way “ours/theirs” merge.

```
jj status
jj log -r 'conflicts() & ::@'
jj resolve --list
jj resolve --tool=:ours
jj resolve --tool=:theirs
jj resolve --tool=meld
```

`:ours` and `:theirs` are convenient only where their interpretation is unambiguous. With a merge commit, “ours” is associated with the destination/first side and “theirs” with the other side according to the command’s merge model; in a multi-sided conflict, choosing one side may discard content from more than one parent. A real merge tool should expose every side, and the result must be tested rather than judged by marker absence alone.

Conflict propagation through descendants

```
before rewrite:
A--B--C--D
  B changes config key
  C changes same key as a dependent test
  D changes docs about C

rewrite B → B':
A--B'--C'[conflict]--D'[conflict or adjusted]
```

Conflict propagation is why `jj` can let work continue on a stack while a lower change is unresolved. `C'` is a valid commit to inspect, describe, or even further rewrite. Its descendant `D'` is likewise present, though its diff may be represented relative to a conflicted parent. Resolve from the lowest semantically understood conflict upward. After resolving `C'`, inspect `D'` because the stored conflict value may have changed even if `D'`’s own files were untouched.

There are two useful strategies. Resolve immediately when the conflict is small and the intended invariant is clear. Or carry the conflict while investigating the stack, then resolve after related changes reveal the intended final interface. The second strategy is safe only if your tests and publication tooling accept conflicted commits; a push or review upload may reject them, and a build workspace may not be able to materialise a useful tree.

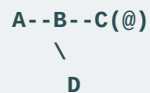
Conflict-aware commands

Operation	What conflict means	Practical check
Rebase	The diff could not be replayed cleanly onto a new parent.	Find the earliest conflicted commit, then inspect its descendants.
Squash	Combining trees or diffs yields a conflict value.	Verify source remainder and destination state before resolving.
Split	A selected portion may carry one side while the residual commit carries another.	Inspect both resulting changes and their conflict predicates.
Restore	Copying content from a conflicted source can preserve conflict structure.	Use <code>show/diff</code> on the source before materialising.
Resolve	Replaces the conflict value with a concrete tree.	Run tests and verify <code>conflicts()</code> no longer selects it.
Push/upload	Server transport may reject conflicted commits or represent them poorly.	Resolve before publication unless the server workflow explicitly supports them.

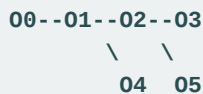
Concurrency, retention, and cross-machine limits

Once the operation model has been established above, these are its boundary conditions. Concurrent operations can branch the operation history, retention and garbage collection affect recovery, and Git transport does not transfer operation IDs to another clone.

commit DAG:



operation DAG:



02 may have rewritten $B \rightarrow B'$, $C \rightarrow C'$ while 04 is an independent operation started from 02. Commit identity and operation ancestry are related but not interchangeable.

Concurrency and operation integration

Two processes can start at one operation and produce independent operations.

`--no-integrate-operation` deliberately leaves a successful operation outside the current integrated head and prints an operation ID. It does not make external side effects transactional and does not protect the filesystem from another writer. `jj op integrate OP` brings that operation into the current operation history; integration may expose divergent repository state or conflicts that were not visible from either isolated view.

```
jj status --no-integrate-operation
# save the printed operation ID
jj op log
jj op integrate OP
jj op diff --operation OP
jj log -G -r 'all()'
```

Use this mode when a supervisor wants to control integration or when testing concurrency. It is not a replacement for a filesystem lock. Editors, build tools, Git, jj, and workspaces can still race over files and refs. Serialise mutation of one worktree and use separate workspaces or clones for concurrent tasks.

Retention and garbage collection

Recovery depends on data that has not been pruned. Abandoning a commit hides it from ordinary views but may leave it addressable; abandoning an operation removes recovery metadata according to the command's semantics; repository GC can reclaim objects no longer considered reachable. Before maintenance, inspect the operation and evolution history, confirm that published refs and temporary rescue bookmarks are no longer needed, and make an independent backup if policy requires one.

```
jj op log
jj evolog -r 'all()'
jj bookmark list --all-remotes
jj util gc
```

The command's successful exit status is not a guarantee that every recovery path remains available. The operation log is local to a clone and is not transferred by Git push/fetch. A clone on another machine can recover commits that were published or fetched there, but it cannot use this clone's operation ID unless the repository state and operation store are shared through an explicitly supported mechanism.

Recovery decision record

Record	Reason
Current operation ID and relevant earlier ID.	Defines the repository views being compared.
Current commit/change IDs and bookmark targets.	Separates exact objects from logical changes and names.
Whether a remote push/fetch occurred.	Local operation tools cannot reverse server side effects.
Whether the filesystem has unrecorded edits.	Historical operation views ignore today's working-copy snapshot.
Chosen recovery operation and why.	Prevents a second recovery action from erasing the first intention.
Post-recovery log/evolution/ref evidence.	Confirms the graph, not only the command's success message.

Part XII — Workspaces and multiple machines

Workspace model in detail

A **workspace** is jj's persistent association between one directory on disk and one working-copy commit in a repository. It is the unit that answers two questions which Git often answers through a worktree and `HEAD`: “which files are materialised here?” and “which commit does this directory currently represent?” Every jj repository has a default workspace, and it may have additional named workspaces. The name is a repository-level identity; the directory is its filesystem location; its working-copy commit is the mutable commit that `@` resolves to when a command runs from that directory.

A workspace is not a bookmark: it is not a publication name and it does not have to be visible to a remote. It is not a clone: workspaces share the same repository commit store, bookmarks, tags, and operation history. It is also not merely another spelling for a Git branch or detached HEAD. The closest Git comparison is a Git worktree with an explicitly recorded, independently mutable checkout commit, but the comparison breaks because jj workspaces participate in the same mutable DAG and operation system rather than being separate ref-oriented checkouts.

```
one jj repository

shared commit store, refs, and operation DAG
|
+-- workspace "default" /home/me/project
|   default@ -> C       files for C
|
+-- workspace "review"  /home/me/project-review
|   review@  -> R       files for R

@ means the workspace-local working-copy commit;
C and R may be unrelated commits.
```

Workspace state is therefore local in one sense and shared in another. The directory, its filesystem contents, its current working-copy commit, and its sparse patterns belong to that workspace. The commit objects those commits point to, the repository's bookmarks and tags, and the operation log are shared. A rewrite performed from `review` can change a commit that `default` is based on; jj may then need to rebase or mark `default`'s working-copy commit stale. Conversely, moving `review`'s `@` does not move `default`'s `@`.

Additional workspaces are created with `jj workspace add DESTINATION`. By default the new working-copy commit is created on the same parent set as the current workspace's working-copy commit. `--revision R` instead makes it a child of the supplied parent commits, with the same graph effect as `jj new R`; `--name NAME` controls the workspace symbol. The command creates another materialisation of the shared graph, not another repository. The remaining workspace lifecycle and sparse-pattern options are covered below after the basic state transition is visible.

before: one workspace

```
A---B---C
      \
      default@
```

```
jj workspace add --name review ../project-review -r B
```

after: two workspace-local tips in one graph

```
A---B---C
  |   \
  |   default@
  |   \
  \   review@
```

The two directories can now be edited and snapshotted independently. A workspace is not a promise that its filesystem is current at every instant: an editor, build tool, or another jj process can change the directory after the last snapshot. Normal jj commands reconcile the live directory with that workspace's recorded working-copy commit; `--ignore-working-copy` deliberately reads the recorded state without doing so. Keep the shared graph, workspace record, last recorded tree, and live filesystem distinct.

The 0.45.1 workspace subcommands are `add`, `forget`, `list`, `rename`, `root`, and `update-stale`. A build workspace can edit a stack while a review workspace remains at a stable commit:

```
jj workspace add ../project-build -r @
jj workspace add ../project-review -r main@origin
jj workspace list
jj -R ../project-build status
cd ../project-build
jj workspace rename ci
jj workspace update-stale
jj workspace forget review
```

Each workspace has its own `@`; commit edits and graph rewrites are shared, so a rewrite can move the commit a second workspace points at. Update stale workspaces deliberately and inspect filesystem state before deleting one. Sparse patterns are workspace working-copy materialisation rules, configured with `jj`

`sparse list, set, edit, and reset`; they do not change the commit tree.

Machines

Clones have independent operation DAGs and workspace pointers. Fetch/push commits and bookmarks through Git remotes; do not expect `@` or `jj undo` on machine A to exist on machine B. To move unfinished work, publish a temporary bookmark, fetch it on the other clone, edit it there, and delete/forget the temporary publication after the handoff. This is safer and more observable than inventing operation-log synchronisation.

Workspace laboratory: one store, several materialisations

A workspace is a named view over a shared repository, with its own working-copy commit and filesystem. It is not a lightweight branch and it is not an independent clone. The commit store and operation history are shared; the workspace pointer, sparse patterns, and files on disk are local to that workspace. This division makes build/review separation powerful and makes concurrent filesystem writes dangerous.

```
shared commit store:  main--A--B--C
workspace dev:        dev@ = C, full tree
workspace build:      build@ = B, sparse src/
workspace review:     review@ = A, full tree
```

A rewrite of A or B is visible to all workspaces because it changes the shared graph. Each workspace then needs its own update policy. A workspace whose recorded working-copy commit is rewritten may need its checkout record updated; a workspace with ordinary unrecorded filesystem edits is not thereby stale. Ask which workspace owns a file before running a formatter, test generator, or `jj` command.

Development, build, and review scenario

```
jj workspace add ../project-build -r @
jj workspace add ../project-review -r main@origin
jj workspace list
cd ../project-build
jj workspace rename ci
jj status
cd ../project-review
jj workspace root
jj log -r @
```

The rename command takes one new name for the current workspace in 0.45.1. It does not take an old and new name pair. Use the workspace's own directory or `jj -R` to select the operating location, then verify the new workspace symbol with `jj workspace list`. Workspace symbols can be used in revsets, but the filesystem directory remains the authority for which process is writing files.

The build workspace can edit or test a particular change without moving the development workspace's `@`. It still shares commits, so a rewrite performed from the build workspace can change what the development workspace sees. A review workspace can remain at an immutable remote boundary while the development workspace advances. This is a graph coordination tool, not a promise that test results remain valid after a later rewrite.

Build and review workspace invariants

A build workspace should normally be boring: pin it to an explicit bookmark or immutable commit, use sparse materialisation only when the build system supports it, and avoid making source edits there. If a build tool generates files, decide whether they are tracked inputs, ignored outputs, or disposable files before allowing a snapshot. A review workspace can remain at an imported remote bookmark while development continues elsewhere; its value is a stable observation point, not a publication name.

```
cd ../project-build
jj edit main@origin
jj status
# run the build; inspect generated files before any snapshot
```

```
cd ../project-review
jj log -r '@ | @-'
```

Do not assume that a workspace at `main@origin` follows later fetches. The remote bookmark may advance while the workspace's working-copy commit remains the previously selected commit. Update it explicitly, and verify the resulting commit ID before trusting a build result.

Stale workspaces and repair

In `jj 0.45.1`, a workspace is stale when its recorded working-copy commit is no longer the current commit for that workspace because another repository operation changed the recorded checkout state. Ordinary edits made by an editor, build tool, or native filesystem command are unrecorded working-copy changes; they do not by themselves make the workspace record stale. A moved directory, missing metadata, or a native Git ref change is a separate location/backend diagnosis, not a synonym for stale state.

Inspect all workspaces before attempting repair:

```
jj workspace list
jj workspace root
jj -R ../project-build status
jj -R ../project-review status
jj -R ../project-build workspace update-stale
```

`update-stale` repairs the stale workspace record/checkout relationship; it is not a general graph repair and it is not a command for incorporating arbitrary filesystem edits. If it reports that the workspace is not stale, leave the files alone and diagnose the live-versus-recorded snapshot instead. If files contain valuable unrecorded edits, snapshot them with an appropriate `jj` command or copy them aside before an operation that will replace the checkout. If the graph target is wrong, choose an explicit commit with `jj edit` or `jj new` in that workspace. If the directory was moved or damaged, repair the filesystem/location problem separately.

Sparse build workspace

Sparse state is per workspace. Use `sparse set` to specify patterns; `sparse edit` opens the external editor for the pattern configuration and takes no fileset argument in 0.45.1.

```
cd ../project-build
jj sparse set --clear --add src --add Cargo.toml
jj sparse list
test -f src/main.rs
test ! -e docs/guide.md
jj file list -- 'root-prefix-glob:docs'
jj sparse reset
```

The commit tree still contains docs and tests while the sparse filesystem does not materialise them. A build that runs successfully in a sparse workspace may therefore omit an input. Make the sparse policy part of the build command's evidence and run a full-tree validation before publication. Changing sparse patterns may add/remove files in the filesystem but does not create a content commit in the same sense as editing a tracked file.

Forget and cleanup

Forgetting a workspace removes its workspace metadata; it does not abandon the commits that workspace's @ pointed at. Check for unrecorded files and useful working-copy changes first. A forgotten workspace's commit may still be reachable from bookmarks or the graph, and its operation history remains part of the shared repository until retention removes it.

```
jj -R ../project-build status
jj -R ../project-build diff
jj workspace forget ci
jj workspace list
jj log -r 'working_copies()'
```

Clean the filesystem directory only according to the repository's workspace management rules. Do not confuse removing a directory with `workspace forget`; one is a filesystem action and the other changes repository metadata. If a stale directory is later reused, inspect its workspace identity and root before assuming it

represents a fresh checkout.

Two machines and unfinished work

Independent clones have independent operation DAGs and independent workspace pointers. Git transport moves commit objects and refs; it does not move @, sparse patterns, workspace names, or operation IDs. The portable handoff for unfinished work is an explicit temporary bookmark:

```
# machine A
jj bookmark set handoff/alice -r @
jj git push --remote origin --bookmark handoff/alice

# machine B
jj git fetch --remote origin
jj log -r 'handoff/alice@origin'
jj new 'handoff/alice@origin'
# continue work, then publish the next handoff explicitly
```

Creating a child on machine B gives that machine its own working-copy commit. The original machine can continue to inspect the remote bookmark, but it must not assume the remote target follows B's new child. After handoff, delete or forget the temporary publication according to whether the server ref should remain available. If both machines edit the same logical change independently, fetching can expose divergent successors; compare evolutions before choosing a winner or converging.

Concurrency rules

Shared	Workspace-local	Concurrency implication
Commit store and graph.	Working-copy commit pointer.	A graph rewrite can affect every workspace's target.
Operation history.	Filesystem and sparse patterns.	Operation inspection is shared; file writes still need serialisation.
Bookmarks/tags.	Current directory and local tools.	Ref moves are repository mutations visible to all workspaces.
Remote observations.	Editor/build process state.	Fetch/import and builds should not race a writer.

A robust multi-workspace routine gives each process one directory, one workspace, and one writer role. Use `--ignore-working-copy` only for a deliberate repository-view read. It prevents the command from reconciling the directory's files, but it does not make a concurrent filesystem process safe.

Part XIII — Configuration

Configuration is TOML and is layered. Built-in defaults are overridden by user config, repository config, workspace config, command-line `--config-file`, then command-line `--config NAME=VALUE` according to the installed precedence rules. On Linux, use XDG locations such as `~/.config/jj/config.toml` and supported `conf.d` directories. Confirm actual paths with `jj config path`; do not assume a distribution's packaging layout.

```
jj config list
jj config get user.name
jj config set --user user.name 'A. Engineer'
jj config set --repo ui.diff-formatter ':git'
jj config set --workspace ui.pager 'less -FRX'
jj config unset --user ui.pager
jj config edit --user
jj config gc
```

The available 0.45.1 config commands are `edit`, `gc`, `get`, `list`, `path`, `set`, and `unset`. The exact scope flags should be taken from local help. In `jj 0.45.1`, `JJ_CONFIG` selects the user/system TOML file or directory set; it is not an inline TOML string. Use `--config` for a one-command reproducible override and `--config-file` to load an additional file.

A maintainable Linux layout

```
~/ .config/jj/  
|-- config.toml  
\-- conf.d/  
    |-- 10-ui.toml  
    |-- 20-revsets.toml  
    |-- 30-templates.toml  
    |-- 40-git.toml  
    |-- 50-gerrit.toml  
    \-- 60-aliases.toml
```

Organise by intent, not alphabetically. Important namespaces include `user.*`; `ui.*` (editor, pager, colour, timestamps, diff and merge presentation); `git.*` (backend, remotes, push/fetch, private commits); `gerrit.*` (including the 0.45.1 keys `default-remote` and `default-remote-branch`); `revsets.*`; `templates.*`; `merge-tools.*`; `fix.tools.*`; `signing.*`; immutable-head settings; aliases; and snapshot/workspace behaviour.

```
[ui]  
default-command = ["log", "-r", "@:: & mutable()"]  
paginate = "auto"  
diff-editor = "meld"  
  
[revsets]  
log = "@:: & mutable() | bookmarks() | remote_bookmarks()"  
  
[revset-aliases]  
stack(x) = "x:: & mutable()"  
  
[templates]  
log = 'builtin_log_compact'  
  
[gerrit]  
default-remote = "gerrit"  
default-remote-branch = "main"
```

Use `jj util config-schema` and the generated config inventory for the exhaustive target-release key surface. Some configuration is security-sensitive: `repo/workspace` config can be untrusted input in a checked-out repository, so 0.38+ layouts moved it outside the repository. Review effective config before using aliases or external tools.

jj 0.45.1 supports conditional configuration with `--when` variables and scoped configuration, as well as ordered files in `~/.config/jj/conf.d/`. Use those mechanisms for host-specific policy when they make the distinction auditable. A robust alternative is explicit command aliases: `jj git push --remote github --bookmark gh/topic` and `jj gerrit upload --remote gerrit`. Never let a convenient default choose a publication remote silently.

Configuration design: build policy in layers

Configuration is executable policy. A useful configuration says what the repository should do when a human invokes a command, what a script may assume, and which decisions must remain explicit at a publication boundary. Keep the user layer ergonomic, the repository layer reproducible, the workspace layer local, and the command line layer temporary.

Layer	Put here	Do not put here
Built-in defaults	Fallback revsets, templates, pager, formatter behaviour.	Organisation policy; defaults vary by release.
User	Name/email, editor, pager, preferred display, personal aliases.	Repository-specific remote or review policy.
Repository	Project revsets, template aliases, fixer/merge-tool definitions, immutable heads.	Secrets; assumptions that should apply to every checkout.
Workspace	Local default command, sparse/workspace-specific display or tool choice.	Policy other contributors must discover implicitly.
Command line	One-run overrides, CI pinning, explicit remote and output controls.	Long-lived aliases; shell secrets embedded in process listings.

On Linux, verify the actual files with `jj config path --user`, `--repo`, and `--workspace`. XDG conventions are useful defaults but packaging can alter paths. `JJ_CONFIG` selects the user/system TOML file or directory set in the target release; it is not an inline TOML string. `--config-file` loads additional input, while `--config NAME=VALUE` applies an inline override.

Progressive setup

```
# identity first
jj config set --user user.name 'A. Engineer'
jj config set --user user.email 'a@example.test'

# safe display and explicit daily scope
jj config set --user ui.pager 'less -FRX'
jj config set --user revsets.log '(@:: & mutable()) | bookmarks() | remote_bookmarks()'

# personal aliases
jj config set --user 'revset-aliases.unpublished()' 'mutable() ~ ::remote_bookmarks()'
jj config set --user 'command-aliases.stack' 'log -r "@:: & mutable()"
```

Use `config edit` for nested TOML and multi-line values; command-line setting must satisfy both the shell and the TOML parser. The last line is illustrative of a command alias and should be checked against the exact alias syntax in the installed schema/help before adoption. A config test should inspect the effective value rather than assuming the write landed in the desired layer.

UI and output policy

`ui.*` settings govern editor, pager, colour, timestamps, diff formatter/editor, merge editor, default command, movement policy, and related presentation. Separate human output from machine output: a human default may use a pager, graph, colour, relative dates, and compact descriptions; a script should specify `--no-pager --color=never`, a bounded revset, and an explicit template.

```
[ui]
pager = "less -FRX"
paginate = "auto"
color = "auto"
default-command = ["log", "-r", "@:: & mutable()"]
movement.edit = true
diff-editor = "meld"
merge-editor = "meld"
```

Do not put a default command that mutates state behind a shell prompt. A default `log` that snapshots is still a mutation in the operation log; if prompt latency and side effects matter, use a prompt-specific command with `--ignore-working-copy`.

Revset and template policy

```
[revsets]
log = '@:: & mutable() | bookmarks() | remote_bookmarks()'
status = 'mutable() & ::@'
fix = 'reachable(@, mutable())'
sign = 'bookmarks()'

[revset-aliases]
stack(x) = 'x:: & mutable()'
unpublished() = 'mutable() ~ ::remote_bookmarks()'
review-candidates() = 'heads(unpublished() ~ empty() ~ conflicts())'

[templates]
log = 'builtin_log_compact'

[template-aliases]
short-id = 'change_id.shortest(10)'
subject = 'description.first_line()'
state = 'separate(" ", if(conflict, "CONFLICT", ""), if(empty, "EMPTY", ""), bookmarks)'
```

Aliases are language code and should be reviewed as such. A revset alias affects selection; a template alias affects rendering. Keep names semantically precise. Document whether an alias includes hidden commits, remote refs, empty changes, or conflicts, because those choices determine whether a publication script is safe.

GitHub and Gerrit policy

```
[git]
push = "github"

[gerrit]
default-remote = "gerrit"
default-remote-branch = "main"
review-url = "https://gerrit.example/review"

[command-aliases]
publish-gh = ["git", "push", "--remote", "github", "--bookmark"]
publish-gerrit = ["gerrit", "upload", "--remote", "gerrit", "--remote-branch", "main"]
```

Use explicit remote arguments in publication aliases. A default push remote is convenient but dangerous in a repository used for both GitHub and Gerrit. `jj 0.45.1` documents the dedicated Gerrit keys shown above. Confirm exact command-alias value shape against `jj help config` and the target schema.

Immutable heads and repository policy

The immutable-head configuration defines the commits `jj` protects from rewrite. A sensible project policy commonly includes imported trunk and tags/release boundaries, but the right set depends on whether the repository permits local rebasing over remote bookmarks. Explain the policy as a revset, test it against a fixture graph, and keep an emergency `--ignore-immutable` procedure separate from normal aliases.

```
[revset-aliases]
"immutable_heads()" = "main@origin | tags() | root()"
```

In 0.45.1 this is the alias that `jj` itself evaluates when it checks rewrite protection. It is deliberately under `[revset-aliases]`, not a made-up `[revsets]` key. Verify the effective policy before relying on it:

```
jj log -r 'immutable_heads()'
jj config get 'revset-aliases.immutable_heads()'
jj rebase -s @ -d main@origin
```

The exact key/value syntax is release-sensitive, but the policy has an observable contract: protected commits appear in the query and an ordinary rewrite that would alter them is rejected. An immutable head is not automatically every server ref, and changing the config does not retroactively make a rewrite safe. Always inspect which commits are protected before using `--ignore-immutable`.

External tools and security

Merge tools, diff editors, fixers, signers, pagers, and command aliases can execute programs. Repository/workspace configuration can therefore alter what a command runs. Review `config list` and the file paths before operating in an unfamiliar repository. Prefer absolute executable paths or trusted `PATH` policy in CI, and do not let a checked-out repository silently replace a user's merge tool.

Family	Questions before enabling
merge/diff tools	What files are passed? Does the tool modify them? What exit status means success?
fix tools	Does it support modified-line ranges? Is it deterministic? Can it rewrite every selected commit?
signing	Which key/agent is used? Does signing rewrite descendants? Is verification available on CI?
aliases	Does the alias mutate, publish, or choose a remote implicitly?
pager/editor	Can a non-interactive environment hang? Are colour and terminal escapes safe?

Configuration diagnosis

```
jj config path --user
jj config path --repo
jj config path --workspace
jj config list ui
jj config list git
jj config list gerrit
jj config get revsets.log
jj util config-schema
```

When behaviour differs between two shells, compare effective config, environment variables, current workspace root, and jj version. When behaviour differs between two repositories, compare repository/workspace config and remote URLs. When behaviour differs between the installed release and future releases, diff the captured schemas and command help rather than guessing from a release note.

Configuration reference conventions

Appendix F groups significant keys by user-facing purpose. Each reference row records the key, TOML type, usual scope, effect, default/absence behaviour, security or performance concern, and version note. The machine-generated schema remains useful for detecting newly added keys, but a schema cannot explain that `git .push` chooses a remote while bookmark tracking chooses a relationship, or that a template alias can shadow a builtin. Those interactions belong in the book.

Configuration laboratory: build a safe Linux configuration

Configuration is executable policy. Build it in layers so that a change in one layer has a visible reason and scope. Start with identity and UI, then add revset/template aliases, then repository transport, then hosting-specific policy.

```
[user]
name = "Ada Example"
email = "ada@example.org"

[ui]
pager = ["less", "-FRX"]
diff-editor = ":builtin"
merge-editor = "diffconflicts"
default-command = ["log"]

[revsets]
log = "builtin_log()"

[templates]
log = "builtin_log_compact"
```

Check the actual accepted value shapes with `jj util config-schema` for the target release; configuration keys and types are version-sensitive. The example illustrates design intent, not a universal drop-in file. Keep the base file free of credentials and repository-specific bookmarks.

Layered files and precedence

On Linux, user configuration normally lives under the XDG configuration directory, with `jj`'s own directory and the ordered `conf.d/*.toml` fragments supported by `jj 0.45.1`. Repository and workspace configuration can narrow policy. Command-line `--config` values are useful for one invocation and for tests; `--config-file` loads a file without pretending it is the user's permanent configuration. In `jj 0.45.1`, configuration commands also expose a direct `--file` target for editing a particular file.

Layer	Good use	Risk
Built-in defaults	Portable baseline.	Defaults can change between releases.
User config	Identity, editor, pager, personal aliases.	Hidden policy affects every repository.
Repository config	Trunk, remotes, fixers, project aliases.	Trust boundary for cloned repositories.
Workspace config	Sparse/build/review-specific behaviour.	Only one workspace sees the setting.
Command line	Tests, temporary overrides, automation.	Harder to audit if embedded in aliases.

When debugging a surprising value, use `jj config list`, `jj config get NAME`, and `jj config path`. Record the invocation's explicit config and environment in automation logs. Avoid aliases that silently add `--ignore-immutable`, select a remote, or permit an unsafe push.

GitHub and Gerrit conditional intent

Hosting-specific settings should be opt-in. A repository that has both remotes should not infer its publication target from whichever remote happens to be listed first. Use explicit `--remote` and `--bookmark` in scripts, and reserve defaults for a repository where the policy is unambiguous.

```
# GitHub publication
jj git push --remote github --bookmark pr/parser

# Gerrit review upload
jj gerrit upload --remote gerrit --remote-branch main -r '@:: & mutable()'
```

For 0.45.1, Gerrit-specific defaults such as `gerrit.default-remote`, `gerrit.default-remote-branch`, and `gerrit.review-url` can reduce repetition. In a shared config, document that those keys are not a guarantee that a GitHub push will use the same remote. Separate command aliases by transport and require a bookmark argument for destructive or public operations.

Aliases as an interface

An alias is a small program in configuration. Name it for its invariant, not its keystrokes. A good read alias prints stable IDs and uses a bounded revset. A risky alias should be explicit about its source, destination, remote, and safety behaviour. Version aliases that use a target-release-specific command or predicate, or gate them with a documented release check.

```
[aliases]
st = ["log", "-r", "@:: & mutable()", "-T", "builtin_log_compact"]
pub = ["git", "push", "--remote", "github", "--bookmark"]
review = ["gerrit", "upload", "--remote", "gerrit", "--remote-branch", "main"]
```

Test aliases in a disposable repository. Configuration parsing errors, command aliases that recursively invoke themselves, and aliases whose revsets return multiple commits are all ordinary failure modes. A useful alias is shorter than the reasoning it makes visible; it should not hide the reasoning.

Configuration reference: the keys that shape daily behaviour

The schema is the authoritative machine-readable inventory, but a human reference must describe which decisions each family controls and what it does not control. The target 0.45.1 schema groups settings as follows; some keys are tables whose contents are repository-specific.

Namespace	Important keys	Decision controlled	Does not control
user	name, email	Author identity defaults.	Gerrit Change-Id or cryptographic identity.
operation	hostname, username	Operation-log attribution.	Commit author/committer.
ui	default-command, color, paginate, pager, editor, diff-editor, merge-editor, conflict-marker-style	Presentation and interactive tools.	Graph semantics and remote ref movement.
fsmonitor	backend, watchman	Filesystem change detection performance.	Which files are logically committed.
colors	label-to-style mappings	Terminal style for semantic labels.	Machine-readable output semantics.
diff	color-words, git, stat	Diff presentation defaults.	Diff ownership.
git	fetch, push, private-commits, sign-on-push, colocate, object-hash	Backend/transport and publication defaults.	Server branch protection and credentials.
remotes	remote tables	Remote URL/name configuration.	Which remote a human intended without an explicit flag.
gerrit	default-remote, default-remote-branch, review-url	Gerrit upload defaults in 0.45.1.	Gerrit server permissions and review identity.
merge	hunk-level, same-change	Merge/conflict heuristics, including individual diff-hunk handling.	Whether a conflict is conceptually acceptable.
revsets	log, fix, run, sign, short-prefixes	Default selections and display policies.	What a command means when an explicit selector is supplied.
snapshot	auto-track, auto-update-stale, max-new-file-size	Filesystem snapshot behaviour.	Files already in a commit's tree.
working-copy	eol-conversion, exec-bit-change	Working-copy representation details.	POSIX file permissions outside jj's snapshot rules.

Namespace	Important keys	Decision controlled	Does not control
<code>fix</code>	<code>tools</code>	Configured fixer programs and file patterns.	Whether a fixer is semantically safe for every commit.
<code>merge-tools</code>	named tool tables	External conflict/diff merge commands.	Correctness of the selected merge result.
<code>templates</code>	log, show, op, bookmark, tag, config, file templates	Human output templates.	Underlying graph/ref state.
<code>aliases</code> , <code>revset-aliases</code> , <code>fileset-aliases</code> , <code>template-aliases</code>	named commands/expressions	Reusable interfaces.	Version compatibility unless explicitly managed.

UI, editors, and pagers

`ui.editor` is used for descriptions and metadata editors; `ui.diff-editor` handles editing at individual diff-hunk granularity; `ui.merge-editor` handles conflict resolution. They are different contracts. A program that can edit a unified diff is not automatically a three-way merge tool. Configure builtin tools first, then add external tools by name in `merge-tools` and test them with a disposable conflict.

```
[ui]
editor = ["nvim", "-f"]
diff-editor = ":builtin"
merge-editor = ":builtin"
pager = ["less", "-FRX"]
paginate = "auto"
color = "auto"
conflict-marker-style = "git"
```

The 0.45.1 documented pager default changes to include `K` in the non-Windows `less` flags. If a printed book or CI log must not contain terminal controls, use `--no-pager` and `--color never` in the command rather than relying on user config.

Git policy and private commits

`git.private-commits` is a revset policy for commits that should not be exported/pushed by default. It is not encryption and does not prevent a user from explicitly selecting a private commit. A private commit can still be part of a local descendant chain and can still be rewritten. Use it to express publication policy, then verify the selected push set.

```
[git]
private-commits = 'description(regex:"(?i)wip|do not publish")'
sign-on-push = true
track-default-bookmark-on-clone = true
```

Do not combine broad private predicates with a script that later uses `--all` and assumes the result is public. Print the push preview. Signing on push can produce a new successor or alter exported metadata according to backend behaviour; test it against the server's verification policy.

Revset defaults

The `revsets` namespace chooses default commit sets for commands such as `log`, `arrange`, `fix`, `run`, `sign`, and `simplify-parents`. Defaults should be readable and bounded. `builtin_log()` is usually a better base than copying a release-specific expansion. A default that uses a private alias or a target-release-specific function must be version-gated.

```
[revsets]
log = '@:: & mutable()'
fix = 'mutable() & ::@'
run = 'mutable() & ::@'
sign = 'mutable() & ::@'
```

Always remember that a default is used only when the user omits the relevant option. It does not override an explicit `-r`. For an operationally important command, include the explicit revset in the script even if the interactive default is convenient.

Fixers and merge tools

Fixer configuration is a table of named tools with command, root, patterns, environment, and working-copy behaviour. A fixer should be deterministic, bounded by a fileset, and tested on conflicted, generated, binary, and sparse trees. Formatters that rewrite every file can accidentally produce enormous successor chains or change line endings.

```
[fix.tools.rustfmt]
command = ["rustfmt", "--edition", "2024", "--emit", "files"]
patterns = ["glob:**/*.rs"]

[merge-tools.meld]
program = "meld"
edit-args = ["$left", "$right", "-o", "$output"]
merge-args = ["$base", "$left", "$right", "-o", "$output"]
```

Use the schema to confirm exact key names and shapes for the installed release. The book's policy is to show the design and test contract, not pretend that every external program uses the same argument protocol.

Conditional configuration

The `--when` conditional configuration table can select repositories, hostnames, workspaces, commands, platforms, and environments. Use it to separate GitHub and Gerrit defaults only when the condition is more reliable than a human's memory:

```
[--when]
repositories = ["~/src/github/**"]
[--when.git]
# repository-specific keys in the supported conditional shape

[--when]
repositories = ["~/src/gerrit/**"]
[--when.gerrit]
default-remote = "gerrit"
default-remote-branch = "main"
```

Check the exact TOML shape in local help/schema; conditional configuration has changed across versions and the illustrative table is not a substitute for that check.

Explicit command flags remain the clearest safety boundary for publication.

Configuration debugging and migration

```
jj config path
jj config list --include-defaults
jj config get revsets.log
jj util config-schema > schema.json
jj config gc
```

When upgrading between installed and future releases, compare schema keys, default templates, default immutable heads, pager defaults, Gerrit keys, and revset functions. Do not run `config gc` as a migration tool without inspecting what it will remove. Keep a copy of the old config and apply changes in one namespace at a time.

Configuration laboratory: construct policy without losing the layers

Configuration is executable policy. A useful `jj` configuration says what an interactive command should do by default, what a script must state explicitly, and which repository boundaries must remain protected. It also has a trust boundary: user configuration is personal, repository configuration may be shared, workspace configuration is local, and command-line configuration is ephemeral. The same key can have a different operational consequence at each layer.

Find the effective value before changing it

```
jj config path
jj config list
jj config get user.name
jj config list --include-defaults
jj config list --include-overridden
```

Use `config get` when a script needs the effective scalar value without the formatting used by `config list`. Use `config list` when precedence and overridden values matter. The exact scope options belong to the installed command surface; in `jj 0.45.1`, the durable scope is chosen by the command's scope flags and an additional file is loaded with `--config-file`. The `config` commands also add

--file PATH targeting for relevant subcommands.

Layer	Best uses	Review question
Built-in	Fallback templates, revsets, pager, safety policy.	Could a release change this default?
User	Name/email, editor, pager, personal aliases.	Would another checkout want this?
Repository	Project aliases, tools, immutable policy, review conventions.	Could a checked-out repository influence an unsafe tool?
Workspace	Local default command, sparse/build preferences.	Does this affect only one working directory?
Command line	CI pinning, explicit remote/output, experiments.	Will the value be visible in process arguments?

Do not store credentials or tokens in aliases or command-line values. A process listing, operation description, shell history, or repository config can expose them. Prefer Git's credential/SSH facilities and the host's agent. `jj`'s role is to select the commit/ref operation and invoke the configured transport.

Progressive Linux layout

```
~/.config/jj/config.toml
~/.config/jj/conf.d/10-identity.toml
~/.config/jj/conf.d/20-ui.toml
~/.config/jj/conf.d/30-revsets.toml
~/.config/jj/conf.d/40-templates.toml
~/.config/jj/conf.d/50-remotes.toml
~/.config/jj/conf.d/60-tools.toml
```

Use `jj config path` to confirm what the build actually reads. XDG paths are a useful Linux convention, not a substitute for checking the installed binary and distribution patches. `jj 0.45.1` supports a configuration directory, so keep its fragments ordered and independently parseable. Do not invent a second loader around that mechanism.

```
[user]
name = "A. Engineer"
email = "a@example.org"
```

```
[ui]
editor = ["nvim", "-f"]
pager = ["less", "-FRX"]
paginate = "auto"
color = "auto"
default-command = ["log", "-r", "(@:: & mutable()) | bookmarks() | remote_bookmarks()"]
```

Start with identity and output, then add selection aliases, then add transport policy. A broken template or alias in the user file can make every repository unusable; add one family at a time and run a disposable command after each change.

Revset and template defaults

```
[revsets]
log = '(@:: & mutable()) | bookmarks() | remote_bookmarks()'
status = 'mutable() & ::@'
fix = 'mutable() & ::@'
run = 'mutable() & ::@'
sign = 'mutable() & ::@'

[revset-aliases]
'stack(tip)' = 'roots(trunk()..tip)::'
'unpublished(tip)' = 'tip ~ ::remote_bookmarks()'
'reviewable(tip)' = 'heads(unpublished(tip) ~ empty() ~ conflicts())'

[template-aliases]
identity = 'change_id.shortest(10) ++ " " ++ commit_id.short(10)'
state = 'separate(" ", if(conflict, "C", ""), if(empty, "E", ""), if(immutable, "I", ""))'
```

A default revset is invoked only when the command's explicit selector is absent. It is not a security boundary: a user can pass another revset. Keep defaults bounded and name aliases after the set they denote. A template alias changes rendering, not selection. If a default includes remote bookmarks, remember that a fetch changes the result without changing local mutable work.

Immutable heads: the safety setting that must actually work

In 0.45.1 the rewrite-protection policy is the revset alias `immutable_heads()`. The correct TOML location is `[revset-aliases]`, with a key whose literal name includes parentheses:

```
[revset-aliases]
"immutable_heads()" = "main@origin | tags() | root()"
```

Verify both the value and an attempted protected rewrite in a disposable repository:

```
jj config get 'revset-aliases."immutable_heads()"'
jj log -r 'immutable_heads()'
jj rebase -s @ -d main@origin
```

The alias controls the set `jj` checks; it is not a label and does not make every remote ref immutable. `--ignore-immutable` disables the check for one operation but does not change the query's value. A repository can therefore be configured with a public boundary that a normal command protects while an emergency command deliberately crosses it. Record why before using that override.

GitHub and Gerrit remote policy

```
[git]
private-commits = 'description(regex:"(?i)wip|do not publish")'

[gerrit]
# jj 0.45.1 documented defaults; pass flags explicitly in mixed-remote repositories.
default-remote = "gerrit"
default-remote-branch = "main"
review-url = "https://gerrit.example.invalid/c/project/+"

[command-aliases]
push-gh = ["git", "push", "--remote", "github", "--bookmark"]
upload-gerrit = ["gerrit", "upload", "--remote", "gerrit", "--remote-branch", "main"]
```

Use separate aliases for separate publication protocols. A default remote is a convenience, not a permission boundary. In `jj 0.45.1`, a portable Gerrit script should pass `--remote` and `--remote-branch` explicitly. The named defaults can reduce repetition, but explicit arguments remain clearer in a repository with multiple publication protocols.

Conditional configuration is useful when a single clone talks to both hosts. `jj 0.45.1` provides the conditional-configuration mechanism described above; use it only when the repository predicate and resulting policy are easy to audit. A conservative design keeps the host choice in the command alias and requires a bookmark

argument. Do not infer “Gerrit repository” solely from the presence of a remote named `origin`, and do not let a GitHub default send a Gerrit review ref.

Editors, merge tools, fixers, and trust

`ui.editor`, `ui.diff-editor`, and `ui.merge-editor` are separate contracts. A diff editor selects changes between two trees; a merge editor resolves conflict sides; a normal editor writes a description or metadata. Configure a builtin tool first, then add an external executable only after testing it in an untrusted/reproducible environment.

```
[ui]
diff-editor = ":builtin"
merge-editor = ":builtin"

[merge-tools.meld]
program = "meld"
edit-args = ["$left", "$right", "-o", "$output"]
merge-args = ["$base", "$left", "$right", "-o", "$output"]

[fix.tools.rustfmt]
command = ["rustfmt", "--edition", "2024", "--emit", "files"]
patterns = ["glob:**/*.rs"]
```

External commands can read repository content, environment variables, credentials available to the process, and files outside the repository. Do not accept a repository's tool definition without review. A fixer also has graph scope: a broad default can rewrite many mutable commits. Bound both commits and files, capture operation output, and test no-op, changed, conflicted, generated, binary, and sparse cases.

Snapshot and workspace policy

Snapshot-related keys control when filesystem state is noticed, which files are tracked, and how large or ignored files are handled. Workspace settings control movement and defaults; sparse patterns control materialisation. These are not interchangeable. A snapshot setting can change the working-copy commit; a sparse setting can make a file absent without removing it from the commit tree; a movement setting can choose whether a navigation command edits a child or creates another working-copy commit.


```
jj config get snapshot.max-new-file-size
jj config get snapshot.auto-track
jj config get ui.movement.edit
jj sparse list
jj workspace list
```

Before changing snapshot policy in a repository with generated files, establish which files are currently tracked and which are ignored. Before changing workspace movement policy, make a two-workspace test and record each workspace's @. Defaults that feel convenient interact with editor and build processes; document the ownership boundary in the repository's developer notes.

Schema validation and migration

`jj util config-schema` is the machine-readable authority for types and available keys in the installed release. It does not replace a human reference: a schema can say that a value is an array without explaining why a list of command arguments is safer than a shell string. Use the schema to detect removed/added keys, then test the semantic contract.

```
jj util config-schema > schema.json
jj config list --include-defaults --include-overridden
jj util config-schema > schema-0.45.1.json
```

Migrate one family at a time. Preserve the old config, run the binary's config parser, inspect effective values, and run a fixture. In particular compare immutable heads, pager flags, remote/tag predicates, Gerrit defaults, aliases, and command defaults between the installed release and future releases. A successful parse is not proof that a key still has the intended meaning; an unknown key can be accepted as data and simply never be read.

Configuration review checklist

1. Identify the layer and trust level.
2. Read the effective value and overridden values.
3. Check whether the key is a default, a language alias, or a command setting.

4. Check the `jj 0.45.1` schema for the key's spelling and type; when preparing a later-release configuration, repeat that check against the later schema.
5. Keep remote, bookmark, and force decisions explicit at publication boundaries.
6. Test editors/tools in a disposable repository.
7. Run a semantic fixture after changing immutable, snapshot, revset, or template policy.

Part XIV — Template language

Jujutsu templates are a small, typed, functional rendering language. They are not format strings with a bag of magic substitutions. A template is parsed, evaluated against a context object, and rendered as labelled output. In `jj log` the context is a `Commit`; in `jj op log` it is an `Operation`. A keyword such as `description` is shorthand for a zero-argument method on that context object.

Target surface. This chapter describes the template help captured from `jj 0.45.1`. Template details are release-sensitive: when building scripts, run `jj help -k templates` for the installed binary and treat a parse error as a compatibility signal. The examples below use only the target-release surface.

1. The evaluation model

There are three stages:

1. **Parsing:** `jj` parses the expression and checks syntax and many type constraints before walking commits.
2. **Evaluation:** the expression is evaluated once for each selected object. A method can return another typed object, a list, an optional value, or a renderable template.
3. **Rendering:** the result is converted to terminal output. Colour labels and hyperlinks are metadata around output, not ordinary characters; `stringify()` deliberately removes those labels.

This explains why a template can fail before printing any commits, why a missing optional value can be tested but not blindly dereferenced, and why parsing JSON with a shell tool is safer when the template emits explicit separators and disables colour.

```
jj log -r '::~@' -T 'change_id.short() ++ " " ++ description.first_line() ++ "\n"'\njj log -r '::~@' --color=never -T 'commit_id ++ "\t" ++ change_id ++ "\t" ++ description.first_line'
```

In the first command the log command supplies one Commit object per selected commit. In the second, full IDs and a JSON-safe description are emitted with tabs. The graph display is independent of -T; add -G when you explicitly want graph characters in a custom log.

2. Literals, shell quoting, and grammar

Use single- or double-quoted string literals. Double quotes support `\\`, `\\\\`, `\\t`, `\\r`, `\\n`, `\\0`, `\\e`, and `\\xHH`. Single-quoted literals have no escape syntax, which makes them convenient for a template containing double quotes and inconvenient for an embedded apostrophe. The shell adds another quoting layer: the outer shell quotes must preserve the inner template exactly.

```
# Shell single quotes; template double-quoted literals\njj log -T 'change_id.short(12) ++ "\t" ++ description.first_line() ++ "\n"'\n\n# Shell double quotes; template single-quoted literals\njj log -T "change_id.short(12) ++ ' ' ++ description.first_line() ++ '\\n'"
```

Templates use method calls (`x.f()`), function calls (`f(x)`), unary operators, arithmetic and comparisons on integers, logical operators, and concatenation. Parenthesise mixed expressions even when the precedence is known; it makes a durable configuration reviewable.

Binding	Forms	Meaning and consequence
strongest	<code>x.f()</code> , <code>f(x)</code>	Method/function call. <code>description.first_line()</code> is evaluated before concatenation.

Binding	Forms	Meaning and consequence
	<code>-x, !x</code>	Integer negation and Boolean negation.
	<code>p:x</code>	String-pattern or pattern-alias syntax where a pattern argument is required.
	<code>* / %</code>	Integer multiplication, division, remainder.
	<code>+ -</code>	Integer addition/subtraction; do not confuse with template concatenation.
	<code>>= > <= <</code>	Integer comparison.
	<code>== !=</code>	Equality for Boolean, byte string, integer, and string-compatible values.
	<code>&&</code>	Short-circuiting Boolean and.
	<code> </code>	Short-circuiting Boolean or.
weakest	<code>x ++ y</code>	Concatenates template output. It is not integer addition.

For example, `if(parents.len() > 1, "merge", "linear")` compares an integer and chooses a string. The equivalent-looking `parents.len() ++ 1` is not integer addition. Write `(parents.len() + 1)` for arithmetic, or stringify the result intentionally.

3. Values, conversions, and optional values

The type system matters at boundaries. A `String`, `ByteString`, `List`, and `Option` can be tested as Boolean. A non-empty string/list and a set option are true; an empty value or unset option is false. A `Commit` cannot itself be printed, but its methods can produce templates, strings, lists, or other objects.

Type family	Can become	Use in practice
Boolean, Integer	Template/Serialize	Conditions, arithmetic, counts, JSON.
String, ByteString	Template, Stringify, Serialize	Descriptions, paths, IDs, JSON-safe output.
List/AnyList	Boolean and often Template	Map/filter/join; empty lists can suppress separators.
Option<T>	Boolean; methods of T when set	Test optional signatures, remotes, keys, roots before use.
Commit, Operation	Serialize but not directly Template	Call methods; use <code>json()</code> where serialisation is intended.

```
jj log -r @ -T 'coalesce(description.first_line(), "(no description)") ++ "\n"'
jj log -r @ -T 'if(signature, signature.status(), "unsigned") ++ "\n"'
```

`try()` catches runtime failures in its alternatives but not parse/type errors.

`coalesce()` chooses the first non-empty rendered value, useful for a missing description or absent bookmark. Use `if(option, option.method(), "fallback")` when the branch itself should be explicit.

4. Global functions

The following functions are the compositional toolkit. A function's argument type is part of its contract; a value that merely prints as text may not be accepted where a parse-time string literal is required.

Function and signature	Semantics	Application and edge case
<code>fill(width, content)</code>	Wraps lines to a display width.	Descriptions in show; control sequences require care.
<code>indent(prefix, content)</code>	Prefixes non-empty lines.	Indent a multi-line description without prefixing blank lines.
<code>pad_start(width, content, [fill])</code>	Left-pads a one-line value.	Align counts/IDs; newlines are invalid.

Function and signature	Semantics	Application and edge case
<code>pad_end(width, content, [fill])</code>	Right-pads a one-line value.	Align subjects; use one display-width fill character.
<code>pad_centered(width, content, [fill])</code>	Pads both sides; odd remainder goes right.	Terminal report headers.
<code>truncate_start(width, content, [ellipsis])</code>	Removes leading characters to fit.	Keep a path suffix; ellipsis appears only after truncation.
<code>truncate_end(width, content, [ellipsis])</code>	Removes trailing characters to fit.	Keep subject prefix; do not use on multi-line content.
<code>hash(content: Stringify)</code>	Returns a hexadecimal digest.	Fingerprint generated output; not a commit identity.
<code>label(label, content)</code>	Applies a space-separated colour/style label.	Mark conflicts or refs; custom labels need configured style.
<code>hyperlink(url, text, [fallback])</code>	Uses OSC 8 when colour is enabled, otherwise fallback/text.	Link commit IDs; terminals and PDFs may discard OSC 8.
<code>raw_escape_sequence(content)</code>	Preserves escape sequences and strips labels.	Terminal control only; visible output breaks wrapping.
<code>replace(pattern, content, lambda)</code>	Replaces matches while preserving untouched formatting.	Lambda receives <code>RegexCaptures</code> .
<code>stringify(content)</code>	Converts to string and removes labels.	Useful before hashing or embedding output.
<code>json(value)</code>	Serialises a supported value as JSON.	Object output; serialised fields are not an eternal API.
<code>if(condition, then, [else])</code>	Conditionally evaluates a branch.	Guard optional values and suppress empty punctuation.
<code>try(expr, fallback...)</code>	Returns the first successful expression.	Catches runtime errors, not static type errors.
<code>coalesce(content...)</code>	Returns first non-empty rendered content.	Defaults for descriptions and refs.
<code>concat(content...)</code>	Equivalent to repeated <code>++</code> .	Readable aliases with many fragments.
<code>join(separator, content...)</code>	Separates every argument.	Use when empty arguments are meaningful.

Function and signature	Semantics	Application and edge case
<code>separate(separator, content...)</code>	Separates non-empty arguments only.	Optional bookmark/tag/signature fields.
<code>surround(prefix, suffix, content)</code>	Wraps non-empty content.	Render parentheses only when content exists.
<code>config(name)</code>	Looks up config as an option.	Extract with <code>as_boolean()</code> , <code>as_integer()</code> , <code>as_string()</code> , or <code>as_string_list()</code> .
<code>git_web_url([remote])</code>	Best-effort remote-to-HTTPS URL.	Defaults to origin; empty on failure; SSH aliases unsupported.

```
jj log -T 'separate(" ",
  if(conflict, label("conflict", "CONFLICT"), ""),
  if(empty, label("empty", "EMPTY"), ""),
  bookmarks, tags, change_id.shortest(8)
) ++ "\n"
```

`separate` is preferable to concatenating spaces inside each branch. The rendered value of `bookmarks` or `tags` can be empty, so output remains stable when a commit has no names.

5. Strings, bytes, and patterns

`String` is UTF-8 text; `ByteString` preserves arbitrary bytes and is used for some repository paths or raw content. Both provide `len`, `contains`, `match`, prefix/suffix tests and removal, trimming, substring, first-line, line splitting, splitting by a string pattern, replacement, upper/lower conversion. String indices are UTF-8 byte offsets, not Unicode scalar indexes. Do not use byte slicing as a human-character counter.

```
jj log -T 'description.first_line().substr(0, 48) ++ "\n"'
jj log -T 'description.lines().filter(|line| line.contains("TODO")).join("\n") ++ "\n"'
jj log -T 'description.replace("API", "Interface") ++ "\n"
```

A bare string pattern literal is case-sensitive substring matching. Explicit pattern kinds make intent clearer. A pattern argument is normally written directly at the call site rather than stored as a dynamic value. `String.escape_json()` is useful for a

field; `json(value)` is useful for a serialisable object.

6. Lists and lambdas

Lists are where templates become a language rather than a pretty-printer. The target surface provides `len()`, `join()`, `filter()`, `map()`, `any()`, `all()`, `first()`, `last()`, `get()`, `reverse()`, `skip()`, and `take()`. Lambda parameters are typed by the list being traversed.

```
jj log -r 'merges()' -T 'parents.map(|p| p.commit_id().short()).join(", ") ++ "\n"'
jj log -T 'trailers.filter(|t| t.key() == "Change-Id").map(|t| t.value()).join(", ") ++ "\n"'
jj log -T 'bookmarks.take(3).join(", ") ++ "\n"'
```

`first()`, `last()`, and `get()` fail on an empty/out-of-range list. Wrap them in `try` when emptiness is normal. `map` returns an `AnyList`; it can usually be joined, but is not a typed list with every original method.

7. Commit context and identity

In a commit template, all zero-argument methods of `Commit` are available as keywords. The following table is the core of a useful daily log.

Keyword/method	Result	Meaning and trap
<code>commit_id</code> , <code>change_id</code>	ID objects	Use <code>.short()</code> , <code>.shortest()</code> , or full rendering. Rewrites change the former, normally not the latter.
<code>description()</code>	String	Usually has a trailing newline when non-blank; use <code>first_line()</code> for one-line output.
<code>trailers()</code>	List<Trailer>	Preserves order and duplicate keys; do not assume one Change-Id.
<code>parents()</code>	List<Commit>	More than one parent means a merge; parent order can matter in display.

Keyword/method	Result	Meaning and trap
<code>author()</code> , <code>committer()</code>	Signature	Author and committer can differ; each has name, email, timestamp.
<code>signature()</code>	<code>Option<CryptographicSignature></code>	Presence is cheap; status/key/display can verify cryptography.
<code>mine()</code>	Boolean	Matches author's email against current user.
<code>working_copies()</code> , <code>current_working_copy()</code>	List/ref and Boolean	Distinguishes any workspace at this commit from this workspace's @.
<code>bookmarks()</code> , <code>local_bookmarks()</code> , <code>remote_bookmarks()</code>	<code>List<CommitRef></code>	Use the specific method when tracked-remote presentation matters.
<code>tags()</code> , <code>local_tags()</code> , <code>remote_tags()</code>	<code>List<CommitRef></code>	Tags have the analogous tracked-remote display rule.
<code>divergent()</code> , <code>hidden()</code> , <code>immutable()</code> , <code>conflict()</code> , <code>empty()</code> , <code>root()</code>	Boolean	State predicates. The template method is <code>conflict()</code> ; the revset function is <code>conflicts()</code> .
<code>change_offset()</code>	<code>Option<Integer></code>	Useful for divergent evolution display; may be absent.
<code>contained_in(revset)</code>	Boolean	Argument must be a parse-time literal such as <code>"mutable()"</code> .
<code>diff([files])</code> , <code>files([files])</code>	<code>TreeDiff/List<TreeEntry></code>	<code>TreeDiff</code> itself is not printable; call <code>summary()</code> , <code>stat()</code> , or <code>git()</code> .
<code>conflicted_files()</code>	<code>List<TreeEntry></code>	Useful for conflict path/count reports.

`CommitId` and `ChangeId` both offer `short([len])` and `shortest([min_len])`. The shortest form returns a `ShortestIdPrefix` with `prefix()` and `rest()`. Change IDs have `normal_hex()` for conventional hexadecimal rather than jj's canonical reversed alphabet.

```
jj log -T 'change_id.shortest(8).prefix() ++
label("dim", change_id.shortest(8).rest()) ++ " " ++
commit_id.short(10) ++ "\n"
```

Short IDs are a display convenience, not an interchange format. A short prefix can become ambiguous after another object enters the repository.

8. References, tracking, and publication state

`CommitRef` provides `name()`, an optional `remote()`, presence/conflict tests, `normal/added/removed` targets, tracking tests and `ahead/behind` hints, and `synced()`. This lets a log distinguish a local publication bookmark from the remote state it tracks:

```
jj log -T 'bookmarks.map(|b|
  separate("",
    if(b.remote(), "@" ++ b.remote(), ""),
    b.name(),
    if(b.conflict(), "!", ""),
    if(b.tracked() && !b.synced(), "*", ""))
  )
).join(" ") ++ "\n"
```

The `!` and `*` are presentation choices; they do not alter refs. A conflicted ref has multiple target sets, so `normal_target()` may be unset and `added_targets()/removed_targets()` are safer forensic fields.

9. Signatures, timestamps, paths, and diffs

A `Signature` has `name()`, `email()`, and `timestamp()`. Email values support `local()` and `domain()`. A timestamp supports `ago()`, `format(strftime)`, `utc()`, `local()`, `after(date)`, `before(date)`, and `since(timestamp)`. A timestamp range has `start()`, `end()`, and `duration()`.

```
jj log -T 'author.name() ++ " <" ++ author.email() ++ "> " ++
  author.timestamp().local().format("%Y-%m-%d %H:%M %Z") ++ "\n"
jj log -T 'committer.timestamp().ago() ++ " " ++
  author.timestamp().since(committer.timestamp()).duration() ++ "\n"
```

For reproducible machine output prefer UTC and an explicit format. Signature verification is separate: testing `signature()` for presence is cheap, but `status()`,

`key()`, and `display()` can invoke GPG or SSH tooling.

`RepoPath` offers `absolute()`, `display()`, and optional `parent()`. `FsPath` offers `absolute()` and `relative()`. Preserve repository-relative paths when output is consumed by another `jj` command; display paths are for humans and can depend on the current directory.

A `TreeDiff` can render `summary()`, a Git-style diff with `git([context])`, a colour-word diff with `color_words([context])`, or statistics with `stat([width], [max_bar_width])`. Its `files()` entries have path, status, status character, source, target, and display path. A `DiffStats` has per-file entries and total added/removed counts.

```
jj show -r @ -T 'description ++
"\nFiles:\n" ++ diff().summary() ++
"\nStat:\n" ++ diff().stat(72, 24) ++ "\n"
```

A template is evaluated for each commit. A large inline diff is consequently expensive. Keep daily logs to summaries and reserve `diff().git()` for `show` or a tightly bounded `reset`.

10. Operation and workspace templates

In `jj op log`, the context is an `Operation`. Its methods are `current_operation()`, `description()`, `id()`, `attributes()`, `time()`, `user()`, `snapshot()`, `workspace_name()`, `root()`, and `parents()`. Operation IDs support `short([len])`. Compose scalar methods; the operation object cannot be printed directly.

```
jj op log -T 'id.short(12) ++ " " ++
if(current_operation, "*" " ") ++
if(snapshot, "snapshot ", "") ++
workspace_name ++ " " ++ description ++ "\n"
```

Keep operation ID, user, workspace, and description visible during recovery. An operation template describes local repository history; it does not synchronise

operation IDs to another clone.

`WorkspaceRef` has `name()`, `target()`, and optional `root()`. The root can be absent for old records or stale/moved paths:

```
jj log -r 'working_copies()' -T 'working_copies.map(|w| w.name() ++ ":" ++ w.root()).join(" ") ++
```

11. Colour labels, styles, and hyperlinks

`jj` automatically attaches labels based on command, context, and method names. Discover exact generated labels with `--color=debug` rather than assuming a stable taxonomy. Use `label("name", content)` for an explicit semantic label, then configure its style in UI colour configuration.

```
jj log -T 'label("bookmarked", bookmarks) ++  
if(conflict, label("conflict", "✖"), "") ++ "\n" ' --color=debug
```

For scripts use `--color=never`. `stringify()` strips labels; `raw_escape_sequence()` bypasses sanitisation and should be limited to terminal control sequences. `hyperlink()` emits OSC 8 only when colour output is enabled. Keep a visible ID fallback:

```
jj log -T 'if(git_web_url(),  
  hyperlink(git_web_url() ++ "/commit/" ++ commit_id, commit_id.short(10)),  
  commit_id.short(10)  
) ++ "\n" '
```

`git_web_url()` is best-effort and returns an empty string if conversion fails. It defaults to `origin` and does not resolve SSH host aliases. Terminals, pagers, PDF output, and machine readers may discard OSC 8 metadata.

12. JSON and machine-readable output

Templates can emit JSON, but JSON validity is the author's responsibility when fields are concatenated manually. Use `json()` for serialisable objects and `escape_json()` for scalar strings. Emit one record per commit with a newline delimiter and disable colour/pager:

```
jj log --color=never --no-pager -r '::@' -T \
'"{\"commit_id\": \" ++ json(commit_id) ++
\", \"change_id\": \" ++ json(change_id) ++
\", \"description\": \" ++ json(description) ++
\", \"empty\": \" ++ json(empty) ++
\", \"conflict\": \" ++ json(conflict) ++ }\"\\n'
```

Use a real JSON parser to consume this output. The serialised representation of underlying `jj` objects can change across releases even when ordinary human fields remain available. Pin `jj` and test fixture output if it feeds a build or publication gate.

13. Template aliases

Aliases are configuration-defined expressions in `[template-aliases]`. They can define keywords, functions, and pattern aliases. Functions can be overloaded by parameter count, but an alias with a builtin name shadows the builtin; avoid such names in shared configuration.

```
[template-aliases]
short-id = 'change_id.short()'
'format-field(key, value)' = 'key ++ ": \" ++ value ++ \"\\n\"'
'json-line' = 'json(commit_id) ++ \"\\n\"'
'pat:x' = 'x ++ \"\\n'
```

Alias names may be exposed to completion with documentation:

```
[template-aliases]
compact-id = { definition = 'change_id.shortest(10)', doc = 'Short unique change ID' }
```

Keep aliases small and composable. A repository's template policy is easier to review when ID, ref-state, subject, and JSON aliases each have one responsibility.

If an alias embeds a revset through `contained_in()` or a value through `config()`, document the required scope and release assumptions beside it.

14. Building a daily advanced log

Build a template from stable fields rather than trying to clone every glyph of the builtin log. First create an unambiguous record:

```
jj log -G -r '@:: & mutable()' -T \  
'change_id.shortest(10) ++ " " ++  
  commit_id.short(10) ++ " " ++  
  description.first_line() ++ "\n"
```

Then add graph-relevant state and publication names:

```
jj log -G -r '(@:: & mutable()) | bookmarks() | remote_bookmarks()' -T \  
'change_id.shortest(10) ++ " " ++ commit_id.short(10) ++  
  " " ++ if(conflict, label("conflict", "CONFLICT "), "") ++  
  if(empty, label("empty", "EMPTY "), "") ++  
  description.first_line() ++ separate(" ", bookmarks, tags) ++ "\n"
```

Finally add author/date only when the output is being used for review:

```
jj log -G -r '(@:: & mutable()) | bookmarks() | remote_bookmarks()' -T \  
'change_id.shortest(10) ++ " " ++  
  if(current_working_copy, "@", " ") ++ " " ++  
  commit_id.short(10) ++ " " ++ author.timestamp().ago() ++  
  " " ++ description.first_line() ++  
  separate(" ", bookmarks, tags) ++ "\n"
```

This is a display template, not a new revset alias. Keep the revset and template independently configurable so a log can change its search domain without changing its fields.

15. Debugging, failure modes, and performance

Symptom	Likely cause	Repair
Parse error before output	Unbalanced quote/parenthesis, unsupported method, or invalid operator.	Reduce to a literal, add one method at a time, and run installed keyword help.
Type error	Arithmetic used where concatenation was intended, or non-printable object emitted.	Use <code>+</code> for integers, <code>++</code> for templates, and call a scalar method.
Inline runtime error	<code>first()/get()</code> on empty list or method on an unset option.	Test with <code>if</code> or use <code>try</code> ; do not hide parse errors with <code>try</code> .
Columns misalign	Newlines, wide Unicode, colour control, or raw escape sequence in padding.	Use one-line values with <code>pad_*</code> ; keep raw escapes invisible.
Pipeline contains escape codes	Colour or OSC 8 enabled.	Pass <code>--color=never --no-pager</code> and explicit delimiters.
Template is slow	Large diffs, signature verification, URL work, or broad revset.	Narrow <code>-r</code> , render summaries, and avoid verification in routine logs.
Works on one machine only	Alias/config absent or release surface differs.	Version-control aliases and run fixture template tests after upgrades.

Debug in layers: `jj log -r @ -T 'commit_id'`, then one method at a time. Test empty descriptions, merge commits, conflicts, unsigned commits, no bookmarks, divergent refs, non-UTF-8 paths, and a commit with multiple workspaces. A large inline diff, cryptographic verification, remote URL conversion, and a broad commit set can each make a template expensive.

16. Design rules

1. Keep graph selection (`-r`) separate from rendering (`-T`).
2. Use full IDs for interchange and short IDs for human scanning.
3. Use `first_line()` for one-line fields; descriptions normally carry a trailing newline.
4. Use `separate` and `surround` for optional punctuation.

5. Guard optional values and empty lists before calling methods that require an element.
6. Prefer summaries to full diffs in a multi-commit log.
7. Disable colour and the pager in automation.
8. Treat template aliases as code: review, test, document, and version them.
9. Do not treat a template hash, short ID, or rendered label as repository identity.

Appendix E repeats the type/method inventory as a lookup reference. The language is documented for jj 0.45.1 and corroborated by the official template documentation. The book paraphrases the rules and uses original examples.

Template laboratory: from a field to a review dashboard

Templates become manageable when developed as typed expressions. Begin with one value, add a separator, then introduce a conditional or list operation. Do not begin with a full replacement for the builtin log: a syntax error in a large template hides which value or method was wrong.

```
jj log -r @ -T 'commit_id ++ "\n"'
jj log -r @ -T 'commit_id.short() ++ " " ++ change_id.short() ++ "\n"'
jj log -r @ -T 'description.first_line() ++ "\n"'
jj log -r @ -T 'if(conflict, "CONFLICT ", "") ++ description.first_line() ++ "\n"'
```

Keywords in a commit template are typed properties. The form `commit_id` yields an ID object; `commit_id.short()` invokes its method and yields a shorter rendering. `description` is a description value with methods such as `first_line()`. A method call is not interchangeable with a global function call: `signature()` is not the right spelling in a commit context when the language exposes `signature` as a keyword. When in doubt, consult the type-specific help and test the smallest expression.

Formatting a record

```
jj log --no-graph -r '@:: & mutable()' -T '  
commit_id.short() ++ "\t" ++  
change_id.short() ++ "\t" ++  
if(empty, "E", "") ++  
if(conflict, "C", "") ++  
if(mutable, "M", "I") ++ "\t" ++  
author.email() ++ "\t" ++  
description.first_line() ++ "\n"  
'
```

This is useful for a terminal report because the output has an explicit delimiter and a stable field order. It is not automatically safe as a TSV format: descriptions and emails can contain tabs or newlines, and labels can introduce escape sequences. For machine consumption, use JSON helpers or an unambiguous encoding policy and test descriptions containing control characters.

Conditional output and optional values

Use `if` for presence and state, not for string concatenation that can be achieved by a list separator. A conditional should make the empty case intentional:

```
if(bookmarks, " [" ++ bookmarks ++ "]", "")  
if(remote_bookmarks, " <" ++ remote_bookmarks ++ ">", "")  
if(conflict, "\n conflict: " ++ conflict, "")
```

Some values are optional or collection-valued. Rendering an optional value directly may yield an empty string, while calling a method that requires a concrete object can fail. The `try`, `coalesce`, and collection functions are useful for controlling this boundary. Test a template on root, merge, conflicted, empty, signed, unsigned, bookmarked, unbookmarked, and working-copy commits; the ordinary linear non-empty case exercises almost none of the interesting type surface.

Lists, lambdas, and separators

Parents, bookmarks, remote bookmarks, and tags are collections. Use list methods and lambda expressions to transform elements, then use `join`, `separate`, or `surround` to render punctuation once:

```
parents.map(|p| p.commit_id().short()).join(", ")
bookmarks.map(|b| b.name()).join(", ")
separate(" ", change_id.short(), author.email(), description.first_line())
surround(" [", "]", bookmarks)
```

The precise element methods depend on the object type. A list of commit objects is not a list of strings; call the ID or name method before joining. This is the same discipline as a typed programming language: if a method is missing, inspect the element type rather than adding arbitrary string conversion until the output happens to look right.

Commit, operation, and workspace contexts

Log templates run with a commit as the current object. Operation templates run with an operation object, whose fields include operation ID, description, timestamp, snapshot and workspace information as documented in the installed template help. Workspace-oriented commands may expose workspace records. A template written for one context cannot assume that commit keywords exist in another.

```
# Commit context
jj log -r 'all()' -T 'commit_id.short() ++ " " ++ description.first_line() ++ "\n"'

# Operation context
jj op log -T 'id.short() ++ " " ++ description ++ "\n"'

# Workspace-aware commit output
jj log -r 'working_copies()' -T 'working_copies.map(|w| w.name()).join(",") ++ "\n"'
```

When designing a monitoring report, choose the context first. A repository health report may need two commands—one for operation history and one for commit state—because forcing both concepts into one template makes the output ambiguous.

Labels, colours, styles, and hyperlinks

Labels are semantic annotations that the UI can style. They are better than embedding raw terminal escape sequences because output policy can then decide whether colour is enabled. Functions such as `label`, `raw_escape_sequence`, and `hyperlink` should be used with awareness of output destinations. A terminal can interpret OSC hyperlinks; a PDF or log file may display the escape bytes or remove them inconsistently.

```
label("change_id", change_id.short())
label("conflict", if(conflict, "CONFLICT", ""))
hyperlink(git_web_url(), commit_id.short())
```

Keep machine output unlabelled and use `--color never` where the command supports it. Use a separate human template for colour and hyperlinks. Labels are also useful in the default log template because the graph renderer and pager can apply consistent styling to bookmarks, remote bookmarks, change IDs, and conflict indicators.

JSON and safe extraction

JSON is a boundary protocol, not a decoration. Quote and escape values with the template's `JSON` function rather than hand-building a JSON string. Emit one complete object per record only if the consuming tool expects newline-delimited JSON; otherwise emit an array with deliberate separators. Test quotes, backslashes, Unicode, empty descriptions, and multiline descriptions.

```
jj log --no-graph -r '@:: & mutable()' -T '
{"commit":' ++ json(commit_id) ++
', "change":' ++ json(change_id) ++
', "subject":' ++ json(description.first_line()) ++ '}' \n
'
```

The exact JSON helper's return type and escaping behaviour are versioned; test it against the binary that will run the automation. Do not parse the ordinary `jj log` graph output in scripts. The graph contains visual layout decisions and can change

with labels, topology, and terminal width.

Aliases and composition

Template aliases let a repository define a named rendering function or reusable fragment. Use aliases to give stable names to concepts such as “review marker”, “compact identity”, or “publication refs”. Keep aliases small enough that an error points to one concept. A large alias that embeds a complete log, revset, and colour policy becomes difficult to override.

```
[template-aliases]
'book-id' = 'change_id.short() ++ " " ++ commit_id.short()'
'review-mark' = 'if(conflict, "C", "") ++ if(empty, "E", "")'
'publication' = 'separate(" ", bookmarks, remote_bookmarks, tags)'
```

When an alias accepts parameters, document the parameter types and call syntax in the repository’s configuration comments. A parameterised alias is a small function, not a textual macro: quoting and type checking still apply. Test aliases with a direct `-T` invocation before making them the default template.

Reconstructing and replacing the default log

The default log is a composition of graph rendering, change identity, description, author/date metadata, ref labels, working-copy state, and conflict/empty markers. Reconstructing it is an educational exercise: first print the commit ID and description, then add change ID, author, refs, parents, state flags, and labels. Compare against `builtin_log()` or the documented builtin template in the target release. Do not copy an internal builtin into a long-lived config without recording the version it came from.

```
jj log -r '@:: & mutable()' -T '
change_id.short() ++ " " ++
commit_id.short() ++ " " ++
if(conflict, "[conflict] ", "") ++
if(empty, "[empty] ", "") ++
description.first_line() ++
```

```
if(bookmarks, " " ++ bookmarks, "") ++  
"\n"  
,
```

A compact daily template should privilege decisions: which logical change is this, what immutable object is checked out, is it conflicted or empty, and what publication names refer to it? A forensic template should add parents, timestamps, signatures, and remote refs. One template cannot optimise simultaneously for a one-line prompt, code review, and audit export.

Template debugging checklist

1. Confirm the command's template context: commit, operation, or workspace.
2. Test the field without methods.
3. Test the method on a commit known to provide the value.
4. Test an empty, merge, conflict, root, and unsigned commit.
5. Add one operator or function at a time.
6. Run with `--no-pager` and colour disabled when comparing output.
7. Use explicit separators and inspect trailing whitespace.
8. Pin aliases and rerun validation after upgrading jj.

A failed template is often valuable evidence about the type model. Treat “unknown keyword”, “method not found”, and “expected one value” as distinct errors. The repair is usually to change context, call a method on the correct object, or map a collection—not to add more quoting blindly.

Template language laboratory: types, evaluation, and stable output

The template language deserves the same discipline as a small typed programming language. A keyword has a type, a method returns another type, operators have domains, and evaluation can fail at runtime for an empty option or list. The fact that a template is written inside a shell command adds another language boundary. Develop it incrementally and keep the output contract visible.

Contexts and values

In a commit template, zero-argument methods on the current `Commit` object are available as keywords. Thus `commit_id` is shorthand for `self.commit_id()`. In an operation template the current object is an `Operation`, so commit keywords such as `author` do not become available merely because the operation caused a commit. This explains many “unknown keyword” errors: the expression is valid in a different context.

Context	Useful values	Typical consumer
Commit	<code>commit_id</code> , <code>change_id</code> , <code>description</code> , <code>parents</code> , <code>signatures</code> , <code>refs</code> , <code>state predicates</code> , <code>diff/tree methods</code> .	<code>jj log</code> , <code>jj show</code> , <code>jj diff</code> .
Commit evolution	Current commit, predecessor list, operation that created it, inter-diff.	<code>jj evolog</code> .
Operation	Operation ID, <code>description</code> , <code>timestamp</code> , <code>user</code> , <code>snapshot flag</code> , <code>workspace</code> , <code>parents</code> .	<code>jj op log</code> , <code>jj op show</code> .
Tree/diff	<code>Paths</code> , <code>status</code> , <code>conflict sides</code> , <code>line/file statistics</code> , <code>formatted diff</code> .	<code>file</code> , <code>diff</code> , <code>annotation</code> .

Start with a context probe:

```
jj log -r @ -T 'self ++ "\n"'
jj log -r @ -T 'commit_id ++ " " ++ change_id ++ "\n"'
jj op log -n 3 -T 'id ++ " " ++ description.first_line() ++ "\n"'
```

If a value is not printable directly, call the method that converts it to a display value. An ID object has `short()` and `shortest()`; a signature has `name()`, `email()`, and `timestamp()`; a list has `map`, `filter`, and `join`. A template that happens to print through implicit conversion is less explicit than one that states the desired representation.

Operators and evaluation order

Method and function calls bind most tightly. Negation and logical not apply to the next expression. Pattern syntax is used in places that expect a string pattern. Integer arithmetic uses `*`, `/`, `%`, `+`, and `-`; comparisons use integer or scalar values; `&&` and `||` short-circuit; `++` concatenates template values. The difference between `+` and `++` is not stylistic: the former requires integers while the latter builds rendered output.

```
1 + 2 * 3
(1 + 2) * 3
if(conflict, "C", "")
commit_id.short() ++ " " ++ description.first_line()
parents.len() == 2 && parents.first().empty()
```

Use parentheses when mixing arithmetic, Boolean tests, and concatenation. Short-circuiting is useful for guards, but it is not a substitute for understanding optional values. A method called on the right side of `&&` is not evaluated when the left side is false; the same expression may fail when the guard is removed.

Strings, bytes, and escaping

The language distinguishes `String`, `ByteString`, and `Template`. A string is textual and has methods such as `trim`, `first_line`, `lines`, `split`, `replace`, `upper`, and `lower`. A byte string has byte-oriented length and slicing, which matters for commit descriptions or paths that are not guaranteed to be valid UTF-8. A template is rendered output and can carry labels. `stringify()` turns a stringifiable value into a string and strips colour labels; `++` preserves template composition.

```
description.first_line().trim()
description.lines().take(3).join(" | ")
description.replace(regex:"\\s+", " ")
description.substr(0, 40)
if(description.contains("WIP"), "[wip] ", "")
```

String literals recognise escaped quote, backslash, tab, carriage return, newline, NUL, escape, and hexadecimal byte forms. The shell parses its own quotes first. For a template containing double quotes, either use a single-quoted shell argument

or escape both layers. A command that works interactively can fail in a shell script when the script changes quoting context.

```
# shell single quotes protect the template's double quotes
jj log --no-pager -T 'description.replace(regex:"\\s+", " ") ++ "\n"'
# a literal newline is a template escape, not a shell line break
jj log -T 'commit_id ++ "\n"'
```

Lists, mapping, and empty collections

Collections are not strings. `parents`, `bookmarks`, `remote_bookmarks`, `tags`, `files`, and `trailers` contain typed elements. `join` can render elements, while `map` first transforms each element. `filter` retains elements satisfying a Boolean lambda; `any` and `all` reduce a list to a Boolean. `first`, `last`, and `get` require a valid element and can fail on an empty list or bad index.

```
parents.map(|p| p.commit_id().short()).join(", ")
bookmarks.map(|b| b.name()).join(", ")
files.filter(|f| f.conflict()).map(|f| f.path().display()).join(", ")
if(parents.len() > 1, "merge", "linear")
if(bookmarks.any(|b| b.remote()), "remote-ref", "local-only")
```

To render optional punctuation, prefer `separate` or `surround`:

```
separate(" ", change_id.short(), author.email(), description.first_line())
surround(" [", "]", bookmarks)
surround(" <", ">", remote_bookmarks)
coalesce(description.first_line(), "(no description)")
```

These functions differ in a way that matters for empty values. `join` puts separators between all supplied content; `separate` omits empty content; `surround` emits both delimiters only for non-empty content; `coalesce` chooses the first non-empty value. Use them to prevent dangling brackets and spaces in root, empty, or unbookmarked commits.

Commit identity, refs, and review state

`commit_id` and `change_id` are different objects. A commit ID's short form is an exact-object display convenience; a change ID's short form represents the logical change. `shortest(n)` asks for a unique prefix with at least a requested length, while `short(n)` truncates to a requested display length. Use full IDs in machine output and `shortest` or `short` IDs only in human output.

```
commit_id
commit_id.short(12)
commit_id.shortest(10)
change_id
change_id.normal_hex()
change_id.shortest(10)
```

Reference collections contain `CommitRef` values. A ref exposes its name, optional remote, presence, conflict status, normal target, added/removed targets, tracking state, ahead/behind counts, and synced state. Rendering the ref object directly is not the same as rendering its name. The following separates local names, remote names, and tracking status:

```
bookmarks.map(|b| b.name()).join(", ")
remote_bookmarks.map(|b| b.remote() ++ "/" ++ b.name()).join(", ")
if(bookmarks.any(|b| b.synced()), "synced", "not-synced")
if(bookmarks.any(|b| b.conflict()), "REF-CONFLICT", "")
```

A conflicted bookmark can have more than one target. Do not call `normal_target()` without guarding or checking `conflict()`. A remote bookmark's presence is not proof that the server is current; it is the local observation produced by `fetch/import`. Templates can display that distinction, but they cannot fetch.

Signatures, timestamps, and diffs

A signature has a name, email, and timestamp. The email object can expose local and domain portions. Timestamps can be rendered as relative age, UTC, local time, or a format string, and can be compared with date-pattern methods. Use UTC for logs that cross machines and relative time for interactive scanning:

```
author.name() ++ " <" ++ author.email() ++ ">"
author.email().domain()
author.timestamp().utc().format("%Y-%m-%dT%H:%M:%SZ")
committer.timestamp().ago()
```

Diff values are structured as well as formatted. A tree diff can produce a summary, Git or colour-word output, statistics, or changed entries. A diff-stat entry carries path, status, status character, bytes, added/removed lines, and display path. Prefer structured statistics in dashboards and formatted diffs only in human review output:

```
diff.stat().files().map(|f|
  f.status_char() ++ " " ++ f.path().display()).join("\n")
diff.stat().total_added() ++ " +" ++ diff.stat().total_removed()
diff.summary()
```

Diff methods may be relatively expensive, especially over a broad commit set. A daily prompt should not compute complete diffs for every visible commit. Build separate “scan”, “review”, and “forensic” templates and use the narrowest one for the task.

Conflicts, signatures, and operations

State methods such as `conflict()`, `empty()`, `immutable()`, `hidden()`, `divergent()`, `mine()`, and `root()` are Booleans. They are excellent for labels but should not be confused with a server's review state. A cryptographic signature is an option; inspect its status, key, and display string rather than merely printing whether an object exists.

```
separate(" ",
  if(conflict, "CONFLICT", ""),
  if(empty, "EMPTY", ""),
  if(immutable, "IMMUTABLE", ""),
  if(divergent, "DIVERGENT", ""),
  if(signature, "SIGNED", "UNSIGNED"))
if(signature, signature.status() ++ " " ++ signature.key(), "")
```

Operation templates have a different identity. An operation ID identifies a repository transformation in this clone. Its description, user, time, snapshot flag, workspace name, and parent operations answer questions that cannot be answered by a

commit template:

```
jj op log -T '  
id.short(12) ++ " " ++  
if(snapshot, "snapshot ", "") ++  
workspace_name ++ " " ++ description.first_line() ++ "\n"
```

Keep the plus signs out of the actual command when copying this example; they are line-continuation markers in the printed template source only. A production template should be written as one valid expression, for example:

```
jj op log -T 'id.short(12) ++ " " ++ if(snapshot, "snapshot ", "") ++ workspace_name ++ " " ++ de
```

Formatting, labels, and terminal boundaries

`fill` wraps lines, `indent` prefixes non-empty lines, padding functions align single-line values, and truncation functions control width with an optional ellipsis. Padding counts display width rather than merely source bytes where the formatter can determine it; colour labels and raw escape sequences must not be treated as visible text by a custom terminal sequence.

```
pad_start(12, change_id.short())  
pad_end(14, author.name())  
truncate_end(60, description.first_line(), "...")  
indent(" ", diff.summary())  
fill(72, description)
```

Labels are semantic styling. `label("conflict", "CONFLICT")` allows the colour configuration to decide presentation, while raw escape sequences bypass sanitisation and can corrupt a file or PDF. Hyperlinks use OSC 8 when colour output is enabled, but a pager, terminal, log file, or PDF may not support them. Use a plain fallback for automation:

```
label("change_id", change_id.short())  
hyperlink(git_web_url(), commit_id.short(), commit_id.short())
```

Do not put raw escape sequences in JSON or TSV output. Use `--color=never` and `--no-pager` for machine consumers. A template can be syntactically correct

yet unsafe for a downstream parser because a label or hyperlink introduced control bytes.

JSON and aliases

`json(value)` serialises a serialisable value; `String.escape_json()` escapes a scalar string. Use one complete JSON object per output record and test quotes, backslashes, Unicode, empty values, and newlines. Do not parse ordinary graph output as JSON-like text.

```
jj log --no-graph --color=never -r '@:: & mutable()' -T '  
{"commit\":" ++ json(commit_id) ++  
", "change\":" ++ json(change_id) ++  
", "subject\":" ++ json(description.first_line()) ++  
"}\n'
```

The printed example is JSONL: each commit emits one object and one newline. If the consuming interface expects a JSON array, the template must deliberately emit brackets and separators; JSONL is usually simpler for streams. Pin the `jj` release when this is an API, because serialisation of object values is a versioned language surface.

Template aliases belong under `[template-aliases]`. Keep them small and typed by convention:

```
[template-aliases]  
identity = 'change_id.shortest(10) ++ " " ++ commit_id.short(10)'  
state = 'separate(" ", if(conflict, "C", ""), if(empty, "E", ""), if(immutable, "I", ""))'  
refs = 'separate(" ", bookmarks, remote_bookmarks, tags)'  
'field(name, value)' = 'name ++ "=" ++ value ++ "\n''
```

An alias name can shadow a builtin, so use a project prefix for shared configuration when necessary. A parameterised alias is not textual shell interpolation; its arguments are parsed as template expressions. Test it directly with `-T` before making it a default. The reference appendix lists the target-version methods; the laboratory approach explains how to combine them without losing type or output discipline.

Part XV — Exhaustive CLI reference for jj 0.45.1

The jj 0.45.1 command tree contains 47 top-level commands and 121 public command paths including subcommands. This section is the reader-facing reference: it presents the important syntax, options, cardinality, state effects, and traps. The entries are deliberately interpreted rather than presented as a raw help dump.

Top-level commands

Family	Commands	Mutation/notes
History	<code>abandon</code> <code>absorb</code> <code>arrange</code> <code>bisect</code> <code>commit</code> <code>describe</code> <code>diffedit</code> <code>duplicate</code> <code>edit</code> <code>interdiff</code> <code>metaedit</code> <code>new</code> <code>parallelize</code> <code>prev</code> <code>next</code> <code>rebase</code> <code>restore</code> <code>revert</code> <code>run</code> <code>show</code> <code>simplify-parents</code> <code>split</code> <code>squash</code>	Most snapshot first; rewrite effects propagate to descendants.
Inspection	<code>diff</code> <code>evolog</code> <code>log</code> <code>status</code> <code>root</code> <code>version</code>	Read commands still snapshot unless <code>--ignore-working-copy</code> .
Refs	<code>bookmark</code> <code>tag</code>	Names and publication relationships, not local branch-centric development.
Git	<code>git</code> , <code>gerrit</code>	Transport, ref import/export, colocation, and review upload.
Repository state	<code>operation</code> <code>undo</code> <code>redo</code>	Operation DAG recovery and concurrent-operation control.
Workspace/content	<code>workspace</code> <code>sparse</code> <code>file</code> <code>fix</code> <code>resolve</code> <code>sign</code> <code>unsign</code>	Filesystem views, conflict tools, fixers, signing, and file metadata.
Configuration/tools	<code>config</code> <code>help</code> <code>util</code>	Configuration layers, help keywords, completion, schema, GC, snapshot.

Command entries and traps

Command path	Purpose	Key arguments/options and trap
<code>abandon</code>	Abandon commits.	Accepts a set; abandoning @ creates a new empty working copy.
<code>absorb</code>	Distribute changes into mutable ancestors.	Review the chosen stack and fileset; content may move farther than one parent.
<code>arrange</code>	TUI reorder/abandon.	Interactive graph surgery; inspect operation log after leaving.
<code>bisect</code> , <code>bisect run</code>	Automated bisection.	Script exit status and commit range are the contract; keep tests deterministic.
<code>bookmark</code> and subcommands	Manage names and tracking.	<code>advance</code> <code>create</code> <code>delete</code> <code>forget</code> <code>list</code> <code>move</code> <code>rename</code> <code>set</code> <code>track</code> <code>untrack</code> ; tracking is not auto-follow.
<code>commit</code>	Describe current change and create child.	Snapshot occurs first; unlike Git, @ already is a commit.
<code>config</code> and subcommands	Read/edit layered TOML.	<code>edit</code> <code>gc</code> <code>get</code> <code>list</code> <code>path</code> <code>set</code> <code>unset</code> ; inspect scope and precedence.
<code>describe</code>	Change description/metadata.	Exactly one commit; no new child unless combined with workflow.
<code>diff</code> , <code>interdiff</code>	Tree or diff comparison.	Commit range and fileset are independent; <code>interdiff</code> compares diffs.
<code>diffedit</code>	Touch up content.	External diff editor and fileset options are config-sensitive.
<code>duplicate</code>	Copy content to new changes.	New change identity; it is not a successor.
<code>edit</code>	Move @ to one commit.	Exactly one; updates filesystem and may snapshot old @ first.
<code>evolog</code>	Show change evolution.	Use for predecessors, successors, and divergence.
<code>file</code> <code>annotate</code> / <code>chmod</code> / <code>list</code> / <code>search</code> / <code>show</code> / <code>track</code> / <code>untrack</code>	File operations.	Search pattern defaults/options changed historically; quote filesets and patterns.

Command path	Purpose	Key arguments/options and trap
<code>fix</code>	Run configured fixers.	May rewrite many changes; line-range support is release-sensitive.
<code>gerrit upload</code>	Upload changes/stacks.	Verify 0.45.1 server/protocol options; jj change ID is not Gerrit Change-Id.
<code>git clone/init</code>	Create Git-backed repositories.	Choose <code>--colocate</code> deliberately.
<code>git colocation enable/disable/status</code>	Change backend layout.	Native Git tools may stop seeing a worktree after disable.
<code>git fetch/import/export/push</code>	Synchronise Git objects/refs.	Fetch imports; push publishes refs. Remote names and bookmark safety matter.
<code>git remote add/list/remove/rename/set-url</code>	Manage remotes.	Qualify same-name bookmarks by remote.
<code>git root</code>	Show Git root.	Useful for colocated scripts; <code>jj root</code> shows workspace root.
<code>log, show, status</code>	Inspect graph/diff/state.	Default revsets/templates are config; stable scripts specify <code>-r</code> , <code>-T</code> , and colour.
<code>metaedit</code>	Rewrite metadata only.	Commit ID changes; content does not.
<code>new</code>	Create child/merge.	Multiple parents create a merge; <code>@</code> normally moves.
<code>operation diff/integrate/log/show/restore/abandon/revert</code>	Inspect/manage operation DAG.	Operation IDs are local; restore affects repository state, not remote servers.
<code>parallelize</code>	Make commits siblings.	Graph-changing rewrite; descendants may rebase.
<code>rebase</code>	Move selected commits.	<code>-r -s -b -o -A -B</code> ; selection and destination are separate dimensions.
<code>resolve</code>	Invoke external merge tool.	Stored conflicts may be multi-sided.
<code>restore, revert</code>	Copy content; apply inverse.	Neither is a ref reset; distinguish tree copy from inverse diff.

Command path	Purpose	Key arguments/options and trap
<code>run</code>	Run a command across commits.	Each selected change gets a private working copy; effects may be one operation.
<code>sign/unsign</code>	Manage cryptographic signatures.	Signing metadata and Git server verification are separate policies.
<code>sparse edit/list/reset/set</code>	Manage present paths.	Changes materialisation only; commit trees remain complete.
<code>split</code>	Divide a change.	Interactive/fileset selection; successors and descendants are rewritten.
<code>squash</code>	Move selected content into destination.	Destination and fileset determine what remains in source.
<code>tag</code> <code>delete/list/set/track/untrack</code>	Manage tag names.	Tags are not moving review bookmarks.
<code>undo/redo</code>	Reverse/reapply operations.	Not a Git reset; inspect operation log when state is surprising.
<code>util</code> <code>backend/config-schema/completion/exec/gc/install-man-pages/markdown</code>	Low-level utilities.	Some are machine/admin oriented; <code>debug snapshot:history</code> is release-sensitive.
<code>workspace</code> <code>add/forget/list/rename/root/update-stale</code>	Multiple working directories.	@ is workspace-local; commit graph is shared.

How to read the command entries

Each entry below is organised around semantics rather than a transcription of help output. “Snapshot” means the normal command boundary at which jj records the current filesystem in the workspace commit. “Rewrite” means new successor commits may be created and descendants may be rebased. “One commit” means a command requires a single resolved commit; a revset can still be syntactically valid and operationally invalid if it returns zero or several commits.

Short options are retained because they are useful at a terminal, but durable aliases and documentation should prefer the long option when it makes the dimension

explicit. Global options apply to every entry: `-R/--repository`, `--ignore-working-copy`, `--no-integrate-operation`, `--ignore-immutable`, `--at-operation/--at-op`, `--debug`, `--color`, `--quiet`, `--no-pager`, repeatable `--config` and `--config-file`, and `help/version` options. Their interaction is explained after the command entries.

Revision arguments: positional aliases versus real options

The spelling `-r` does not identify a special kind of revset. It has two different command-line roles in the 0.45.1 CLI, and the command's help tells you which role applies.

For some commands, a revision is the command's primary subject and the positional argument has `-r` as an alias. These forms are equivalent:

```
jj show C
jj show -r C

jj describe C
jj describe -r C

jj edit C
jj edit -r C

jj abandon C
jj abandon -r C
```

In this case `-r` is not adding another selection dimension. It is simply an alternate spelling of the positional REVSET. The help expresses this with wording such as `[aliases: -r]` beside the positional argument.

Other commands have a genuine revision-selection option because their positional arguments are filesets, or because they need several independently named revision roles:

```
jj log -r 'mutable() & ::@' -- 'glob:src/**'
jj diff -r 'C:D' -- 'root-glob:src/**'
jj rebase -r C --onto B
jj rebase --source C --onto B
```

```
jj squash -r C
jj squash --from C --into B
```

These are not all interchangeable. In `jj log` and `jj diff`, `-r` selects the commits to display or compare while the positional arguments select paths. In `jj rebase`, `-r/--revisions` means “rebase exactly this selected set”; `-s/--source` includes the selected commits' descendants, and `-b/--branch` selects the branch-shaped set defined by the command. In `jj squash`, `-r` selects the change whose content is moved into its parent, whereas `--from` and `--into` name the source and destination independently.

The practical rule is to read the argument heading and the option description together. A positional `REVSET` is the command's primary subject; an option-valued `REVSET` occupies a named role such as displayed revisions, source, destination, or parent. Both values are parsed by the same revset language, but the command determines their cardinality, default, and effect on the graph.

Alphabetical command reference entries

These reader-facing entries are ordered by canonical command path. The order is deliberately independent of the conceptual grouping used in the teaching chapters; use the browser's find function or the printed contents to reach a command, then follow the related-command links for semantics and workflows.

`jj abandon`

Purpose and model. Hides the selected commits from the ordinary visible graph and reconnects their children to the abandoned commits' parents. It is a graph operation, not deletion of immutable Git objects. The default commit is `@`; the argument is a revset and may select several commits.

Syntax/options. `jj abandon [REVSETS...]`; `--retain-bookmarks` keeps bookmarks by moving them to parent commits, and `--restore-descendants` preserves children's content rather than preserving their diff relative to the changed

parents. The latter can prevent a surprising descendant rewrite when the purpose is to discard an ancestor's content.

State effects. `jj` snapshots first, creates an operation, and may create a new empty working-copy commit if the current `@` is abandoned. Descendants normally change because their parent edge and diff context change; conflicts can be produced. Bookmarks pointing at abandoned commits are normally deleted or handled according to the command's policy unless retained. Native Git refs are not the unit of selection, but exported bookmark movement can later affect Git.

```
jj log -r '@:: & mutable()'
jj abandon 'change_id(abc...)'
jj op log
jj evolog -r 'change_id(abc...)'
```

Trap and recovery. Do not use `abandon` as a private equivalent of “reset branch”: it changes visible graph topology and can rewrite descendants. Inspect the operation immediately; `jj undo`, `jj op restore`, or `jj evolog` can recover the intended change. Immutable commits require `--ignore-immutable`, which should be treated as an exceptional audit event.

`jj absorb`

Purpose and model. Distributes the current change's file-level modifications into suitable mutable ancestors. It is an inference-driven stack operation: `jj` compares the source diff with ancestor changes and moves an individual diff hunk to the nearest compatible destination, rather than blindly squashing all content into the parent.

Syntax/options. `jj absorb [FILESETS...]`; `-f/--from REVSET` selects the source (default `@`), `-t/--into REVSETS` bounds candidate destinations (default `mutable()`, only ancestors of the source), `-i/--interactive` chooses parts, and `--tool NAME` selects the diff editor and implies interactive mode.

State effects. The source and one or more mutable ancestors can be rewritten; descendants of every changed ancestor are rebased. The source may become

empty or retain changes that could not be attributed. Change IDs normally follow their logical evolutions; commit IDs change for every rewritten node. A fileset restricts candidate content, not the commit domain.

```
jj absorb --from @ --into '@:: & mutable()'
jj absorb --interactive -- 'glob:src/**'
jj log -r '@:: & mutable() & conflicts()'
```

Trap. “Earliest compatible” is a semantic choice, not a promise that every individual diff hunk goes to the immediate parent. Review the diff and evolution log. Use `jj squash` when the intended destination is explicit, and `jj undo` if attribution was wrong.

jj arrange

Purpose and model. Opens the interactive graph arrangement interface for the selected commits. It is intended for visual reordering and abandoning of a graph region; it is not a text editor for commit messages and not a replacement for understanding the resulting parent edges.

Syntax/options. `jj arrange [REVSETS...]`; the default selection comes from `revsets.arrange` if no commit is supplied, and `-r` is the commit alias. There are no command-specific rewrite switches in 0.45.1. Global repository, immutability, colour, and operation-view options still apply.

State effects. The TUI can create several graph transformations in one operation. Mutable commits and descendants may receive new commit IDs; bookmarks can move or become conflicted according to the resulting graph. Snapshotting occurs at the normal command boundary. Review `jj op show` after leaving the interface because the visual action is otherwise hard to audit.

```
jj arrange -r '@:: & mutable()'
jj op show
jj log -r 'visible_heads() | bookmarks()'
```

Trap. Use a narrow revset. A broad arrangement selection can expose immutable boundaries or make the intended graph difficult to recognise. For repeatable automation use `rebase`, `parallelize`, `new --insert-after`, or explicit bookmark operations.

`jj bisect` and `jj bisect run`

Purpose and model. The parent command is a namespace for commit bisection and requires a subcommand in 0.45.1. Bisection searches a commit range using a user-defined test rather than moving a Git branch pointer. Candidate checkouts are temporary working copies; the result is a graph commit selected by bisection state and test outcomes.

Syntax/options. `jj bisect run --range REVSET [COMMAND] [ARGS...]`. The range is required; arguments beginning with a dash should follow `--`. The run subcommand accepts normal global repository/view options.

State effects. A zero/non-zero test status has the meaning defined by the local bisection help and script contract. Make the predicate deterministic and independent of the developer's primary filesystem. Do not assume a candidate checkout is the current `@` or that a discovered commit should automatically become a bookmark.

```
jj bisect run --range 'good..bad' -- ./test-regression.sh
jj log -r 'bisect(@)'
```

Trap. A test that mutates files, depends on network state, or returns a status for an unrelated failure invalidates the search. Record the range and command; inspect the result with `jj show` and `jj diff` before applying a fix.

`jj bisect run`

Runs a test command over candidate commits in the required `--range REVSET`. Put command arguments after `--` when they begin with a dash. The command's exit status is the test result; keep it deterministic and independent of the primary workspace.

Selection and lifecycle. The range is a revset describing the known-good/known-bad corridor; it is not a bookmark movement and does not mean “all commits between two Git branches” unless the revset says so. Each candidate is materialised for the test, then the bisection state records the result. Make the script distinguish a product failure from an unavailable dependency or infrastructure failure; otherwise the search can label a candidate incorrectly.

```
jj bisect run --range 'good..bad' -- ./test-regression.sh
```

Candidate working copies are temporary and are not publication bookmarks. The result does not change the meaning of `@` in the primary workspace. Inspect the discovered commit with `jj show`, verify its full ID and diff, then create a bookmark or new change only if that is the intended next operation. If the predicate was wrong, rerun with a corrected range rather than trying to repair the graph with `restore`.

`jj bookmark`

Purpose and model. Names a commit for local navigation or publication and manages relationships with remote bookmarks. The parent command requires a subcommand; its aliases include `b`. A bookmark is a ref-like name over the commit DAG, not the unit of ordinary local work.

Subcommands. 0.45.1 provides `advance`, `create`, `delete`, `forget`, `list`, `move`, `rename`, `set`, `track`, and `untrack`. Each is described below. Names usually accept glob-like string patterns; commit options accept revsets with the cardinality required by the operation.

State effects. Bookmark operations change local ref state and create operations, but do not rewrite commit content. A later Git push exports the selected local bookmark; tracking is a relationship and does not make a bookmark follow @.

```
jj bookmark list --all-remotes
jj bookmark create review -r @
jj bookmark set review -r @ --allow-backwards
jj bookmark track review --remote origin
```

Trap. Always qualify same-name remote refs in review scripts and specify the remote on publication. “Bookmark changed” and “commit changed” are different facts.

jj bookmark advance

Moves matching bookmarks forward to a target selected by `-t/--to REVSET`, defaulting to `revsets.bookmark-advance-to`. Names are patterns and can match several bookmarks. It is intended for advancing bookmarks along descendant history, not for arbitrary sideways replacement.

```
jj bookmark advance --to @ review
jj bookmark advance --to 'main@origin::@' 'pr/*'
```

It changes ref state only; selected commits are not rewritten. If the target or bookmark set is ambiguous, narrow the name/revset. Use `bookmark set` when an intentional backwards or sideways move is required and review its safety option.

jj bookmark create

Creates one or more local bookmarks. `-r/--revision REVSET` (alias `--to`) supplies the target and defaults to `@`. The target must resolve as required by the command; creating several names at one target is useful when a server policy needs aliases.

```
jj bookmark create github/pr-42 -r @
jj bookmark create base feature -r A
```

Creation does not move @, rewrite commits, or create tracking by itself. A later push may automatically establish tracking for an untracked bookmark. Avoid creating a publication name before the graph boundary is intentional.

jj bookmark delete

Deletes local bookmarks matching the supplied name patterns. It changes local ref state; it does not erase the target commits. A tracked deletion can be exported as a remote deletion by an appropriate push.

```
jj bookmark delete 'github/pr-*'  
jj bookmark delete review
```

Use `forget` when the goal is to discard local knowledge rather than express a deletion to the remote. Check tracking and server state before deleting a publication bookmark. If descendants or another bookmark still identify the commits, deleting this name changes neither their content nor reachability through those other names.

jj bookmark forget

Forgets local bookmark and, optionally, remote-bookmark knowledge.

`--include-remotes` also forgets corresponding remote bookmarks; a future fetch recreates still-existing remote refs. Forgetting is local bookkeeping and is not a remote deletion request.

```
jj bookmark forget review  
jj bookmark forget --include-remotes 'old/*'
```

Use this after a merged or closed review when the remote state should remain untouched. If the server ref must be deleted, use `bookmark delete` and push the deletion. Forgetting a remote observation also removes local comparison evidence until the next fetch; it is not a privacy or server-cleanup operation.

jj bookmark list

Displays local and selected remote bookmark state. Names are optional patterns. `-a/--all-remotes` includes synchronized remote bookmarks; `--remote REMOTE` limits a remote; `--tracked`, `--conflicted`, `-r/--revision REVSETS`, `-T/--template`, and output filters provide the important inspection dimensions in 0.45.1.

```
jj bookmark list
jj bookmark list --all-remotes
jj bookmark list --remote origin --tracked
jj bookmark list -T 'name ++ " " ++ if(synced, "synced", "changed") ++ "\n"'
```

Listing does not move refs but normally snapshots unless ignored. The default view can omit synchronized remote targets; use `--all-remotes` when auditing all server knowledge. A conflicted bookmark needs its target sets inspected before push.

jj bookmark move

Moves existing bookmarks to a target. Names are patterns, or `-f/--from REVSETS` selects bookmarks whose targets are in a commit set; `-t/--to REVSET` supplies the destination and defaults to `@`. It is a ref move, not a commit rewrite.

```
jj bookmark move review --to @
jj bookmark move --from 'heads(mutable())' --to @
jj bookmark move 'pr/*' --to 'main@origin::@'
```

Moving sideways or backwards may require the command's safety confirmation. Review local/remote divergence after a move; a tracked remote bookmark is not changed until push. The `--from` form selects bookmarks by their current targets, so print the matched names before using a broad pattern.

jj bookmark rename

Renames one local bookmark from OLD to NEW. `--overwrite-existing` permits replacing an existing name where the command's safety policy allows it. Target commit and content are unchanged; remote publication may need a new push/delete sequence.

```
jj bookmark rename review github/pr-42
jj bookmark rename --overwrite-existing old current
```

Renaming is not the same as creating a second name: tracking relationships and remote names need explicit review. Use `list` before and after.

jj bookmark set

Creates or updates bookmarks to a target. `-r/--revision REVSET` (alias `--to`) defaults to `@`; `-B/--allow-backwards` permits backwards or sideways movement that is otherwise protected.

```
jj bookmark set review -r @
jj bookmark set --allow-backwards review -r 'main@origin'
```

Set is convenient for a publication boundary but can hide whether a name was new or already existed. For an auditable existing-name move use `bookmark move`. Neither operation changes the commit DAG. In jj 0.45.1, `--allow-backwards` is a ref-safety override for the bookmark operation: it does not make the resulting commit immutable or force a remote update.

jj bookmark track

Creates a relationship between a local bookmark and a remote bookmark. Arguments can be bookmark patterns with `--remote REMOTE` or exact `BOOKMARK@REMOTE` symbols. Tracking does not move either target and does not make the local bookmark follow `@`.

```
jj bookmark track review --remote origin
jj bookmark track main@origin
```

```
jj bookmark list --remote origin --tracked
```

Tracking supplies comparison and push safety information: ahead, behind, synced, and expected remote target. It is especially important when a repository has GitHub and Gerrit remotes with the same branch names. The local name and remote name need not have been created by the same command, and tracking does not cause automatic movement when a rewrite changes the commit at the local tip.

jj bookmark untrack

Removes tracking relationships from local bookmark/remote bookmark pairs. It accepts the same pattern/remote forms as `track`, including `BOOKMARK@REMOTE`.

```
jj bookmark untrack review --remote origin
jj bookmark untrack main@github
```

Untracking is local metadata; it neither deletes the remote bookmark nor moves the local one. After untracking, a push may treat the name as a new publication, so inspect the push preview and specify the remote. Re-track only after confirming that the local and remote names refer to the intended pair; same-name bookmarks on another remote are independent.

jj commit

Purpose and model. Describes the current working-copy change and creates a new empty child on top of it. In `jj`, the current filesystem is already associated with `@`; `commit` is a boundary in the local patch stack, not the act of transferring files into a hidden index.

Syntax/options. `jj commit [FILESETS...]`; a fileset commits only selected paths, `-i/--interactive` selects diff hunks, `--tool NAME` chooses the diff editor and implies interactive mode, `-m/--message` supplies a description, and `--editor` opens the editor even after a message was supplied. Global snapshot controls apply.

State effects. `jj` snapshots first, updates `@` with selected content and metadata, then creates a new empty child working-copy commit. Unselected filesystem changes remain in the new child. The old commit's change identity is preserved by the description operation; the new child receives a new change ID. A bookmark at the old `@` does not automatically follow.

```
jj commit -m 'API: validate quota'
jj commit --interactive -- 'glob:src/**'
jj describe -m 'API: split validation'
jj status
```

Trap. Do not use `jj commit` to save an arbitrary ancestor: use `jj edit`, `jj describe`, or `jj squash` according to intent. An accidental child is recoverable with `jj undo`.

`jj config`

Purpose and model. Reads and writes layered TOML configuration. The parent namespace provides `edit`, `gc`, `get`, `list`, `path`, `set`, and `unset`. Scope is part of the command contract: user, repository, and workspace files are different state locations.

Safety. Config can invoke editors, external tools, aliases, fixers, merge tools, and transport behaviour. Treat repository/workspace config as input, not trusted code. Inspect effective values with `config list` before running a command from an unfamiliar checkout.

```
jj config list
jj config get ui.pager
jj config set --user user.name 'A. Engineer'
jj config path --repo
```

Configuration changes create operations only when they affect repository state according to the command; they do not rewrite commits by themselves. The full Linux design and grouped key reference appear in Part XIII and Appendix F.

jj config edit

Opens a selected configuration file in the configured editor. Exactly one of `--user`, `--repo`, or `--workspace` selects the scope in 0.45.1. The editor is an external side effect; `--config-file` can add input but is not the same as choosing the file to edit.

```
jj config edit --user
jj config edit --repo
jj config edit --workspace
```

Use this for multi-line templates, aliases, and tool definitions where shell/TOML quoting is risky. In 0.45.1, `--file PATH` is available for selecting a specific file; mark that as target-release-specific in shared instructions.

jj config gc

Garbage-collects obsolete configuration files or fragments according to the target release's config storage policy. It is maintenance, not a commit operation, and should be run only after understanding which generated/config files are no longer referenced.

```
jj config gc
```

Do not confuse config GC with repository object GC. Version-control important config and take a backup before maintenance on a non-standard layout.

jj config get

Reads one named configuration value. `jj config get NAME` reports the effective value after layers are applied; use scope options from local help when you need a particular layer. It is useful in shell guards because it tests the configuration `jj` itself will use.

```
jj config get revsets.log
jj config get git.push
jj config get gerrit.default-remote
```

Absent values and typed TOML values are different from empty strings. Use `jj config list` or `jj config path` to explain surprising results.

`jj config list`

Lists effective configuration, optionally restricted to a name/prefix. It is the best first diagnostic for precedence, aliases, templates, remotes, and tool definitions.

```
jj config list
jj config list ui
jj config list revset-aliases
```

Inspect before executing repo-provided aliases or external tools. Values can come from several files; use `path/edit` commands to locate and change the intended layer.

`jj config path`

Prints the path of a selected config scope. Exactly one scope flag is required in the 0.45.1 surface: `--user`, `--repo`, or `--workspace`.

```
jj config path --user
jj config path --repo
jj config path --workspace
```

Use it in diagnostics and backup scripts rather than assuming an XDG path. Repository and workspace config are intentionally separated from the repository in newer layouts to reduce accidental execution of untrusted checked-in configuration.

`jj config set`

Sets one TOML value in a selected scope. Syntax is `jj config set --user | --repo | --workspace NAME VALUE`. The value is a TOML expression; strings, arrays, booleans, integers, and multi-line strings therefore need both TOML and shell quoting.

```
jj config set --user ui.pager 'less -FRX'
jj config set --repo revsets.log '(@:: & mutable()) | bookmarks()'
jj config set --workspace ui.default-command '["log", "-r", "@:: & mutable()"]'
```

Use `config edit` for complex values. In 0.45.1, `--file PATH` can target a specific config file. Do not assume a string without quotes has the desired TOML type.

`jj config unset`

Removes a named value from a selected user/repository/workspace layer. It exposes lower-precedence values again; it does not necessarily mean the effective value becomes absent.

```
jj config unset --user ui.pager
jj config unset --repo revsets.log
```

Check `config get` before and after. In 0.45.1, `--file PATH` can target a specific config file; mark scripts using it as version-specific.

`jj converge 0.45.1`

Purpose and model. Groups visible commits by change ID in a selected search space and reconciles the multiple visible revisions of one logical change into one replacement revision. Divergence usually means that the same change was rewritten independently in two operation histories, so `jj` can see, for example, `X1` and `X2` with the same change ID but different commit IDs. Convergence is therefore not a lateral squash: it does not move the diff owned by one independent change into another change. It resolves competing evolutions of the same change, normally preserving that change's identity while choosing or combining their content, descriptions, parents, and other metadata.

Syntax/options. Use `jj converge -r REVSET`; the default search space is controlled by the `jj 0.45.1` revset configuration. `--no-interactive` suppresses prompts and asks the heuristic convergence to proceed without interactive decisions. Check the target local help for the complete option surface.

State effects. The selected divergent revisions are replaced by one new revision for that change; if other visible revisions for the same change lie outside the revset,

those remain divergent. The solution may have file conflicts even when the inputs were conflict-free. Descendants of every replaced revision are rebased onto the solution, and local bookmarks pointing at those revisions are moved to it. This is not a remote merge and does not merge Gerrit Change-Ids or GitHub pull requests. Review `jj op show -p`, `jj evolog`, the resulting diff, and `conflicts()`; use `jj undo` if the selected convergence was not intended.

```
jj converge -r 'divergent()'
jj converge --no-interactive -r 'mutable()'
jj op show
jj evolog
```

before:	after:
P--X1--D1	P--X*--D1'
\-X2--D2	\-D2'

X1 and X2 share a change ID but are different visible revisions.
X* is the replacement; its parent shape and content depend on the heuristic or interactive choices. It is not a squash from X2 into X1.

Version note. This command is part of the `jj 0.45.1` target surface. Keep the search revset explicit in automation and inspect the resulting operation and evolution.

`jj describe`

Purpose and model. Rewrites the description and selected metadata of one or more commits, defaulting to `@`. It preserves logical change identity unless metadata is explicitly changed by another command; the commit ID changes because the commit object changes.

Syntax/options. `jj describe [REVSETS...]`; `-m/--message` avoids the editor, `--stdin` reads the description, and `--editor` forces editing after either. Multiple commits receive the same supplied description. The `-r` alias selects the commit set.

State effects. This is a metadata rewrite, but descendants can still be rewritten because their parent commit IDs change. The filesystem normally remains the same tree. Bookmarks follow successor graph locations; Git export sees a new Git

commit when the changed commit is exported.

```
jj describe -m 'API: introduce quota limits'
jj describe --stdin -r 'mutable()'
jj log -r '@:: & mutable()' -T 'change_id ++ " " ++ description.first_line() ++ "\n"
```

Trap. Supplying one message to a set is intentional but often wrong for a stack. Use one commit or a per-commit workflow. Use `jj metaedit` for author/timestamp/signature metadata.

jj diff

Purpose and model. Compares trees and renders a diff. Commit selection and path selection are independent: `-r`, `--from/--to`, and positional filesets describe separate domains.

Syntax/options. `jj diff [FILESETS...]; -r/--revisions REVSETS` shows the total change across a set, `-f/--from REVSET` and `-t/--to REVSET` compare endpoints, and no selector defaults to `@`. Important presentation options include `--summary`, `--stat`, `--types`, `--name-only`, `--git`, `--color-words`, `-T/--template`, context controls, and external diff controls.

Semantics. A multi-commit set must be connected: a gap such as `A|C` in a linear chain is not a diff range. A merge commit compares its tree with the automatic merge of its parents. A range expression such as `B::D` is not the same as selecting two unrelated trees.

```
jj diff -r @
jj diff --from 'main@origin' --to @ -- 'glob:src/**'
jj diff -r 'B::D' --stat
jj diff -r @ --name-only
```

Trap. A diff is not proof that a commit is single-parent; merges and conflicts have special tree semantics. For scripts prefer `--name-only` or controlled templates with colour disabled. Use `jj interdiff` for successive diff versions.

jj diffedit

Purpose and model. Touches up the content delta of an existing commit through a configured diff editor. It changes the diff while preserving the selected change's logical identity; unlike `restore`, it is expressed as editing a diff rather than copying an entire tree.

Syntax/options. `jj diffedit [FILESETS...]; -r/--revision REVSET` edits one commit (default `@`), `--from/--to` choose comparison endpoints, `--tool NAME` chooses the diff editor, and `--restore-descendants` controls descendant content. Positional filesets leave unmatched paths unchanged.

State effects. The target and descendants can receive successor commit IDs. The filesystem is updated to the resulting working-copy tree when the target is current; conflicts may be stored if the edited diff no longer applies cleanly.

```
jj diffedit -r A -- 'glob:src/**'  
jj diffedit --tool meld -r @  
jj log -r '::~@ & conflicts()'
```

Trap. `--from/--to` here define the diff shown to the editor, not rebase source/destination. Use `jj describe` for metadata and `jj squash` for selected-content movement.

jj duplicate

Purpose and model. Creates a new change whose content is equivalent to a selected commit, placed at chosen parents. It is a copy, not a successor: the new change has a new change ID even if its tree and description match.

Syntax/options. `jj duplicate [REVSETS...]`; source defaults to `@` and accepts `-r`. Use repeated `-o/--onto REVSETS` (alias `-d/--destination`), `-A/--insert-after`, or `-B/--insert-before`. Repeated destinations can create a merge.

State effects. The original remains. The duplicate and descendants at the insertion site can be rewritten; move publication bookmarks explicitly. A workspace generally stays at the resulting current location according to normal movement policy.

```
jj duplicate -r A -o 'main@origin'
jj duplicate -r A -o X -o Y
jj log -r 'A | descendants(A) | mutable()'
```

Trap. Continue an existing logical change with `jj rebase` or `jj edit`; publish an inverse with `jj revert`. Use duplicate when starting a new evolution is intentional.

`jj edit`

Purpose and model. Moves the current workspace's working-copy commit to exactly one existing commit. It is how an advanced user resumes or amends an earlier change; it does not create a bookmark or move another workspace's @.

Syntax/options. `jj edit REVSET` or `-r REVSET`; exactly one commit is required. `--ignore-working-copy` is usually inappropriate because edit is intended to update the filesystem. `--at-op` is for historical inspection, not continuation.

State effects. `jj` snapshots the old workspace first, then checks out the selected tree as the new @. The old working-copy commit remains unless abandoned. Filesystem edits made while another change was current become part of the pre-move snapshot.

```
jj edit 'change_id(abc...)'
jj edit 'bookmarks(exact:"review")'
jj status
jj new
```

Trap. “Edit A” is not Git checkout of a branch: A remains mutable and unnamed. Use `jj new A` to leave A unchanged and create a fresh child.

jj evolog

Purpose and model. Shows the evolution of a logical change: successive commit versions and their predecessor relationships. It answers what happened to one change, not what every repository operation did.

Syntax/options. `jj evolog [OPTIONS]`; `-r/--revisions REVSETS` follows changes (default `@`), `-n/--limit` bounds entries, `--reversed` reverses order, `-G/--no-graph` flattens output, `-T/--template` renders an evolution entry, and `-p/--patch` shows an inter-diff.

State effects. Observational, but normal snapshot rules apply unless disabled. It does not restore a commit or move a bookmark. Evolution entries link to operations, allowing a rewrite to be correlated with `jj op log`.

```
jj evolog -r 'change_id(abc...)'
jj evolog -p -r @
jj evolog --reversed -n 10
```

Trap. Evolution is not a ref reflog and does not record every unrelated abandoned change. Use operation history for repository-wide recovery.

jj file

Provides file-oriented inspection and metadata commands: `annotate`, `chmod`, `list`, `search`, `show`, `track`, and `untrack`. These operate on filesets and often a single commit; they are not Git's index commands.

A fileset selects repository paths, while a commit option selects the tree/commit. File tracking changes whether ignored/large paths participate in snapshotting; it does not create a commit by itself until a snapshot records the filesystem.

jj file annotate

Shows line attribution for one path. Syntax is `jj file annotate PATH`; `-r/--revision REVSET` selects the starting commit and `-T/--template` renders `AnnotationLine` values, including commit, content, line numbers, original line numbers, and diff-hunk boundaries.

```
jj file annotate src/quota.rs
jj file annotate -r main@origin -T 'line_number() ++ " " ++ commit().change_id() ++ " " ++ content'
```

It is observational and can snapshot. The path is a single path argument, not a general fileset. Attribution follows jj's graph/evolution data and may differ from Git blame after rewrites.

jj file chmod

Changes executable metadata for filesets in a selected commit. Syntax is `jj file chmod MODE FILESETS...`; mode `n/normal` clears executable and `x/executable` sets it, while `-r/--revision REVSET` defaults to `@`.

```
jj file chmod x scripts/release.sh
jj file chmod n --revision A -- 'glob:bin/**'
```

This rewrites the target's tree metadata and can rebase descendants. It is not a filesystem `chmod` of every checkout; inspect the resulting tree and working copy. Use template `TreeEntry.executable()` to verify.

jj file list

Lists paths in the selected tree. Positional filesets filter output, `-r/--revision REVSET` selects the tree (default `@`), and `-T/--template` renders tree-entry metadata in jj 0.45.1.

```
jj file list --revision @
jj file list -r main@origin -- 'glob:src/**'
jj file list -- 'root-glob:docs/**/*md'
```

Use it to validate a fileset before passing that fileset to restore, split, squash, or fix. It is observational but may snapshot.

jj file search

Searches file contents in a selected commit. Syntax requires `--pattern PATTERN`; `-r/--revision REVSET` defaults to `@`, positional filesets restrict file prefixes, and `--name-only` prints paths rather than matching lines. The pattern defaults to regex when no string-pattern kind is supplied.

```
jj file search --pattern 'quota' -- 'glob:src/**'  
jj file search --pattern 'glob:*TODO*' --name-only  
jj file search -r main@origin --pattern 'regex:^use ' -- 'glob:*.rs'
```

A glob pattern applies to the whole line, so `glob:*foo*` is different from `glob:foo`. This command searches a tree; it does not search the working filesystem independently.

jj file show

Prints file contents from a selected tree. Filesets are required; `-r/--revision REVSET` defaults to `@`, and `-T/--template` renders `TreeEntry` metadata.

```
jj file show -r main@origin -- src/quota.rs  
jj file show -- 'glob:docs/**/*md'
```

It is read-only repository inspection but normal snapshot policy still applies. A conflicted entry is not ordinary file content; inspect conflict metadata and resolve or select a side explicitly.

jj file track

Marks filesets for tracking even when ignored or too large only when requested. `--include-ignored` overrides ignore and maximum-file-size restrictions.

```
jj file track --include-ignored generated/schema.json  
jj file track -- 'glob:vendor/**'
```

Tracking affects snapshot eligibility and can change a later working-copy commit; it does not itself publish or commit. Use sparingly because large/ignored content has performance and policy implications.

jj file untrack

Stops tracking selected paths. It affects future snapshots and working-copy materialisation, not historical trees already committed.

```
jj file untrack 'glob:generated/**'  
jj status
```

Check status and sparse configuration afterward. Do not confuse untracking with deleting a file from a commit; use restore or filesystem deletion for content changes.

jj fix

Purpose and model. Runs configured fixer programs against files in selected commits. It is a history rewrite operation whose input is a commit domain and whose file scope is a fileset/configured fixer policy.

Syntax/options. `jj fix [FILESETS...]; -s/--source REVSETS` selects commits and descendants, defaulting to `revsets.fix` or `reachable(@, mutable())`. `--include-unchanged-files` fixes files even when not changed in the target; `-a/--all-lines` asks for all lines rather than modified lines when the fixer supports ranges.

State effects. Fixers can rewrite many commits and trigger descendant rebasing/conflicts. A failing fixer may leave a useful operation to inspect, but do not assume partial output is a valid stack. Fix configuration supplies commands, environment, file patterns, and range protocol; the working-copy commit is not special except as a selected commit.

```
jj fix -s '@:: & mutable()' -- 'glob:src/**'  
jj fix --all-lines -s 'mutable()'  
jj log -r 'mutable() & conflicts()'
```

Trap. Run first on a small revset and inspect the diff. A formatter that does not understand modified-line ranges can rewrite entire files. For reproducible automation, pin the fixer executable/version and use `run --ignore-changes` for checks that should not rewrite history.

`jj gerrit` and `jj gerrit upload`

Purpose and model. Provides Gerrit-aware upload on top of `jj`'s mutable change graph. It is not a replacement for Gerrit's server identity: `jj` change IDs identify local evolution, while Gerrit uses a message `Change-Id: I...` footer or server/plugin policy to associate patch sets.

Syntax/options. The parent requires `upload`. In 0.45.1, `jj gerrit upload` accepts `-r/--revision REVSET`, `-b/--remote-branch BRANCH`, `--remote REMOTE`, `-n/--dry-run`, reviewer/CC/label/topic/hashtag/message options, edit/WIP/ready/private/remove-private, comment publication, notification, merged, validation, attention-set, deadline, custom and repeated `-o/--option` push options. Run local help for the exact spelling of server options.

State effects. Upload exports commits and review metadata; it can update remote-tracking state but does not make Gerrit a local bookmark manager. Rewriting an earlier `jj` change produces new commit IDs while a preserved Gerrit footer associates the new commit with its existing review. Descendants are automatically rebased locally before re-upload.

```
jj gerrit upload --dry-run --remote gerrit --remote-branch main
jj gerrit upload --revision 'roots(main@origin..@)' --remote gerrit --remote-branch main
git push gerrit 'HEAD:refs/for/main%topic=quota'
```

Boundary. Local `jj` transport and footer handling are testable with fixtures; Gerrit server review, patch-set, topic, permission, and submit behaviour is server-dependent. Never claim that a `jj` change ID is a Gerrit Change-Id.

`jj gerrit upload`

Uploads the selected commit stack to a Gerrit remote and branch. The principal selectors are `-r/--revision REVSET`, `-b/--remote-branch BRANCH`, `--remote REMOTE`, and `-n/--dry-run`. Reviewer, CC, label, topic, hashtag, WIP/ready/private, notification, validation, attention-set, deadline, custom, and repeated `--option` flags alter server-facing review metadata.

Cardinality and publication contract. The revision revset may describe a stack, but the upload must be bounded to the commits intended for one Gerrit target. Before uploading, render the exact set and its parents, check `conflicts()`, and inspect each commit's trailers. An existing Change-Id requests an update to that Gerrit review; a missing one is generated from the `jj change` ID for the uploaded object in 0.45.1. `--dry-run` validates local selection and command construction; it cannot prove server permissions, review association, or submit policy.

Rewrite and ref effects. Upload does not create or move a local bookmark unless the selected workflow does so separately. A rewritten ancestor gives descendants new commit IDs. Re-uploading an existing footer requests a new patch set; a missing footer causes `jj` to add a derived footer only to the uploaded commit, so the uploaded commit ID can differ from the local one and later fetch can expose divergence. Configure `templates.commit_trailers` when the footer should be present locally before upload. A rejected push leaves the local `jj` graph intact, so inspect the remote state before choosing a force or replacement action.

```
jj gerrit upload --dry-run --remote gerrit --remote-branch main --revision 'roots(main@gerrit..@)
jj gerrit upload --remote gerrit --remote-branch main --revision 'roots(main@gerrit..@)::'
```

Local `jj change` IDs do not equal Gerrit Change-Id footers, although `jj` 0.45.1 can derive a missing upload footer from the local `jj change` ID. Server acceptance, patch-set association, and permissions remain Gerrit-dependent.

jj git

Purpose and model. Operates the Git backend, remotes, Git-compatible refs, and colocation. The parent is a namespace for `clone`, `colocation`, `export`, `fetch`, `import`, `init`, `push`, `remote`, and `root`.

jj metadata and the Git object database are related but not identical. A commit can have a jj change ID and operation history that Git does not know. A bookmark can be exported as a Git ref; an operation ID and workspace pointer cannot.

```
jj git root
jj git remote list
jj git fetch --remote origin
jj git import
jj git export
```

Native Git commands are useful in a colocated checkout, but after Git-side ref changes explicitly import them and avoid two writers changing the worktree concurrently. The detailed backend model appears in Part VII.

jj git clone

Clones a Git repository into a jj repository. Syntax is `jj git clone SOURCE [DESTINATION]`. In 0.45.1 the result is colocated by default unless `git.colocate=false`; `--no-colocate` selects the hidden backing layout and `--colocate` makes the default an explicit script precondition. Important options include `--remote REMOTE_NAME`, fetch branch/tag filters, and checkout/workspace options in the installed help.

```
jj git clone --colocate ssh://git@example.test/project.git project
jj git clone --remote upstream https://example.test/project.git project
```

A local source path is resolved to an absolute path. Colocation makes the directory usable by native Git and jj; non-colocation keeps jj metadata separate. The clone's operation DAG is new and independent of the source clone.

jj git colocation

Controls whether jj and Git share the repository/worktree layout. Subcommands are `enable`, `disable`, and `status`. Colocation is a backend layout property, not a branch or workspace mode.

```
jj git colocation status
jj git colocation enable
jj git colocation disable
```

Enable for Git-native tooling; disable only after checking scripts, Git roots, hooks, and worktree state. Existing Git objects/refs and jj metadata are not interchangeable merely because they reside nearby.

jj git colocation disable

Disables colocation. Git-native tools may no longer see the expected top-level Git repository, and scripts using `git -C` need revisiting. Do this only with a backup and an explicit transport plan.

```
jj git colocation disable
jj root
jj git root
```

jj git colocation enable

Enables Git colocation for a compatible repository. Native Git commands can then use the same worktree, but an export/import boundary still exists.

```
jj git colocation enable
jj git colocation status
```

jj git colocation status

Reports whether the current repository is colocated. It is a useful preflight for `git`, `gh`, hooks, and CI scripts.

```
jj git colocation status
```

jj git export

Exports jj bookmarks/tags and compatible commit information to Git refs. It is useful after local ref changes when a native Git consumer or hosting tool needs the Git view updated.

```
jj bookmark create review -r @
jj git export
git -C "$(jj root)" show-ref
```

Export does not publish to a remote; `git push` or `jj git push` is the network operation. Operation history, change IDs, conflict structure, and workspace pointers are not made into ordinary Git concepts.

jj git fetch

Fetches Git objects and remote refs, then imports them as remote bookmarks/tags. `--remote REMOTE` selects a named remote; `-b/--branch` repeats branch patterns, `-t/--tag` repeats tag patterns, and the command has options for remotes, pruning, depth, and parallelism according to local help.

```
jj git fetch --remote origin
jj git fetch --remote upstream --branch 'main'
jj git fetch --remote origin --tag 'v*'
```

Fetch does not rebase your mutable stack automatically. It moves remote bookmarks; rebase explicitly onto the imported commit. Branch filters use the fetch command's documented pattern rules, which are not necessarily identical to `reset` string patterns.

jj git import

Imports Git-side ref changes into jj's view without fetching from a network remote. It is the explicit bridge after native Git creates, moves, or deletes refs in a colocated repository.

```
git -C "$(jj root)" branch temp
jj git import
```

```
jj bookmark list --all-remotes
```

Import can alter remote bookmark/tag state and create an operation, but it does not turn a Git branch into a local jj working-copy commit or move @. Inspect the imported ref and rebase/publish deliberately.

jj git init

Initialises a Git-backed jj repository in the destination, defaulting to the current directory. In 0.45.1, `--colocate` creates/uses a top-level `.git` and is the default unless `git.colocate=false`; `--no-colocate` explicitly puts the backing Git repository under the jj metadata layout. `--git-repo PATH` selects an existing backing Git repository and is mutually exclusive with `--colocate`. There is one important same-path rule: if PATH is the same working directory as the jj destination, so that both `.jj` and `.git` are in that directory, the result is a colocated workspace and both native Git and jj commands work there. If PATH names a different directory, the jj destination is non-colocated and native Git commands must be run against the separate backing repository. `jj git root` reports the underlying Git directory, commonly the `.git` directory; it is not necessarily the workspace root. The destination is created if necessary.

```
jj git init --colocate
jj git init project          # colocated in 0.45.1 by default
jj git init --no-colocate hidden-project
mkdir -p same-path backing
git -C same-path init
jj git init --git-repo "$PWD/same-path" "$PWD/same-path" # same-path, colocated
git -C backing init
jj git init --git-repo "$PWD/backing" "$PWD/project"      # different path, non-colocated
git --git-dir="$PWD/backing/.git" show-ref
```

Choose colocation when GitHub/Gerrit tools and Git transport must share a worktree, but establish a single-writer rule for filesystem operations. Check `jj root` and `jj git root` after initialisation.

`jj git push`

Purpose and model. Publishes Git refs selected from local bookmarks/tags. It is the boundary where unnamed local work must acquire a publication name unless a release-specific change push mode is used.

Syntax/options. `--remote REMOTE` selects a named remote; `-b/--bookmark PATTERN` repeats bookmark patterns, `-t/--tag PATTERN` repeats tags, `--all` pushes all bookmarks/tags including new ones, `--tracked` pushes tracked refs, and `--deleted` pushes deletions. `-r/--revision REVSETS` selects bookmarks/tags pointing to revisions, `--named NAME=REVISION` creates or updates a named bookmark for publication, and `-c/--change REVSETS` creates a bookmark whose default name is based on the change ID. `--allow-empty-description`, `--allow-private`, and `--allow-conflicts` relax eligibility checks; `--dry-run` previews the operation. These are the `jj 0.45.1` options; new-bookmark publication uses `--named` or `--change`, not an additional permission flag.

Safety. An untracked selected bookmark can become tracked automatically. A new bookmark is requested with `--named` or `--change`, not with a force or “allow new” switch. A backwards update is subject to `jj`'s lease-like safety check: the remote must still match the state `jj` last fetched before the update is accepted. Review local and remote targets, specify the remote, and use `jj 0.45.1`'s `--dry-run` before publishing.

```
jj git push --remote origin --bookmark review
jj git push --remote github --bookmark 'gh/*' --dry-run
jj git push --remote gerrit --named review=@
jj git push --remote origin --change 'change_id(abc...)' --dry-run
```

State effects. Push changes the remote server, not the commit DAG. It can update local remote-tracking state and operations. A later fetch observes server-side merge/rewrite. `--no-integrate-operation` only changes operation integration; it does not prevent the network push.

`jj git remote`

Manages named Git remotes. Subcommands are `add`, `list`, `remove`, `rename`, and `set-url`. Remote names qualify remote bookmarks and select push/fetch destinations; they do not identify a workspace.

```
jj git remote add github git@github.com:me/project.git
jj git remote add gerrit ssh://gerrit.example/project
jj git remote list
jj git remote set-url github git@github.com:me/project.git
jj git remote rename origin upstream
```

Inspect full URLs before pushing. A two-remote repository should use explicit `--remote` in scripts and qualify `main@github` versus `main@gerrit` in revsets.

`jj git remote add`

Adds a named remote with a URL. It changes Git remote configuration; no commits are fetched until `jj git fetch`.

```
jj git remote add origin ssh://git@example.test/project
jj git git remote add github git@github.com:me/project.git
```

The second line is intentionally wrong: the command is `jj git remote add`, not `jj git git remote add`. Shell aliases should not obscure the remote name.

`jj git remote list`

Lists configured remote names and URLs. It is read-only repository configuration inspection and is the first command to run before a dual-host push.

```
jj git remote list
```

`jj git remote remove`

Removes a configured remote. Existing imported remote bookmarks can remain as local knowledge until forgotten or pruned according to the command's policy. Removing a remote does not delete the server repository.

```
jj git remote remove old-origin
jj bookmark forget --include-remotes 'main@old-origin'
```

jj git remote rename

Renames a configured remote and therefore changes the qualifier of its remote bookmarks. Review tracking relationships and scripts after renaming.

```
jj git remote rename origin github
jj log -r 'main@github'
```

jj git remote set-url

Changes the URL for an existing remote. It does not fetch or push and does not rewrite commits. Use it when moving from HTTPS to SSH or changing a fork URL, then verify with `remote list`.

```
jj git remote set-url origin git@github.com:me/project.git
jj git remote list
```

jj git root

Prints the Git repository root used by the backend. It can differ in meaning from `jj root`, which prints the current workspace root. Use the result instead of assuming the current directory is a Git root.

```
git -C "$(jj git root)" show-ref
GIT_DIR="$(jj git root)/.git" git status
```

In a non-colocated repository, native Git use may be invalid or point at another repository. In scripts, first inspect colocation status.

jj help

Prints help for the root or a command path. `jj help --help` documents the help command itself; `jj help command` and `jj help command subcommand` select paths. Keyword search exposes language and conceptual material in the installed documentation.


```
jj help rebase
jj help bookmark track
jj help -k revsets
jj help -k templates
```

Help is the normative installed-CLI authority for syntax/defaults. It may link to a current documentation site; use the local version for behaviour. In a release audit capture root help recursively rather than relying on memory.

jj interdiff

Compares the diff changes between two commit versions. Use `--from REVSET` and `--to REVSET` as required by the 0.45.1 syntax, plus positional filesets and diff-formatting options. It is the right comparison for successive versions of one logical change whose parent context changed.

```
jj interdiff --from A --to B
jj interdiff --from A --to B --stat -- 'glob:src/**'
```

It does not rewrite commits or move refs, but normal snapshot policy still applies. Use `diff` for endpoint tree comparison and `evolog -p` when the commits are known successors of one change.

jj log

Purpose and model. Displays a graph projection of a commit set. It does not show “branches” by default; it shows the selected visible commits, their topology, and a template rendering for each. The default revset and template are configuration, so a stable script supplies both.

Syntax/options. `jj log [FILESETS...]; -r/--revision REVSETS` selects commits, `-n/--limit N` limits after filtering and topological ordering, `--reversed` shows older first, `-G/--no-graph` produces a flat list, `-T/--template` renders commits, `-p/--patch` includes patches, and `--count` prints a count. A fileset restricts commits that modify those paths.

```
jj log -r '@:: & mutable()'
jj log -r 'mutable() & conflicts()' -p
jj log -G --color=never -T 'commit_id ++ "\n"'
jj log -n 20 --reversed -r 'main@origin..@'
```

State effects. Normal log snapshots the working copy first; use `--ignore-working-copy` for a prompt or monitoring read. It never moves bookmarks or rewrites commits. The graph can contain hidden/divergent information only when the selected revset includes it.

Trap. Do not parse graph glyphs or assume the default revset is a complete repository view. For automation use explicit revsets, templates, colour, and pager options. A fileset is applied to changed-path history, not merely to the displayed tree.

`jj metaedit`

Purpose and model. Rewrites commit metadata without changing file content. It is the precise tool for author, timestamp, description, change ID, and related metadata edits.

Syntax/options. The commit set defaults to `@`. Important 0.45.1 options include `--update-change-id`, `-m/--message`, `--update-author-timestamp`, `--update-author`, `--author`, `--author-timestamp`, and committer controls shown by local help. The `-r` alias selects commits.

State effects. The tree is unchanged, but metadata changes the commit ID. Descendants may be rewritten because their parent ID changes; bookmarks follow successors. `--update-change-id` intentionally breaks local evolution continuity and should not be used merely because a commit ID changed.

```
jj metaedit --author 'A. Engineer <a@example.test>' -r @
jj metaedit --update-author-timestamp
jj metaedit --update-change-id -r 'mutable()'
```

Trap. Author/committer identity, `jj` change ID, and Gerrit Change-Id are separate. Use `jj describe` for editor-driven descriptions and `jj sign` for cryptographic

signing.

`jj new`

Purpose and model. Creates an empty working-copy change with supplied parents. One parent creates a child; multiple parents create a merge. It is the ordinary boundary between patch-stack changes.

Syntax/options. `jj new [REVSETS...]` defaults to `@`; `-m/--message` describes it, `--no-edit` leaves the workspace where it is, `-A/--insert-after` inserts after selected commits and rebases their children, and `-B/--insert-before` inserts before selected commits. `-o/--onto` is an alias for parent selection in 0.45.1.

State effects. The old `@` becomes a parent and remains intact. Insert modes are graph surgery and can rewrite descendants. Bookmarks at the old commit do not automatically move to the new child.

```
jj new main@origin -m 'API: introduce quota'
jj new @- @--
jj new --no-edit A
jj new --insert-after A
```

Trap. Multiple commits are explicit merge parents, not alternatives. Use `jj edit` when continuing an existing change.

`jj next`

Purpose and model. Moves from the current working-copy commit toward a child, optionally jumping to the next conflicted descendant. It can create a new child or edit the target directly according to movement policy.

Syntax/options. `jj next [OFFSET]` defaults to one step. `-e/--edit` edits the target; `-n/--no-edit` creates a new working-copy commit on the target; `--conflict` searches for the next conflicted descendant. These override `ui.movement.edit`.

State effects. Normal snapshotting records current edits before movement. Forks may not have a unique next child. No bookmark is moved merely by navigation.

```
jj next
jj next --edit
jj next --conflict
jj next 2 --no-edit
```

Trap. “Next” is graph navigation, not next commit on a branch. Use an explicit revset with `jj edit` or `jj new` when precision matters.

`jj operation` / `jj op`

Purpose and model. Manages the operation DAG, the local history of repository views. Subcommands are `abandon`, `diff`, `integrate`, `log`, `restore`, `revert`, and `show`. Operation IDs identify repository transformations in this clone; they are not commit IDs, change IDs, or remote refs, and they do not transfer through Git.

Cardinality and recovery contract. Operation selectors identify one operation unless a command explicitly accepts endpoints or a set. Inspect the operation graph before choosing a recovery action: `undo` moves to the previous integrated view, `op restore` creates a new view copied from a selected operation, and `op revert` constructs an operation-level inverse. None of them retracts a push, editor side effect, or test result outside the repository.

```
jj op log
jj op show
jj op diff --from OP1 --to OP2
jj undo
```

Use operation commands when repository state surprises you, especially after a rewrite, concurrent operation, or ref movement. The command family is explained in the recovery chapter and laboratory.

jj operation abandon alias: `jj op abandon`

Abandons an operation from the operation DAG. It is a history-maintenance action on local operation records, not the same as abandoning a commit. Do not use it until recovery is complete and retention policy is understood.

```
jj op abandon OP
jj op log
```

Once operation evidence is abandoned, recovery may become harder. Keep an external record of important operation IDs before pruning.

jj operation diff alias: `jj op diff`

Compares repository state between operations. `--operation OP` compares an operation with its parent; `-f/--from` and `-t/--to` to compare endpoints; `-G/--no-graph` flattens changes and `-p/--patch` includes rebased diff detail. The endpoints are operation IDs, not commit revsets: use `jj diff` when the question is about two trees in the current repository view.

```
jj op diff --operation OP
jj op diff --from OP1 --to OP2 --patch
```

It is especially useful when later operations have changed commit IDs: compare repository views, not only remembered IDs. It is observational and does not restore either state.

jj operation integrate alias: `jj op integrate`

Integrates an operation created with `--no-integrate-operation` into the current operation head. It is the explicit convergence point for concurrent/unintegrated local operations; it does not mean converge the commit revisions of one change, which is the separate `jj converge` command.

```
jj status --no-integrate-operation
# record the printed operation ID
jj op integrate OP
```

Integration can expose conflicts or divergent operation history. It does not suppress or reverse external side effects performed by the original command.

jj operation log *alias: jj op log*

Displays operation history as a graph. Options include operation selection, `-n/--limit`, `--reversed`, `-G/--no-graph`, `-T/--template`, and diff/detail output according to local help. The rendered operation graph is not a reflog of one bookmark: it records repository-wide views, including rewrites, imported refs, and workspace metadata.

```
jj op log
jj op log -n 20
jj op log -T 'id.short(12) ++ " " ++ description ++ "\n"
```

Operation log is the first recovery command. Record the current head before attempting repair; a later operation can make an earlier state harder to recognise but does not necessarily remove it. Use `--at-operation` to inspect a historical repository view without changing the live one, then use `op show` or `op diff` before restoring it.

jj operation restore *alias: jj op restore*

Restores repository state to an operation. The operation must be selected explicitly; inspect it with `jj --at-op OP log` first.

`--what` can repeat and, in the experimental 0.45.1 surface, selects `repo` and/or `remote-tracking`. Omitting remote-tracking state can preserve the ability to push after undo; restoring it can make the local view match the old state exactly.

```
jj --at-op OP log -r 'all()'
jj op restore OP
jj op restore --what repo OP
```

Restore creates a new operation representing the restoration; it does not move a server and does not make the filesystem historical unless the workspace is updated. Treat it as a deliberate state change, then inspect the new operation.

jj operation revert alias: `jj op revert`

Creates a repository operation that reverses the effect of a selected operation while retaining later state as appropriate. It is not `commit jj revert`: the former operates on repository metadata/graph state, the latter creates an inverse commit.

```
jj op revert OP
```

Use `operation diff` first. `Operation revert` is a recovery tool for an operation-level mistake, not a way to undo a public remote push.

jj operation show alias: `jj op show`

Shows one operation's metadata and repository changes. Select an operation ID from `jj op log`; options include `diff` and `graph` formatting.

```
jj op show OP
jj op show -p OP
```

Use it to answer what a rewrite actually changed. It does not change state. For the state itself, use `jj --at-op OP log`; for a controlled restore, use `jj op restore OP`.

jj parallelize

Purpose and model. Makes selected commits siblings by removing the edge that serialises them. It is useful when changes were accidentally stacked but should be independent.

Syntax/options. `jj parallelize [REVSETS...]`; `-r` is the commit alias and there are no additional 0.45.1 command-specific options.

State effects. New parent edges cause selected commits and descendants to receive successors. A child that depended on removed content may conflict or change diff. Bookmarks follow affected successors; remote publication does not happen automatically.

```
jj parallelize -r 'B|C'  
jj log -r '::~@'  
jj log -r 'conflicts()'
```

Trap. Parallelisation is not reverting a dependency's files. Check every descendant; use `jj rebase` when an explicit destination describes the intention better.

jj prev

Purpose and model. Moves backward from the current working-copy commit. It can create a new child on the selected parent or edit the parent directly, mirroring next.

Syntax/options. `jj prev [OFFSET]` defaults to one parent. `-e/--edit` edits the parent; `-n/--no-edit` creates a new working-copy change; `--conflict` jumps to the previous conflicted ancestor.

State effects. Current edits are snapshotted first. A merge may have no unique parent, and an offset is graph navigation rather than a first-parent promise. Existing commits are not rewritten unless subsequently edited.

```
jj prev  
jj prev --edit  
jj prev --conflict  
jj prev 2 --no-edit
```

Trap. Use `jj edit` for a non-linear review target. A new child protects the parent; edit mode makes later edits part of it.

jj rebase

Purpose and model. Rewrites parent edges for a selected set while preserving each selected commit's diff as far as possible. Source selection and destination placement are independent dimensions.

Syntax. `jj rebase <destination option> [source option]`. Source options are `-r/--revisions REVSETS` for explicit commits, `-s/--source REVSET` for a

commit and descendants, and `-b/--branch REVSET` for the branch relative to the destination (default `@`). Exactly one destination mode is required: `-o/--onto` (aliases `-d/--destination`), `-A/--insert-after` (alias `--after`), or `-B/--insert-before` (alias `--before`).

Mode	Transformation	Typical intent
<code>-r X -o Y</code>	Rewrite exactly X onto Y.	Surgical move; useful with holes in a stack.
<code>-s X -o Y</code>	X and descendants onto Y.	Move a whole stack.
<code>-b X -o Y</code>	Branch of X relative to Y onto Y.	Update a current line without naming every node.
<code>-A Y</code>	Insert selected changes after Y and rebase Y's descendants.	Insert between Y and its children.
<code>-B Y</code>	Insert before Y and rebase Y and descendants.	Place a change before a known commit.
repeat <code>-o</code>	Multiple destination parents.	Create an intentional merge.

State effects. Rebased commits get successor commit IDs; change IDs normally persist. Descendants automatically rebase. Conflicts are stored in the resulting commits. Bookmarks follow rewritten graph targets, but remote publication requires an explicit push.

```
jj rebase -b 'roots(main@origin..@)' -o main@origin
jj rebase -r 'A|C' -o B
jj rebase -s A -A X
jj rebase -r D -o K -o M
```

Trap. Several source roots can produce several direct children, and a destination must satisfy its own cardinality. Inspect log, evolution, and operation state after a non-trivial rebase. There is no Git-style continuation session: resolve the stored conflict or perform another rewrite.

jj redo

Reapplies the most recently undone operation when redo state is available. It is repository-operation recovery, not replay of a Git reflog entry.

```
jj undo
jj redo
```

Any unrelated mutating operation can affect the redo chain. Inspect operation log before assuming redo is available or safe.

jj resolve

Purpose and model. Resolves stored file conflicts with a merge tool. It changes the conflict value in a selected commit; it is not a Git index continuation phase.

Syntax/options. `jj resolve [FILESETS...]; -r/--revision REVSET` defaults to `@`, `-l/--list` lists paths instead of resolving, and `--tool NAME` selects a configured three-way merge tool. Built-in `:ours` and `:theirs` choose sides one and two for applicable conflicts.

State effects. The selected commit is rewritten with resolved file content; descendants that depend on its tree may be rebased and conflicts can disappear or move. The filesystem is materialised for the current workspace, but the stored conflict value is authoritative. A fileset limits conflicted paths.

```
jj resolve --list -r @
jj resolve --tool meld -r @ -- 'glob:src/**'
jj resolve --tool :ours
jj log -r '::-@ & conflicts()'
```

Trap. Multi-sided conflicts are not necessarily two-way, and marker files are a rendering. Inspect the resulting diff and conflict predicate. Resolving a child does not necessarily resolve an independent conflict in its descendant.

jj restore

Purpose and model. Restores selected file contents from one commit into another, or undoes the diff represented by a commit. It is a content operation, not a ref reset and not the same as `revert`. Ask whether the source tree, destination tree, or a commit's visible changes define the operation.

Syntax/options. `jj restore [FILESETS...]; -f/--from REVSET` is the source, `-t/--into REVSET` the destination, and `-c/--changes-in REVSET` undoes changes visible in a commit compared with the merge of its parents. Without options the default is effectively `--changes-in @`. `-i/--interactive` and `--tool NAME` select individual diff hunks; `--restore-descendants` preserves descendant content during rebasing.

State effects. The destination is rewritten with selected paths copied from the source or parent-derived inverse. Descendants may be rebased and conflicts may be stored. A fileset narrows paths, not commit selection. Restoring into `@` changes the working-copy tree at the next snapshot.

```
jj restore --from main@origin --into @ -- 'glob:src/**'
jj restore --changes-in A
jj restore --from @- --into @ --interactive
```

Trap. Use `revert` for a review-visible inverse; use `restore` from a parent for local cleanup. Verify with `jj diff` before publishing.

jj revert

Purpose and model. Creates a new change that applies the reverse of selected commits. It preserves original history and expresses the reversal as a reviewable diff.

Syntax/options. `jj revert --revision REVSETS --onto REVSETS`, or use `-A/--insert-after/-B/--insert-before`. `-r/--revision` is required; `-o/--onto` (alias `-d/--destination`) supplies the destination. Multiple destinations can create a merge.

State effects. Original commits are not rewritten. A new change with a new change ID is created; conflicts can occur when the inverse no longer applies. Bookmarks move only if they point at the result or are advanced explicitly.

```
jj revert -r A -o @
jj revert --revision 'A::C' --onto main@origin
jj log -r '::-@'
```

Trap. Revert is not abandon and does not erase a public change. For unpublished cleanup, restore/squash may communicate intent better.

jj root

Prints the current workspace root and is a shortcut for `jj workspace root`. It is useful for scripts that need to locate the repository from a subdirectory.

```
cd src/lib
git -C "$(jj root)" status
```

It reports the workspace root, not necessarily the Git backend root; use `jj git root` for the latter. The command is observational but follows global snapshot policy unless ignored.

jj run

Runs a command over selected commits. `-r/--revision REVSETS` selects the set, `-j/--jobs N` controls parallelism, `--root` changes the working directory, `--clean` discards reusable private workspaces, `--passthrough` connects output and requires one job, and `--ignore-changes` prevents rewriting commits.

Execution contract. Each selected commit is tested in a private materialisation; the selected revset is not a shell loop over the current workspace. A non-zero command status stops or reports the run according to the command mode, while files written without `--ignore-changes` become candidate tree rewrites. Parallel jobs require isolated caches and temporary directories. Use a bounded revset, a deterministic command, and a template or log record containing the tested commit ID.

Graph effects. With changes enabled, a formatter or generator can create successors and automatically rebase descendants. With `--ignore-changes`, output is evidence only and should not be mistaken for a clean working-copy or a committed fix. The operation log records the repository effects, but it cannot make an external test side effect reproducible.

```
jj run --ignore-changes --clean -r 'main@origin..@' -- ./check.sh
jj run -r '@:: & mutable()' -- ./format-and-test.sh
```

Without `--ignore-changes`, files written by the command can become commit rewrites and rebase descendants. The full execution model is in Part XVI.

jj show

Shows metadata and the diff of selected commits compared with their parent(s), defaulting to `@`. `--reversed` changes ordering, `-T/--template` renders metadata, and diff formatting includes `--summary`, `--stat`, `--types`, `--name-only`, `--git`, and context/diff options from local help.

```
jj show -r A
jj show -r @ --stat
jj show -r @ -T 'change_id ++ "\n" ++ description'
```

For a merge, the displayed change is relative to the automatic merge of parents. Show is observational but can snapshot; it does not create a new commit. Use `diff` to compare arbitrary endpoints and `evolog` to compare versions of one logical change.

jj sign

Purpose and model. Adds or replaces a cryptographic signature on selected commits. Signing rewrites commits because the signature is metadata; it is separate from a Git hosting server's verification policy.

Syntax/options. `jj sign` selects commits with `-r/--revision` `REVSETS`, defaulting to `revsets.sign`, and accepts `--key` `KEY`. Commits are re-signed even

if they appear already signed; hardware-key workflows should account for that.

State effects. Signed commits receive new commit IDs and may cause descendant successors. Change IDs normally persist. Bookmarks follow successors locally; pushing signed commits still requires the relevant bookmark/ref and server policy.

```
jj sign -r 'bookmarks(exact:"release")'  
jj sign --key KEYID -r @  
jj log -r @ -T 'if(signature, signature.status(), "unsigned") ++ "\n"'
```

Trap. Presence does not prove a good signature. Template status/key/display can be expensive because verification may invoke GPG or SSH tooling.

jj simplify-parents

Purpose and model. Removes redundant parent edges while preserving the selected commits' tree meaning. It is graph normalisation: it does not turn a merge into an arbitrary squash and should be used when parent reachability makes an edge unnecessary.

Syntax/options. `jj simplify-parents; -s/--source REVSETS` simplifies commits together with their descendant trees, while `-r/--revision REVSETS` limits the operation to specified commits. If neither is supplied, 0.45.1 uses `revsets.simplify-parents` or `reachable(@, mutable())`.

State effects. Parent edges and therefore commit IDs can change; descendants may be rewritten. Filesystem content should remain semantically equivalent, but conflicts and merges must be inspected. Bookmarks follow target successors.

```
jj simplify-parents -r @  
jj simplify-parents --source 'mutable()'  
jj log -r 'merges()'
```

Trap. Use `rebase` when you want a particular destination, and do not treat parent simplification as a content cleanup. Compare before/after graph and diff.

jj sparse

Manages which repository paths are materialised in the current workspace. Subcommands are `edit`, `list`, `reset`, and `set`. Sparse state changes the filesystem view, not the complete commit tree.

A sparse workspace can therefore run a build or review focused on one subtree while commits still contain paths outside the materialised set. A rewrite can update absent paths in the commit without writing them into the sparse filesystem.

jj sparse edit

Opens the editor for the current workspace's sparse patterns. The editor is external and operates on configuration-like path patterns.

```
jj sparse edit
jj sparse list
```

Use it for complex patterns; use `sparse set` for repeatable commands. Sparse editing can update the filesystem and therefore should not run concurrently with another process writing the workspace.

jj sparse list

Lists sparse patterns for the current workspace. It is the first diagnostic when files appear absent from a checkout.

```
jj sparse list
```

An absent file may be sparse rather than deleted. Compare `jj file list` against filesystem listing before restoring or committing.

jj sparse reset

Resets sparse patterns to the default/full materialisation policy. It changes workspace files, not commit trees.

```
jj sparse reset
jj sparse list
```

Use before a full test when a sparse workspace could hide inputs. It may be expensive in a large repository and can overwrite local filesystem materialisation; snapshot or inspect status first.

jj sparse set

Sets sparse patterns directly. The fileset expressions describe paths to materialise; all other paths remain absent from the filesystem view.

```
jj sparse set --clear --add src --add tests
jj sparse set --clear --add Cargo.toml --add Cargo.lock
```

Patterns are workspace state, not history filters. A command such as `jj diff` can still see complete commit trees; use filesets separately when the diff itself should be narrowed.

jj split

Purpose and model. Divides one commit into two changes. The first result contains selected content and the second contains the remainder; this is a graph replacement, not merely a file move. By default the target is `@`.

Syntax/options. `jj split [FILESETS...]; -r/--revision REVSET` chooses the commit, `-i/--interactive` and `--tool NAME` select diff hunks, `-o/--onto/-d/--destination` and `-A/-B` place the resulting changes, `-m/--message` supplies a description, `--editor` opens the editor, and `-p/--parallel` controls parallel handling in the installed surface. A fileset selects paths for the first side when non-interactive.

State effects. The original commit receives successors representing the split. Descendants are rebased onto the second resulting change and can conflict. The split changes have distinct change IDs; neither should be assumed to be the original change's sole continuation without checking evolution. Bookmarks at the

original target follow the command's successor policy, but publication bookmarks may need to be moved.

```
jj split -r @ -- 'glob:tests/**'
jj split --interactive -r A
jj split -r @ -m 'API: implementation'
jj log -r '::~@'
```

Trap. A fileset is a path selection, not a semantic diff-hunk classifier. Check both diffs and descriptions. If the operation is wrong, undo before further edits; later, use evolution to identify the split successors.

jj squash

Purpose and model. Moves content from one or more source commits into a destination, normally the parent of the current change. It can fold all or selected paths and can perform experimental insertion-style rewrites in 0.45.1.

Syntax/options. `jj squash [FILESETS...]`; ordinary mode uses `-r/--revision REVSET` to squash a commit into its parent, or `-f/--from REVSETS` and `-t/--into REVSET` to name source and destination. `-i/--interactive/--tool NAME` select content. `-m/--message` sets the destination description, `-u/--use-destination-message` discards source descriptions, and `--editor` opens the editor. Experimental `-o/--onto`, `-A`, and `-B` insert the new commit.

State effects. Source and destination are rewritten; source may become empty and descendants receive new commit IDs. Change IDs of the surviving logical changes can persist, but moving content between changes changes which change owns the diff. Bookmarks at source/destination follow the resulting graph; remote bookmarks are not updated until push.

```
jj squash                                # current diff into parent
jj squash --from @ --into @- -- 'glob:src/**'
jj squash --use-destination-message
jj squash --interactive --tool meld
```

Trap. `--revision` mode is incompatible with experimental insertion options. Distinguish “copy these paths into a tree” (`restore`) from “move this diff ownership” (`squash`). Review descriptions and empty changes after a partial squash.

`jj status`

Reports high-level workspace status: current @, parent relationship, working-copy modifications, conflicts, and ignored/untracked information according to the workspace. Positional filesets restrict the status display.

```
jj status
jj status --ignore-working-copy
jj status -- 'glob:src/**'
```

By default status snapshots, so it can turn filesystem edits into a new working-copy commit. This is a major difference from a passive Git status check. Use the global ignore option when a prompt or monitor must not mutate repository state. Status does not move bookmarks or rewrite ancestors, except for the snapshot of @.

`jj tag`

Manages tag names, which are release/reference names rather than moving review bookmarks. Subcommands are `delete`, `list`, `set`, `track`, and `untrack`.

Tags can be local or remote and may be tracked, but their expected immutability and publication policy differ from a review bookmark. Use tags for release identity and bookmarks for moving work.

`jj tag delete`

Deletes local tags matching names/patterns. It changes local tag state; a tracked deletion can be pushed according to Git/tag policy. It does not erase the tagged commit.

```
jj tag delete 'v0.45.1-*'
```

jj tag list

Lists tags, optionally filtered by names, remote, tracking, or commit and rendered with a template according to local help. Use it to audit release names and remote tag state before a force-sensitive operation.

```
jj tag list
jj tag list --all-remotes
```

jj tag set

Creates or updates tags to a target commit. `-r/--revision REVSET` (alias `--to`) defaults to `@`; `--allow-move` permits moving an existing tag. Moving a release tag is a policy event, not ordinary diff editing.

```
jj tag set v1.2.0 -r @
jj tag set --allow-move v1.2.0 -r release-candidate
```

jj tag track

Associates local tag state with a remote tag. It changes tracking metadata, not the tag target or server.

```
jj tag track v1.2.0@origin
```

jj tag untrack

Removes a local/remote tag tracking relationship without deleting either tag. Review push behaviour after untracking because a later tag publication may be interpreted as new.

```
jj tag untrack v1.2.0@origin
```

jj undo

Undoes the most recent integrated operation by creating a reverse repository operation. It is the fast recovery path for a mistake noticed immediately.

```
jj undo
jj op log
jj redo
```

Undo does not automatically undo an already-pushed remote change, and it may not be the desired repair when several operations have happened. Use operation `restore/diff` for a named historical state and evolution for one change's identity.

`jj` **unsign**

Purpose and model. Removes cryptographic signatures from selected commits. It is metadata rewriting, not a content or ref operation.

Syntax/options. `jj unsign` accepts `-r/--revision REVSETS`; selection follows the command's default when omitted. Global immutability and repository-view options apply.

State effects. The affected commits and dependent descendants can receive new IDs while their trees remain unchanged. Change identities normally persist; bookmarks move with local successors and remotes remain unchanged until publication.

```
jj unsign -r 'mutable() & signed()'
jj log -r @ -T 'if(signature, "signed", "unsigned") ++ "\n"'
```

Trap. Do not unsign imported/public immutable history casually. Treat it like any other metadata rewrite and inspect the operation log.

`jj` **util**

Contains infrequently used administrative and generation commands: `backend`, `completion`, `config-schema`, `exec`, `gc`, `install-man-pages`, `markdown-help`, and `snapshot`. Treat this namespace as an implementation/tooling surface; commands can be version-sensitive and some can affect repository maintenance or external files.

```
jj util completion bash
jj util config-schema
jj util backend name
```

jj util backend and jj util backend name

Inspects backend information. The `name` subcommand prints the active backend name. These are diagnostics, not commands for changing commit content.

```
jj util backend name
```

Use this in bug reports and environment audits. A backend name does not describe colocation; check `jj git colocation status` separately.

jj util backend name

Prints the active backend name for diagnostics. It does not report colocation, Git remote configuration, or the current workspace. Pair it with `jj git colocation status` and `jj version` in an environment report.

```
jj util backend name
```

jj util completion

Generates shell completion for the requested shell. It is output generation, not repository mutation.

```
jj util completion bash > ~/.local/share/bash-completion/completions/jj
jj util completion zsh > ~/.zfunc/_jj
jj util completion fish > ~/.config/fish/completions/jj.fish
```

Install the generated file according to the shell's search path, then start a new shell or source it. Regenerate after a `jj` upgrade because command and option surfaces change.

jj util config-schema

Prints the machine-readable configuration schema for the installed version. It is the validation authority for key types and structure, not a human explanation of how keys interact.

```
jj util config-schema > config-schema.json
jj util config-schema | jq '.properties | keys'
```

The book's Appendix F interprets significant keys for humans; keep this schema in the research artefacts as the jj 0.45.1 machine-readable baseline. Do not treat every schema implementation detail as a stable public user-facing contract.

jj util exec

Executes a command in the repository/tooling context used by jj utilities. It is intended for internal or advanced integration use; ordinary shell commands should be run directly or through `jj run` when commits are the intended domain.

```
jj util exec -- command --arg
```

Read the installed help before scripting because argument forwarding and environment details are implementation-sensitive. It may have external side effects but does not by itself define a commit rewrite.

jj util gc

Runs repository object/storage garbage collection. It can remove unreachable data that would otherwise help recovery, so it is a maintenance boundary rather than a routine cleanup after every abandon.

```
jj op log
jj util gc
```

Run only after operation/recovery retention and remote publication state are understood. Garbage collection is distinct from commit abandonment, config GC, and Git remote pruning.

jj util install-man-pages

Installs generated manual pages for the current jj binary. It changes files outside the repository and should be run with an explicit destination/permission policy as shown by local help.

```
jj util install-man-pages --help
```

Regenerate after upgrading jj. Man pages are convenience documentation; local `jj help` remains the exact installed command authority.

jj util markdown-help

Generates Markdown help documentation from the installed command tree. This is useful for release audits and offline documentation, but generated output is not itself the authored book.

```
jj util markdown-help > jj-help.md
```

Keep it separate from the first-class HTML source. Compare generated help across binaries rather than silently mixing versions.

jj util snapshot

Forces or inspects a working-copy snapshot according to the installed utility help. It is useful in automation that deliberately owns snapshot timing, but it bypasses the convenient assumption that any ordinary jj command will establish the desired boundary.

```
jj util snapshot  
jj status --ignore-working-copy
```

Coordinate with editors, build tools, and other jj processes. Snapshotting can create an operation and rewrite the working-copy commit; use the operation log when diagnosing unexpected changes.

jj version

Prints jj version/build information. Use it as the first line of a bug report, validation log, and version-sensitive script.

```
jj version
jj --version
```

The book's normative baseline is jj 0.45.1, which is separately inventoried from the installed binary. Never infer feature availability from current online documentation alone.

jj workspace

Purpose and model. Manages multiple working directories over one repository commit/operation store. Subcommands are `add`, `forget`, `list`, `rename`, `root`, and `update-stale`. Each workspace has a local `@`; a graph rewrite is shared. A workspace is therefore a filesystem view plus a pointer, not a second repository or a Git branch.

Safety contract. A graph rewrite can make another workspace's working-copy commit stale even though its files are untouched. Inspect `workspace list` after rewriting shared ancestors, update or repair the affected workspace deliberately, and do not let two processes snapshot the same directory concurrently. Sparse patterns belong to the workspace's materialisation policy; they do not remove paths from commit trees.

```
jj workspace list
jj workspace add ../build -r @
jj workspace root
jj workspace update-stale
```

Use workspaces for build/review separation, not as lightweight bookmarks. Filesystem state and workspace metadata need a single-writer discipline.

jj workspace add

Creates a workspace directory. The destination is required. `--name NAME` chooses the workspace symbol; `-r/--revision REVSETS` supplies zero or more parents for its working-copy commit; `-m/--message` describes it; `--sparse-patterns copy|full|empty` controls sparse state and defaults to `copy`.

```
jj workspace add ../project-build --name build -r @ --sparse-patterns full
jj workspace add ../project-review --name review -r main@origin
jj workspace add ../project-ci -r @ --sparse-patterns empty
```

With no commits, the new working-copy commit shares the current workspace's parents; with explicit commits, it is like `jj new r1 r2 ...`. The new workspace has a distinct `@` and can be independently edited.

jj workspace forget

Forgets a workspace's metadata. It does not necessarily remove the directory or erase commits; inspect the filesystem and repository state separately before cleanup.

```
jj workspace list
jj workspace forget review
```

Do not forget a workspace whose unrecorded filesystem changes matter. A temporary bookmark or operation record can preserve graph recovery, but neither preserves uncommitted external files automatically. Forgetting the record also removes the convenient name used to select that workspace; it does not publish a deletion and does not alter a bookmark.

jj workspace list

Lists workspaces and their working-copy commits. `-T/--template` renders `workspaceRef` fields such as name, target, and optional root.

```
jj workspace list
jj workspace list -T 'name ++ " " ++ target.commit_id() ++ "\n"'
```

Use it to find which workspace owns a commit before rewriting or updating stale state. A root path can be absent or stale. Listing is also the quickest check that `@` in a command refers to the workspace you think it does; qualify a revision with `workspace@` when comparing workspaces.

jj workspace rename

Renames a workspace symbol. The directory path and commit graph are separate concerns; a renamed workspace changes symbols such as `review@`, not commits.

```
cd ../project-build
jj workspace rename ci
jj workspace list
```

Update scripts and templates that refer to the old workspace name. The operation remains in local operation history.

jj workspace root

Prints the root directory of the current workspace, equivalent to top-level `jj root`. It is useful from a nested subdirectory and differs from `jj git root` in a non-trivial backend layout.

```
jj workspace root
cd src && jj workspace root
```

jj workspace update-stale

Updates workspaces whose working-copy state is stale after shared graph changes. It is a repair/materialisation command, not a rebase selection command.

```
jj workspace update-stale
jj workspace list
jj -R ../project-build status
```

Inspect filesystem edits before updating. A stale workspace may contain useful unrecorded files, and update policy may materialise a new tree or conflict. Use a temporary copy/bookmark when the state is valuable. Updating the workspace

pointer does not rewrite unrelated commits, move publication bookmarks, or synchronise another clone.

Global option semantics

Option	Operational meaning	Use/caution
<code>-R/--repository PATH</code>	Selects a repository from any directory.	Useful for workspaces/scripts; it does not select a Git remote.
<code>--ignore-working-copy</code>	Does not snapshot or update the filesystem.	Reads a possibly stale commit; implied by <code>--at-op</code> .
<code>--no-integrate-operation</code>	Creates an operation without integrating it into the current op head.	Prints an ID for later integration; external side effects still occur.
<code>--ignore-immutable</code>	Disables immutable rewrite protection.	Use only with explicit audit/recovery intent.
<code>--at-operation OP</code>	Loads a historical repository view.	Read-only investigation; cannot snapshot today's filesystem into that view.
<code>--color MODE</code>	Controls colour/debug output.	Use never in scripts; debug reveals labels.
<code>--no-pager</code>	Disables pager.	Use for automation and captured fixture output.
<code>--config NAME=VALUE</code>	Applies an inline TOML override.	Repeatable; shell and TOML quoting both apply.
<code>--config-file PATH</code>	Loads an additional config file.	It is input, not necessarily the scope file selected by <code>config edit</code> .
<code>--debug</code>	Enables diagnostic output/behaviour.	Useful for bug reports, not stable machine output.

Global options are parsed before/alongside command options according to jj's CLI parser. Repeatable options such as `--config` accumulate. When combining a historical view, `--ignore-working-copy` is implied and filesystem state is deliberately not reconciled. When using `--no-integrate-operation`, the command's repository operation exists but another process may still advance the integrated operation head before you integrate it.

Global options

The 0.45.1 global surface includes `-R/--repository`, `--ignore-working-copy`, `--no-integrate-operation`, `--ignore-immutable`, `--at-operation/--at-op`, `--debug`, `--color`, `--quiet`, `--no-pager`, repeatable `--config`, and repeatable `--config-file`, plus `-h/--help` and `-V/--version`. The installed help is the option authority because defaults and accepted values are version-sensitive.

Version-specific notes

Verified against jj 0.45.1. This edition documents the command, configuration, `revset`, `fileset`, `template`, and default surfaces of jj 0.45.1. In particular, the target release provides `jj converge`, `direct --file PATH` targeting for relevant configuration commands, tracked and untracked `remote-tag` predicates, the documented Gerrit defaults, and the non-Windows pager default `less -FRXK`. Re-run the local inventory after a future upgrade because these are versioned interfaces.

Command contracts: how to predict side effects

The command reference is easiest to use when every invocation is classified along four axes: snapshot, selection, rewrite, and publication. Before running a command, ask four questions:

1. Will `jj` read the filesystem and possibly create/update the workspace commit?
2. What is the commit cardinality: zero-or-one default, exactly one, or arbitrary set?
3. Does the command rewrite selected commits, their descendants, or only metadata?
4. Can it move a bookmark, import/export a Git ref, or perform network I/O?

The answer predicts most side effects. For example, `jj log -r 'all()'` is a set-valued observation and normally does not rewrite commits. `jj edit A` is singleton-valued, snapshots before moving the workspace, and changes filesystem

state. `jj rebase -s 'B::D' -d main@origin` is set-valued source selection, graph-rewriting, descendant-affecting, and non-networked. `jj git push --bookmark review --remote origin` is ref-publication and networked; it does not rewrite the local commit graph merely because the server accepts it.

Dimension	Inspection example	Mutation example	Question to record in a script
Filesystem snapshot	<code>status, diff</code>	<code>new, describe, commit</code>	Are unsaved edits part of the intended operation?
Commit graph	<code>log, show</code>	<code>rebase, split, abandon</code>	Which source set and destination set?
Content ownership	<code>diff, interdiff</code>	<code>squash, restore, absorb</code>	Which tree contributes which paths?
Local refs	<code>bookmark list</code>	<code>bookmark move/set/delete</code>	Which name should point where after the operation?
Remote state	<code>git fetch, remote bookmarks</code>	<code>git push, gerrit upload</code>	Which remote and publication ref are explicit?
Operation state	<code>op log/show/diff</code>	<code>undo, op restore/revert</code>	Can this be reversed and how will it be identified?

Rewriting commands: source, destination, and ownership

`rebase, new, duplicate, parallelize, and simplify-parents` change parent relationships. `squash, absorb, restore, diffedit, and split` change tree/diff ownership. `describe, metaedit, and signing` commands change metadata. The categories overlap in descendant effects, but keeping them separate prevents the wrong mental shortcut.

For a parent-edge operation, write the old and desired edge before selecting flags:

```
old:  A--B--C--D
want: A--C--B--D      (reorder)
want: A--B----D      (remove C contribution)
want: A--B--C
      \--D      (fork/parallelise)
```

Then choose the operation that expresses that edge change. Do not start with “which Git command resembles this?” Git’s interactive rebase todo is a user interface for several transformations; `jj` exposes those transformations separately and applies them to commit sets.

Rebase destination modes

In 0.45.1, `-o/--onto` and its destination spelling place source commits on a new parent. `-A/--insert-after` and `-B/--insert-before` express relative placement. The aliases in the local help are important because examples copied from different `jj` releases can use different long names. Use the long form in the book’s prose and include the short form only where it improves terminal ergonomics.

```
jj rebase --revisions 'C::D' --onto main@origin
jj rebase --source C --insert-after B
jj rebase --branch C --insert-before D
jj rebase -r 'C::D' -d main@origin
```

A source set and a destination set can each contain multiple commits. Multiple destinations can create a merge-shaped graph; multiple sources can move a disconnected set only when the command’s topology rules permit it. Inspect the result with `jj log -r 'all()'` and a parent template. The visual graph alone can hide that a node gained two parents.

```
jj log -r 'C::D' -T '
commit_id.short() ++ " parents=" ++
parents.map(|p| p.commit_id().short()).join(",") ++ "\n"
'
```

Content commands: tree endpoints versus diff endpoints

`restore` copies selected file contents; `squash` moves a diff into another change; `diffedit` changes a selected diff through an editor. The commands may produce similar final files but differ in which logical change owns the content and which description/evolution receives the successor.

```

B tree: implementation
C tree: implementation + tests

restore from C into B:
  B' tree = implementation + tests
  C may be unchanged

squash C into B:
  B' owns implementation + tests
  C becomes empty/removed according to operation

diffedit C:
  C' diff is edited; B is not destination

```

When a user says “bring these changes into B”, ask whether the source should remain, whether its logical review should survive, and whether the selected paths are a diff or a complete tree. Those answers determine the command.

Inspection and stable output

`log` is a graph-oriented view; `show` is a commit-oriented view; `diff` is a tree/diff view; `status` is a workspace/filesystem view. They answer different questions even when they display overlapping information.

Need	Command	Why not the nearest alternative?
Find a topology or ref	<code>jj log -r REVSET</code>	<code>show</code> focuses on one commit and may hide the surrounding graph.
Inspect one commit's metadata/tree	<code>jj show REV</code>	<code>log</code> may omit the full diff and file state.
Understand diff ownership	<code>jj diff --from X --to Y</code>	<code>status</code> only sees workspace differences.
Understand disk versus @	<code>jj status</code>	<code>log</code> can snapshot or omit the filesystem dimension.
Understand a rewrite	<code>jj op diff</code> and <code>jj evo log</code>	Commit IDs alone do not explain successor relationships.

For machine output, use a template with explicit field delimiters and a controlled revset. Include full IDs when the output is evidence; use short IDs only for human

reports. Never parse graph characters, colour labels, or the default description wrapping. The default template is a user interface and can change between releases.

Global options and their combinations

`--ignore-working-copy` is appropriate for a monitoring process that must not snapshot a half-written build tree. It can also make a human answer stale if unrecorded filesystem edits matter. `--at-operation` provides a historical repository view; combining it with the live filesystem is usually misleading because the files on disk belong to the current workspace, not the historical operation.

```
jj --ignore-working-copy log -r 'all()'
jj --at-operation OP log -r 'all()'
jj --at-operation OP --ignore-working-copy show @
```

`--ignore-immutable` changes the permitted rewrite boundary and should be visible in logs. `--no-integrate-operation` changes operation-head integration and should be used only by tooling that records the resulting operation.

`--repository` changes the repository/workspace context; it does not turn a path into a revset. `--config` and `--config-file` alter interpretation and are therefore part of the command's reproducibility record.

Command failure taxonomy

Most `jj` errors fall into a small set of categories:

- **Resolution:** a symbol is unknown or a prefix is ambiguous. Inspect refs, IDs, and aliases.
- **Cardinality:** a command expected one commit but got zero or several. Narrow the revset and print IDs.
- **Policy:** a rewrite reaches an immutable commit or a push violates ref safety. Do not bypass until intent is established.

- **Tree/fileset:** a path is absent, sparse, conflicted, or outside the selected tree. Inspect with `file list` and `diff`.
- **Editor/tool:** an external program failed or returned an invalid result. Re-run with a builtin or explicit tool.
- **Transport:** credentials, remote URLs, server permissions, or non-fast-forward state failed. Fetch and inspect before retrying.
- **Operation concurrency:** an operation was created outside the integrated head or another writer changed state. Inspect operation history and integrate deliberately.

A failed command may have performed an automatic snapshot before reporting its main error. Check `jj op log` rather than assuming failure means no repository state changed. The operation record is the authoritative place to determine what was committed locally.

Version-aware CLI use

Scripts should probe the binary before depending on a release-sensitive feature. Do not test for a feature by running a destructive command and interpreting its error as a capability probe. The safe pattern is to compare `jj version`, inspect help in installation tests, and select a command alias or argument set accordingly.

```
case "$("$JJ" version | sed -n 's/^jj \([0-9][^ ]*\).*/\1/p')" in
  0.45.1.*) # this book's target release
    ;;
  *)      printf 'unsupported jj version\n' >&2; exit 2 ;;
esac
```

For this edition, `jj 0.45.1` is normative. After upgrading, compare the installed help and schema because command options, builtin templates, and schemas are not frozen interfaces.

Command contracts: how to read an advanced jj command

A command reference is useful only when it describes the state transition, not just the spelling. For each command, ask four questions: what repository value is selected, what is rewritten or observed, which workspace is affected, and which refs or remote state remain unchanged. The same global option can alter several commands: normal snapshotting may turn an apparently read-only invocation into an operation, `--ignore-working-copy` reads a stale recorded tree, `--at-operation` selects a historical repository view, and `--ignore-immutable` changes only rewrite protection.

Contract field	Question to answer	Evidence to show
Selection	Does the command accept one commit, a set, a range, or a fileset?	Syntax and a cardinality/error example.
Mutation	Does it create, rewrite, hide, or merely render?	Before/after commit IDs and operation entry.
Descendants	Will parent identity changes rebase children?	Graph showing successor IDs or conflicts.
Workspace	Does it move/edit @, materialise files, or leave the filesystem?	Workspace-local log and filesystem check.
Refs	Does it move bookmarks/tags or only alter commits?	Local/remote ref listing before and after.
Network	Does it contact Git/Gerrit or change only local observations?	Remote bookmark/ref state and explicit remote option.
Recovery	What operation ID and evolution entry does it create?	<code>op show</code> and <code>evo log</code> evidence.

Rewrite commands

`jj describe` changes the description of one mutable commit, usually `@`. It preserves the tree but changes commit metadata and therefore the commit ID; descendants can receive successors. `jj metaedit` performs metadata surgery such as author/timestamp and can intentionally update the change ID. `jj edit` moves one workspace pointer to one existing commit; it does not create a successor until a snapshot or rewrite occurs. `jj diffedit` edits a diff through a diff editor. These commands look similar from a Git perspective but have different state targets.

```
jj describe -r 'change_id(API_CHANGE)' -m 'API: reject negative quota'
jj metaedit -r 'change_id(API_CHANGE)' --update-author-timestamp
jj edit 'change_id(API_CHANGE)'
jj diffedit -r 'change_id(API_CHANGE)' -- 'root-prefix-glob:"src"'
```

`jj squash` moves the current or selected diff into a destination change. A partial fileset makes it a content-ownership operation; an unselected diff hunk stays at the source boundary. The destination's description and change identity are not the source's by default. Descendants of the destination can be rebased, and the source may become empty. If the source should remain as a separate logical change, use `duplicate` or `restore` instead of `squash`.

`jj split` turns one change into multiple changes. It is a boundary operation: the first resulting change receives selected content and the second receives the remainder according to the mode. Existing descendants are rebased over the new sequence. Treat the interactive editor as a content selector, then verify with two diffs and a parent-edge query. A split is not “make a branch”; it changes the diff stack's granularity.

`jj absorb` routes working-copy changes into suitable mutable ancestors. It is powerful precisely because it is a policy-driven multi-target rewrite. Before using it, inspect which ancestors are mutable, which files each owns, and whether the current change contains unrelated work. Use a narrow revset/fileset and an operation checkpoint. If an individual diff hunk matches several ancestors or no

ancestor, the command's behaviour follows its configured applicability rules; do not promise ownership without inspecting the result.

`jj duplicate` creates a new logical change with copied content. It intentionally breaks evolution continuity. `jj abandon` removes a commit from ordinary visible work and reconnects or updates dependants according to graph rules. `jj restore` restores selected file contents or reverses a selected change. `jj revert` creates a new inverse change. The last two are both content operations but differ in whether the inverse is applied to an existing destination or represented as a new commit.

Rebase as a two-dimensional contract

`jj rebase` has a source dimension and a destination dimension. Source `-r/--revisions X` selects explicit commits; `-s/--source X` selects X and descendants; `-b/--branch X` selects the branch of X relative to the destination. Destination `-o/--onto Y` places the selected source on Y; repeating `-o` in `jj 0.45.1` creates multiple parents; `-A/--insert-after Y` inserts selected changes after Y; `-B/--insert-before Y` inserts them before Y. The names are not cosmetic: changing `-r` to `-s` can turn a one-commit repair into a whole-stack rewrite.

before: base--A--B--C

 \
 X--Y

<code>-r B -o X:</code>	<code>base--A--B'--C'</code>	and X--B' if destination is X
<code>-s B -o X:</code>	<code>base--A--B'--C'</code>	all descendants selected/rebased
<code>-r B -A X:</code>	<code>base--A--X--B'--C'</code>	insert the selected diff after X
<code>-r B -B X:</code>	<code>base--A--B'--X'--Y'</code>	rebase X and descendants after B'

The diagrams show the shape, not a promise that every shorthand combination is accepted by every release. Check local help and inspect the selected set before execution. A source with holes may preserve an edge to an unselected parent, may be rejected, or may cause a split of the selected roots according to the rebase algorithm. This is why `jj log -r SOURCE`, `jj log -G -r SOURCE`, and a temporary bookmark are useful preconditions.

After a rebase, compare four things: each selected change ID, each new commit ID, each descendant's diff, and every affected bookmark. Then query conflicts and operation detail:

```
jj log -G -r 'SOURCE | descendants(SOURCE)'
jj evolog -r 'SOURCE'
jj log -r 'conflicts() & descendants(SOURCE)'
jj bookmark list --all-remotes
jj op show -p
```

Inspection and diff commands

`jj log` accepts a many-valued revset and renders a graph or a template. It normally snapshots first, so “inspection” is not synonymous with “no operation”. Use `--ignore-working-copy` for a stable recorded view and `--no-pager` `--color=never` for automation. `jj show` focuses on one commit's metadata, tree, and diff; it is a good cardinality check before editing. `jj diff` compares a commit's diff, endpoints, or selected set and accepts a fileset independently.

```
jj log -G -r 'roots(main@origin..@)::'
jj show -r 'exactly(change_id(API_CHANGE), 1)'
jj diff --from main@origin --to @ -- 'root-prefix-glob:"src"'
jj diff -r @ --summary --no-pager
```

`jj interdiff` compares two versions of a logical change, not two arbitrary branch tips. Use it after a rewrite when the commit IDs differ but the review question is “what changed between patch set 1 and patch set 2”. `jj evolog -p` answers a related question by following a change's evolution and showing predecessor/inter-diff information.

Bookmarks and tags

`bookmark create` creates a local name at a commit; `set` creates or moves a name; `move` moves an existing name; `advance` moves it to a descendant under the command's safety rules; `rename` changes the local name; `delete` expresses local deletion suitable for publication; `forget` removes local knowledge without being the same as a remote delete; `track/untrack` manage a relationship with a remote bookmark; and `list` renders current state. None of these commands makes a bookmark follow @ automatically.

```
jj bookmark create pr/cache -r @
jj bookmark set pr/cache -r 'change_id(CACHE_CHANGE)'
jj bookmark move pr/cache --to @
jj bookmark track pr/cache --remote origin
jj bookmark list --all-remotes
jj bookmark forget pr/cache
```

Tags are release-oriented names with their own command family. A tag's presence in an immutable-head policy can protect an older commit even when no local bookmark reaches it. Fetch and tag tracking affect local observations; moving a tag is an intentional release/ref change. Display local and remote target states separately before any deletion or force publication.

Git transport commands

`jj git clone` creates a jj repository from a Git remote; `git init` initialises the backend and can colocate with an existing Git repository; `git fetch` imports remote objects and updates remote bookmarks; `git push` exports selected bookmarks/commits; `git import` notices Git-side ref changes; and `git export` writes jj refs/objects to the Git view where required. `git root` identifies the Git root and `git remote` manages the transport configuration. Colocation makes native Git visible but creates a second writer boundary.

```
jj git remote list
jj git fetch --remote origin
jj git import
jj git push --remote origin --bookmark pr/cache
GIT_DIR=$(jj git root) git show-ref
```

A fetch changes local remote observations, not the server and not the local mutable graph. A push is external side effect and may be rejected; inspect tracking and commit ancestry first. Explicit remote and bookmark selectors are safer than broad defaults, especially when GitHub and Gerrit coexist.

Operation, configuration, and workspace contracts

Operation commands act on the operation DAG. `op log` lists transformations; `op show` explains one; `op diff` compares views; `op restore` restores selected repository state; `op revert` reverses a selected operation; `op abandon` prunes an operation record; and `op integrate` joins an unintegrated operation. The `op` spelling is an alias, but the canonical family is `operation` in the 0.45.1 command tree.

`config get/list/path` are read operations with different output contracts; `set/unset/edit` mutate configuration; `gc` removes obsolete config entries according to its rules. A config command can target a scope or load an additional file. In jj 0.45.1, `--file` provides direct file targeting for relevant subcommands.

`workspace add` creates a second working directory over the shared store; `list` displays records; `root` reports the current directory's workspace root; `rename` changes the current workspace name and takes one new name in 0.45.1; `update-stale` repairs stale workspace state; and `forget` removes metadata without abandoning its commits. Treat workspace filesystem updates as a separate side effect from graph rewrites.

Content, conflict, and maintenance commands

`file list/show/search/annotate` inspect trees and content; `file track/untrack` change snapshot tracking; `file chmod` changes executable metadata. `sparse set/list/edit/reset` controls materialisation, with `set` accepting patterns and `edit` opening an editor without positional patterns. `resolve` operates on stored conflicts; `fix` applies configured fixers; `sign/unsign` rewrite cryptographic metadata; `bisect` and `bisect run` perform graph search using script exit status; and `run` executes a command over selected commits, potentially rewriting them unless changes are ignored.

Administrative commands have external or destructive boundaries. `util gc` can remove recovery data; `util completion` writes shell output; `util install-man-pages` writes files outside the repository; `util config-schema` produces a machine-readable schema; `util snapshot` controls snapshotting; and `util exec` forwards commands according to its own contract. Always read the installed help for a utility before placing it in automation.

A reader's command validation protocol

1. Run the exact command with `--help` under the target `jj` binary.
2. Identify revset and fileset positions and quote them as one shell argument.
3. Build the smallest graph that exposes source, destination, descendants, refs, or conflicts.
4. Record filesystem state and the working-copy commit before invocation.
5. Run the command and capture the operation ID.
6. Compare graph edges, commit IDs, change IDs, bookmarks, remote bookmarks, and conflicts.
7. Inspect `op show` and `evolog` when the command rewrites.
8. Repeat the experiment after an upgrade whenever the release notes or help show a changed surface.

This protocol is deliberately more demanding than a successful exit status. Use it whenever a command selects or rewrites more than one graph component, especially in automation.

Part XVI — Advanced mechanics

Immutability and safety boundaries

Configure immutable heads to protect imported/public history. The check applies to rewrites; the revset/template notions of immutable state remain observable even with `--ignore-immutable`. Hidden commits and divergent change IDs are useful forensic states, not garbage to delete reflexively.

Evolution, concurrency, and operation merging

Run concurrent read commands with a clear snapshot policy.

`--no-integrate-operation` lets automation create an operation without changing the integrated operation head; it does not suppress side effects such as a network push. Inspect and integrate deliberately.

`jj fix` and `jj run`

Fix tools are configured programs that transform files. `jj fix` can operate over a commit set and may rewrite a stack. In `jj 0.45.1`, `jj run` uses private working copies for selected commits, allowing checks or formatters to update changes and propagate conflicts. Treat command exit status and operation boundaries as part of your automation contract.

```
jj run -r '@:: & mutable()' -- cargo test
jj run -r 'mutable() & description(substring:"API")' -- cargo fix
jj fix -s 'mutable()'
```

Bisect, tags, signing, completion, and maintenance

`jj bisect run` automates commit bisection; tags provide stable release names; `jj sign/unsig` manage supported cryptographic metadata; `jj util completion` generates shell completion. Use `jj util gc` and backend maintenance only after understanding operation/recovery retention. In large repositories, narrow revsets, use immutable remote boundaries, avoid rendering enormous diffs, and use `--ignore-working-copy` for stable monitoring reads.

Machine-readable output

Templates plus `--color never`, explicit separators, and a controlled revset are the most portable interface. Do not parse the human graph glyphs. Pin aliases and templates in a versioned config, and run the inventory/fixture checks after upgrading.

Advanced mechanics in practice

Immutable heads are a policy boundary

Immutability is rewrite protection expressed as a revset. It is not the same thing as committed, pushed, on a branch, or old. A mutable commit can be published, and an immutable commit can be local. The useful policy question is which graph boundary must not be rewritten by an ordinary command.

```
jj log -r 'immutable_heads()'
jj log -r 'mutable() & ::@'
jj rebase -b 'roots(main@origin..@)' -o main@origin
```

If a source set reaches an immutable commit, `jj` protects it unless `--ignore-immutable` is supplied. That flag is an explicit override, not a repair to the graph. Save the operation ID, explain which boundary is being crossed, and verify the result with operation and evolution logs.

State	Ordinary rewrite	Publication meaning
Mutable, unpublished	Usually permitted.	No remote ref necessarily exists.
Mutable, published	Locally permitted; push may need a safe non-fast-forward update.	Server sees a Git ref, not jj mutability.
Immutable imported trunk	Rejected without explicit override.	Remote bookmark is an observation and likely safety boundary.
Hidden predecessor	Not in ordinary visible selection.	May remain recovery evidence.

Hidden commits, divergent changes, and evolution

Abandoning a commit hides it from ordinary visible history but does not immediately erase its objects. Rewriting a change creates successors, and independent successors can be divergent. These states let jj preserve evidence instead of forcing every rewrite to destroy it.

```
jj log -r 'hidden()'
jj log -r 'divergent()'
jj evolog -r 'change_id(abc...)'
jj log -r 'all() & present(commit_id("prefix"))'
```

A hidden predecessor may be reachable from an operation view or evolution entry. A divergent change ID can have several visible versions. Use an explicit offset or full commit ID when a command requires one commit. Do not select an ambiguous change ID merely because a log can display the set.

Concurrency and operation integration

jj operations support concurrent processes, but that is not permission for multiple processes to mutate one working directory without coordination.

`--no-integrate-operation` creates an operation and prints its ID without advancing the integrated operation head.

```
jj status --no-integrate-operation
# record OP_PRINTED
jj --at-op OP_PRINTED log -r 'all()'
jj op integrate OP_PRINTED
```

The operation can still have external side effects: a Git push reaches a server, an editor writes files, and a fixer modifies a private working copy. Integrating later does not roll those effects back. A concurrent operation can advance the current head before integration; inspect `jj op log` and `jj op diff` first.

For concurrent read-only automation, use `--ignore-working-copy` with explicit templates. For concurrent filesystem writers, assign one process to one workspace. A build workspace and a development workspace can run simultaneously because their filesystems differ; two processes snapshotting the same workspace remain a race.

Fixers and modified-line protocols

A fixer is a configured program invoked over commits, often with a protocol describing changed lines. This can update a stack without rewriting unrelated historical lines, but only when the program supports the protocol correctly.

```
[fix.tools.rustfmt]
command = ["rustfmt", "--emit", "stdout"]
patterns = ["glob:*.rs"]

jj fix -s 'roots(main@origin..@)::' -- 'root-prefix-glob:"src/**"'
```

Validate the exact fixer keys and protocol against the target schema and help. Use `--all-lines` when the tool cannot safely act on modified ranges. Use `--include-unchanged-files` only when a repository-wide invariant is intentional: it can rewrite substantially more history.

Fix can affect many mutable commits and descendants. Run it on a small stack, inspect changed paths and descriptions, record the operation ID, and undo before another fix if the result is too broad.

jj run as a graph-aware test harness

jj run has two distinct modes. With `--ignore-changes` it tests commit states. Without it, files written by the command are recorded back into selected commits, so it is a transformation tool. Reused private working copies preserve build artefacts; `--clean` removes that state.

Goal	Controls	Reason
Check commits	<code>--ignore-changes</code> <code>--clean</code> <code>--passthrough</code>	No rewrite; clean state; interactive output.
Apply formatter	Narrow <code>-r</code> , intentional rewrite, explicit tool.	Scope and graph effects are visible.
Parallel tests	<code>-j N</code> , captured output.	Avoid interleaved logs.
Interactive tool	<code>--passthrough</code> , one job.	TTY-like output; stdin is not inherited.

```
jj run --ignore-changes --clean -r 'main@origin..@' -- ./ci/check.sh
jj run -j 4 --ignore-changes -r 'heads(mutable())' -- ./ci/check.sh
jj run -r '@:: & mutable()' -- ./format-current-tree.sh
```

The current directory is the invocation directory unless `--root` is supplied. Use `--` before command arguments beginning with a dash. Capture exit status and inspect the operation when changes are permitted.

Bisect, signing, and release boundaries

Bisection assumes a monotonic predicate over the selected graph range. Merge-heavy graphs, non-deterministic tests, and environment-dependent failures violate that assumption. Define the range, make the script exit status unambiguous, and preserve the discovered commit before leaving the investigation.

```
jj bisect run --range 'good..bad' -- ./test-regression.sh
jj show -r 'bisect(@)'
```

jj sign and jj unsign rewrite metadata. Signing an ancestor changes descendant commit IDs even when tree content is unchanged. Presence of a signature is cheaper to query than status/key/display, which can invoke GPG or

SSH verification.

```
jj sign -r 'heads(bookmarks())'  
jj log -r 'signed()' -T 'commit_id ++ " " ++ if(signature, signature.status(), "unknown") ++ "\n"  
jj unsign -r 'mutable() & signed()'
```

Tags name release points. Treat them as immutable policy even when

`--allow-move` exists. Remote tag tracking is separate from bookmark tracking; `jj 0.45.1` provides tracked/untracked remote-tag predicates, and target-release scripts can use the supported forms from local help.

```
jj tag set v2.4.0 -r @  
jj tag list --all-remotes  
jj git push --remote origin --tag 'v2.4.0'
```

Performance and machine-readable output

Cost is driven by selected graph size, tree/diff materialisation, external tools, signature verification, remote URL work, and rendering. Narrow the revset before optimising the template. A template that calls `diff().git()` for every commit can be much slower than one that renders IDs and subjects.

Concern	Technique
Prompt latency	<code>--ignore-working-copy -r @</code> , no diff, cached output.
Large log	<code>-n</code> , <code>heads()</code> , explicit template, no full diff.
Slow signatures	Query <code>signed()</code> ; verify selected commits only.
Large fix/run	Narrow commits/fileset; control jobs and clean/reuse.
Recovery retention	Do not GC until operation/evolution evidence is no longer needed.

For automation use explicit revsets, templates, `--color=never`, `--no-pager`, a pinned `jj` binary, and a fixture test. Human graph glyphs and default templates are presentation, not a stable API.

Fixers as controlled graph transformations

`jj fix` is not merely a formatter command. It can run configured tools over selected commits and turn their filesystem changes into successor commits. Its safe use therefore combines a revset, a fileset/pattern policy, a tool protocol, and a review of descendant effects.

```
selected stack:
A--B--C--@      each has Rust files

fix over 'mutable() & ::@':
A'-B'-C'-@'     if each fixer result is clean
                or
A'-B'[conflict]--C'[conflict]--@'
```

Fixers should be idempotent: running them twice should produce no new content after the first successful run. They should preserve files outside their patterns, return a meaningful exit status, and not depend on the user's editor or network. A tool that rewrites generated files or line endings must be explicitly scoped. Run it on one commit in a fixture before applying it to `mutable()`.

```
jj fix -s @
jj diff -r @
jj op show
jj fix -s 'mutable() & ::@' -- 'root-glob:src/**/*.rs'
jj log -r '::-@ & conflicts()'
```

The exact fixer table belongs in configuration and the command's accepted fileset form is version-sensitive. The invariant is independent: select commits first, select paths second, run a deterministic tool, then inspect the full evolved stack.

jj run: commit-scoped automation

`jj run` provides a controlled way to execute a shell command in private working copies for a commit set. This is useful for tests, formatters, and checks that need each commit's tree rather than the current workspace. The private workspaces are an implementation mechanism; the important contract is whether filesystem changes are imported back as commit rewrites.

Mode	Use	Graph risk
<code>--ignore-changes</code>	Run a read-only check or discard tool writes.	Low graph mutation; tool can still affect external state.
default	Allow command changes to update selected commits.	Successors and descendant rebases are possible.
<code>--clean</code>	Do not reuse prior private workspaces.	More predictable, slower, avoids stale generated files.
<code>-j N</code>	Run independent commits concurrently.	Output interleaves; tools must not share mutable external files.
<code>--passthrough</code>	Expose one job's live output.	Incompatible with multi-job presentation.
<code>--root PATH</code>	Run command at a chosen root.	Relative paths and generated artefacts can differ.

```
jj run --ignore-changes --clean -j 4 -r 'main@origin..@' -- ./ci/check.sh
jj run --clean -r '@:: & mutable()' -- ./format.sh
```

Do not use an ordinary workspace path as a lock-free coordination directory for parallel jobs. A test that writes to `/tmp/result`, a shared compiler cache with unsafe locking, or a generated file outside the private root can make independent commits interfere. Capture each job's status and map it back to the full commit ID. A successful command may still leave a failed graph rewrite if the tool's exit status was misreported.

Bisect as a predicate search

Bisection is a search over a commit set given a monotonic predicate: there is a boundary where commits are good on one side and bad on the other. The graph need not be linear, but the selected range and the meaning of “between” must be clear. A test that is flaky or depends on an unrelated descendant invalidates the result.

```
jj bisect run --range 'good..bad' -- ./regression-test
# inspect the candidate
jj log -r 'bisect(@)'
jj show 'bisect(@)'
```

For a merge-heavy history, choose a first-parent or release corridor if the bug is associated with integration order. A query over all ancestors can find a commit that introduced the file change but not the first integration that made the system fail. Document the test environment, dependency versions, and whether the command mutates the candidate tree.

Signing and cryptographic evidence

Signing operates on a specific commit object. Rewriting description, metadata, parent, or tree produces a different object that needs independent verification/signing. A signature does not make a change immutable and does not make a bookmark safe to force-update. In a review workflow, sign immediately before publication or sign every successor as policy requires.

```
jj sign -r @ --key KEY
jj log -r @ -T 'commit_id ++ " " ++ if(signature, signature.status(), "unsigned") ++ "\n"'
jj unsign -r @
```

Separate cryptographic evidence into three questions: does the object have a valid signature, does the key belong to the expected identity, and does the hosting server trust that identity? `jj` can answer the first and expose metadata for the second; GitHub/Gerrit policy answers the third. Hardware tokens and agents are external dependencies; test them in the target environment and verify the resulting signature

with the policy used by the host.

Tags and release boundaries

Tags name release graph positions and are deliberately less fluid than review bookmarks. Treat a tag as an immutable publication boundary in policy even if the underlying ref can technically be moved. Fetch tags explicitly when a release query depends on them, and distinguish local tags, remote tags, tracked remote tags, and untracked remote tags in jj 0.45.1.

```
jj tag list
jj log -r 'tags()'
jj log -r 'remote_tags()'
# 0.45.1:
jj log -r 'tracked_remote_tags()'
jj log -r 'untracked_remote_tags()'
```

In jj 0.45.1 the default immutable-head definition includes untracked remote tags, and the corresponding tracked/untracked tag predicates are available. This affects rewrite protection, not just display. A release script should use an explicit release tag and record the binary version rather than assuming that a future release has the same default immutable set.

Large repositories and performance

Performance problems often come from asking for a broad graph, rendering a large diff, or repeatedly snapshotting a large working tree. Narrow the revset, use bounded limits, separate graph selection from diff display, and use `--ignore-working-copy` for repository-only monitoring. Do not solve a slow query by making its result semantically broader.

```
jj log -r 'heads mutable() & ::@)' -n 30 --no-pager
jj log -r 'main@origin..@' --no-graph -T 'commit_id.short() ++ "\n"'
jj diff --stat --from main@origin --to @
jj status --ignore-working-copy
```

Filesystem monitoring, sparse working copies, and private run workspaces can improve latency, but they introduce state that must be inspected when results are surprising. Measure with representative repositories. A prompt that runs an all-history template on every shell command is a design problem even if jj itself is fast.

Debugging and bug reports

A reproducible jj bug report should include version, backend, repository layout (colocated or not), workspace name/root, operation ID, exact command with redacted paths, revset/fileset, configuration layers, and whether the filesystem contained unrecorded changes. Include `jj op show` and a minimal graph rendered with full IDs if the issue is a rewrite.

```
jj version
jj root
jj util backend name
jj git colocation status
jj workspace list
jj op log -n 10
jj config list --include-defaults
jj log -r 'all()' --no-graph -T 'commit_id ++ " " ++ change_id ++ "\n"'
```

Use `--debug` when requested by jj's diagnostics, but redact credentials, remote URLs containing tokens, private descriptions, and filesystem paths. The operation log may contain descriptions and usernames; it is repository data, not automatically safe to attach publicly.

Advanced mechanics laboratory: automation, signing, and maintenance

Running commands over commits

`jj run` provides temporary working environments for running a command against a commit set. It is useful for testing a stack, applying a deterministic fixer, or collecting a result at several graph points. It is not merely a loop around `jj edit`: the command can use private workspaces and, unless changes are ignored, files written by the process may become commit rewrites.

```
jj run --ignore-changes --clean -r 'main@origin..@' -- ./check.sh
jj run --ignore-changes -j 4 -r 'heads(mutable() & ::@)' -- ./test.sh
jj run --passthrough -r @ -- ./interactive-check.sh
```

Option/policy	Effect	Use when
<code>--ignore-changes</code>	Do not record files written by the command as commit changes.	Read-only tests or tools whose outputs are disposable.
<code>--clean</code>	Do not reuse existing private workspaces.	Reproducible runs that must not inherit files.
<code>-j N</code>	Run selected commits concurrently in <code>jj 0.45.1</code> .	Independent deterministic tests; not a shared mutable worktree.
<code>--passthrough</code>	Connect command output and constrain execution mode.	Interactive diagnostics, not machine aggregation.
<code>--root</code>	Choose the command's working root.	Scripts that must not depend on the caller's <code>cwd</code> .

Parallel execution requires test isolation. A test that writes to a shared cache, local database, or fixed temporary filename can make a graph query appear nondeterministic. Pin environment variables, use per-commit temporary directories, and record the selected commit in output. If the command is intended to rewrite commits, run one small fixture first and inspect its operation/evolution results.

Fixers are graph rewriters

`jj fix` applies configured fixer tools to selected commits and paths. A formatter is not automatically a safe no-op: line-ending changes, generated output, executable bits, and tool version changes alter commit trees and therefore commit IDs. A fixer that runs over an ancestor can trigger automatic descendant rebasing. Treat fixer configuration as both a tool policy and a graph-selection policy.

```
jj fix -s 'mutable() & ::@ & ~conflicts()' -- 'root-glob:src/**/*.rs'  
jj log -r '::-@' -T 'change_id.short() ++ " " ++ commit_id.short() ++ "\n"  
jj op show -p
```

Test a fixer in four states: unchanged input should produce no successor or a clearly explained no-op; changed input should have the expected paths; a conflict should be skipped or handled according to policy; and a sparse workspace should not silently imply that absent paths were checked. Keep generated files out of a source formatter unless the generator is part of the intended operation.

Bisect as graph search

Bisect narrows a commit set using a predicate supplied by the user or a script. The script's exit status is the test contract, so distinguish “known good”, “known bad”, and “infrastructure failure” in the script. A flaky or environment-dependent test can produce a plausible but wrong boundary. Keep the bisection range explicit and save the discovered commit before leaving the temporary checkout.

```
jj bisect run --range 'main@origin..@' -- ./test-regression.sh  
jj show REVISION
```

There is no separate `jj bisect log` or `jj bisect reset` command in `jj 0.45.1`. Inspect the candidate with ordinary `jj log -r 'bisect(@)'` or `jj show -r 'bisect(@)'`, and use `jj op log/jj undo` when you need to inspect or recover the operation sequence. A bisect result is not a diff to publish: it is evidence about graph history. If it identifies an ancestor that has since been rewritten, use change/evolution information to locate the current successor and verify the test at that object.

Signing and verification

Signing rewrites commit metadata and can therefore create successor commit IDs. It does not make a change immutable, publish it, or establish a Gerrit review identity. Verification status is a property of a cryptographic signature and key environment; a template can display it, but a successful local signing command does not prove that a remote host will trust the key.

```
jj sign -r 'bookmarks() & mutable()'
jj log -r 'signed()'
jj show -r @
jj unsign -r 'change_id(CHANGE)'
```

Define whether signing applies before or after review publication. Signing an already pushed commit may require a force update because the exported commit object changes. In a stack, signing an ancestor changes descendant commit IDs as well. Test with a disposable key or a controlled agent, keep private material out of repository config, and record the host's verification policy separately.

Tags as release boundaries

A tag names a release point rather than serving as a daily work pointer. Treat tag movement as an exceptional publication change, even when the local command permits it. A release query should name the tag explicitly and record the target commit ID, version, and binary used to compute the set.

```
jj tag set v2.4.0 -r @
jj log -r 'v2.4.0:: & ::main@origin'
jj tag list
jj tag track v2.4.0 --remote origin
```

Remote tags are local observations after fetch. jj 0.45.1 provides tracked/untracked remote-tag predicates and includes untracked remote tags in the documented default immutable-head expression. A tag can be a policy boundary without being a bookmark and without making every descendant immutable.

Maintenance and debugging

Repository maintenance has a recoverability cost. Before `jj util gc`, inspect operation history, evolution records, local rescue bookmarks, and remote state. Config GC is a different operation: it removes obsolete configuration entries, not unreachable commit objects. Git maintenance and jj maintenance can operate on related object stores in a colocated repository; use the backend's documented tooling and avoid concurrent writers.

```
jj op log
jj evolog -r 'hidden()'
jj bookmark list --all-remotes
jj util gc
jj util backend name
jj version
```

For a bug report, capture `jj version`, backend/colocation status, effective relevant configuration, command line, operation IDs, and a minimal graph that reproduces the result. Do not include credentials or private repository content. A machine-readable template should disable pager and colour and use full IDs. A human log can use labels, relative times, and short IDs, but it is not stable interchange.

Completion and Linux integration

Generate completion from the same binary whose commands are being documented:

```
jj util completion bash > ~/.local/share/bash-completion/completions/jj
jj util completion zsh > ~/.zfunc/_jj
jj util completion fish > ~/.config/fish/completions/jj.fish
```

Static completion becomes stale after a release upgrade. Keep it under the shell's supported directory, start a new shell or source it as appropriate, and regenerate when the 0.45.1 command tree differs. A prompt should use a bounded, no-pager query and should not run a broad diff on every redraw:

```
jj --ignore-working-copy --no-pager --color=never log -r @ -T \  
'change_id.shorttest(10) ++ " " ++ if(conflict, "C", "") ++ " " ++ bookmarks'
```

Shell functions should preserve exit status. If a prompt suppresses errors, it should do so outside publication and recovery commands; never wrap a push, rewrite, or restore in a fallback that hides failure.

Part XVII — Workflow map and complete real workflows

The eleven entries below are a map of the complete narratives that follow, not substitutes for them. Each map entry identifies the authoritative worked case; read that case for the starting graph, commands, resulting graph/ref state, identity consequences, conflicts, recovery, and cleanup. The detailed language and command explanations remain in Parts IV–XV; the cases apply them in sequence.

1. Branchless daily development — map

See Branchless day: from fetch to publication. Fetch; create a child of the imported trunk; work in @; use `jj log -r '@:: & mutable()'`; make changes with describe/squash/split/absorb; create a publication bookmark only at the boundary; push; then forget it when its server purpose ends.

2. GitHub feature and stacked PRs — map

See Case study: a branchless GitHub feature through review and stacked GitHub review. Use one bookmark per PR boundary, rebase the mutable stack onto main@origin, move bookmarks, push, and let GitHub merge. Treat server merge and local graph update as separate fetch steps.

3. Gerrit patch stack — map

See Gerrit patch stack over two review iterations. Keep Gerrit Change-Id footers in descriptions, use jj change IDs for local evolution, edit an early review, let descendants rebase, then upload the series again. Fetch landed changes and rebase the remaining stack.

4. Rewriting an early change after review — map

See Case 4: rewrite an early reviewed change.

```
jj edit A
jj describe -m 'API: corrected quota model'
jj squash --from @ --into @- -- 'glob:src/**'
jj log -r '::~@'
jj git push --bookmark review --remote origin
```

Review the operation log before and after. The review bookmark must be moved to the successor tip; the change ID is the local continuity signal.

5. Upstream synchronisation — map

See Upstream movement with a conflict branch.

```
jj git fetch --remote origin
jj rebase -b 'roots(main@origin..@)' -o main@origin
jj log -r '@:: & mutable() & conflicts()'
jj resolve
```

Choose `-b` for a whole relative stack and `-r` for surgical selection. Do not substitute a Git branch mental model for the source/destination semantics.

6. Two remotes — map

See Case study: two remotes and one local DAG. Fetch GitHub and Gerrit independently, qualify symbols, name publication bookmarks explicitly, and make each push command specify its remote.

7. Moving unfinished work — map

See Two machines and a temporary publication. Create a temporary bookmark, push to a private remote, fetch on the second clone, edit or rebase there, and delete the temporary bookmark after verifying the receiving clone.

8. Build/review workspaces — map

See Case study: build, review, and development workspaces. Add separate workspaces at the build and review commits, use sparse patterns where appropriate, and remember that a shared rewrite can make either workspace stale.

9. Recovery — map

See Recovery and operations. Stop, record `jj op log`, inspect with `--at-op`, then undo or restore the operation. Do not attempt recovery by guessing commit IDs alone.

10. Git in a colocated repository — map

See Native Git in a colocated checkout. Use Git for inspection, hooks, and transport when useful; import ref changes before relying on them; never let independent worktree writers race.

11. Revset-driven daily work — map

See Revset-driven daily gate.

```
alias jj-stack='jj log -r "@:: & mutable()"'
jj log -r 'mutable() ~ ::remote_bookmarks()'
jj diff -r 'roots(@:: & mutable())..@'
jj run -r '@:: & mutable() & conflicts()' -- test -f CONFLICTS
```

Revssets let policy be expressed once and reused across log, diff, run, rebase, and publication checks.

How to read a complete workflow

A workflow in this book is a state transition, not a list of commands. Read each narrative with two graphs in mind. The commit graph records the changes, parents, conflict values, and publication refs. The operation graph records how the repository view got there. A command can leave the commit graph looking familiar while still creating a new operation, changing the working-copy commit, or importing a remote ref. Conversely, a server-side merge or review action can change the remote graph without creating a local jj operation at all. The boundary matters whenever the next command depends on what is actually known locally.

The compact notation `C1`, `C2`, and `C3` names commit IDs in a diagram; it does not name jj change IDs. A rewritten commit normally receives a new commit ID while its change ID can continue through the successor relationship. A bookmark is a name pointing at one commit, a remote bookmark is imported remote state, and a tracking relationship records which local and remote names correspond. None of those names makes `@` follow automatically. The current workspace's working-copy commit is the only meaning of `@`, and another workspace can have a different `@` in the same repository.

At each checkpoint, ask what was snapshotted, which commits were selected, where the destination is, and whether the command is a content operation, a graph rewrite, a ref operation, or an operation-log operation. That classification predicts most surprises. For example, `jj restore` changes recorded content; it is not an operation undo and it does not move a bookmark. `jj undo` changes the repository view to the prior operation; it does not mean “remove the diff from this commit”. `jj`

`git push` publishes a Git ref and may update tracking state; it does not publish every mutable descendant merely because the commits are reachable.

Checkpoint	Record before continuing	Why it is useful
Before editing	workspace, @, selected stack, bookmarks, remote bookmarks, conflicts, and <code>jj op log -n 3</code>	Separates the local starting view from an already-imported remote change.
Before a rewrite	source revset, destination revset, fileset, and whether the source is mutable	Prevents a broad revset or an unintended fileset from changing an ancestor or a publication tip.
After a rewrite	new commit IDs, preserved change IDs, successor/descendant graph, conflict values, and moved refs	Shows whether the operation preserved the intended logical change and which publication names now need movement.
Before publishing	exact bookmark(s), remote, remote target, and the diff relative to the imported base	Turns “push my work” into an auditable ref update, especially when GitHub and Gerrit share a repository.
After publishing	local bookmark, remote bookmark, tracking relation, server-side review state, and local operation ID	Distinguishes a successful local ref export from a review that has actually been accepted or merged.

The narratives use this ledger selectively rather than printing the same five-row table every time. When a workflow says “the descendants rebase”, read that as a graph claim: `jj` creates successor commits whose parent edges and commit IDs may differ, then keeps the logical stack connected. When it says “the review bookmark must move”, read that as a ref claim: the bookmark remains a separate name and will not necessarily be carried to the successor by the rewrite. When it says “resolve and continue”, read that as a content claim: the conflict value is replaced in the affected commit and its descendants may then be rewritten again. These distinctions are the reason the workflows are not Git branch recipes with `jj` spelling.

Primary route and later lookup

The primary route through this Part is deliberately short at the map and long in the linked narratives immediately below. The authoritative workflow prose is here: the later Part XVIII casebook is a practice and lookup mode, not a prerequisite chapter. The route is therefore: read the map, choose a narrative, follow its links to the detailed operation or language explanation only when a command's semantics need review, then use the appendices for spelling and option lookup. This keeps a reader who is solving a publication or recovery problem from having to infer the workflow from an alphabetical command list.

For review work, the GitHub narrative is the authoritative local-to-publication story and the Gerrit narrative is the authoritative patch-set story. For graph recovery, the operation laboratory is authoritative; for a file-level content rollback, the restore chapter is authoritative. The same subject can appear later because it serves a different purpose—practice, synthesis, or lookup—but the cross-reference should always point back to the teaching route rather than silently establishing a competing definition.

End-to-end workflow narratives

Branchless day: from fetch to publication

Start with an imported trunk and no local bookmark for the feature. This is not an unfinished branch; it is an intentional unnamed graph boundary.

```
main@origin -- T
    \
    @_0  empty mutable working-copy commit
```

```
jj git fetch --remote origin
jj new main@origin -m 'Cache: bound memory use'
# implement the change
jj status
jj diff
jj describe -m 'Cache: bound memory use'
jj new -m 'Cache: expose metrics'
```

```
# implement dependent change
jj log -r 'roots(main@origin..@)::'
jj log -r 'mutable() ~ ::remote_bookmarks()'
```

At this stage the graph is useful without a bookmark. The first change can be revised by selecting it through a change ID or description; the dependent change automatically follows. The publication boundary appears only when a server review is ready:

```
jj bookmark create gh/cache -r 'roots(main@origin..@)'
```

```
jj git push --remote origin --bookmark gh/cache
```

```
gh pr create --head gh/cache --base main
```

If there are two changes intended as one PR, point the bookmark at the stack tip and use a server/UI description that explains the stack. If they are intended as two stacked PRs, create two bookmarks. The graph decides; the names are interfaces.

Review iteration and early rewrite

```
reviewed state:  main -- A -- B -- C
                  ▲      ▲
                gh/base gh/tip
```

feedback on A:

```
main -- A' -- B' -- C'
```

same logical A; all commit IDs after A may differ

```
jj edit A
# edit files
jj describe -m 'API: correct quota invariant'
jj log -r 'gh/base | gh/tip | ::@'
jj diff --from 'gh/base@origin' --to gh/base
jj bookmark set gh/base -r A
jj bookmark set gh/tip -r C
jj git push --remote origin --bookmark gh/base --bookmark gh/tip
```

Do not move only the tip if the base review has its own bookmark. Rewriting A can affect both server refs. If the push is rejected, fetch and compare expected remote targets before deciding whether the local rewrite should replace a remote update.

Upstream movement with a conflict branch

```
jj git fetch --remote origin
jj rebase -b 'roots(main@origin..@)' -o main@origin
jj log -r '::-@ & conflicts()'
jj resolve --list
jj resolve --tool meld
jj log -r '::-@ & conflicts()'
jj bookmark list --all-remotes
```

Resolve before moving the publication bookmark. If a conflict is in an ancestor, inspect descendants even if only the ancestor's file was changed. A review bookmark that remains at a conflicted predecessor can make the push appear to succeed while publishing the wrong graph boundary.

Gerrit stack over time

```
day 1: main -- A(I-A) -- B(I-B) -- C(I-C)
day 2: main'-- A_2(I-A) -- B_2(I-B) -- C_2(I-C)
day 3: main"-- A(landed) -- B_3(I-B) -- C_3(I-C)
```

```
jj new main@gerrit -m 'API: introduce quota'

Change-Id: Iaiaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
jj new -m 'API: wire quota'

Change-Id: Ibmbmbmbmbmbmbmbmbmbmbmbmbmbmbmbmbmbmbmbb
jj new -m 'API: expose metrics'

Change-Id: Icscscscscscscscscscscscscscscscscscscscsc
jj gerrit upload --dry-run --remote gerrit --remote-branch main
jj gerrit upload --remote gerrit --remote-branch main
# later: edit the first change
jj edit 'subject(exact:"API: introduce quota")'
jj describe --editor
jj gerrit upload --remote gerrit --remote-branch main
# later: fetch landing
jj git fetch --remote gerrit
jj rebase -b 'roots(main@gerrit..@)' -o main@gerrit
```

At each review iteration, local `jj` change identity and Gerrit footer identity answer different questions. Record both in a review ledger if a team needs to audit patch sets. The local stack can be revised continuously; upload is a publication operation over the selected stack.

Two machines and a temporary publication

```
machine A: local unnamed @
          |
          \- temporary handoff bookmark → origin/handoff

machine B: fetch origin/handoff → new local @
```

```
# A
jj bookmark create handoff -r @
jj git push --remote origin --bookmark handoff

# B
jj git fetch --remote origin
jj new 'handoff@origin'
jj log -r '::~@'
jj bookmark forget handoff@origin
```

The handoff transfers commits and a Git bookmark observation. It does not transfer A's operation DAG, workspace-local @, uncommitted filesystem edits, or editor state. If the work is unfinished, commit a deliberate WIP change rather than relying on files that another machine cannot see.

Multiple workspaces in one repository

```
shared DAG: root -- A -- B -- C
workspace dev:   dev@ = C (active edits)
workspace build: build@ = B (repeatable build)
workspace review: review@ = A (review baseline)
```

```
jj workspace add ../build --name build -r B --sparse-patterns full
jj workspace add ../review --name review -r A --sparse-patterns copy
jj workspace list
jj -R ../build run -- cargo test
jj -R ../review show -r @ --stat
jj workspace update-stale
```

The build command's private or workspace filesystem must not be confused with dev's current @. A rewrite of A changes the commit graph observed by all three workspaces. Update stale workspaces only after checking for unrecorded files and decide whether a stable build should instead use a temporary clone.

Native Git in a colocated checkout

```
jj bookmark create review -r @
jj git export
git -C "$(jj root)" show-ref
git -C "$(jj root)" log --oneline --all
git -C "$(jj root)" fetch origin
jj git import
jj bookmark list --all-remotes
```

Git is valuable for ecosystem tooling and transport, but it cannot show jj change IDs, operation history, or stored multi-sided conflict values. A Git-side ref change becomes visible to jj only at the import boundary. A Git-side worktree reset while jj is snapshotting can discard or misattribute filesystem changes; establish a single writer.

Revset-driven daily gate

```
[revset-aliases]
unpublished() = 'mutable() ~ ::remote_bookmarks()'
review-candidates() = 'heads(unpublished() ~ empty() ~ conflicts())'

# human inspection
jj log -r 'review-candidates()'

# machine gate
jj log --no-pager --color=never --count -r 'review-candidates()'

# inspect each candidate
jj show -r 'review-candidates()' --stat
```

The gate is intentionally a policy expression rather than a command alias that hides selection. Teams may choose to allow empty or conflicted WIP reviews; the important property is that the policy is explicit, testable, and reviewed as code.

Case study: a branchless GitHub feature through review

Start with a fetched trunk and no local feature bookmark. The developer creates a child of the imported remote tip and works in the workspace:

```
main@origin--A--B--@
```

```
jj git fetch --remote origin
jj new main@origin
jj describe -m "Parser: report malformed input"
# edit files and run tests
jj diff
jj status
```

The working-copy commit is already the reviewable logical change. If the tests reveal that documentation and implementation should be separate, split the change before publication:

```
jj split -r @ 'root-glob:src/**/*.rs'
jj log -r '@:: & mutable()'
```

```
main@origin--A--B--C(API)--D(docs)--@
```

Create a bookmark at the boundary GitHub should expose. If the two changes are one PR, use one bookmark at D. If they should be stacked PRs, create names at C and D and publish both explicitly. The bookmark is a ref for GitHub; it is not the local unit of editing.

```
jj bookmark create pr/parser-api -r C
jj bookmark create pr/parser-docs -r D
jj git push --remote origin --bookmark pr/parser-api
jj git push --remote origin --bookmark pr/parser-docs
```

Suppose review changes C. Edit C, amend its tree and description, and inspect the successors of D and @. Because C is an ancestor, the descendants receive updated commit IDs. Move only the publication bookmark that represents C and push it explicitly; the dependent PR bookmark must also be moved if its target changed.

```
jj edit C
# edit src/parser.rs
jj describe -m "Parser: report malformed input"
jj log -r 'C::@'
jj bookmark set pr/parser-api -r C
jj bookmark set pr/parser-docs -r D
jj git push --remote origin --bookmark pr/parser-api
jj git push --remote origin --bookmark pr/parser-docs
```

If GitHub rejects the update, fetch first and compare the remote bookmark. A rejection may indicate that another actor moved the ref, that the local tracking relationship is stale, or that server-side branch protection disallows the update. Do not respond with a blind force push. Determine whether the local graph is the intended successor of the remote commit and whether the GitHub branch policy allows replacement.

Case study: main moves under a stack

```
old: main@origin--A--B
      \--C--D
new remote after fetch: main@origin--A--B--E--F

local stack before rebase:
      C--D
      /
main@origin(old)--B
```

```
jj git fetch --remote origin
jj log -r 'main@origin'
jj rebase -s 'roots(main@origin..D)' -d main@origin
jj log -r 'D:: & mutable()'
jj diff --from old-main --to main@origin
```

Choose the source set carefully. A broad source selection can include an unrelated fork; a narrow selection can leave a dependent child behind. Record the old trunk commit before the fetch if you need a precise comparison. After the rebase, inspect changed commit IDs and run tests at the stack head. Update publication bookmarks only after the graph is correct, then push them explicitly.

Case study: Gerrit's patch-set lifecycle

Gerrit recognises a patch set by its commit message's Gerrit Change-Id footer, while jj recognises an evolving logical change by its own change ID. They can coexist in one description. A rewrite normally changes the Git commit ID, may preserve the jj change ID, and should preserve the Gerrit footer if the intent is a new patch set for the same Gerrit review.

```
origin/main--A--B_1--C_1--D_1
      |      |
      review review
      Change-Id Iabc  Change-Id Idef
```

```
jj gerrit upload --remote gerrit --remote-branch main -r 'B_1::D_1'
```

Use the command's supported commit and metadata options to set reviewers, topic, work-in-progress/private state, and push options. The local upload transport and the server's review interpretation are separate layers: validate the former with `jj's` dry-run/verbose facilities and treat the latter as Gerrit behaviour that requires server documentation or a real Gerrit instance.

Now a reviewer requests a fix to B_1:

```
jj edit B_1
# edit and test
jj describe -m 'API: validate parser limits'

Change-Id: Iabc...'
jj log -r 'B_1::D_1'
jj gerrit upload --remote gerrit --remote-branch main -r 'B_1::D_1'
```

C_1 and D_1 are rebased descendants with new commit IDs. Their `jj` change IDs continue to identify their logical changes when the rewrite is an evolution; their Gerrit footers continue to identify their server reviews if descriptions were preserved. If a footer was absent at upload, `jj 0.45.1` generated it on the uploaded object, not in the local description, so fetching can reveal a divergent uploaded successor. In that case either abandon the local duplicate and continue from the fetched review object, or rebase the local change onto the appropriate trunk with `jj rebase --skip-emptied ...`, as the target help describes. If a footer is missing or duplicated in the actual uploaded description, inspect the commit trailers and server response rather than assuming `jj` metadata supplies the server identity.

Case study: landed lower change, remaining upper stack

```
before server integration:
main@origin--A--B
      \--C(review 1)--D(review 2)--E(review 3)

after review 1 lands:
main@origin--A--B--C'
local reviews:      D--E (still based on old C)
```

Fetch the new remote branch, identify whether the landed commit is equivalent to the local C by change ID, diff, or server metadata, then rebase the remaining stack onto the fetched trunk. Do not simply delete C and assume D/E will find the right parent: abandoning or rebasing the wrong node can produce duplicate changes or conflicts.

```
jj git fetch --remote gerrit
jj log -r 'main@origin | C::E'
# choose the remaining stack explicitly
jj rebase -s 'roots(D::E)' -d main@origin
jj log -r 'E:: & mutable()'
```

When a review is abandoned, a local change can be retained as a new change, moved to another publication bookmark, or abandoned after creating a safety bookmark. The correct action depends on whether the code is still wanted; server review state is not a command to delete local graph state.

Case study: two remotes and one local DAG

```
local graph:
upstream/main@origin--A--B--C--@--D

refs:
origin/main      = B
gerrit/main      = B
github/pr-parser  = C
```

Qualify all remote symbols. Use `origin/main` or the exact `jj` remote-bookmark spelling shown by `jj log`, and pass `--remote` on push/upload. A local bookmark called `main` can obscure which publication channel is intended; avoid it unless the

repository convention gives it one unambiguous meaning.

```
jj git fetch --remote origin
jj git fetch --remote gerrit
jj log -r 'main@origin | main@gerrit | github/pr-parser'
jj git push --remote origin --bookmark github/pr-parser
jj gerrit upload --remote gerrit --remote-branch main -r '@:: & mutable()'
```

One commit can be reachable from multiple publication systems, but server policy is not shared. A GitHub branch protection rule does not protect a Gerrit review ref, and a Gerrit upload option does not affect a GitHub push. Keep credentials, aliases, and remote names explicit.

Case study: moving unfinished work safely

Unfinished work has value but also a publication risk. Create a temporary bookmark with a name that says why it exists, push only to an access-controlled remote, and remove it after the receiving machine verifies the graph:

```
jj bookmark create transfer/2026-09-parser -r @
jj git push --remote origin --bookmark transfer/2026-09-parser
# machine B
jj git fetch --remote origin
jj log -r 'transfer/2026-09-parser@origin'
jj new transfer/2026-09-parser@origin
# after verification on both machines
jj bookmark delete transfer/2026-09-parser
jj git push --remote origin --deleted
```

Do not treat a temporary bookmark as a private stash unless the remote and access controls actually make it private. If the graph is sensitive, transfer via an encrypted private remote or another controlled mechanism. The operation log itself is not the transport protocol between independent clones.

Case study: build, review, and development workspaces

Use one repository with three workspaces when the tasks need different filesystem states but share object storage:

```
jj workspace add --name build -r main@origin ../repo-build
jj workspace add --name review -r 'bookmarks(exact:"pr/parser")' ../repo-review
jj workspace list
```

In the development workspace, rewrite the stack freely. In review, inspect the publication target and run review-specific tools. In build, run a clean build against the pinned commit. After a rewrite, check all three workspaces; a stale review or build workspace should be updated deliberately, not silently moved by a prompt hook.

Workflow casebook: maintain the graph across a working week

The following cases use the same operating rule: maintain a local graph that represents the work, and create or move a publication name only when an external system needs a ref. Each case records the graph, the publication boundary, and the recovery boundary. The commands are examples for jj 0.45.1; hosting behaviour is marked separately.

Case 1: branchless daily development

```
Monday after fetch:
main@origin--A--B(@)
      B = API change, unnamed

Tuesday review preparation:
main@origin--A--B--C(@)
      C = test/docs tail

Thursday upstream moves:
main@origin--A--B--C
      \
new main@origin-----B'--C'(@) after rebase
```

1. Fetch and inspect the imported trunk.
2. Create a child and do not name it while the change is private.
3. Use `jj new` for a new logical boundary and `jj describe` for the current change.
4. Use `jj log`, `jj diff`, and a bounded revset to inspect the stack.

5. Create a bookmark only when a review or handoff needs a Git ref.
6. After upstream movement, rebase mutable roots, resolve stored conflicts, and republish explicit bookmarks.

```
jj git fetch --remote origin
jj new main@origin -m 'API: enforce quota'
# edit and test
jj new -m 'API: add quota tests'
jj log -G -r 'roots(main@origin..@)::'
jj bookmark create review/quota -r @
jj git push --remote origin --bookmark review/quota
```

The local history never required a branch to move the working copy. The bookmark was an interface introduced at publication time. If the review is abandoned, keep or forget the bookmark according to server state; abandoning a local commit is a separate decision.

Case 2: stacked GitHub review

```
main@origin--A--B--C--D(@)
      ▲      ▲
    pr/api  pr/metrics
```

Publish B and D as boundaries only after verifying that B is the intended base PR and D is the intended dependent PR. When review changes A, edit by change ID. Automatic descendant rebasing produces B', C', and D'. Compare each local bookmark with its remote counterpart, update all affected bookmarks, and push explicitly. When GitHub merges the base PR, fetch before rebasing the remaining stack. The server's PR base and merge strategy are not encoded by jj's operation log.

```
jj log -r 'pr/api | pr/metrics | pr/api@origin | pr/metrics@origin'
jj edit 'change_id(API_CHANGE)'
jj describe -m 'API: correct invariant'
jj log -r 'roots(main@origin..@)::'
jj diff --from pr/api@origin --to pr/api
jj bookmark set pr/api -r 'change_id(API_CHANGE)'
jj bookmark set pr/metrics -r 'change_id(METRICS_CHANGE)'
jj git push --remote origin --bookmark pr/api --bookmark pr/metrics
```


Case 3: Gerrit patch stack over two review iterations

```
iteration 1: main@gerrit--B--C--D
            Iapi          Itest          Imetrics

iteration 2: main@gerrit--B'--C'--D'
            same footers, new commit IDs,
            new Gerrit patch sets when server accepts
```

Use one jj change per reviewable change and preserve a Gerrit Change-Id footer in each description. Upload a selected stack to the review ref according to Gerrit policy. On an early review comment, edit the jj change by change ID; do not create a local fixup branch solely to obtain a new commit. Inspect descendants, resolve any stored conflict, and re-upload. jj can prove the footer and local IDs; only Gerrit can prove that the server associated the object with the intended review and patch set.

```
jj evolog -r 'change_id(API_CHANGE)'
jj log -r 'roots(main@gerrit..@)::' -T 'change_id ++ " " ++ commit_id ++ "\n"'
jj gerrit upload --dry-run --remote gerrit --remote-branch main \
  --revision 'roots(main@gerrit..@)::'
jj gerrit upload --remote gerrit --remote-branch main \
  --revision 'roots(main@gerrit..@)::'
```

When B lands, fetch the new Gerrit main and rebase C/D, not the remote bookmark. If a submit strategy changes the effective ancestry, use the fetched remote bookmark as the new base after inspecting it. Abandoning a Gerrit review does not abandon the local change; clean the local graph only after preserving any needed footer and operation evidence.

Case 4: rewrite an early reviewed change

```
before: main--A--B--C--D(@)
        A reviewed and published

after:  main--A'--B'--C'--D'(@)
        A/B/C/D change IDs follow successors;
        all affected commit IDs change
```

Make a safety record with `op log` and a temporary bookmark. Edit A by change ID, make the smallest semantic repair, run focused tests, and inspect the entire descendant stack. The fact that B' is clean does not prove its tests still test the same contract. Compare old and new diffs with evolution/inter-diff output. Move every review bookmark and publish with an explicit safety decision.

```
jj bookmark create before-early-rewrite -r @
jj op log -n 3
jj edit 'change_id(A_CHANGE)'
# edit, test, and snapshot
jj evolog -r 'change_id(A_CHANGE)' -p
jj log -G -r 'roots(main@origin..@)::'
jj log -r 'conflicts() & ::@'
```

Case 5: GitHub and Gerrit from one clone

```
      +-- github/pr-cache
main@github --A-+
      \-- gerrit review stack
main@gerrit ----A--B--C(@)
```

One local commit can be exported to both systems, but the systems may have different trunk objects, review identities, ref namespaces, permissions, and merge policies. Keep two publication stories. Fetch each remote explicitly; qualify revsets such as `main@github` and `main@gerrit`; create a GitHub bookmark only for the GitHub ref; and invoke Gerrit upload with its explicit remote/branch. Do not assume a Gerrit footer controls GitHub review or that a GitHub branch name supplies a Gerrit topic.

```
jj git fetch --remote github
jj git fetch --remote gerrit
jj log -G -r 'main@github | main@gerrit | ::@'
jj bookmark create github/cache -r @
jj git push --remote github --bookmark github/cache
jj gerrit upload --remote gerrit --remote-branch main \
  --revision 'roots(main@gerrit..@)::'
```

Case 6: move unfinished work between machines

Operation IDs and @ do not transfer through Git. Publish a temporary bookmark, fetch it on the second clone, and create a new local child there. The first machine can continue to observe the handoff ref but must not assume it follows the second machine's new working-copy commit. If both machines edit the same logical change, fetching can expose divergent successors. Compare `evo log` and select a policy before force-updating the handoff ref.

```
# A
jj bookmark set handoff/alice -r @
jj git push --remote origin --bookmark handoff/alice

# B
jj git fetch --remote origin
jj new 'handoff/alice@origin' -m 'Continue cache work'
# after handoff:
jj bookmark set handoff/bob -r @
jj git push --remote origin --bookmark handoff/bob
```

Case 7: recovery after a wrong rewrite

Stop after noticing the mistake. Do not create several “repair” operations while guessing. Record the current operation, use `op show` and `--at-op` to identify the last good state, use `undo` only when it is the immediate last operation, and use named `restore/revert` when later operations must remain meaningful. If a remote push occurred, perform a separate `server/ref` repair and fetch afterward.

```
jj op log -n 20
jj op show OP -p
jj --at-operation OP log -G -r 'all()'
jj op diff --from GOOD --to CURRENT
jj op restore GOOD
jj op log -n 5
jj log -G -r 'all()'
```

After the repair, verify the commit graph, operation graph, bookmarks, remote bookmarks, working-copy files, and any published review identities. A command returning zero is not the recovery assertion; the assertion is that the repository is now in the intended state and that the choice is recorded.

Case 8: revset-driven automation

Use revsets to define a bounded domain and templates to emit stable records. Discovery should be read-only and cardinality-aware. Mutation should receive full IDs or explicit commit selectors, run with a known configuration, and leave an operation record. Never parse the graph glyphs of ordinary `jj log` in automation.

```
domain='mutable() & ::@ ~ ::remote_bookmarks() ~ conflicts()'  
jj log --no-pager --color=never -r \"$domain\" \  
-T 'change_id ++ \"\\t\" ++ commit_id ++ \"\\t\" ++ description.first_line() ++ \"\\n\"'  
jj diff --no-pager -r \"$domain\" --summary  
# apply a selected, reviewed operation only after checking cardinality
```

Run this style of automation in CI with an explicit `JJ_CONFIG` or `--config-file`, a pinned binary, and no implicit remote. Make the script fail when the domain is empty or unexpectedly broad if either condition would be unsafe. Keep a fixture that exercises a fork, merge, conflict, remote bookmark, and duplicate subject so the query is tested against the cases that make shortcuts fail.

Part XVIII — Practice, reference, and fieldbooks

Part XVII closes the linear teaching route. Part XVIII is an intentional second mode of use: the advanced field manual applies the model to day-to-day decisions, the language and command sections are lookup references, the labbook and curriculum provide controlled practice, and the hosting/recovery sections supply incident and publication procedures. The headings below are grouped here so that the reader does not mistake a reference atlas for another competing introduction.

Read this part selectively. A first pass can stop after Part XVII. A reader investigating a difficult graph can enter the field manual; a reader writing automation can enter the language or command reference; a reader preparing a review can enter the hosting fieldbook; and a reader recovering from an error can enter the recovery recipes. The cross-links point back to the primary teaching chapters whenever a detail depends on an earlier invariant.

Subject	Authoritative teaching	Practice or workflow	Lookup reference
Graph and rewriting	Parts II–III	Rewrite and mechanics laboratories	Appendices A, B, G
Revsets, filesets, templates	Parts IV, V, XIV	Language laboratories and revset-driven case	Appendices C, D, E
Bookmarks and Git transport	Parts VI–VII	GitHub/Gerrit workflows	Appendices H, I
Conflicts and recovery	Parts IX–X	Conflict and recovery laboratories	Appendix J
Workspaces and configuration	Parts XI–XII	Workspace and Linux configuration laboratories	Appendix F, K
CLI	Relevant semantic chapters	Part XVII workflows and command laboratories	Part XV and Appendices A–B

The rule is simple: the Parts named in the second column teach the subject; the later sections in the third column exercise it; the locations in the fourth column are lookup aids. A later fieldbook may combine subjects, but it is not a second canonical definition. When a fieldbook appears to disagree with a teaching chapter, follow the target-version command entry and the cited local validation before treating it as a new rule.

The advanced field manual — practice and decision procedures

Use this field manual when a `jj` operation crosses several state boundaries at once. A single operation can snapshot a filesystem, rewrite a set of commits, move local names, update a workspace, and leave an operation record. Looking at only one of those layers is how apparently impossible states are produced. The useful question is not “which command is the `jj` equivalent of my Git command?” but “which state transition do I want, and which of the other layers must remain invariant?”

Every investigation in this chapter uses the same notation. A capital letter is a symbolic commit label in a disposable fixture; `cid(A)` is its commit ID; `chg(A)` is

its change ID; @ is the current workspace working-copy commit; and op_n is an operation-log node. A line such as bookmark review → C is a local name, while review@origin → C_0 is a remote bookmark observed after fetch. These are not interchangeable identifiers.

State accounting before action

Before a risky operation, write down a small state ledger. This is faster than recovering from a mistaken broad revset and it makes a code review of shell automation possible.

Layer	Question	Evidence command	What a mismatch means
Filesystem	Are there edits not yet in the recorded working-copy tree?	<code>jj diff --summary</code> , then <code>jj status</code>	The next command may snapshot more than the operator thinks.
Working copy	Which workspace owns @ and which commit is it?	<code>jj log -r @</code> , <code>jj workspace list</code>	A second workspace may be the one that becomes stale.
Graph	What are the exact parents, descendants, and heads?	<code>jj log -r 'all()'</code> with a parent template	A visual linear stack may conceal a merge or fork.
Identity	Is this a commit ID, change ID, bookmark, tag, or operation ID?	<code>jj show</code> , <code>jj evolog</code> , <code>jj op log</code>	Using a short ID from the wrong namespace can select nothing or the wrong object.
Immutability	Which commits are protected by <code>immutable_heads()</code> ?	<code>jj log -r 'immutable_heads()'</code>	A failed rewrite may be protection working correctly.
Publication	Which local and remote names point into this graph?	<code>jj bookmark list</code> , <code>--all-remotes</code> , <code>jj tag list</code>	A local rewrite may require an explicit force update or a new review ref.
Operations	What operation created the state, and is it integrated?	<code>jj op log</code> , <code>jj op show</code>	Recovery should start from the operation boundary, not guesswork.

Record the ledger before and after. In a real repository, use full IDs in the record but symbolic labels in discussion. A shell script should discover full IDs, validate its assumptions, perform one operation, and discover the resulting state again. It should not assume that a commit ID remains valid after a rewrite merely because the change ID is stable.

Read-only observation and snapshot policy

There are two different meanings of “read-only”. A command may not intentionally rewrite history, yet it may snapshot the working copy. For a human at a terminal that is normally a convenience: the graph catches up with the editor before the log is displayed. For a diagnostic script, it is an implicit write to repository state and can race another process editing the same workspace.

Use ordinary commands when the current filesystem is part of the question. Use `--ignore-working-copy` when the question is specifically about the last recorded commit, or when another process owns the workspace. Use `--at-operation OP` when the question is about a historical repository view; it implies that current filesystem state must not be inserted into that view.

```
# Human inspection: include edits in the snapshot boundary.
jj status
jj log -r '@:: & mutable()'
```



```
# Stable inspection of the recorded commit, without taking a snapshot.
jj --ignore-working-copy log -r @ -T 'commit_id ++ " " ++ description.first_line() ++ "\n"'
```



```
# Historical inspection: OP is an operation ID from jj op log.
jj --at-operation OP log -r 'all()'
```

Do not infer that `--ignore-working-copy` makes a mutating command safe. It prevents snapshotting; it does not turn rebase, abandon, bookmark movement, or operation restoration into observation. The four questions remain separate: what state is loaded, what set is selected, what mutation is requested, and which workspace or ref is updated.

Graph invariants as tests

A robust workflow states invariants in terms of relationships rather than display order. For example, after rewriting an early change in a three-change stack:

- the edited change has the same change ID before and after;
- its commit ID is different if its tree, parents, metadata, or conflict value changed;
- the descendant changes still exist and are successors of their prior evolutions;
- the local publication bookmark points to the intended new stack tip;
- the remote bookmark has not moved until an explicit push;
- the operation log contains one operation describing the rewrite and any automatic rebases.

These assertions survive changes to the default log template. Assertions based on “the third line begins with @” do not. In a fixture, print a tab-separated record with full commit ID, change ID, parent IDs, bookmark names, conflict state, and description. Compare the fields separately so an expected commit-ID change is not mistaken for a lost change.

Publication boundaries in practice

Local jj development can remain unnamed, but a Git server consumes refs. The publication boundary is therefore an interface, not a preference. The local graph may be:

```
main@origin -- A -- B -- C -- D -- @
                ↑           ↑
            review/base  review/tip (local only)
```

At publication, choose which node represents the server-facing boundary. If the review should contain C..D, create or move a bookmark to D and push that name. If it should contain the whole stack from A, the server's base and the pushed ref define that interpretation; a local bookmark is not a substitute for checking the server's base.

After a push, local and remote bookmarks are two observations. A tracking relationship lets `jj` report whether the local name is ahead, behind, synchronised, or conflicted with the corresponding remote name. It does not make a local bookmark follow the working-copy commit. Editing files after a push changes `@`, not a bookmark that happens to have pointed at the old `@`.

A safe publication protocol

1. Fetch the target remote and inspect `main@remote`, the local bookmark, and the candidate stack.
2. Choose a publication name that identifies the server-facing review, and set it to an exact commit only after inspecting that commit's parents.
3. Run a dry-run or preview where the command provides one. Otherwise print the candidate ref and its remote counterpart with full IDs.
4. Push one named bookmark to one explicitly named remote. Avoid an unqualified push in a repository with GitHub and Gerrit remotes.
5. Fetch again if the server transforms or creates refs, then inspect the resulting remote bookmark.
6. After review feedback, rewrite the local change, verify change and commit identity, and push the same publication name. The server-side interpretation of that update is host-specific.

The protocol is intentionally more explicit than “push `@`”. `@` is a workspace symbol, and its value can change on every snapshot or `jj new`. A bookmark is a durable interface choice that can be reviewed, tracked, and published.

Same bookmark names on several remotes

Suppose a repository has `github` and `gerrit` remotes, each with a `main` bookmark. The unqualified symbol `main` denotes a local bookmark; `main@github` and `main@gerrit` denote different remote observations. They may have identical commit IDs today and diverge tomorrow. A local bookmark can track one corresponding remote relationship; it cannot mean “the main branch of the organisation” without naming which remote supplies that fact.

```

jj git fetch --remote github
jj git fetch --remote gerrit
jj log -r 'main@github | main@gerrit'
jj bookmark list --remote github
jj bookmark list --remote gerrit

# Publication remains explicit.
jj git push --remote github --bookmark github/pr-17
jj gerrit upload --remote gerrit --remote-branch main -r 'roots(gerrit-main..@)'

```

Do not use a config default to conceal this distinction in a shared script. A personal alias can choose a remote for interactive work, but the alias should print or encode the remote and should refuse to publish if the selected bookmark is absent, conflicted, or unexpectedly behind its tracking target.

Colocation and native Git boundaries

A colocated repository has a Git directory and jj metadata associated with the same working directory. This is valuable for an advanced Git user because Git-native tools can inspect exported commits and refs while jj provides mutable graph operations. It also means two tools can change related state. The safe model is “one filesystem owner at a time, explicit import after Git-side ref changes”.

Action	Git object/ref effect	jj effect	Follow-up
Edit a file	None until Git writes a commit	Next jj snapshot sees it	Decide which command owns the next snapshot.
<code>git commit</code>	Creates a Git commit and moves a Git ref	jj metadata is not automatically a matching logical change	Use the supported Git import path, then inspect the jj graph.
<code>git update-ref</code>	Moves a Git ref	jj's remote/local bookmark view may be stale	Run <code>jj git import</code> and inspect remote bookmarks.
<code>jj bookmark set</code>	May export through backend policy	Moves a jj bookmark	Use <code>jj git export</code> when the local Git view must be current.
<code>jj rebase</code>	Creates/exportable Git objects for successors	Rewrites mutable jj commits and descendants	Push the intended bookmark, not an obsolete commit ID.

Action	Git object/ref effect	jj effect	Follow-up
<code>git reset --hard</code>	Moves a Git ref and overwrites files	Can leave workspace state stale or create surprising next snapshots	Prefer jj workspace commands; if used, stop other writers and reconcile.

Git object reachability and jj operation reachability are different graphs. Git garbage collection knows about Git refs and Git objects; it does not preserve an unexported jj operation history merely because that history is important. Conversely, jj operation recovery can restore a repository view whose commits have not yet been exported to Git. Do not run aggressive Git maintenance without knowing which objects and refs provide recovery.

Ref import/export experiment

In a disposable colocated fixture, create a bookmark in jj, export it, and compare the Git ref. Then move the Git ref directly and import:

```
jj bookmark create experiment -r @
jj git export
git show-ref --verify refs/heads/experiment

# This changes Git state without moving the jj bookmark first.
git update-ref refs/heads/experiment "$(jj log -r 'root()' -T commit_id)"
jj git import
jj bookmark list --all-remotes
```

The exact exported ref names and handling of imported Git refs are part of the Git backend contract, not an invitation to edit internal jj files. Always use supported import/export commands. The experiment establishes a habit: after a Git-side ref mutation, refresh jj's view before reasoning about a remote bookmark or immutability boundary.

Review identities: one local change, several server views

Review systems commonly associate successive Git commits with a review by a message footer, a ref, or server metadata. jj's change ID is a local identity for an evolving change. These identities can coexist without being equal. A rewrite may preserve the jj change ID while changing the commit ID; a Gerrit upload may preserve a `Change-Id` footer while creating a new patch set; a GitHub pull request may continue to use the same branch ref while its tip commit changes.

```
jj change Q
|- commit q_0, footer I123, pushed at patch set 1
\~ commit q_1, footer I123, pushed at patch set 2

server review identity      jj identity
review ref / Change-Id     change ID Q
particular object          commit ID q_0 or q_1
```

When debugging a review update, record all three: old and new commit IDs, the jj change ID, and the host-side review identifier. If one is missing, the update may be correct but difficult to explain. A command that only prints a short commit ID is insufficient evidence for a review audit.

Rebase surgery: choosing the smallest transformation

Rebase is often described as moving commits, but in jj its source selection and destination selection are independent. Start by stating whether parent edges, tree deltas, or both should change.

Intent	Primary operation	What remains invariant	Inspect afterwards
Move a stack to a new base	<code>rebase -s STACK -d BASE</code>	Selected changes and order, subject to conflicts	Parents, successors, bookmark tip, conflicts
Move one independent change	<code>rebase -r CHANGE -d BASE</code>	Unselected graph edges as far as descendant rebasing permits	Whether descendants followed the rewritten parent
Put a change after another	<code>rebase -r CHANGE -A DEST</code>	Change diff, unless context creates a conflict	Exact parent list and merge shape

Intent	Primary operation	What remains invariant	Inspect afterwards
Make one change own selected files	squash, restore, or split	Depends on whether source remains and a new change is made	Diff of both source and destination
Apply a change elsewhere	duplicate	Content intent, not change identity	New change ID and parent edges
Remove a change from visible history	abandon	Unselected commits and recovery records	Descendants, operation log, bookmark targets

The source selection is not “the commit I clicked”. `-r` selects commits individually; `-s` selects a source commit and its descendants; `-b` selects a branch-like set according to the command's definition. A source set with a hole is not a contiguous stack: descendants that remain selected may be rebased around the hole, while the omitted commit stays in place. Use a parent template to verify the result.

```
jj log -r 'all()' -T '
commit_id.short() ++ " " ++ change_id.short() ++
" parents=[" ++ parents.map(|p| p.commit_id().short()).join(",") ++ "]" " ++
description.first_line() ++ "\n"
```

Ancestor rewrite with a publication bookmark

Consider a published two-change stack:

```
main@origin -- A -- B -- C
                ↑      ↑
            review/base review/tip
```

If B is edited, the new B' has a new commit ID. Descendant C cannot retain its old parent edge, so jj creates a successor C' rebased onto B'. A local bookmark pointing at C is moved to its successor as part of the rewrite. The remote bookmark remains at its previously exported commit until a push. The review server therefore sees a ref update containing new objects, while the local user experiences one logical edit to B and automatic maintenance of the stack.

If the rebase cannot apply cleanly, C' can contain a conflict. The correct response is not to abandon B' merely because the descendant is conflicted. Inspect the conflict

value, materialise it if needed, resolve it in the descendant, and rerun tests at the stack tip. The operation log records the rewrite and resolution as separate boundaries, making “before the rebase” and “after resolution” recoverable states.

Three languages, three failure classes

jj's commit language, fileset language, and template language often appear in the same command, but they are evaluated in different domains:

Language	Value	Typical failure	Diagnostic
Revset	Set of commits	A range includes a merge ancestor or has the wrong cardinality	<code>jj log -r 'EXPR' -T commit_id</code>
Fileset	Set of repository paths	Shell expands a glob or a relative path uses the wrong base	<code>jj file list 'EXPR'</code>
Template	Typed rendering value	Method is called on the wrong type or an optional value is unset	<code>jj log -T '...' in a fixture</code>

Do not fix a revset problem by changing the template, or a fileset problem by changing the commit selector. First render the selected commits with one full ID per line. Then render selected paths against one known tree. Only then combine both in a mutating command.

Fileset command boundaries

A fileset can limit a display without limiting a graph operation, or it can define exactly which diff paths move. `jj diff -r R -- root-glob:src/**` restricts output. `jj restore --from R --into S -- root-file:src/config.toml` copies one path from one tree to another. `jj squash --from R --into S -- root-glob:tests/**` changes diff ownership for selected paths. The same syntax describes a path set, but the command supplies the semantics of applying that set.

When a command accepts both a commit option and fileset arguments, prove both independently. If `R` is a source commit and `paths` selects tests, first run `jj diff -r R -- paths`. Next check that resulting source/destination diffs differ only under `paths`. A successful command is not proof that the intended fileset was parsed; an empty selection can be a valid no-op.

Operation recovery as forensic work

The operation log should be treated as a forensic record, not a magical undo button. First identify the bad operation, then decide whether recovery is a new operation that reverses it or a restoration of the repository view as it existed then. These differ: restoring a historical operation can discard later integrated changes from the current view, while a revert-like operation creates a new state that preserves later history.

Question	Preferred inspection	Likely recovery
Was the immediately previous operation wrong?	<code>jj op show</code> , then compare current and parent	<code>jj undo</code>
What did the repository look like before a named operation?	<code>jj op show OP</code> , <code>jj op diff OP</code>	Read with <code>--at=operation</code> ; restore only after review.
Was useful work abandoned?	<code>jj evolog</code> , hidden/explicit commit lookup	Duplicate or restore useful evolution into current work.
Did a bookmark move incorrectly?	<code>jj bookmark list --all-remotes</code> , operation diff	Set/move the bookmark to the intended commit.
Did another process advance the operation head?	<code>jj op log</code> and operation parents	Inspect competing operation before undoing.

Do not run repository garbage collection while an abandoned commit or old operation is your only recovery path. Preserve the operation ID, full commit IDs, and any untracked files outside the workspace before a large repair. An operation record cannot restore an editor buffer or a file that was never snapshotted.

Recovery exercise: wrong ancestor

1. Make a disposable three-change stack and record `jj op log`, the current tip, and every change ID.
2. Edit the middle change when the intention was to edit the tip. Do not immediately continue working.
3. Use `jj op show` and `jj op diff` to identify the operation that rewrote the ancestor.
4. Use `jj undo` once. Confirm that the middle and tip commit IDs, bookmark target, and filesystem are back at the recorded state.
5. Repeat the mistake, then perform one unrelated description edit before noticing. Compare `undo` with inspecting the historical operation and using an explicit `restore/revert` strategy.

The exercise teaches an important limitation: “undo the thing I meant” is not a semantic operation. `undo` operates on the latest integrated operation. Once other operations have been integrated, choose the historical operation deliberately and account for their changes.

Workspace ownership and staleness

A workspace gives another directory its own working-copy commit, not another copy of the commit store. Two workspaces can therefore have different `@` values while sharing ancestors, bookmarks, and operation history. A graph rewrite seen by one workspace can make another workspace stale. The stale workspace's filesystem is still real; updating it is a reconciliation action, not a harmless refresh.

```
shared store
  A -- B -- C'
      \
workspace dev:  dev@ = C'
workspace build: build@ = C (stale working-copy successor)
```

Use a development workspace for editing and a build workspace for long-running tests when the build workspace can tolerate being updated. Before rewriting an

ancestor, check `jj workspace list` and quiesce other writers. After the rewrite, inspect each workspace's filesystem edits before `jj workspace update-stale`. If build output is valuable, keep it outside the workspace or make the build reproducible; do not expect an update to preserve arbitrary generated files.

Moving unfinished work between machines

Clones do not share an operation log. They share commits and refs only when those objects are transferred through the Git backend. The portable unit is therefore an explicitly published bookmark, even if that bookmark exists for only a few minutes.

```
# Machine A
jj status
jj diff --stat
jj bookmark set handoff -r @
jj git push --remote origin --bookmark handoff

# Machine B
jj git fetch --remote origin
jj log -r 'handoff@origin' -T 'commit_id.normal_hex() ++ "\n"'
jj new handoff@origin
jj status

# Machine A, after confirmation
jj bookmark delete handoff
jj git push --remote origin --bookmark handoff
```

Do not promise that `jj undo` on machine B recovers an operation performed on machine A. It cannot. If the unfinished work was not exported, the second clone cannot discover it from Git. If a publication bookmark was moved twice, fetch and inspect both the current remote target and the originating machine's operation/evolution history before deciding which state is authoritative.

Book-wide case study: one change through its entire life

The following narrative deliberately follows one logical change rather than one command. It is a useful model for review notes, incident reports, and CI tooling because it keeps the change ID, commit IDs, graph edges, workspace, operation log, and host ref visible at each stage.

Stage 0: imported base

```
root -- A -- B
      ↑
    main@origin
workspace dev: @ = B + empty working-copy commit
```

After fetch, `main@origin` is an observation of a remote bookmark. The local repository may have an ordinary local `main` as well, but that is a separate ref. The working-copy commit is a child of the selected base even if it is empty. It has a commit ID and change ID, and its workspace association is local metadata. A branch-oriented Git user may describe this as checking out a branch and then creating an index/worktree state; a detached-HEAD-oriented user may describe it as detached HEAD plus an index. The `jj` description is more exact for both habits because the working-copy commit is part of the commit DAG and is updated through snapshot operations.

At this stage, the safe audit is: `render @, @-, main@origin`, full IDs, and parents; run `jj status`; and record `jj op log`. Do not create a bookmark just to make the working copy feel named. The absence of a local publication name is intentional.

Stage 1: first edit and description

Edit `src/quota.rs`. The filesystem changes immediately; the commit tree does not change until a snapshot boundary. Running `jj status` ordinarily snapshots the edit and updates the working-copy commit. The new working-copy commit has a new commit ID because its tree changed, while its change ID identifies the same evolving working-copy change. Use `jj describe -m` to add a subject. That metadata change can create another successor, so a script should not cache the commit ID across both operations.

```
jj status
jj describe -m 'add quota API'
jj log -r @ -T '
change_id.normal_hex() ++ "\t" ++ commit_id.normal_hex() ++ "\t" ++
parents.map(|p| p.commit_id().normal_hex()).join(",") ++ "\n"
```

The working-copy commit is not a staging area. There is no need to ask which diff hunks are “staged” before creating a new change. If the edit belongs in a separate logical change, use `jj new` before editing or use `split/diffedit` after the fact. The choice is about graph and diff ownership, not the durability of the current files.

Stage 2: dependent change

Run `jj new` and add tests. The old API change remains a parent, and the new working-copy commit is an empty or partially edited child while you work. Once the tests are snapshotted, the graph is:

```
root -- A -- B -- Q -- R
      main@origin API tests
                  @ = R
```

Q and R are separate change identities. The fact that R is a child of Q is the dependency relationship. If a test change also needs a documentation file, that is diff ownership within R; it does not require a second bookmark. The query `Q : R` returns both commits, while `Q . R` excludes Q and expresses the diff range above Q. Using the wrong one in a review tool can change the displayed diff even though

both look like “the changes between Q and R”.

Stage 3: publication

For GitHub, create `github/quota` at R. For Gerrit, the upload command can select the range without a local bookmark, although a local name can help human review. The publication choice is not a graph operation: Q and R keep their IDs. The push may export Git objects and create a remote bookmark, after which the local tracking relationship records the remote target.

```
jj bookmark create github/quota -r @
jj git push --remote github --bookmark github/quota
jj bookmark list --remote github --tracked
jj gerrit upload --dry-run --remote gerrit --remote-branch main -r 'Q::R'
```

The GitHub ref and the Gerrit upload are different interfaces to the same local graph. A GitHub PR normally names one head ref and one base ref. Gerrit commonly receives each commit in a selected range through a review ref and associates it using message metadata. The local fact that Q is the parent of R is shared; the host-side display and review identities are not.

Stage 4: review feedback on the ancestor

The reviewer finds an API problem in Q. Use `jj edit Q`, modify the file, and snapshot. `jj` rewrites Q to Q' and automatically rebases R to R'. The graph now has successors:

```
old: root -- A -- B -- Q -- R
new: root -- A -- B -- Q' -- R'
                ↑      ↑
            same change same change
            IDs normally IDs normally
```

Q' and R' have new commit IDs. Q and R retain their logical change IDs in the ordinary evolution. The local GitHub bookmark follows the local successor of R. The remote GitHub bookmark remains at R until pushed. The Gerrit footer in Q' should

remain the same if the intended result is a new patch set of the same server review. This is the exact point at which a Git user who thinks only in branch tips may miss the fact that the dependent review also changed.

Run an interdiff or a pair of diffs for Q/Q' and R/R' . The first shows the review response; the second mostly shows the parent-context update but can contain conflict-resolution or test changes. Record both so reviewers can distinguish intended feedback from automatic descendant consequences.

Stage 5: trunk movement

While the review is open, upstream adds C and the remote main moves. Fetch it. Do not move the local publication bookmark to C; it is a review boundary, not trunk. Rebase the local stack onto the fetched trunk and inspect the new parent edges. If the server has merged Q in the meantime, fetch its actual landed result instead of assuming C is the new parent.

```
jj git fetch --remote github
jj log -r 'main@github'
jj rebase -s 'roots(main@github..@)::' -d main@github
jj log -r 'main@github..@' -T '
commit_id.short() ++ " " ++ change_id.short() ++ " " ++
description.first_line() ++ "\n"
```

A merge from upstream can cause conflicts in Q' or R' . The conflict is part of the new commit value. Continue to inspect, materialise, resolve, snapshot, and test; there is no separate rebase continuation token to preserve. If a stack contains multiple publication boundaries, move and push them in a way that matches the host review dependency order.

Stage 6: closure and cleanup

When GitHub merges the PR, fetch main and compare the fetched commit with the reviewed tip. The server may have used squash, merge, or rebase; the new remote main is the authority for the next local base. The feature bookmark can be forgotten locally after its remote ref is no longer needed. If deleting the remote ref is desired, express and push that deletion separately. Closing a GitHub PR, deleting its ref, forgetting a local bookmark, and abandoning a jj change are four different actions.

When Gerrit submits Q but R remains, fetch Gerrit main, rebase R onto the landed server result, preserve R's Change-Id footer, and upload a new patch set. The local jj evolution of R is still useful, but it cannot predict the exact commit that Gerrit created when Q landed. This is why host integration always begins with fetch.

What to record in a lifecycle review note

Field	Example	Reason
Target versions	jj 0.45.1, Git 2.53.0	Syntax and default policies are versioned.
Base ref	main@github at full ID	Explains the diff range and host comparison.
jj change IDs	Q and R full IDs	Tracks logical evolution through rewrites.
Commit IDs	q_0/r_0 then q_1/r_1	Identifies concrete uploaded objects.
Publication refs	github/quota, remote target	Explains what a push changed.
Server identity	Gerrit Change-Id: I... or PR number	Maps local change to host review.
Conflict state	Commit, path, resolution operation	Distinguishes a stored conflict from a clean rewrite.
Operation IDs	rewrite, resolve, push-related operations	Provides local recovery and evidence.

A review note like “rebased branch and force-pushed” is too lossy for mutable history. The fuller record states which logical changes survived, which concrete objects were replaced, which server ref moved, and which host identity remained stable.

Edge-case catalogue

Empty commits

An empty commit has no tree delta relative to its parents, but it can carry a description, a bookmark, a signature, a workspace target, and descendants. It may be an intentional review boundary or an accidental result of a squash. Query `empty()` within the domain you care about. Do not remove every empty node from a stack without checking whether its message records a meaningful change or whether a publication workflow uses it as a boundary.

Merge commits

A merge has multiple parents, and many operations treat its parent list as a set rather than a single base. `parents(G)` returns both sides. `roots`, `heads`, `ranges`, first-parent traversal, and descendant selection all produce different results around a merge. Before a linear-only command, query `merges()` & `domain`. If it is non-empty, state whether the operation should preserve, flatten, or deliberately introduce the merge.

Hidden commits

Hidden does not mean unreachable from every mechanism. An explicit full commit ID can identify a hidden predecessor; an ordinary visible query may omit it. If a hidden ID is introduced into a traversal, its ancestors can enter the search space. This is useful for recovery and surprising in a broad query. When comparing two results, state whether the search was current-visible or explicitly seeded with a hidden commit.

Divergent changes

A divergent change has multiple visible successors for one logical change ID. A command requiring one commit may reject the bare change ID. Use offsets or an explicit query that chooses by topology, bookmark, description, or operation provenance. Do not pick the first displayed successor solely because it appears first; the display order is not an authorship or recency guarantee unless the template says so.

Bookmark conflicts

A bookmark conflict is ref state with competing target information, usually after incompatible local/remote updates. It is not a file conflict and is not resolved by editing a conflict marker. Fetch authoritative remote state, inspect target sets, decide whether to retain, move, or delete the name, and publish only after the result is explicit. Use `bookmark list` with remote and tracking filters rather than a commit log alone.

Private commits

Private is a publication policy defined by a revset, not a filesystem attribute or a second commit type. A private commit can be mutable, conflicted, bookmarked, or visible locally. Ordinary push excludes it according to configuration; an explicit allow option changes that safety policy for one invocation. Always render the private set and the candidate push domain before overriding it.

Working-copy edge cases

The working-copy commit can be empty, conflicted, bookmarked, immutable after a push, stale in another workspace, or based on multiple parents. A command that creates a new child can leave the old working copy as a useful change. A command that edits an ancestor can cause the current working copy to become a successor or a conflict. Query `@`, `@-`, `working_copies()`, and `conflicts()` when the prompt alone is ambiguous.

Shell edge cases

There are at least three parsers in a complicated invocation: the shell, jj's revset/fileset/template parser, and any external editor/tool. Quote the whole expression, use `--` when the command's help documents it, and avoid command substitutions that turn an empty result into a missing argument. A path containing whitespace is not evidence that a fileset is wrong; an unquoted glob being expanded before jj sees it is.

Automation edge cases

Automation must decide whether it owns snapshotting, whether it can tolerate multiple commits, whether external tools may write files, and whether network publication is allowed. Use explicit binaries, config, commit sets, filesets, templates, remotes, and colour/pager modes. Capture operation IDs and exit statuses. Fail closed on unexpected cardinality, a conflicted publication target, an untracked remote, or a changed remote target.

Language fieldbook — practice: deriving selections and output

Revsets, filesets, and templates are all functional languages, but their values live in different universes. A revset value is a set of commit nodes. A fileset value is a set of repository-relative paths. A template expression evaluates against a typed object and emits formatted output. The three languages meet at command boundaries, not in one shared evaluator. That distinction explains why an expression can be syntactically valid in one position and meaningless in another.

Revset evaluation workbench

Use this graph for the evaluations in this section. The arrows point from parent to child. The merge has two parents; F is not a head of the set containing G because F is an ancestor of G.



bookmarks:
main@origin → B
topic → H
release → G

Expression	Intermediate reasoning	Result
<code>B::</code>	Start at B; follow every child path.	<code>{B, C, D, E, G, H}</code>
<code>::G</code>	Start at G; follow both parent paths, including their common ancestors.	<code>{root, A, B, C, D, E, G}</code>
<code>B..G</code>	Ancestors of G minus ancestors of B.	<code>{C, D, E, G}</code>
<code>B::G</code>	Descendants of B intersect ancestors of G.	<code>{B, C, D, E, G}</code>
<code>heads(B::G)</code>	Remove every member that is an ancestor of another member.	<code>{G}</code>
<code>roots(B::G)</code>	Remove every member with a parent also in the set.	<code>{B}</code>
<code>(C D)..</code>	Not ancestors of C and not ancestors of D; do not distribute <code>..</code> over the union.	<code>{E, G, H}</code> subject to the visible search domain.
<code>C.. D..</code>	Union two complements of ancestry sets.	Usually larger than <code>(C D)..</code> ; calculate each side first.
<code>topic ~ ::B</code>	Start at H and remove all ancestors of B.	<code>{H}\`</code>

The exact set printer can be made independent of graph glyphs:

```
jj log --no-pager --color never -r 'B::G' \
-T 'description.first_line() ++ "\t" ++ commit_id.normal_hex() ++ "\n"'
jj log --no-pager --color never -r 'heads(B::G)' \
-T 'description.first_line() ++ "\n"'
```

When checking a worked example, do not use only the visible order in `jj log`. A merge is normally shown with graph indentation, but the indentation is a rendering. The semantic test is the parent list. Render each selected commit's parents and verify the closure or boundary by hand.

Set algebra and complement

`x | y` is union, `x & y` is intersection, and `x ~ y` is relative difference. Prefix `~x` is the complement of `x` in the revset search domain. The domain matters: hidden commits are generally absent unless explicitly introduced, and `all()` is not an unrestricted database query. If a hidden ID is mentioned, its ancestors become available to the expression; this can change what a later traversal includes.

Relative difference is the usual way to express “my set, except the published part”:

```
mutable() ~ ::main@origin
mutable() & ::@
(::@ ~ ::main@origin) & ~conflicts()
```

These are not interchangeable in every graph. The first removes every ancestor of remote main from the mutable set. The second first narrows to ancestors of the current workspace and then removes remote ancestry. The third additionally excludes conflicted nodes. State the intended universe before choosing the algebra.

A complement is best used inside a bounded expression. `~bookmarks()` means every visible commit that is not named by a bookmark, not “the commits in my current stack that have no bookmark”. The latter is `(@:: & mutable()) ~ bookmarks()` or another explicitly bounded set. This is one of the most common ways a correct operator produces an unsafe mutation.

Reachability and ranges

The double-colon operators are closures; the double-dot operators are ancestry differences. `x::y` is a bounded path-shaped intersection: descendants of `x` that are ancestors of `y`. `x..y` is ancestors of `y` excluding ancestors of `x`. The endpoints do not have to be related. In a fork, `left..right` can include the right branch and common descendants while excluding the common base; it is not a list of edges on one unique path.

Question	Derivation	Why the obvious spelling can fail
What changed since base?	<code>base..tip</code>	Includes all ancestors of tip not also ancestors of base, including side branches merged into tip.
What is on the ancestry path from base to tip?	<code>base::tip</code> then remove the base if the diff should exclude it	<code>base..tip</code> has different endpoint and fork semantics.
What is the complete stack below the current change?	<code>@:: & mutable()</code> or an explicitly configured alias	<code>@::</code> includes descendants of @, while the mutable filter excludes immutable descendants.
What is reachable from either publication ref?	<code>::(main@origin release)</code>	<code>::main@origin release</code> has the same result only if the grouping and parser precedence are understood.
What remains unpublished in a bounded topic?	<code>(topic:: & mutable()) ~ ::main@origin</code>	<code>mutable() ~ ::main@origin</code> can include unrelated work.

For an expression with a range on the left, always parenthesise unions:

```
(A | B)..      # not ancestors of A and not ancestors of B
A.. | B..      # commits not ancestors of A OR not ancestors of B
(A | B).. = A.. & B..
```

This identity follows from De Morgan's law applied to ancestry complements. It is not a jj peculiarity; it becomes visible because `..` is defined as a complement of reachability rather than as a path enumerator.

Symbol resolution in detail

Resolution is deliberately ordered. Tags take priority over bookmarks, which take priority over commit-ID or change-ID interpretation. This makes a short symbol convenient for humans but potentially unstable in automation. Use `commit_id("...")`, `change_id("...")`, `bookmarks(exact:"...")`, or `tags(exact:"...")` when the namespace matters.

A full commit ID identifies one object. A unique prefix identifies one object until a later object makes the prefix ambiguous. A full change ID identifies the visible evolution of a logical change; a divergent change ID requires an offset or an explicit

query over the divergent commits. A short change ID copied from a review message is therefore not a safe immutable handle. Record the full commit ID for an exact object and the full change ID for the evolving change.

```
# Treat a string as a literal symbol when punctuation is meaningful.
jj log -r '"x-"'
jj log -r 'commit_id(exact:"a1b2...")'
jj log -r 'change_id(exact:"qwer...")'

# Qualify remote state.
jj log -r 'main@origin'
jj log -r 'main@github | main@gerrit'
```

Shell quoting is an additional parser. In `jj log -r 'x-'`, the shell passes one string and `jj` interprets the suffix as the parents operator. In `jj log -r '"x-"'`, `jj` receives a revset containing a quoted string symbol. A script should quote the complete expression, not merely the individual words that happen to be visible in an example.

Cardinality and command contracts

Revsets always produce sets, even when a command wants one commit. The command applies a cardinality contract after parsing. `log`, `diff`, and many reporting commands accept arbitrary sets. `edit`, `describe`, and other operations that need one current target generally reject zero or multiple results. Some commands accept several explicit commits but impose topology constraints. Read the command's help and test the exact invocation.

```
candidate='heads(mutable() & ::@)'\njj log -r "$candidate" -T 'commit_id.normal_hex() ++ "\\n"'\ncount=$(jj log -r "$candidate" -T 'commit_id ++ "\\n" | wc -l)\ntest "$count" -eq 1\njj edit "$candidate"
```

The shell guard is valuable because a query that was singleton yesterday may become multi-valued after a fork or a new workspace. If the operation is safe over several heads, use a set-valued command intentionally. Do not force a singleton with `first()` merely to make an error disappear; that converts an explicit

ambiguity into an arbitrary policy.

Function families and their contracts

The function inventory is easiest to learn by the fact it computes. Topology functions transform or inspect graph reachability. State predicates select stored properties. Reference functions select names and remote relationships. Content predicates inspect trees or diffs. Metadata predicates match typed fields. The following tables state the contract, not just the spelling.

Function	Signature/arguments	Semantics and edge case	Operational use
<code>parents</code>	<code>parents(x[, depth])</code>	Parents at exactly the requested depth; merges return multiple parents; root becomes empty.	Find the base of a stack without assuming one parent.
<code>children</code>	<code>children(x[, depth])</code>	Direct or depth-limited children; a merge's parents can have several children.	Find immediate successors after a rewrite.
<code>ancestors</code>	<code>ancestors(x[, depth])</code>	Inclusive parent closure; depth zero is the starting set according to the release contract.	Bound a review or build dependency.
<code>descendants</code>	<code>descendants(x[, depth])</code>	Inclusive child closure; branch and merge paths are all followed.	Estimate rewrite blast radius.
<code>first_parent</code>	<code>first_parent(x[, depth])</code>	Follows first-parent edges rather than all ancestry; important for merge-shaped history.	Produce a release-line view without selecting side branches.
<code>connected</code>	<code>connected(x)</code>	Connected graph component containing the set under the documented visibility domain.	Check whether a proposed range spans unrelated roots.

Function	Signature/arguments	Semantics and edge case	Operational use
<code>reachable</code>	<code>reachable(sources, domain)</code>	Walk from source commits but include only nodes satisfying the domain; a domain is not merely an output filter in every graph shape.	Find commits reachable from refs while excluding immutable or private regions.
<code>roots</code>	<code>roots(x)</code>	Members with no parent in x; a merge-boundary set can have more than one root.	Find patch-stack bases.
<code>heads</code>	<code>heads(x)</code>	Members with no descendant in x; an ancestor of a selected merge is not a head.	Find publication tips and test endpoints.
<code>merges</code>	<code>merges()</code>	Members with more than one parent.	Intersect with a domain, such as <code>merges() & ::@</code> , before a linear-only operation.

Function family	Members	Interpretation
State	<code>empty()</code> , <code>conflicts()</code> , <code>mutable()</code> , <code>immutable()</code> , <code>divergent()</code>	Predicates over stored commit state; an empty commit can still be published, and a conflicted commit can have descendants.
Visibility	<code>all()</code> , <code>none()</code> , <code>visible_heads()</code> , <code>immutable_heads()</code> , <code>builtin_immutable_heads()</code>	Define the query domain and rewrite boundary; custom immutable heads are configuration, not a new commit type.
Identity	<code>commit_id(pattern)</code> , <code>change_id(pattern)</code> , <code>present(x)</code>	Resolve object or logical-change identity explicitly; inspect successive versions with <code>jj evolog</code> , not a revset function.
References	<code>bookmarks</code> , <code>remote_bookmarks</code> , <code>tracked_remote_bookmarks</code> , <code>untracked_remote_bookmarks</code> , <code>tags</code> , <code>remote_tags</code>	Reference state is not the same as reachability from a branch-like name; inspect native Git refs through the backend when needed.

Function family	Members	Interpretation
Workspace/operation	<code>working_copies()</code> , <code>at_operation(op,x)</code>	Working-copy commits and historical-operation views are queryable; workspace records themselves are managed by <code>jj workspace</code> .
Metadata	<code>author</code> , <code>committer</code> , <code>author_email</code> , <code>committer_email</code> , <code>author_date</code> , <code>committer_date</code> , <code>description</code>	Arguments are string/date patterns; there is no <code>email:</code> pattern kind.
Content	<code>files</code> , <code>diff_lines</code> , <code>diff_lines_added</code> , <code>diff_lines_removed</code>	Tree and diff predicates can be expensive and should be bounded before mutation.

For an unfamiliar function, establish four facts before using it in a destructive command: does it accept a set or a single commit argument; does it include its argument or only its closure; does it search visible commits or explicitly named hidden commits; and does it inspect metadata at one commit or compare two trees? The name alone does not answer these questions.

Fileset language workbench

Create a tree with deliberately similar names:

```
src/main.rs
src/lib.rs
src/generated/schema.rs
tests/api.rs
docs/api.md
Cargo.lock
README.md
build/output.bin
notes/old api.txt
```

By default, an unqualified path is a cwd-relative prefix glob. From the repository root, `src` selects `src` recursively. From a subdirectory, that same spelling is interpreted relative to the current working directory. Root-qualified forms make scripts independent of the caller's location:

Expression	Matches	Recommended use
<code>src</code>	cwd-relative prefix below <code>src</code>	Interactive command from a known directory.
<code>root:src</code>	workspace-relative prefix	Explicit subtree in a command where prefix form is intended.
<code>root-file:src/lib.rs</code>	one exact path	Scripts that must not include neighbours.
<code>root-glob:src/*.rs</code>	Rust files directly below <code>src</code>	Non-recursive glob selection.
<code>root-prefix-glob:src/*.rs</code>	matching files and matching prefixes	Recursive path-oriented glob policy.
<code>root-glob-i:docs/*.MD</code>	case-insensitive glob matches	Repositories with inconsistent case; the <code>-i</code> suffix is supported by <code>jj</code> 0.45.1.

Operators have the same useful shape as revsets: function/pattern, complement, intersection or difference, then union. Thus `'src ~ root-glob:src/**/*.generated/**'` selects the source subtree except generated files. Parenthesise a union before complementing it; `'~a | b'` is not the same as `'~(a | b)'`.

```
jj file list 'root:src ~ root-prefix-glob:src/**/*.generated/**'
jj diff -r @ -- 'root-file:README.md | root-glob:docs/**/*.md'
jj restore --from SOURCE --into @ -- 'root:src & ~root-glob:src/**/*.generated.rs'
```

Quoting has two layers. The shell expands unquoted `*`, `?`, and brackets before `jj` can parse them. Then `jj` parses operators, function calls, and string literals. Quote the complete fileset. If a path contains whitespace or parentheses, quote at both layers as the target syntax requires:

```
jj diff -- 'root-file:notes/old api.txt'
jj diff -- '~"notes/old api.txt"'
jj diff -- 'root-file:"notes/old api.txt"'
```

Use the exact pattern spelling supported by the installed help for paths containing a colon or quote. When a command accepts multiple path arguments, do not assume they are unioned in exactly the same way as one fileset expression; test the

command contract and prefer one explicit expression when the difference matters.

Template language workbench

Start with a scalar and keep the output contract visible:

```
# Typed object rendered through a method.
jj log -r @ -T 'commit_id.short() ++ "\n"'

# A list mapped to strings and joined at the output boundary.
jj log -r @ -T 'parents.map(|p| p.commit_id().short()).join(", ") ++ "\n"'

# A conditional over a Boolean.
jj log -r @ -T 'if(conflict(), "CONFLICT ", "") ++ description.first_line() ++ "\n"'

# Width and truncation for a human log.
jj log -r '@:: & mutable()' -T 'pad_end(12, commit_id.short()) ++ " " ++ truncate_end(72, descrip'
```

Methods are resolved from the value's type. `commit_id.short()` is a method on a commit-ID value. `description.first_line()` is a method on a string-like value. `parents.map` operates on a list and invokes a lambda once per element. Calling a commit method on `commit_id.short()`, or a string method on the commit itself, is a type error, not a formatting preference.

Optional values require a policy. A signature may be absent, a remote ref may not be tracked, and a workspace root may be stale. Use a Boolean predicate, `if`, `try`, or `coalesce` as appropriate. A fallback should identify absence rather than quietly render a misleading zero value.

```
jj log -r 'all()' -T '
if(signature, signature.name() ++ " <" ++ signature.email() ++ ">", "(unsigned)") ++
" " ++ description.first_line() ++ "\n"
```

The output of `json` is a string, so escape and delimit it at the record boundary. A description can contain a newline, tab, or control character. In the 0.45.1 template language there is no general object-literal syntax; emit independently escaped JSON scalar fields or use an external filter to assemble records:

```
jj --no-pager --color never log -r 'all()' -T '  
json(commit_id.normal_hex()) ++ "\t" ++  
json(change_id.normal_hex()) ++ "\t" ++  
json(description.first_line()) ++ "\n"
```

For display templates, labels are semantic style rather than content. A label can be mapped to a configured colour and can be stripped when colour is disabled.

Machine output should avoid labels and use `--color=never`. Raw escape sequences are an escape hatch for terminal control; they should not be used in a file intended for a Kindle PDF or a parser.

Revset reference atlas — lookup

This is a reader-facing reference, not a transcription of help output. Each entry states the input kind, the set returned, and the question it answers. The examples use the query-lab graph; replace the symbolic labels with real commits only after rendering the set. A function that accepts a commit expression does not necessarily require a single commit, and a function that returns a set does not make every consuming command set-valued.

Topology and closure

Function	Input	Result	Example and use
<code>parents(x)</code>	Revset, optional depth	Parents of members of <code>x</code> ; unioned as a set.	<code>parents(@)</code> finds the immediate base; use a parent template for a merge.
<code>children(x)</code>	Revset, optional depth	Direct or depth-limited children of <code>x</code> .	<code>children(B)</code> distinguishes a fork from a linear successor.
<code>ancestors(x)</code>	Revset, optional depth	Inclusive transitive parent closure.	<code>ancestors(@, 2)</code> bounds a local context without including all trunk.
<code>descendants(x)</code>	Revset, optional depth	Inclusive transitive child closure.	<code>descendants(B, 1)</code> estimates immediate rewrite blast radius.
<code>first_parent(x)</code>	Revset, optional depth	First-parent walk, useful for merge-oriented views.	Use on a release line when side-parent changes should not appear as a linear series.
<code>roots(x)</code>	Revset	Members with no parent also in <code>x</code> .	<code>roots(main@origin..@)</code> identifies the base of each unpublished component.
<code>heads(x)</code>	Revset	Members with no descendant also in <code>x</code> .	<code>heads(B::G)</code> is <code>G</code> in the worked merge, not its parent-side ancestor.
<code>merges()</code>	Revset	Members with more than one parent.	<code>merges() & ::@</code> audits whether the current ancestry is actually linear.
<code>connected(x)</code>	Revset	The connected component or components containing the set under the documented graph domain.	Use before a bulk rewrite to detect independent roots.
<code>reachable(s, d)</code>	Source revset and domain revset	Walks from <code>s</code> while restricting the reachable result using <code>d</code> .	<code>reachable(bookmarks(), mutable())</code> asks which mutable nodes are reachable from named refs.

The closure functions are inclusive at depth one: `ancestors(B, 0)` and `descendants(B, 0)` are the useful boundary cases to test against the installed binary because an omitted depth and an explicit depth have different meaning. A merge makes set union visible: `parents(G)` contains both parent sides, and `ancestors(G)` contains their common ancestry only once. Do not interpret a returned set as an ordered path.

`roots` and `heads` are dual boundary operators within a selected set. They are not the repository root and the repository's all-time tips unless their input is the corresponding whole domain. On `B : G`, `roots` returns `B` and `heads` returns `G`. On `: G`, `roots` returns the repository root and `heads` returns `G`. On a union containing unrelated stacks, both can return multiple commits.

State, visibility, and identity

Function	Contract	Edge case	Operational pattern
<code>all()</code>	All visible commits in the current repository view.	Hidden commits are not silently included; explicitly naming one changes the available search space.	Use for fixture inspection, not unbounded mutation.
<code>none()</code>	Empty commit set.	Useful in aliases and tests; commands may reject an empty selection even when parsing succeeds.	Use as a safe default for an optional publication set.
<code>visible_heads()</code>	Heads of the visible commit graph.	Not the same as heads of an arbitrary selected set.	Define the visible traversal boundary explicitly.
<code>immutable_heads()</code>	Configured immutable-head boundary.	The alias is configuration; an inert key with a similar name changes nothing.	Inspect before <code>--ignore-immutable</code> or policy changes.
<code>builtin_immutable_heads()</code>	Built-in immutable boundary before custom policy.	Useful for comparing policy to release defaults.	Audit a repository's safety policy after changing aliases.
<code>mutable()</code>	Commits permitted to be rewritten under the current policy.	A mutable commit may still have an immutable ancestor and a remote bookmark.	Bound local surgery with <code>mutable() & ::@</code> .
<code>immutable()</code>	Commits protected by the current policy.	Protection is a command safety rule, not permanent object storage.	Find the boundary before explaining a rejected rewrite.
<code>conflicts()</code>	Commits containing conflict values.	A clean working directory does not prove this set is empty elsewhere.	Use after rebase, merge, squash, or restore.
<code>empty()</code>	Commits whose tree delta is empty under the command's comparison semantics.	An empty commit is still a graph node and can have descendants or a bookmark.	Audit accidental empty review changes.
<code>divergent()</code>	Commits belonging to divergent change evolutions.	A divergent change ID may require an offset or a set query.	Find local evolution forks before convergence or manual surgery.

Function	Contract	Edge case	Operational pattern
<code>present(x)</code>	Commits in <code>x</code> that are present/visible under the current view.	Useful around optional names; it does not resurrect an arbitrary deleted object.	Guard a publication query against a missing remote ref.
Evolution history	Not a revset function in jj 0.45.1.	Use <code>jj evolog -r x</code> to inspect successive commit versions of a logical change.	Investigate what an ancestor rewrite replaced without pretending the evolution log is a commit set.
<code>root()</code>	The repository root commit.	Its parent set is empty; it is not a normal mutable working change.	Use <code>root()::</code> for a complete ancestry domain.

Visibility and immutability are independent dimensions. A hidden commit can remain immutable, and a visible commit can be mutable. A remote bookmark can point at a mutable commit if local policy permits it; publication does not automatically make every local descendant immutable unless the configured boundary says so. Use separate predicates in a query when both dimensions matter.

Bookmarks, tags, remotes, and workspaces

Function	Arguments	Meaning	Example
<code>bookmarks()</code>	Optional name pattern	Commits targeted by local bookmarks matching the pattern.	<code>bookmarks("pr/*")</code> finds local PR boundaries.
<code>remote_bookmarks()</code>	Optional name and remote patterns	Commits targeted by remote-tracking bookmarks.	<code>remote_bookmarks("main", remote="origin")</code> selects fetched trunk.
<code>tracked_remote_bookmarks()</code>	Optional patterns	Remote bookmarks participating in tracking relationships.	Audit only refs that supply ahead/behind information.
<code>untracked_remote_bookmarks()</code>	Optional patterns	Remote bookmarks without local tracking.	Find fetched refs that a push script must qualify explicitly.
<code>tags()</code>	Optional name pattern	Commits targeted by local tags.	<code>tags("v*")</code> finds release points.
<code>remote_tags()</code>	Optional name and remote patterns	Fetched remote tag targets.	Compare release tags across remotes.
<code>working_copies()</code>	No arguments	Commits serving as working copies across all workspaces.	<code>working_copies()</code> detects a commit held by one or more workspaces; use <code>jj workspace list</code> to identify the workspace names.
<code>at_operation(op, x)</code>	Required operation and revset	Evaluates <code>x</code> in the repository view at operation <code>op</code> .	<code>at_operation(OP, @-)</code> or <code>at_operation(@-, visible_heads())</code> ; both arguments are required.

Reference functions return targets, not the names themselves. To render names, use a commit template's bookmark or tag methods. This separation is useful: `bookmarks()` can select a set of commits, while a template can then show all names attached to each selected commit. If two local names point to the same commit, the selection still contains one commit.

Remote arguments are patterns and can be qualified by labels according to the installed syntax. An unqualified `remote_bookmarks("main")` can select same-named refs from several remotes. In a publication safety check, pass the remote pattern or use an explicitly qualified symbol. In jj 0.45.1, remote tag predicates distinguish tracked and untracked relationships.

Metadata and content predicates

Function	Argument	Matching rule	Example
<code>description(pattern)</code>	String pattern	Matches commit descriptions using explicit pattern kinds.	<code>description(substring:"quota")</code> .
<code>author(pattern)</code>	String pattern	Matches author identity according to the installed pattern semantics.	<code>author(glob:"A.*")</code> .
<code>committer(pattern)</code>	String pattern	Matches committer identity.	Use when import/rewrite actor differs from author.
<code>author_email(pattern)</code>	String pattern	Matches the author email field.	<code>author_email(exact:"dev@example.o</code>
<code>committer_email(pattern)</code>	String pattern	Matches the committer email field.	Distinguish automation from the original author.
<code>author_date(pattern)</code>	Date pattern	Matches author timestamps.	<code>author_date(after:"2026-01-01")</code> .
<code>committer_date(pattern)</code>	Date pattern	Matches committer timestamps.	Find changes rewritten this week.
<code>commit_id(pattern)</code>	String pattern	Resolves commit IDs without bookmark/change-ID precedence surprises.	<code>commit_id(exact:"full-id")</code> .
<code>change_id(pattern)</code>	String pattern	Resolves visible change IDs and their matching evolutions.	Use a full ID or offset for divergence.
<code>files(fileset)</code>	Fileset expression	Selects commits whose changes match the fileset.	<code>files("root-file:src/api.rs")</code> .
<code>files(fileset)</code>	Fileset expression	Selects commits changing/containing paths under the documented predicate.	Bound to a stack before use in automation.
<code>diff_lines(text, [files])</code>	String pattern and optional fileset	Matches changed content line by line in a diff.	<code>diff_lines(substring:"diagnostic"</code> <code>"root:src")</code> .

Function	Argument	Matching rule	Example
<code>diff_contains(pattern)</code> deprecated	Compatibility string pattern	Accepted by jj 0.45.1 but emits a deprecation warning.	Use <code>diff_lines(pattern)</code> in new queries.
<code>diff_lines_added(pattern)</code>	String pattern	Matches added lines.	Audit introduction of an API or marker.
<code>diff_lines_removed(pattern)</code>	String pattern	Matches removed lines.	Find the change that removed a compatibility path.

Pattern arguments are not a single universal string syntax. String patterns support explicit kinds such as `exact:`, `substring:`, `glob:`, and `regex:`. Date patterns have their own forms and ranges. The invalid form `author(email:"...")` fails because `email` is not a pattern kind in 0.45.1; use `author_email(exact:"...")` or the documented `author` pattern semantics.

Content predicates can be expensive. A file predicate compares trees and a diff predicate examines diff content. First narrow the graph with `mutable()` & `::@`, a publication range, or a date window. If the query is used in a bulk rewrite, render the result and record how many commits matched before invoking the mutation.

Patterns, dates, and aliases

A pattern is a typed value passed to a matcher. It is not the same thing as a shell glob and not the same thing as a fileset. The shell must pass the pattern intact; jj then parses the pattern prefix. These examples show the intent:

```
description(exact:"fix quota boundary")
description(substring:"quota")
description(glob:"fix * boundary")
description(regex:"quota[0-9]+")
author_email(exact:"dev@example.org")
author_date(after:"2026-01-01" & before:"2026-02-01")
```

Use exact patterns for identity and release automation. Use substring for interactive search. Use glob when a conventional naming family is deliberate, and regex only

when the additional syntax is worth the portability and escaping cost. A pattern alias can hide that policy; give aliases descriptive names and document whether they match case, punctuation, and whitespace.

Revset aliases are parsed in the context in which they are used. A parameterless alias names a reusable set; a parameterised alias behaves like a function. Keep aliases small enough that `jj log -r ALIAS` remains explainable. Avoid aliases that silently choose the first result or that combine publication and mutability policy without a name that reveals both. Test recursive aliases and aliases that shadow builtin functions in a disposable configuration.

```
[revset-aliases]
"unpublished_stack()" = "(@:: & mutable()) ~ ::remote_bookmarks()"
"owned_work()" = "author_email(exact:\"dev@example.org\") & mutable()"
"release_line()" = "first_parent(tags())"
```

The examples above show the conceptual shape; quote TOML strings and pattern arguments exactly as required by the installed parser. A parameterised alias's argument substitution is not shell interpolation. If an alias is used in a script, include one expansion test in the repository's validation suite.

Derivation exercises

Exercise: derive unpublished work

Start with a graph containing two local stacks and two remote bookmarks. The goal is “mutable commits in the current workspace's ancestry that are not reachable from any remote bookmark”. Begin with `::@`. Intersect with `mutable()`. The publication exclusion is `~::remote_bookmarks()`. The derived expression is `mutable() & ::@ ~ ::remote_bookmarks()`, with parentheses added around the positive domain if it is extended. Verify that a mutable commit on an unrelated branch is excluded, while a mutable unpublished ancestor remains. If the expression returns a merge, decide whether its heads or all changes are wanted.

Exercise: heads of a non-linear query

Using the workbench graph, calculate $B : G$ first. It contains B, both side paths, and G. Test each member for whether it is an ancestor of another member. B, C, D, and E are ancestors of G; G is not an ancestor of another member in the set. Therefore $\text{heads}(B : G)$ is $\{G\}$. Now calculate $\text{heads}((B : G) \sim \text{merges}())$. The merge is removed from the domain, so the heads can change; the answer must be derived from the resulting set, not copied from the earlier result. This is why a heads query should be evaluated after every filter.

Exercise: a range across a fork

Let L and R be sibling tips from a common base. Calculate $L..R$ as $::R \sim ::L$. The result contains R's ancestry that is not also in L's ancestry; it does not represent a unique sequence of edges from L to R. Now add a merge descendant that contains both sides and recalculate. The merge can enter the set even though L is not an ancestor of R. A Git user accustomed to a branch range should write the intended review base and server ref explicitly rather than relying on the visual branch names.

Exercise: precedence proof

Compare $A \mid B \ \& \ C$, $(A \mid B) \ \& \ C$, and $A \mid (B \ \& \ C)$. The first and third are equivalent because intersection binds more tightly than union; the second is different. Then compare $A \sim B \ \& \ C$ with $A \sim (B \ \& \ C)$. Difference and intersection have the same precedence and associate left-to-right in the installed grammar, so the unparenthesised form is $(A \sim B) \ \& \ C$. Do not rely on a language remembered from another tool.

Exercise: cardinality as a changing property

Define `candidate = heads(mutable() & ::@)`. It is singleton-valued in a linear stack. Create a second child or a fork; the same expression becomes multi-valued. A command such as `jj edit "$candidate"` now fails rather than choosing arbitrarily. This is a useful failure. A script may assert one result, or it may use a set-valued command and handle several tips. The query has not become wrong merely because its cardinality changed; the consumer's precondition has changed.

Configuration and template reference — lookup

Configuration is part of `jj`'s execution model. It selects defaults, defines aliases, controls immutability, chooses external tools, and can change which commits an apparently unqualified command touches. A configuration file is therefore closer to a program's policy module than to a collection of cosmetic preferences. Keep the stable personal layer small, keep repository policy reviewable, and treat workspace overrides as local experiments.

The layer model

On Linux, `jj` loads built-in defaults and then the configured user, repository, workspace, and command-line inputs in the order documented by the target release. The exact paths are discovered with `jj config path`; do not hard-code an XDG path in an installer. `--config NAME=VALUE` is an invocation override. `--config-file PATH` adds a file as input; it is not the same as selecting the user/repository/workspace file that `config edit` modifies.

Layer	Typical scope	Good contents	Hazard
Built-in	Every invocation	Safe defaults supplied by <code>jj</code>	Defaults can change between releases.
User	One Linux user across repositories	Identity, pager, editor, personal aliases	A remote or immutable policy can unexpectedly affect unrelated repositories.

Layer	Typical scope	Good contents	Hazard
Repository	One repository	Project revsets, fileset aliases, tool policy	Opening an unfamiliar repository can load policy you did not write.
Workspace	One working directory	Temporary log view, build/review defaults	Another workspace in the same store does not necessarily share it.
Command line	One process	Version gates, dry runs, explicit remote, diagnostic override	Quoting and TOML typing errors are easy to hide in shell history.

For a scalar key, a higher layer normally replaces a lower value. For tables and aliases, merging and replacement rules make it unsafe to assume that “unset the top-level table” removes every child. Diagnose with `config list`, then inspect the scope files. Use a temporary alternative config file when testing a policy; never edit the production user config first.

A progressive Linux configuration

```
# ~/.config/jj/config.toml, discovered with jj config path --user
[user]
name = "A. Engineer"
email = "a.engineer@example.org"

[ui]
pager = "less -FRX"
paginate = "auto"
color = "auto"
default-command = ["log", "-r", "@:: & mutable()"]

[revset-aliases]
"mine()" = "author_email(exact:\"a.engineer@example.org\")"
"my_work()" = "mine() & mutable()"
"published()" = "::remote_bookmarks()"
"unpublished()" = "my_work() ~ published()"

[fileset-aliases]
"LOCKFILES" = "root-glob:**/Cargo.lock | root-glob:**/package-lock.json"

[template-aliases]
"brief_commit(commit)" = "commit.change_id().short() ++ \" \" ++ commit.description().first_line()
```


Start with identity and output. Add revset aliases only after testing them against a non-linear fixture. Add fileset aliases when the same path policy appears in several commands. Add template aliases last, because a typo in a type-sensitive method can make all log output fail. A repository-specific config may redefine the log revset, but a publication alias should name its remote explicitly or accept the remote as a visible argument.

jj 0.45.1 supports splitting user configuration under `conf.d`, loaded in lexical order. Verify the effective result with `config list` and a harmless sentinel value in a disposable configuration. The maintainable principle is independent of the directory name: keep UI, aliases, templates, Git transport, and Gerrit policy reviewable as separate units and document their precedence.

UI, snapshot, and editor policy

Family	Key examples	Design choice	Interaction
Identity	<code>user.name</code> , <code>user.email</code>	Default author/committer identity	Explicit metadata options can override it; operation username/hostname is a separate record.
Output	<code>ui.color</code> , <code>ui.paginate</code> , <code>ui.pager</code> , <code>ui.graph</code> , timestamps	Interactive readability versus stable scripts	Command-line <code>--color=never</code> and <code>--no-pager</code> should win in automation.
Editing	<code>ui.editor</code> , <code>ui.diff-editor</code> , <code>ui.merge-editor</code>	How descriptions, diffs, and conflicts are edited	External tools have their own <code>merge-tools.*</code> configuration and exit-status contract.
Conflict rendering	<code>ui.conflict-marker-style</code>	How stored conflicts are materialised	Changing style changes the working-copy representation, not the underlying conflict value.
Snapshot	<code>snapshot.auto-track</code> , size/EOL/exec-bit policy	Which filesystem paths enter the next tree	Ignored/untracked files can make a command appear to have done nothing.
Movement	working-copy movement/update policy	How commands choose or update @	Workspace-local state and stale-workspace repair must be considered.

The pager is a particularly common source of false failures. A program that pipes `jj log` to a parser may receive a pager process or terminal control sequences unless it explicitly disables both. Conversely, forcing `--no-pager` in every interactive alias can make a large history unusable. Keep human and machine aliases distinct.

Snapshot settings are policy, not a substitute for `.gitignore`-style project discipline. Test a maximum new-file size with a disposable large file, test executable-bit changes on a filesystem that supports them, and test a repository from a subdirectory. Record whether the policy prevents tracking, warns, or refuses an operation. A configuration that “looks right” in TOML can still be ineffective if the

command's default snapshot path never considers the file.

Revsets, aliases, and rewrite safety

Defaults such as the log, status, fix, sign, run, and bookmark-advance revsets are executable policy. A default is evaluated in a graph that changes over time, so a command that was narrow when configured can become broad after a merge or workspace handoff. Before changing a default, run the old and new expressions side by side on the query fixture.

```
old=$(jj config get revsets.log)
jj log -r "$old" -T 'commit_id.short() ++ " " ++ description.first_line() ++ "\n"'
jj log -r '(@:: & mutable()) | main@origin' -T 'commit_id.short() ++ " " ++ description.first_line() ++ "\n"
```

The immutable boundary is a special case. In 0.45.1, the configured function is `revset-aliases."immutable_heads()"`, whose definition can refer to tags, remote bookmarks, and the root. A key called `revsets.immutable_heads` is merely unknown data and does not redefine the function. Demonstrate a safety setting by querying the function and attempting a protected rewrite in a disposable repository.

```
[revset-aliases]
"immutable_heads()" = "builtin_immutable_heads() | main@origin | tags()"
```

Keep parameterised aliases small. An alias that accepts a pattern and inserts it into a regex or fileset can be useful, but it also creates a second quoting boundary and can be a source of injection-like mistakes in shell automation. Prefer fixed aliases for publication and require an explicit commit argument for destructive operations.

GitHub and Gerrit configuration

Git transport configuration should identify credentials and remote URLs, not silently encode a publication decision that differs by repository. Use `jj git remote list` to inspect names and URLs, and use explicit `--remote` in pushes. A repository can have an `origin` GitHub remote and a `gerrit` remote without either being “the” remote in the abstract.

```
[git]
# Keep publication remote explicit in shared scripts.
private-commits = "mutable() ~ ::remote_bookmarks()"

[gerrit]
# Use only keys supported by the installed release; on 0.45.1 pass
# --remote and --remote-branch to gerrit upload.
default-remote = "gerrit"
default-remote-branch = "main"
```

The Gerrit defaults shown are part of the `jj 0.45.1` surface. Pass the remote and branch explicitly in scripts. Configuration can select a default, and `templates.commit_trailers` can generate the footer in local commits; without that template, `gerrit upload` derives a missing footer only for the uploaded object. Preserve or deliberately generate the server footer and inspect both identities before upload.

For conditional configuration, match repository identity or path only when the condition is visible and testable. A safe pattern is to keep the user layer neutral, place GitHub/Gerrit aliases in repository-level policy, and make each alias print a dry-run or require the remote in its invocation. Never make a broad command alias called `push` choose a server based solely on the current directory without showing the choice.

Template reference: types and composition

Templates have values, methods, functions, and a rendering boundary. In a commit template, zero-argument commit methods are available as keywords; in an operation template, operation methods are available. Explicit method calls are clearer when a template is copied between commands.

Value	Important methods/fields	Use	Failure mode
Commit	<code>commit_id()</code> , <code>change_id()</code> , <code>description()</code> , <code>parents()</code> , <code>author()</code> , <code>committer()</code> , <code>state</code> <code>predicates</code>	Log/show records	Printing the object directly or calling a method with the wrong return type.
CommitId/Changeld	<code>short()</code> , <code>shortest()</code> , <code>normal_hex()</code>	Human and exact identities	Confusing a shortest prefix with a stable full identifier.
Signature	<code>name()</code> , <code>email()</code> , <code>timestamp()</code>	Author/committer display	Assuming author and committer are one object.
Timestamp	<code>ago()</code> , <code>format()</code> , <code>utc()</code> , <code>local()</code>	Review age and reproducible reports	Local timezone or clock changes output.
CommitRef	<code>name()</code> , <code>remote()</code> , <code>target/tracking predicates</code>	Bookmark/tag status	Conflicted refs have more than one target state.
List	<code>map</code> , <code>filter</code> , <code>join</code> , <code>first</code> , <code>last</code>	Parents, refs, paths	Empty list and wrong element type.
Option	<code>conditional/unwrap/fallback functions</code>	Signatures and optional refs	Calling contained methods before testing presence.
Template	<code>fill</code> , <code>indent</code> , <code>padding</code> , <code>truncation</code> , <code>labels</code>	Human layout	Widths and escape sequences differ from visible byte length.

Arithmetic operators operate on integers; comparisons operate on compatible scalar values; boolean operators short-circuit; and `++` concatenates template values. Parenthesise mixed arithmetic and concatenation. A style label is metadata attached to rendered content and may disappear when colour is disabled. This is why styles should not be parsed as data.

Building a compact daily log

```
# 1. Identity and subject.
jj log -T 'change_id.short() ++ " " ++ description.first_line() ++ "\n"'

# 2. Add the object ID and graph state.
jj log -T 'change_id.short() ++ "/" ++ commit_id.short() ++ " " ++
if(conflict(), "[conflict] ", "") ++ description.first_line() ++ "\n'

# 3. Add refs and a bounded parent list.
jj log -T 'change_id.short() ++ "/" ++ commit_id.short() ++ " " ++
bookmarks.map(|b| b.name()).join(",") ++ " [" ++
parents.map(|p| p.commit_id().short()).join(",") ++ "]" ++
description.first_line() ++ "\n'
```

The source examples use line-leading operators only inside a code block; indent them in shell files if a diff or code-generation tool treats a leading plus specially. A daily template should remain cheap: avoid full diff rendering, signature verification, and expensive file predicates for every commit. Create a second “forensic” template for those fields.

```
# A forensic record, with colour and pager disabled.
jj --no-pager --color never log -r 'all()' -T '
json(commit_id.normal_hex()) ++ "\t" ++
json(change_id.normal_hex()) ++ "\t" ++
json(description.first_line()) ++ "\n"
'
```

Test each expression against root, merge, conflict, empty, and missing-ref cases. A template that works at @ can fail on root because a parent list is empty, or on an unsigned commit because an optional signature is absent. Optional values are not exceptional data; they are part of the normal graph.

Template debugging and portability

Debug a template by reducing it to `self`, one keyword, one method, and one combinator. Keep a “type probe” template for a fixture rather than trying to infer types from an error message after a complex expression has failed. Use explicit full IDs when output will be used to select a commit again; use short IDs only for human scanning.

When moving a template between the installed release and future releases, compare the local help captures for method names and argument order. A template alias can hide a version difference, so include the alias definition in a release note. If a field is not available in both releases, either omit it from the shared template or supply two clearly named aliases. Never label a target-release-specific method as if it were an installed 0.45.1 builtin.

Standalone references — lookup: keys and command paths

These tables are intended to be used without opening installed help. They describe the significant target-version keys and the canonical command paths that an engineer is likely to use. The human interpretation matters because a schema can state a type without explaining how a key interacts with graph selection, publication, or workspace state.

Configuration key reference

Key	Type/shape	What it controls	Interaction and caution
<code>user.name</code>	String	Default commit identity name	Does not set operation username; explicit author options can override.
<code>user.email</code>	Email string	Default commit identity email	Validate repository policy before signing or publication.
<code>operation.hostname</code>	String	Hostname metadata on operations	Separate from committer metadata and can reveal machine identity.
<code>operation.username</code>	String	Username metadata on operations	Useful for multi-user shared stores; not a Git author field.
<code>ui.default-command</code>	String or command array	Command when no explicit command is given	A default that snapshots or invokes a broad revset should be reviewed as policy.
<code>ui.color</code>	Enum	Colour mode: auto/always/never/debug	Scripts should use an explicit command-line mode.
<code>ui.paginate</code>	Enum/string	Whether output is paged	Disable in captured output; interactive pager choice is separate.
<code>ui.pager</code>	String/command	Pager executable and arguments	the built-in pager default is release-specific; set explicitly when important.
<code>ui.streampager.interface</code>	Enum/config table	Stream pager interface behaviour	Terminal-dependent; do not parse screen output.
<code>ui.streampager.wrapping</code>	Enum	Pager wrapping	Changes human layout, not repository state.
<code>ui.streampager.show-ruler</code>	Boolean	Pager ruler display	Cosmetic and unsuitable as a machine-output assumption.

Key	Type/shape	What it controls	Interaction and caution
<code>ui.diff-instructions</code>	Boolean	Diff-editor instruction display	Helpful interactively; disable or account for it in scripted editors.
<code>ui.graph.style</code>	String/enum	Graph rendering style	Graph glyphs are display, not stable topology data.
<code>ui.log-word-wrap</code>	Boolean/enum	Log description wrapping	Use templates for stable fields instead.
<code>ui.log-synthetic-elided</code>	Boolean	Whether elided nodes appear synthetically	Can alter apparent graph density; query explicit commits for proofs.
<code>ui.editor</code>	Command	Commit-description editor	Environment and exit status are external dependencies.
<code>ui.diff-editor</code>	Command/enum	Interactive diff editor	Different from a merge editor; fileset and commit selection remain jj policy.
<code>ui.diff-formatter</code>	Command/enum	Diff display/formatting	Human rendering can hide binary or conflict details.
<code>ui.merge-editor</code>	Command/enum	Conflict merge editor	External tool failure leaves the stored conflict; inspect exit status.
<code>ui.conflict-marker-style</code>	Enum	Materialised conflict marker style	Changes filesystem representation, not the conflict value.
<code>ui.show-cryptographic-signatures</code>	Boolean	Signature display/verification in output	Verification may invoke external tools and be expensive.
<code>ui.movement.edit</code>	Boolean/policy	Movement commands' edit behaviour	Can change which workspace commit is materialised.
<code>ui.bookmark-list-sort-keys</code>	List	Bookmark list ordering	Ordering is presentation; never use it as a ref selector.
<code>ui.tag-list-sort-keys</code>	List	Tag list ordering	Same name/target caveat as bookmarks.

Key	Type/shape	What it controls	Interaction and caution
<code>fsmonitor.backend</code>	Enum/config	Filesystem monitoring backend	Performance feature; a monitor failure must not be mistaken for a clean tree.
<code>fsmonitor.watchman.register</code>	Boolean	Watchman snapshot integration	Requires external Watchman setup and process coordination.
<code>colors</code>	Table	Semantic label colours	Labels can disappear with colour disabled; data templates should not depend on them.
<code>diff.color-words.conflict</code>	Colour/table	Conflict word diff styling	Presentation only; stored conflict state is queried separately.
<code>diff.color-words.max-inline</code>	Integer	Inline word-diff complexity threshold	Large diffs may fall back to another representation.
<code>diff.color-words.context</code>	Integer	Word-diff context	Does not change the selected fileset.
<code>diff.git.context</code>	Integer	Git-style diff context	Rendering option; avoid parsing line layout.
<code>diff.git.show-path-prefix</code>	Boolean	Git diff path prefixes	Root/cwd path semantics are still determined by the fileset.
<code>diff.stat.context</code>	Integer	Stat context	Only affects summary presentation.
<code>fileset-aliases</code>	Table/alias definitions	Named path sets, functions, docs	Separate parser and namespace from revset aliases.
<code>git.abandon-unreachable-commits</code>	Boolean/policy	Handling of unreachable commits in Git interaction	Can affect visible/ref state; test before enabling in a shared repository.
<code>git.fetch</code>	String/list/table	Fetch defaults and ref behaviour	Fetch updates observations; it does not move local publication bookmarks.

Key	Type/shape	What it controls	Interaction and caution
<code>git.private-commits</code>	Revset	Commits excluded from ordinary push	Push overrides are explicit and should be audited.
<code>git.push</code>	Remote name/string	Default push remote	Tracking does not choose the remote; explicit <code>--remote</code> is safer.
<code>git.record-synthetic-predicates</code>	Boolean	Backend evolution bookkeeping	Affects history diagnostics, not server review identity.
<code>git.sign-on-push</code>	Boolean/policy	Signing of pushed Git commits	Key agents and server verification remain external.
<code>git.track-default-bookmark-on-clone</code>	Boolean	Tracking after clone	Does not select a publication remote for later pushes.
<code>git.write-change-id-header</code>	Boolean/policy	Git commit header/evolution interoperability	Do not confuse generated metadata with Gerrit's Change-Id footer.
<code>git.executable-path</code>	Path/string	Git executable selection	Pinning can help reproducibility; verify the binary version.
<code>git.colocate</code>	Boolean/policy	Colocation default/behaviour	Changes the native Git interoperability boundary.
<code>git.object-hash</code>	Enum	Git object hash format	Interoperability with external tools and remotes is format-dependent.
<code>remotes</code>	Table	Named Git remotes and URLs	Names are local configuration; remote bookmarks are fetched observations.
<code>gerrit.default-remote</code>	String	Default Gerrit upload remote in jj 0.45.1	the target documentation surface; explicit upload flags are portable.
<code>gerrit.default-remote-branch</code>	String	Default Gerrit target branch	Do not infer trunk from a local bookmark of the same name.

Key	Type/shape	What it controls	Interaction and caution
<code>gerrit.review-url</code>	URL/string	Review URL construction/display	Instance-specific; it does not create or verify a review.
<code>merge.hunk-level</code>	Boolean/enum	Merge granularity	Can change conflict shape; validate with real two-sided fixtures.
<code>merge.same-change</code>	Policy	Merge treatment for same logical change	Change identity and commit identity remain distinct.
<code>merge-tools</code>	Table	External conflict tools and arguments	Treat executable paths and arguments as trusted code.
<code>revsets.log</code>	Revset	Default log domain	Defaults change what an unqualified log shows; render explicit sets for scripts.
<code>revsets.fix</code>	Revset	Default fixer domain	A broad value can rewrite many changes.
<code>revsets.run</code>	Revset	Default run domain	Pair with <code>--ignore-changes</code> for checks when appropriate.
<code>revsets.sign</code>	Revset	Default signing domain	Signing can rewrite descendants and publication IDs.
<code>revsets.arrange</code>	Revset	Default arrangement domain	Graph transformation defaults should be inspected before use.
<code>revsets.simplify-parents</code>	Revset	Parent simplification domain	Can alter merge topology; test on a copy.
<code>revsets.bookmark-advance-to</code>	Revset	Bookmark advance destination	Advance is not arbitrary bookmark set/move.
<code>revsets.bookmark-advance-from</code>	Revset	Bookmark advance source	Bound it so unrelated names are not moved.

Key	Type/shape	What it controls	Interaction and caution
<code>revset-aliases.immutable</code>	Read-only alias	Rewrite protection boundary	Correct 0.45.1 location; an unknown similarly named key is inert.
<code>template-aliases</code>	Table/alias definitions	Reusable typed output expressions	Aliases can shadow builtins; test each type and release.
<code>aliases</code>	Command alias table	Shortcuts and workflows	Document side effects, remote selection, and argument forwarding.
<code>run.jobs</code>	Integer	Default parallelism for <code>jj run</code>	Parallel output and shared resources can make tests non-deterministic.
<code>snapshot.auto-track</code>	Boolean/policy	Automatic file tracking	Changes which paths enter a future snapshot.
<code>snapshot.auto-update-stale</code>	Boolean/policy	Stale workspace update policy	Inspect unrecorded files before enabling automatic reconciliation.
<code>snapshot.max-new-file-size</code>	Size/integer	Large-file snapshot policy	Can make a file remain untracked despite appearing in the filesystem.
<code>experimental-advance-branches.enabled</code>	Unimplemented-branches	Experimental automatic branch advancement	Do not confuse with tracking or working-copy movement.
<code>signing.backend</code>	Enum	Cryptographic signing backend	Requires external key infrastructure and verification policy.
<code>signing.key</code>	String/key identifier	Signing key selection	Keep identifiers/private paths out of shared reports.
<code>signing.behavior</code>	Enum/policy	When signing is required/attempted	Can affect push and rewrite workflows.
<code>signing.backends.gpg.program</code>	Path	GPG executable	Validate with a test key; never assume an agent is available.

Key	Type/shape	What it controls	Interaction and caution
<code>signing.backends.ssh.program</code>	Path	SSH signing executable	Allowed-signers/revocation configuration controls verification.
<code>fix.tools</code>	Table	Fixer tool definitions	External commands can rewrite a broad commit/fileset domain.
<code>split.legacy-bookmark-behavior</code>	Boolean/policy	Compatibility behaviour for split/bookmarks	Version-specific migration setting; test before enabling.
<code>hints</code>	Boolean/table	Helpful command hints	Presentation, not a semantic assertion.
<code>templates.*</code>	Template strings	Default output templates for log/show/operation/file/ref commands	Human defaults can change; scripts should pass <code>-T</code> .
<code>working-copy.eol-conversion</code>	Enum/policy	Working-copy line-ending conversion	Can change the snapshot representation and commit ID.
<code>working-copy.exec-bit-check</code>	Policy	Executable-bit handling	Filesystem support and Git backend semantics both matter.
<code>--when.*</code>	Conditional tables	Repository/host/workspace/conditions	Test templates in every intended context.

The reference intentionally groups some table-valued keys. Their child schema is a user-defined namespace: for example, `merge-tools.NAME`, `fix.tools.NAME`, and `template-aliases.NAME`. The correct way to document a new child is to state the executable or expression, its inputs, its output, and whether it can rewrite state. A name alone is not a safe configuration.

Canonical command-path reference

Canonical path	Primary input	Effect class	Reader contract
<code>jj abandon</code>	Revset	Graph rewrite	Can reconnect/rebase descendants; inspect evolution and bookmarks.
<code>jj absorb</code>	Working-copy diff and mutable ancestors	Content ownership/rewrite	Heuristically assigns individual diff hunks; verify every affected diff.
<code>jj arrange</code>	Commit set and arrangement	Graph rewrite	Use for deliberate stack arrangement; inspect parent arrays.
<code>jj bisect</code>	Commit domain and predicate/command	Search state	Requires a meaningful graph order and deterministic test.
<code>jj bisect run</code>	Command	Repeated checkout/test	Record environment and return-code meaning.
<code>jj bookmark advance</code>	Name patterns, destination revset	Local ref movement	Advances under policy; does not arbitrarily replace a name.
<code>jj bookmark create</code>	Name and one target	Local ref creation	No commit rewrite or tracking by itself.
<code>jj bookmark delete</code>	Name patterns	Local ref deletion	Can express remote deletion when pushed.
<code>jj bookmark forget</code>	Name patterns	Local bookkeeping	Does not delete remote state; may include remote observations.
<code>jj bookmark list</code>	Name/remote/commit filters	Inspection	Use all-remotes and tracked filters for a publication audit.
<code>jj bookmark move</code>	Existing names and target	Local ref movement	Backwards/sideways movement has safety checks.
<code>jj bookmark rename</code>	Old/new local name	Local ref metadata	Remote names and tracking need separate review.

Canonical path	Primary input	Effect class	Reader contract
<code>jj bookmark set</code>	Name and target	Local ref create/replace	Convenient but hides whether a name was new.
<code>jj bookmark track</code>	Local name and remote	Relationship metadata	Does not make the name follow @.
<code>jj bookmark untrack</code>	Tracked pair	Relationship metadata	Does not delete either ref.
<code>jj commit</code>	Working-copy state/message	Working-copy transition	Creates a new working-copy child or commits current work according to help.
<code>jj config edit</code>	Scope/file	Config write/external editor	jj 0.45.1 provides direct file targeting alongside scope selection.
<code>jj config gc</code>	Config storage	Maintenance	Do not confuse with storage GC or remote pruning.
<code>jj config get</code>	Key	Inspection	Reports effective typed value.
<code>jj config list</code>	Optional key/prefix	Inspection	Use to diagnose precedence and aliases.
<code>jj config path</code>	Scope	Inspection	Prints the participating scope path.
<code>jj config set</code>	Scope, key, TOML value	Config write	Shell quoting and TOML typing both matter.
<code>jj config unset</code>	Scope, key	Config write	Reveals lower-precedence values; does not always remove the effective key.
<code>jj describe</code>	One commit/message	Metadata rewrite	Commit ID can change; logical change normally persists.
<code>jj diff</code>	Commit endpoints/fileset	Inspection	Fileset limits output; it does not itself limit graph rewrites.
<code>jj diffedit</code>	One commit/editor/fileset	Diff rewrite	Interactive ownership selection creates successors.

Canonical path	Primary input	Effect class	Reader contract
<code>jj duplicate</code>	Source commits/destination	New change	Copies diff intent with a new change identity.
<code>jj edit</code>	One commit	Workspace movement	Moves current @ and filesystem, not bookmarks generally.
<code>jj evolog</code>	Change/commit selection	Evolution inspection	Shows predecessors, successors, and divergence.
<code>jj file annotate</code>	Commit/fileset	Inspection	Line attribution is not a stable machine schema by default.
<code>jj file chmod</code>	Fileset/mode	Working-copy tree change	Snapshot records executable-bit policy.
<code>jj file list</code>	Commit/fileset	Inspection	Best first check for fileset scope.
<code>jj file search</code>	Pattern/commit/fileset	Inspection	Search result text is arbitrary user data.
<code>jj file show</code>	Commit/path	Inspection	Binary/conflict content needs explicit handling.
<code>jj file track</code>	Fileset	Snapshot policy	Changes future tracking, not existing commit content.
<code>jj file untrack</code>	Fileset	Snapshot policy	Does not delete the path from the tree.
<code>jj fix</code>	Commit/fileset/configured tools	External rewrite	Inspect tool, domain, successors, and conflicts.
<code>jj gerrit upload</code>	Revsets/remote/branch/footer	Network publication	Local transport is testable; server review association is instance-dependent.
<code>jj git clone</code>	Git URL/path	Network/clone	Creates an independent operation history.
<code>jj git colocation</code>	Subcommand	Backend layout	Enable/status/disable have native Git safety implications.

Canonical path	Primary input	Effect class	Reader contract
<code>jj git export</code>	Repository state	Backend synchronisation	Makes supported jj refs/objects available to Git.
<code>jj git fetch</code>	Remote/options	Network/import	Updates remote observations; local bookmarks remain local.
<code>jj git import</code>	Git state	Backend synchronisation	Reconciles native Git-side ref changes.
<code>jj git init</code>	Path/options	Repository creation	Initialises backend; does not publish or clone.
<code>jj git push</code>	Remote/bookmark/tag/change/network	Network publication	Tracking provides lease-like safety; remote must be explicit.
<code>jj git remote add</code>	Name/URL	Config write	Does not fetch or validate credentials.
<code>jj git remote list</code>	None	Inspection	Shows names/URLs, not reachability or permission.
<code>jj git remote remove</code>	Name	Config write	Remote observations and server refs may persist elsewhere.
<code>jj git remote rename</code>	Names	Config/ref metadata	Qualifiers in scripts need updating.
<code>jj git remote set-url</code>	Name/URL	Config write	Does not move remote refs.
<code>jj git root</code>	None	Inspection	Git backend root differs from workspace root.
<code>jj help</code>	Topic/path	Discovery	Installed binary is normative for syntax.
<code>jj interdiff</code>	Two commits/fileset	Inspection	Compares diff iterations; not simply a tree diff.
<code>jj log</code>	Revset/template	Inspection/snapshot	Default domain/template are config; graph glyphs are display.

Canonical path	Primary input	Effect class	Reader contract
<code>jj metaedit</code>	Commit/metadata	Metadata rewrite	Parent descendants can receive successors.
<code>jj new</code>	Parents/message	Working-copy graph change	Creates a child/merge and moves @ according to options.
<code>jj next</code>	Movement selection	Workspace movement	Forks and merges make “next” a policy question.
<code>jj operation abandon</code>	Operation selector	Operation graph rewrite	Changes recovery history; does not directly edit a commit tree.
<code>jj operation diff</code>	Operation selector	Inspection	Shows repository-view differences.
<code>jj operation integrate</code>	Operation selector	Operation graph change	Combines an unintegrated/concurrent operation after inspection.
<code>jj operation log</code>	Template/limit	Inspection	Operation IDs are local and distinct from commit IDs.
<code>jj operation restore</code>	Operation selector	Repository-view restore	Can remove later integrated state from the current view.
<code>jj operation revert</code>	Operation selector	Repository-view inverse	Creates a new recovery operation where possible.
<code>jj operation show</code>	Operation selector	Inspection	Read the exact operation before restoring it.
<code>jj parallelize</code>	Commit selection	Graph rewrite	Creates independent parallel parents/children; inspect merge implications.
<code>jj prev</code>	Movement selection	Workspace movement	Merge parents require deliberate choice.
<code>jj rebase</code>	Source/destination revsets	Graph rewrite	Selection and destination are independent; descendants follow.

Canonical path	Primary input	Effect class	Reader contract
<code>jj redo</code>	Operation convenience	Repository-view restore	Review operation state before redoing after other work.
<code>jj resolve</code>	Conflict/fileset/tool	Conflict resolution	Stored conflict becomes a resolved tree after snapshot.
<code>jj restore</code>	From/into/fileset	File-content rewrite	Restores selected file contents; does not mean reset or squash.
<code>jj revert</code>	Commit/destination	New inverse change	Preserves the original event; can conflict.
<code>jj root</code>	None	Inspection	Prints workspace/repository root.
<code>jj run</code>	Commit set/command/jobs	Repeated checkout/rewrite	Use <code>--ignore-changes</code> for checks and inspect env variables.
<code>jj show</code>	Commit/template/fileset	Inspection	Shows selected commit and its diff.
<code>jj sign</code>	Commit/key/backend	Metadata rewrite	External key infrastructure and descendants matter.
<code>jj simplify-parents</code>	Commit set	Graph rewrite	Can change merge topology; test on a copy.
<code>jj sparse edit</code>	Editor	Sparse config edit	Opens an editor; does not take fileset patterns in 0.45.1.
<code>jj sparse list</code>	None	Inspection	Prints patterns; verify filesystem separately.
<code>jj sparse reset</code>	None	Sparse materialisation	Restores full/default sparse state.
<code>jj sparse set</code>	Fileset patterns	Sparse materialisation	Replaces/sets patterns; does not alter commit trees.
<code>jj split</code>	Commit/fileset/editor	Graph/content rewrite	Creates a boundary and rebases descendants.

Canonical path	Primary input	Effect class	Reader contract
<code>jj squash</code>	From/into/fileset	Diff ownership rewrite	Source may become empty; inspect both diffs.
<code>jj status</code>	None	Inspection/snapshot	Filesystem cleanliness is not graph cleanliness.
<code>jj tag delete</code>	Name patterns	Local tag change	Remote deletion needs publication.
<code>jj tag list</code>	Name/remote filters	Inspection	Local and remote tags are separate.
<code>jj tag set</code>	Name/target	Local tag creation/move	Use a tested immutable release target.
<code>jj tag track</code>	Name/remote	Relationship metadata	Tracking tags is not bookmark tracking.
<code>jj tag untrack</code>	Name/remote	Relationship metadata	Does not delete a remote tag.
<code>jj undo</code>	None/operation convenience	Repository-view recovery	Primarily reverses the latest integrated operation.
<code>jj unsign</code>	Commit set	Metadata rewrite	Can change IDs and publication objects.
<code>jj util backend name</code>	None	Inspection	Identifies storage backend.
<code>jj util completion</code>	Shell	Generated installation file	Regenerate per release and shell.
<code>jj util config-schema</code>	None	Generated schema	Machine authority, not a human policy guide.
<code>jj util exec</code>	Command/args	External execution	Argument/environment contract is version-sensitive.
<code>jj util gc</code>	None/options	Storage maintenance	Can remove recovery data; inspect op/evolution first.
<code>jj util install-man-pages</code>	Destination/options	External file write	Installation action outside repository state.
<code>jj util markdown-help</code>	None	Generated documentation	Useful evidence, not the authored book.

Canonical path	Primary input	Effect class	Reader contract
<code>jj util snapshot</code>	None/options	Working-copy snapshot	Explicitly owns the snapshot boundary.
<code>jj version</code>	None	Inspection	Record before comparing inventories.
<code>jj workspace add</code>	Path/name/commits/sparse	Workspace creation	Creates workspace-local @ over shared graph.
<code>jj workspace forget</code>	Workspace name	Workspace metadata	Does not delete directory or commits automatically.
<code>jj workspace list</code>	Template	Inspection	Use before graph rewrites to find stale writers.
<code>jj workspace rename</code>	New name	Workspace metadata	0.45.1 renames current workspace; it is one positional new name.
<code>jj workspace root</code>	None	Inspection	Prints current workspace root.
<code>jj workspace update-stale</code>	Workspace selection/options	Workspace reconciliation	Inspect unrecorded files before updating.

This table uses canonical operation names even where daily use favours the `op` alias. The canonical path is what a release inventory must cover. The per-command entries earlier explain syntax and examples; this atlas supplies the cross-command effect classification for selection, graph, refs, workspace, and backend state.

Command reference atlas — lookup: the remaining surface

The 0.45.1 command inventory contains more than the history-rewriting core. These commands are smaller individually but matter in a Linux field workflow because they form the inspection, transport, and maintenance boundary. The canonical names below are the names used by the installed help; short aliases such as `op` for `operation` are shown only as conveniences.

Inspection and navigation

Command	Arguments/options that matter	Semantic contract	Typical trap
<code>jj status</code>	Global repository, snapshot, pager and colour controls	Reports working-copy state and normally snapshots first.	Treating “clean” as proof that every graph commit is conflict-free.
<code>jj log</code>	<code>-r/--revisions</code> , <code>-T/--template</code> , limit/reverse/graph/output controls	Renders a commit set; the default revset and template are configuration.	Parsing graph glyphs or assuming the displayed order is a topological proof.
<code>jj show</code>	Commit selector, template, diff/output options	Shows commit metadata and tree difference for selected commits.	Confusing the displayed diff with the complete tree or with a fileset-limited diff.
<code>jj diff</code>	Commit endpoints, fileset arguments, format/stat/template controls	Compares trees or working-copy state and limits paths through a fileset.	Using an unquoted glob and letting the shell choose the paths.
<code>jj interdiff</code>	Two commit/change endpoints and output/fileset controls	Compares the diffs represented by two commits, useful for review iterations.	Using it as a tree diff when the two endpoints have different bases.
<code>jj root</code>	Repository selection	Prints the workspace/repository root used by jj.	Confusing it with <code>jj git root</code> , which answers a backend Git-directory question.
<code>jj next</code>	Movement options and optional commit selection	Moves the workspace through graph children according to the command's selection policy.	Assuming it moves a bookmark or chooses the only visual child in a fork.
<code>jj prev</code>	Movement options and optional commit selection	Moves the workspace through graph parents.	Assuming a merge has one parent or that movement rewrites history.
<code>jj edit</code>	Exactly one commit in normal use; movement/allow options	Moves the current workspace's working-copy pointer to an existing commit and materialises its tree.	Expecting another workspace's @ or a bookmark to move.

For stable evidence, separate the selected set from rendering. A log command may show an abbreviated ID, labels, or a graph; a diagnostic template can render the full commit ID and parent IDs. A diff command may select a set of commits but render them in a chosen order. An inspection command can still snapshot unless the global working-copy option is used.

File subcommands

Command	Purpose	Selection	State effect
<code>jj file annotate</code>	Blame-like line attribution for a commit/tree	Commit plus fileset/path arguments	Inspection; normally no graph rewrite, but snapshot policy still applies.
<code>jj file chmod</code>	Change executable-bit state	Fileset plus executable/non-executable option	Changes working-copy tree on the next snapshot.
<code>jj file list</code>	List files in a tree	Commit and fileset	Inspection; ideal for testing fileset parsing.
<code>jj file search</code>	Search text in a tree/workspace	Pattern and optional commit/fileset	Inspection; output can contain arbitrary user text.
<code>jj file show</code>	Print one file from a commit	Commit and path/fileset contract	Inspection; binary and conflict values need explicit handling.
<code>jj file track</code>	Mark paths for snapshot tracking	Fileset	Changes tracking state, which affects a future snapshot.
<code>jj file untrack</code>	Stop tracking paths	Fileset	Changes tracking state; it is not deletion of the file from a commit.

File commands are where the distinction between a path and a fileset has practical consequences. `file show` generally wants a path whose content can be printed; `file list` can display a set; `file track` changes what the next snapshot will consider. A command can successfully parse a path and still find no matching file. Test a positive and a negative case in the fileset fixture.

Git backend and remote subcommands

Command	Purpose	Network?	Important state distinction
<code>jj git init</code>	Initialise a jj repository with a Git backend	No	Creates/associates Git storage; it is not a remote clone.
<code>jj git clone</code>	Clone a Git repository into a jj working copy	Yes	Creates a new clone with an independent operation history.
<code>jj git fetch</code>	Fetch Git refs and update remote-bookmark observations	Yes	Remote names are observations until local tracking/publication policy uses them.
<code>jj git push</code>	Export and push selected bookmark refs	Yes	Publishes refs; it does not push the operation log or workspace pointer.
<code>jj git import</code>	Import Git ref changes into jj	No	Reconciles native Git-side ref changes with jj's view.
<code>jj git export</code>	Export jj ref/commit state to Git storage	No	Makes local jj state available to Git-native tools.
<code>jj git root</code>	Print Git backend root/directory	No	Not necessarily the same path as the workspace root.
<code>jj git colocation status</code>	Report whether Git and jj are colocated	No	Answers backend layout, not whether refs are synchronised.
<code>jj git colocation enable</code>	Enable colocated Git state	No	Adds interoperability surface; establish one-writer discipline.
<code>jj git colocation disable</code>	Disable colocation	No	Changes backend arrangement; check native Git tooling before use.

Remote management has its own namespace: `remote add` records a name and URL, `remote list` displays them, `remote remove` removes a configured remote,

`remote rename` changes the local name, and `remote set-url` changes its URL. None of those operations fetches or pushes by itself. A remote URL can be valid while its credentials or server permissions are not.

```
jj git remote add github git@github.com:org/project.git
jj git remote add gerrit ssh://review.example.org/project
jj git remote list
jj git fetch --remote github
jj git fetch --remote gerrit
```

After a native Git fetch in a colocated repository, run `jj git import` before querying remote bookmarks. After a `jj bookmark update`, `jj` exports the Git ref; a Git-native tool can then inspect the new ref. Concurrently changing the worktree with Git and `jj` is unsafe even if the two commands target different refs; snapshot ownership is a filesystem concern.

Tag commands

`jj tag set` creates or moves a local tag at a selected commit; `tag list` inspects tags; `tag delete` changes local tag state; `tag track` and `tag untrack` manage relationships to remote tags. Tag names are publication labels, not mutable stack organisation. A release tag should be placed only after the exact commit and parents have been tested, and the release policy should make the tag part of the immutable-head boundary.

```
jj tag set v2.4.0 -r release-candidate
jj tag list
jj tag track v2.4.0 --remote origin
jj git push --remote origin --bookmark release-candidate
```

In `jj 0.45.1`, `jj git push --tag NAME` publishes a selected tag; a local tag operation alone does not publish it. `jj 0.45.1` also provides tracked/untracked remote tag revset predicates. Use those predicates in a target-release shared alias only when their policy meaning is intentional.

Utilities and maintenance

Command	Use	Why it deserves a contract
<code>jj version</code>	Print binary version	Use in every reproducibility report and version-gated script.
<code>jj help</code>	Discover command tree and topics	Help is the installed release's normative syntax surface.
<code>jj util completion</code>	Generate shell completion	Completion can expose aliases and descriptions but is not a substitute for help.
<code>jj util config-schema</code>	Emit machine-readable config schema	Schema validates types and keys but not human policy interactions.
<code>jj util backend name</code>	Identify the backend	Useful when a repository's storage assumptions are unclear.
<code>jj util gc</code>	Garbage-collect jj storage	Can reduce recovery options; run only after op/evolution retention review.
<code>jj util snapshot</code>	Explicit snapshot utility	Changes the normal command-boundary assumption; coordinate workspace ownership.
<code>jj util exec</code>	Utility-level command execution	Argument forwarding and environment are release-sensitive; do not use as an undocumented shell wrapper.
<code>jj util markdown-help</code>	Generate installed help material	Useful for research; generated help is not the authored book.
<code>jj util install-man-pages</code>	Install local man pages	Writes outside the repository and should be treated as an installation action.

Maintenance commands are deliberately absent from the everyday alias layer. A daily alias may call `status`, `log`, or a bounded diff; it should not call storage GC, config GC, or an operation-abandoning command as a side effect. Keep repository maintenance explicit and independent of ordinary inspection aliases.

Global options as an explicit contract

`-R/--repository` selects a repository. It does not select a Git remote, workspace, or operation. `--ignore-working-copy` prevents snapshot/update of the filesystem. `--at-operation` loads a historical repository view and therefore implies that current filesystem edits must not be inserted into it. `--ignore-immutable` bypasses rewrite protection and should be rare in a script. `--no-integrate-operation` changes operation integration and creates a concurrency/recovery responsibility.

`--color`, `--no-pager`, and `--quiet` affect output. Repeatable `--config` and `--config-file` affect input. `-h/--help` and `-V/--version` are discovery/version controls. The fact that a flag appears globally does not make its semantic effect identical in every subcommand; the command-specific help remains authoritative.

Mechanics synthesis: why the graph changes

The commands in `jj` are easier to predict if the underlying transformations are understood. This chapter describes the conceptual algorithms without pretending to expose an implementation API. The commit store, working-copy state, reference state, and operation store are separate pieces of repository state. A command can update more than one, but the updates have different invariants.

Rewrite as a successor function

Let a commit be a tuple containing parent IDs, tree ID, author/committer metadata, description, and conflict values. Its commit ID is a function of that tuple. A rewrite changes at least one component. The new tuple receives a new commit ID. `jj` also records that the new tuple is a successor of the old logical change. Descendants whose parent tuple contains the old commit ID cannot retain their old commit object unchanged, so `jj` derives successors for them too.

```
old B = (parent A, tree tb, metadata mb) → cid(B)
new B = (parent A, tree tb', metadata mb) → cid(B')
```

```
old C = (parent B, tree tc, metadata mc) → cid(C)
new C = (parent B', rebased tree, metadata mc) → cid(C')
```

The logical change ID is attached to the evolution rather than recomputed as the commit ID. This is why a commit message can refer to a change after several rewrites, while a CI system that stores only the commit ID must update its record. If two processes derive B' and B'' independently from B, both successors can remain visible and the change becomes divergent.

The successor relationship is not a promise that the diff is textually identical. A descendant rebase may resolve cleanly, may alter context, may introduce a conflict value, or may require a new merge shape. The invariant is graph evolution: the descendant is the successor of the old descendant under the changed parent context. Review the diff to determine whether the logical change survived as intended.

Automatic descendant rebasing

Automatic descendant rebasing is the consequence of maintaining parent identity, not an optional convenience. If an ancestor changes, leaving descendants pointing at the old object would produce a stale branch of the graph. jj instead rewrites affected descendants so their parent edges point at the new successors. This can be surprising when editing a small early change changes many commit IDs, but it is what keeps a stack coherent.

Ancestor operation	Descendant condition	Typical result
Description/meta edit	Descendant directly or transitively depends on changed commit	New descendant commit IDs with equivalent or rebased trees.
Diff edit	Descendant's diff applies to new ancestor	Clean successor with changed parent and often changed tree ID.
Diff edit	Descendant's diff cannot apply	Successor contains a stored conflict.
Ancestor abandon	Descendant depends on abandoned node	Reparenting/rebase according to topology; possible conflict.

Ancestor operation	Descendant condition	Typical result
Ancestor split	Descendant points at old change	Successor points at the new split tip.
Ancestor duplicated elsewhere	Original descendant does not depend on duplicate	Original stack unchanged; duplicate has independent identity.

Bookmarks are updated as references to local successors where the operation can determine the corresponding evolution. A remote bookmark is not silently changed because it represents a fetched external state. A workspace working-copy pointer can also be updated or become stale depending on whether its commit was rewritten and whether its filesystem has unrecorded content.

To reason about a large rewrite, compute the descendant closure before the command:

```
jj log -r 'descendants(TARGET)' -T '
commit_id.normal_hex() ++ "\t" ++ change_id.normal_hex() ++ "\t" ++
description.first_line() ++ "\n"
```

Then run the operation and compute it again. The set of exact commit IDs will differ; the set of intended logical changes and their parent relationships is the useful comparison. If the new closure includes unrelated work, the source set or destination was too broad.

Trees, diffs, and context

A commit stores a tree, while a review usually discusses its diff relative to a parent. The same tree can represent different diffs under different parents. Rebase preserves a change's intended tree transformation as far as possible, but its diff can change when context changes. Restore copies selected file contents; squash changes which commit's parent-to-tree difference owns content; diffedit edits a diff; and split creates new boundaries through one tree delta.

```
parent P:  config = old, code = old
change Q:  config = new, code = new
```

```
restore Q → R:
  R receives selected tree values
  Q remains a separate change

squash Q into P:
  P' owns the diff
  Q may become empty

split Q:
  P -- Q_1 -- Q_2
  Q_1 and Q_2 compose to Q's original tree delta
```

When a path is selected, ask whether its value is present in the source tree, whether it differs from the source parent, and whether the destination has a conflicting value. A fileset that selects an unchanged path can be a valid empty diff selection. A path missing from a sparse workspace can still be present in the stored tree and must be reasoned about with a commit/fileset command rather than a filesystem listing.

Diff context is also why automatic descendant rebase can produce a conflict without losing the descendant's logical change. The source side, destination side, and common ancestor are retained in the conflict value. The resolution must choose a tree that satisfies the domain invariant; “take ours” is only correct when the parent orientation and desired semantics make it correct.

Conflict values as algebra

A conflicted tree entry records sides and a base relationship. Materialising the working copy displays a representation of that value. Resolving it replaces the conflict value with a normal tree value. The commit can therefore move through these states without an interrupted transaction:

```
clean Q
| rebase/merge
▼
conflicted Q' -- edit/resolve/snapshot --> clean Q''
|                                     |
|-- descendant R' may also be conflicted +
```

Conflict propagation follows graph operations. A descendant created on top of a conflicted parent can remain meaningful, but a subsequent rewrite may materialise or propagate the conflict. Query `conflicts()` over the full review stack, not only the current workspace. If a merge editor resolves a path in one workspace, other workspaces may need stale update/materialisation before they see the resolved tree.

Conflict marker style controls presentation. The “diff” style can show base and sides differently from “git” style, and experimental styles can change between releases. Do not store marker text as the conflict identity. The stable evidence is the conflicted predicate and the resulting tree after resolution.

The operation DAG model

The commit DAG answers “what commits exist in this repository view?” The operation DAG answers “how did this repository view change?” One operation can update refs without changing commits. Another can rewrite commits and update workspace state. A third can recover a historical view. The commit graph and operation graph therefore have different nodes, edges, and recovery semantics.

```
commit DAG:      A -- B -- C'
                  \
                  D

operation DAG:    op_0 -- op_1(rewrite B)
                  | - op_2(snapshot)
                  \- op_3(move bookmark)
```

Operation descriptions are useful but should not be treated as a complete semantic diff. Use `op show` and `op diff` to inspect the repository-view changes. A snapshot operation may rewrite the working-copy commit; a bookmark operation may change only a ref; a push has external side effects that the local operation record cannot roll back at the server.

An operation created without integration can be inspected at its operation ID. Integrating it is a deliberate change to the current operation head. Concurrent operation nodes can be compared and possibly integrated. If two operations rewrite the same logical change differently, the operation graph may be easy to merge while the commit evolution becomes divergent. These are separate conflicts.

Recovery granularity

Recovery tool	Granularity	Use when	Limit
<code>undo</code>	Latest integrated operation	The last operation was clearly wrong.	Later operations change what “latest” means.
<code>redo</code>	Previously undone operation	The undo itself was a mistake and the operation remains redoable.	Not a general commit reapplication mechanism.
<code>op restore</code>	Repository view	You want the whole repository state at a known operation.	Can remove later integrated state from the current view.
<code>op revert</code>	Repository-view inverse	You want a new operation that reverses a selected operation.	Not every external side effect is reversible.
<code>evolog</code>	Logical change evolution	You need an old or divergent form of a change.	Does not restore unrecorded files or remote server state.
Git remote	Exported commits/refs	You need to move work to another clone.	Does not transfer operation history or filesystem buffers.

Recovery should preserve evidence. Before a destructive-looking command, create a full-ID record and note the current operation. After a mistake, stop making changes until the wrong operation is identified. If a server push already happened, treat the server state as a separate incident; local undo cannot erase a remote review or ref update.

Git backend object boundary

The Git backend gives jj a way to exchange commit trees and refs with Git. It does not turn all jj metadata into Git metadata. Commit IDs can be represented as Git objects; bookmarks can be exported as refs; operation IDs, workspace symbols, change evolution, and conflict representation are jj concerns. A Git-native tool sees the exported object graph and refs, not the complete local jj state.

Colocation is therefore a coordination contract. After `git update-ref`, import. After jj changes a bookmark, export before a Git-native inspection. After a Git commit, import through supported mechanisms and inspect how it entered the jj graph. Do not edit `.jj` internal files or assume the Git index is a jj staging area.

Object reachability also affects recovery. An unexported jj commit may not be available in another clone. An abandoned local commit may be retained by jj operation/evolution storage but not by a Git remote. Git GC and jj GC have distinct roots. Preserve a temporary bookmark or an exported ref before maintenance if the object matters.

Synthesis: proof-driven jj practice

The most useful way to close the gap between knowing individual commands and operating a difficult repository is to treat every non-trivial jj action as a small proof. The proof is not formal verification in the theorem-prover sense; it is a disciplined record of the state that makes the intended transformation unambiguous. This matters more in jj than in a branch-centred workflow because a single command can rewrite an ancestor, rebase several descendants, move local bookmarks, update a workspace, and create a new operation while leaving remote observations untouched.

Begin by naming the question in graph terms. “Put the tests with the API change” is underspecified. It might mean move the test diff into the API change, copy the test tree into the API change, split one mixed change, or create a new dependent change. Those are different tree and identity transformations. A useful precondition

record therefore contains the selected commit set, the destination commit set, the selected fileset, and the intended diff owner. It also records whether the operation is allowed to cross an immutable head and whether a publication bookmark currently names any affected commit.

Proof item	Question	Typical observation
Domain	Which commits can the expression see?	<code>jj log -r 'visible() & ::@'</code>
Selection	Which exact commits will the command consume?	<code>jj log --no-graph -r 'roots(@:: & mutable())'</code>
Tree delta	Which paths and diff hunks are actually changing?	<code>jj diff -r R -- fileset</code>
Destination	Which parent edge or tree endpoint will change?	<code>jj log -r 'parents(R) destination'</code>
Identity	Which commit IDs and change IDs may evolve?	<code>jj log -T 'commit_id ++ " " ++ change_id'</code>
Publication	Which local and remote names are involved?	<code>jj bookmark list --all-remotes</code>
Recovery	Which operation and temporary ref preserve the before-state?	<code>jj op log</code> plus a temporary bookmark

After the command, repeat the observations rather than trusting the success message. For a rewrite, compare parent edges and the diff at every affected change. For a bookmark operation, compare the local target with the imported remote target. For a workspace operation, inspect both the workspace symbol and the filesystem. For a push, record the remote response separately: the local operation log can show what `jj` requested, but it cannot undo a server-side ref update or remove a review patch set.

```
proof loop:
  state description
    |
    v
  source/destination/fileset query
    |
    v
```

```
operation 0_n
|
+-- commit DAG and evolution
+-- workspace materialisation
+-- bookmarks and remote observations
+-- operation DAG
|
v
postcondition checks
```

This loop also explains why a command that merely “works” can still be wrong. A broad `-r` expression may be accepted and produce a coherent graph, but if it includes a sibling stack the operation has violated the intended precondition. A fileset may parse and select existing paths while selecting no changed diff hunks. A bookmark may be moved locally while the user believes a server ref has moved. A template may render a plausible empty field for an absent optional value. Proof-driven practice makes these errors visible at the boundary where they are cheapest to correct.

Rewrite proof: diff ownership, topology, and identity

Consider a stack with a public base and two private changes:

```
main@origin
|
A public API change
|
B tests and compatibility
|
C(@) documentation and examples
```

Suppose the intent is to move only the compatibility tests from B into A while keeping the documentation as a separate change. The operation is not “amend A”. First determine whether the tests are complete files, diff hunks within a file, or a diff that is already shared with C. A path-level squash can move a complete path; a diff editor is needed for diff-hunk-level ownership. The precondition is that A and B are mutable, that the selected paths are present in B’s diff, and that C is a descendant whose diff can be reapplied over the new B.

```
jj log -r 'A|B|C' -T 'change_id.short() ++ " " ++ commit_id.short() ++ " " ++ description.first_line()'
jj diff -r B -- root-glob:tests/**
jj diff -r A -- root-glob:tests/**
jj op log --limit 3
jj squash --from B --into A -- root-glob:tests/**
```

The postcondition is not simply that one file appears in A. A's successor must contain the intended test diff, B's successor must retain the unselected compatibility diff, and C's successor must have the same documentation diff relative to its new parent. If the test diff touched a file that A already changed, compare the combined tree rather than comparing file presence. A path can exist in both commits while contributing no delta. If C now conflicts, the conflict is evidence that its old diff depended on context changed by the ownership transfer; it is not evidence that squash failed to perform the transfer.

Identity gives a second independent postcondition. The rewritten A and B normally retain their respective jj change IDs while receiving new commit IDs. C also receives a new commit ID because its parent identity changed, even if its logical diff is unchanged. A bookmark at C may follow the local successor, while `C@origin` remains the fetched remote observation. A Gerrit footer in the commit message is a third identity channel and must be inspected independently. This is why a review note should record all three values when a published stack is rewritten.

The same proof pattern applies to split and rebase. For split, prove that the two new changes partition the original diff and that the descendant is rebased onto the second result. For rebase, prove the source set before the destination: `-r`, `-s`, and `-b` describe different source policies. Then prove every new parent edge. A visual log is useful, but a template printing full IDs and parents is stronger evidence when a graph has several forks or merges.

Publication proof: local graph, review ref, and remote observation

Publication is best modelled as a state machine with three layers. The local graph contains changes and their current commit successors. The local bookmark is a name chosen for a publication boundary. The remote bookmark is the last imported observation of a server ref. A tracking relationship connects the local and remote names but does not cause either to follow @. A GitHub pull request adds a server-side base/head view; Gerrit adds a review identity usually carried by a commit-message footer. None of those server objects is the jj change ID.

```
local graph:      A -- B -- C(@)
                  |
local ref:         review -> C
                  |
tracking pair:     review <-> review@origin -> B
                  |
server request:    update review@origin from B to C
```

In this state, the local bookmark is ahead of its imported remote target. The safe publication procedure is to fetch first when the observation may be old, verify the remote name and URL, compare B with the currently fetched target, and then push the explicit bookmark. The command is a request to update a Git ref; it is not a request to publish “the current change” in the abstract. If another contributor has moved the server ref, the expected old target no longer matches and the operator must decide whether to rebase, merge, or deliberately replace the ref through the hosting policy; jj 0.45.1's `jj git push` has no force flag.

```
jj git remote list
jj git fetch --remote origin
jj bookmark list review --all-remotes
jj log -r 'review | review@origin' -T 'commit_id.normal_hex() ++ " " ++ change_id.normal_hex() ++ '
jj diff --from review@origin --to review
jj git push --dry-run --remote origin --bookmark review
jj git push --remote origin --bookmark review
```

A rejected push has a small diagnostic decision tree. If the bookmark is absent remotely, determine whether creation is allowed and whether the chosen name is correct. If the remote target has advanced, fetch and inspect the new ancestor

relation. If the local target is behind, a normal fast-forward push may be inappropriate; moving the local bookmark is a ref decision, not a rebase. If the server rejects the update because the branch is protected, the local graph is still valid but the publication protocol requires a pull request, a Gerrit review ref, or an authorised force update. If the commit cannot be exported, inspect private-commit policy and backend state before trying another ref.

GitHub and Gerrit can share one local graph without sharing publication semantics. A GitHub bookmark might name the head of a PR stack, while Gerrit upload selects a range and sends commits to `refs/for/main`. The same local commit can therefore be viewed as a GitHub branch tip and as one patch set in a Gerrit series. Keep remote names explicit, and use distinct local prefixes when that reduces operator ambiguity. Do not make a local bookmark track two server concepts merely because their visible names happen to be equal.

Cleanup is another state transition, not an aesthetic operation. After a GitHub PR merges, fetch and determine whether the merged commit is reachable from the imported base. Only then forget or delete the local publication bookmark according to whether the server ref should remain. After a Gerrit change lands, fetch the target branch and check whether the local change is now an ancestor of it; a server-side rebase or squash can mean that commit IDs differ while the diff is integrated. Preserve a temporary ref until that comparison is complete.

Conflict and recovery proof: preserve both graphs

A conflict drill should distinguish three events: conflict creation, conflict materialisation, and conflict resolution. Rebase or merge can create a conflicted successor in the commit DAG. A command can materialise that value as marker files in one workspace. Editing and snapshotting creates a successor with a resolved tree. These are not stages of an interrupted Git transaction; they are ordinary repository states connected by operations. A descendant may be created while an ancestor is conflicted, and a later rewrite may propagate or replace the conflict.

```

before:      A -- B -- C(@)
              B changes shared API
rewrite B onto X:
              X -- B'! -- C'!
              conflict value propagates
resolve B':  X -- B'' -- C''(@)
              clean tree; new successors

```

When the conflict appears, record the current operation and identify the conflicted commits with a revset rather than assuming that the current workspace is the only location:

```

jj op log --limit 8
jj log -r 'conflicts() & ::@' -T 'change_id ++ " " ++ commit_id ++ "\n"'
jj log -r 'conflicts() & descendants(@-)' -G
jj status

```

Materialise only after deciding which workspace owns the resolution. If a build or review workspace is also present, record its workspace-local `@` before updating it. Inspect the conflict sides and the parent list; “ours” and “theirs” are poor descriptions in a multi-parent graph. The domain invariant—such as preserving both API behaviours or retaining generated output policy—should decide the merged file. After editing, run a command that snapshots the result, then query `conflicts()` again and compare the tree and diff of every descendant.

Recovery has the same evidence requirement. `jj undo` is appropriate when the latest integrated operation is clearly the mistake. It is not a general reflog selector. If later operations occurred, inspect `jj op log`, use `jj op show` and `jj op diff` to identify the historical view, and decide between restoring that view, reverting its effects with a new operation, or recovering only a logical change from `evolog`. If work must move to another clone, create a temporary bookmark and push it; operation IDs and unrecorded filesystem contents do not travel through Git.

Incident	First safe action	Evidence before repair
Wrong ancestor edited	Stop ordinary commands; inspect the latest operation.	Old/new commit IDs, change IDs, descendants, bookmark targets.

Incident	First safe action	Evidence before repair
Useful change abandoned	Query evolution and operation history before creating replacements.	Change ID, predecessor/successor chain, whether a remote copy exists.
Bookmark moved too far	Compare local ref with fetched remote ref.	Old target, new target, tracking state, server response if pushed.
Several mistakes accumulated	Make a recovery note and stop mutating the graph.	Operation IDs and diffs for each candidate boundary.
Other machine needs unfinished work	Publish an explicit temporary bookmark.	Full commit ID, change ID, remote URL, and intended cleanup owner.

Do not use recovery to erase evidence prematurely. An operation restore changes the current repository view and can make later local work disappear from ordinary reachability. A remote push is outside the operation DAG. A workspace's unrecorded files are outside both graphs. The correct repair may therefore be a new graph operation, a filesystem backup, a server-side review action, or a combination of them.

Language and automation proof: revsets, filesets, templates

Revsets, filesets, and templates are three different languages that often appear in one command line. A revset selects commits; a fileset selects paths relative to the command's tree/filesystem context; a template renders typed values. Shell quoting happens before any of the three languages sees the expression. A reliable automation contract documents all four boundaries.

```
revset='heads(mutable() & ::@)'  
fileset='glob:src/**/*.*rs & ~glob:src/generated/**'  
template='commit_id.normal_hex() ++ "\t" ++ description.first_line() ++ "\n"  
jj --no-pager --color never log -r "$revset" -T "$template"  
jj diff -r @ -- "$fileset"
```

The two commands above do not necessarily consume the same kind of value. `log -r` accepts a commit set and can print many commits. `diff -r` accepts a commit expression whose tree comparison has its own endpoint semantics, then consumes a fileset after the `--` boundary. A command such as `edit` generally requires one commit; a query that returns two heads should fail rather than silently choose one.

Cardinality is therefore part of the script's precondition. Use an explicit assertion or a command designed for sets when the result can branch.

For revsets, calculate closures before reductions. For example, given two independent stacks:

```
root -- A -- B
              \
              C(@)
root -- X -- Y
              \
              Z(@2)
```

`heads.mutable()` may return both C and Z. `heads.mutable() & ::@` restricts the domain to the current workspace's ancestry before reducing it.

`heads.mutable() & ::@` reduces globally and intersects afterward; these operations can differ when a global head is an ancestor of a current-workspace head or when a filter removes the descendant that made a node non-maximal. Parenthesise the policy and inspect the result in a template before handing it to a rewrite command.

For filesets, evaluate the path set against the actual tree. A glob that matches a generated file is not automatically the same as a glob that matches a changed generated file. A relative literal is interpreted from the relevant working-directory or repository-root coordinate, depending on the command and prefix. Quote metacharacters so that Bash does not expand them first. If a fileset is intended to move diff hunks, prove it with a diff editor or a before/after diff comparison; `file list` proves names, not diff ownership.

For templates, keep typed values typed until the output boundary. A commit ID's `short()` method is not interchangeable with a string method; an absent signature is not an empty signature string; a list must be mapped or joined before concatenation. Machine output should disable colour and use an unambiguous record separator. JSON escaping belongs at the field boundary, and a template regression should test control characters, absent optionals, merge parents, and

conflicts rather than only checking that the command exits successfully.

Versioned automation should pin the target `jj` binary and test the effective syntax. A release-specific function, option, or command should be checked against that binary rather than assumed from online documentation. Keep a small fixture with a merge, a fork, a conflicted commit, a bookmark, a remote bookmark, a divergent change, and descriptions containing whitespace. Store expected symbolic results and assert full IDs only where the fixture is deterministic. This turns automation examples into executable contracts and makes release upgrades a deliberate review rather than a silent change of query meaning.

Workspace and concurrency proof: local materialisations are not branches

Workspaces complicate the word “current” in a productive way. The commit DAG and operation DAG are shared by the repository, but each workspace has its own working-copy commit and filesystem. A development workspace can be editing C while a review workspace is inspecting B and a build workspace is materialising a clean candidate. These are not three branches, and moving one workspace's `@` does not move the others. The correct unit of coordination is a workspace symbol plus its working-copy commit, not a branch name inferred from the directory.

```
shared DAG:      A -- B -- C -- D
                  |           |
development:      dev@      |
review:           review@ --+
build:            build@ at B

each @ is workspace-local; A, B, C, D are shared commits
```

Before a rewrite, list workspaces and record their working-copy commits. If a workspace has unrecorded filesystem edits, updating it after an ancestor rewrite can either snapshot those edits into a successor or require explicit stale-state repair. A build workspace should normally be treated as disposable: reproduce its commit from a full commit ID or bookmark, run the build, and avoid using its `@` as a publication handle. A review workspace may intentionally remain at an old commit while the development workspace evolves; that can be valuable evidence during a

code-review comparison.

```
jj workspace list
jj log -r 'working_copies()' -T 'change_id ++ " " ++ commit_id ++ "\n"'
jj -R ../project-review log -r @
jj -R ../project-build status
jj workspace update-stale
```

Concurrency adds a second distinction. Two processes can begin with the same repository operation head and create operations that have the same logical parent but different effects. Operation integration combines repository views according to jj's operation model; it does not guarantee that two independent rewrites of one change become one unambiguous diff. The commit evolution may still be divergent. Inspect operation parents and change evolution separately, and do not call a clean operation integration proof that the resulting review stack is semantically correct.

When an automated job runs against a shared repository, choose one of three contracts. It can use `--ignore-working-copy` and inspect a stable commit without owning snapshotting. It can own the workspace and allow normal snapshot boundaries, provided no editor or other jj process writes there. Or it can use an independent clone and exchange commits through an explicit remote bookmark. The third contract is usually easiest for CI because it separates filesystem ownership, operation history, and publication. A shell loop that happens to pass in a quiet checkout is not evidence that two writers are safe together.

Reference-reading protocol: from a question to a safe command

The command reference is most useful when read as a contract rather than a catalogue of flags. Start with the object being changed: a commit tree, a diff boundary, a bookmark, a remote observation, a workspace, a configuration layer, or an operation view. Then identify the selector language and its cardinality. Finally identify what jj will snapshot or rewrite at the command boundary. This order prevents a familiar option such as `-r` from hiding the fact that the selected values are commits while a nearby path expression is a fileset.

Question	Reference section to consult	Minimum proof
Which changes should move?	Rebase, squash, split, absorb, and reset sections.	Print source roots/heads and affected descendants.
Which content should move?	Fileset reference and tree-endpoint command contract.	Show the selected diff, not only matching filenames.
Which review should update?	Bookmark, GitHub, or Gerrit fieldbook.	Print local target, fetched remote target, and review identity.
Why did a commit disappear?	Evolution and operation reference.	Compare operation views and the change's predecessor/successor chain.
Why did output change?	Template and configuration references.	Disable colour/pager and test the typed field or effective key.
Can another machine continue?	Git backend, workspaces, and recovery recipes.	Export a reachable commit/ref; do not rely on an operation ID or dirty files.

This protocol also supplies a stopping rule. If the command's source set is not printed, the remote destination is not explicit, the fileset has not been evaluated, or the current operation cannot be identified, the next action should be inspection rather than mutation. That is not excessive caution: jj makes broad rewrites cheap and recoverable, but external publication and unrecorded filesystem state remain expensive to reconstruct. A clear precondition is often faster than diagnosing a valid command applied to the wrong graph.

For readers moving from either conventional branch-based Git or a detached-HEAD Git practice, the resulting routine is intentionally light. Keep local work unnamed while it is private when that suits the task; retain or create a bookmark when a team convention or publication boundary needs a name. Use change IDs to follow mutable evolution, and use commit IDs plus the operation/evolution history where a detached-HEAD user would previously reach for the reflog. Use revsets to describe the graph region and filesets to describe the tree region. Use templates to make the evidence legible. Use the operation log before recovery, and use Git remotes when work must cross a clone or machine. The point is not to reproduce Git's branch choreography with new verbs; it is to make the graph, its identities, and its

interfaces explicit enough that either Git habit can be replaced by deliberate jj state accounting.

Template debugging as typed observation

Templates deserve the same proof discipline as graph operations because a misleading display can cause a correct command to be applied to the wrong commit. Build a template in layers: first print one stable scalar, then add identity, then add topology, then add optional state, and only then add labels or formatting. Run with `--no-pager --color never` while debugging. This separates a template-language error from a pager, terminal-colour, or shell-quoting problem.

```
jj log -r @ -T 'commit_id.normal_hex() ++ "\n"'
jj log -r @ -T 'commit_id.short() ++ " " ++ change_id.short() ++ "\n"'
jj log -r @ -T 'parents.map(|p| p.commit_id().short()).join(", ") ++ "\n"'
jj log -r @ -T 'if(conflict(), "CONFLICT ", "clean ") ++ description.first_line() ++ "\n"'
```

If the second command fails, the problem is identity or method syntax; if the third fails, inspect the parent value's type and the lambda's receiver; if the fourth renders an unexpected state, query `conflicts()` independently. A template should not be used to infer a Boolean property from a pretty symbol when the corresponding predicate can be queried directly. Conversely, a human display can deliberately combine several fields, provided its labels make absence, divergence, and remote state visible.

Templates also define an interface for scripts. Stable machine output should use full hexadecimal IDs, explicit separators, disabled colour, and a documented treatment of empty descriptions and absent signatures. Do not parse graph indentation or the default log template. If a release changes the default display, that is a presentation change; if a method changes its type or disappears, it is a compatibility change that belongs in the version matrix. Keep one fixture for both cases so your templates remain executable observations rather than attractive but unverifiable output.

One final distinction is useful when auditing a result: an observation can be locally correct and still operationally incomplete. A template can show that `@` has a

particular commit ID without showing that a second workspace has another working-copy commit. A revset can show all visible mutable heads without showing hidden predecessors that explain divergence. A bookmark listing can show a local target without proving that the target was exported to Git. A clean conflict query can describe the current commit while a stale workspace still contains marker files from its previous materialisation. Compose observations across state components whenever the question crosses a boundary.

That compositional habit is the book's durable reference algorithm: identify the state component, choose the language that selects or renders it, make the version and scope explicit, perform the smallest operation, and inspect every coupled component that the operation could have changed. It is equally applicable to a one-line daily edit and to a Gerrit stack with several descendants, multiple review identities, a moving trunk, and a conflict. The command syntax is release-specific; the accounting discipline is the `jj` skill.

For publication notes, write the before and after in the same vocabulary as the repository: “local bookmark `review` moved from C to C’; fetched `review@origin` remained at C; change ID `qqq` continued; commit ID changed; descendants R and S were rebased; operation `O_n` recorded the rewrite.” This sentence is more useful than “the branch was force-pushed” because it states which facts are local, which are remote, which identities evolved, and which consequences require testing. It also gives a future maintainer enough information to reproduce the decision without reconstructing it from a terminal scrollbar buffer.

Use the same record for negative results. “No bookmark moved” is different from “the remote bookmark was not fetched”; “no conflict is materialised here” is different from “no commit is conflicted”; “the command selected one commit” is different from “the selected commit is the intended change”. `jj` rewards this precision because its separate state machines let work continue in more situations, but that flexibility means an operator must state which absence was actually observed.

In practice, these records become lightweight review artefacts: a shell transcript with full IDs, a saved revset, a temporary bookmark name, and the operation ID. They are especially valuable around force publication, immutable-head overrides, Gerrit patch-set replacement, and cross-machine handoff. Delete the temporary bookmark only after the receiving ref or fetched trunk proves that the intended state is reachable elsewhere.

That final reachability check closes the loop: the work is no longer dependent on one workspace, one operation head, or one operator's memory. It is represented by a reachable commit and, where publication is intended, by an explicitly named ref whose remote observation has been fetched and recorded.

If publication was not intended, the same proof ends at a local bookmark or a documented reachable commit. This is the practical meaning of branchless work: no name is required for every intermediate state, but every state worth preserving must have a deliberate reachability story.

The discipline is deliberately symmetric: before an operation, prove selection; after it, prove reachability. Between those points, jj may change many commit IDs, but the operator can still account for every intended change, descendant, workspace, and publication boundary.

That is the shortest reliable description of advanced jj work: selection is a query, rewriting is a graph transformation, publication is an explicit ref transaction, and recovery is evidence-led state selection.

Keep that sentence beside the terminal until it becomes instinctive.

It is a compact operational model for every command and every difficult history.

It keeps local graph work, filesystem materialisation, publication, and recovery visible as separate but connected responsibilities.

That separation is the practical safeguard against confusing a valid local rewrite with a completed publication.

Every boundary should be named and independently inspected.

That is deliberate.

Advanced mechanics labbook — practice

The unusual features of jj become useful only when they are treated as mechanics rather than slogans. This labbook gives experiments for the features that most often alter an expert user's operating assumptions: immutable heads, hidden commits, divergent changes, concurrent operations, sparse working copies, fixers, signing, and machine-readable output.

Immutable heads: a policy boundary, not a property of a commit

The immutable set is computed from a revset alias. A commit is not born immutable because it has a particular author, description, or age. It is protected because it is reachable from the configured immutable heads under jj's rewrite rules. This makes the policy inspectable and changeable, but it also means that a configuration change can materially alter which commands are permitted.

```
[revset-aliases]
"immutable_heads()" = "builtin_immutable_heads() | main@origin | tags()"
```

In the target 0.45.1 configuration syntax, query the resulting set with `jj log -r 'immutable_heads()'`, then compare it with `builtin_immutable_heads()`. Create a disposable child below a protected commit and try an edit or rebase. The expected failure is evidence that the alias is active. Change an unrelated key named `revsets.immutable_heads` and repeat: the function result should not change. This negative experiment is important because unknown TOML keys are not necessarily rejected.

Do not make every remote bookmark immutable by reflex. A repository that routinely rewrites review bookmarks may need a boundary at fetched trunk and release tags while leaving local topic commits mutable. Conversely, a release

repository may intentionally include all remote tags. In jj 0.45.1, untracked remote tags participate in the default immutable-head policy; inspect the effective function before relying on it.

Hidden commits, evolution, and divergence

When a mutable commit is rewritten, its predecessor may become hidden from ordinary visible queries. The predecessor is not the same thing as a deleted Git commit: it remains addressable through its full commit ID, an evolution query, or a historical operation view while retention permits. The successor is the current visible form of the logical change. A divergent change occurs when two visible successors exist for one change identity, commonly because two independent operations rewrote the same predecessor and were later brought together.

```
      q_0 (predecessor, hidden)
    /   \
q_1 (visible) q_2 (visible)
  same jj change ID
  divergent evolution
```

Use `jj evolog` to see the evolution rather than relying on `jj log -r 'all()'`. A change ID alone can be ambiguous in the divergent state; use a change offset or an explicit set of successors. The commit IDs distinguish the two concrete trees and parent relationships. If one successor is the intended one, inspect its descendants and bookmarks before abandoning the other. jj 0.45.1 includes `converge`, which can assist with reconciling divergent evolutions, but it remains a rewrite whose result must be reviewed.

Evolution is a local explanatory history, not a publication protocol. A remote server sees exported Git objects and refs, not the full jj predecessor/successor relation. Preserve a mapping in review notes when a server-side review is being updated. The mapping is also useful when a test report or CI system still mentions the old commit ID.

Concurrent operations and integration

jj operations are serialised through an operation head, but concurrent processes can create operation nodes from a common parent. One process may use `--no-integrate-operation` or otherwise finish while another process has advanced the integrated head. The operation graph then records more than one line of activity. This is not the same as two commit branches: the competing nodes describe repository-view transitions, not necessarily different commit parents.

```
      op-A: rebase stack
     /
op_0 ----+
     \
      op-B: update bookmark

integrate or inspect both before choosing a current view
```

When diagnosing concurrency, identify the common operation parent, inspect each operation's diff, and determine whether their changes touch disjoint repository state. An operation that changed a bookmark and an operation that rewrote a commit may be integrable, but this is a property of their repository transformations. Do not infer it from process start order. The operation integration command and its conflict behaviour are documented by the installed help; test them in the recovery fixture.

A single-writer rule is still the simplest policy for a workspace filesystem. Operation integration does not make simultaneous editors safe. Two processes snapshotting the same files can race before the operation graph has a chance to record the difference. Separate workspaces for build/test processes reduce that risk; separate clones eliminate shared filesystem ownership but also create independent operation histories.

Sparse working copies

Sparse patterns define which files are materialised in a workspace. They do not change the commit tree, the revset result, or the publication ref. A file absent from the sparse working copy can still exist in the commit and can still be selected by a root-qualified fileset. This is the correct mental model for a large monorepo: sparse state is a materialisation choice, not a history filter.

```
# Inspect the current pattern set.
jj sparse list

# Replace it with an explicit root-relative selection.
jj sparse set --clear --add src --add tests

# Restore the full materialisation when needed.
jj sparse reset
```

`sparse edit` opens the sparse-pattern configuration editor; it does not accept a fileset argument in 0.45.1. Use `sparse set` for command-line patterns and `sparse reset` for the full default state. A successful sparse command should be verified by checking both `sparse list` and actual filesystem presence. Test with at least one selected file and one excluded file so a no-op or inverted pattern cannot pass on exit status alone.

Fileset matching and sparse materialisation interact in a predictable but easy-to-miss way. `jj file list -r @ -- root-file:docs/api.md` can report a path that is not materialised. A subsequent external tool may fail because the file is absent locally, while `jj` can still reason about it in the stored tree. Use a full workspace for a tool that needs the entire tree or materialise the required sparse paths first.

Fixers as controlled rewrites

A fixer is an external transformation associated with a fileset and a commit selection. The tool should be deterministic, preserve the intended file encoding, and return a meaningful exit status. The fix command can rewrite selected commits, which means an innocent-looking formatter run may create successors throughout a stack. Treat fixer configuration as executable code: review the command, arguments, environment, and selected paths.

```
[fix.tools.rustfmt]
command = ["rustfmt", "--edition", "2021"]
patterns = ["root-glob:**/*.rs"]

[revsets]
fix = "mutable() & ::@"
```

The exact schema fields and command capabilities must be taken from the target version's config schema. The example illustrates the design, not an assertion that every tool accepts every field. First run the fixer on one disposable commit and one fileset. Record the diff before and after, inspect its change ID and descendants, then expand the commit set. If the tool changes generated files, keep generated-file policy separate from source formatting policy.

Fixers and [absorb](#) solve different problems. A fixer computes a transformation from an external program; absorb assigns existing working-copy diff hunks to ancestor changes. Running a formatter and then absorb may produce a clean stack, but it can also distribute formatting changes across reviews in a way the reviewer did not expect. Choose one ownership policy and inspect the resulting diff boundaries.

Running commands over commits

`jj run` is a graph-oriented facility for invoking a command over selected commits. The crucial question is whether the command is a predicate, an inspection, or a mutation. A test command should leave no meaningful filesystem mutation; a generator or fixer can intentionally alter the commit context. The commit domain must be printed before running, and the command's exit status should be associated with the selected commit in an artefact.

```
domain='main@origin..@ & mutable()'  
jj log -r "$domain" -T 'commit_id.normal_hex() ++ "\n"  
  
# The exact run options are target-version specific; inspect jj run --help.  
jj run -r "$domain" -- ./ci/check-one-commit  
jj op log
```

Do not build a shell loop around `jj edit` unless you explicitly need one. Such a loop changes workspace state, can snapshot files at each iteration, and is easy to interrupt in a state that the operation graph does not describe as one conceptual test. If the command is intended to run once per commit, use `jj`'s commit-oriented interface and capture its documented environment. If it is intended to test the current stack as a whole, run it once at a deliberate workspace commit instead.

Signing lifecycle

Signing belongs to the commit metadata layer. It is not a property of the change ID, and it is not equivalent to a remote host's verified badge. A signing operation can rewrite a commit and its descendants. If a local publication bookmark points at the affected tip, the bookmark can move locally while the remote ref stays at the old object until push.

```
jj log -r 'mutable() & ::@' -T '  
commit_id.short() ++ " " ++ change_id.short() ++ " " ++  
if(signature, "signed", "unsigned") ++ "\n"  
'  
jj sign -r 'mutable() & ::@'  
jj log -r 'mutable() & ::@' -T '  
commit_id.normal_hex() ++ " " ++ change_id.normal_hex() ++ "\n"
```

Configure a signing backend and key only after testing verification outside jj. GPG and SSH agents, key trust, hardware tokens, and server acceptance are external dependencies. Do not include secret material in configuration snapshots or research output. When signing a published stack, record the old commit IDs and verify that the review system accepts the new exported objects before moving a release tag.

Lab: machine-readable output

Human output is a user interface. Machine output should define a schema. Use full IDs, JSON or a delimiter that cannot occur unescaped, explicit colour/pager settings, and a version field when output crosses process boundaries. A template that prints a short ID and description is suitable for a human log but not for a database key.

```
jj --no-pager --color never log -r 'all()' -T '
json("jj-0.45.1") ++ "\t" ++
json(commit_id.normal_hex()) ++ "\t" ++
json(change_id.normal_hex()) ++ "\t" ++
json(description.first_line()) ++ "\n"
'
```

Descriptions may contain newlines, tabs, quotes, and Unicode. JSON encoding handles these at the record boundary; a hand-rolled tab format must escape or reject them. Parent order is meaningful for a merge, so preserve the list rather than converting it to an unordered set. Remote and bookmark names can also contain characters that make naive whitespace parsing unsafe.

For a script that performs mutation, use a two-phase protocol. Phase one renders and validates the candidate set and all preconditions. Phase two repeats the selection in the actual command and records the operation ID and resulting state. The graph can change between phases, so a long-running automation process should either hold an appropriate repository lock through the command or fail if its

preconditions no longer match.

Shell completion and debugging

Generate completion from the installed binary with `jj util completion` and install it according to the shell's conventions. Bash, Zsh, and Fish have different completion file locations and evaluation rules; test the generated file in a clean shell. Completion describes command and alias syntax, but it is not a compatibility promise across releases. Regenerate it after a `jj` upgrade.

For a bug report, include `jj version`, the command, the exact revset/fileset/template, whether the repository is colocated, the relevant operation ID, and a reduced graph. Use `--debug` when requested by the installed help or issue instructions, but do not treat debug output as a stable API. Redact remote URLs, credentials, private descriptions, and signing identifiers before sharing.

Practice curriculum — twenty graph-first labs

These labs are designed for an experienced Git user who wants to internalise `jj`'s semantics. Each lab starts with a disposable repository and ends with an assertion. Run them with the target binary and a clean environment; use the symbolic descriptions in the expected state rather than hard-coded generated IDs. The point is to observe the graph and operation transitions, not to memorise a transcript whose IDs will differ on every run.

Lab 1: snapshot boundary

Initialise a repository, create a file, and inspect `jj status`. Then edit the file without running `jj` and execute `jj log -r @`. The ordinary command snapshots the edit before rendering the log. Repeat with `jj --ignore-working-copy log -r @` before and after a deliberate snapshot. The recorded tree differs from the filesystem in the first observation and agrees after the snapshot. The experiment separates a filesystem edit from a commit-tree edit and proves why a read command can create an operation.

```
printf 'one\n' > value.txt
jj status
printf 'two\n' > value.txt
jj --ignore-working-copy diff
jj diff
jj --ignore-working-copy log -r @ -T 'commit_id.normal_hex() ++ "\n"'
```

Do not use `git status` as the only comparison in a colocated repository: Git's index/worktree view and `jj`'s working-copy commit are different observations. Record the operation log before and after each command. A snapshot-only operation is still relevant when diagnosing a race.

Lab 2: identity through description changes

Create and describe a change, record its commit and change IDs, then run `jj describe` again. The commit ID changes because metadata is part of the commit identity. The change ID normally remains the same because the operation is an evolution of one logical change. A child of the commit receives a successor because its parent object changed. Render all three values before and after, and use `evolog` to connect them.

Repeat with a metadata change on an ancestor that has a local publication bookmark. Verify that the local bookmark follows the successor while the remote bookmark remains unchanged. This is the minimal experiment that prevents a Git expert from treating commit IDs as long-lived change identifiers in a mutable

repository.

Lab 3: edit versus a bookmark

Create a bookmark at a child and then run `jj edit` on its parent. The bookmark remains at the child; only the current workspace's `@` moves. Edit the parent, snapshot, and inspect the child successor. The bookmark's target may follow the rewritten local evolution, but its name and the workspace pointer remain separate state. Add a second workspace and repeat the edit there to see that each workspace has a local `@`.

The assertion is not “the prompt changed”. It is: the workspace symbol resolves to the expected commit, the bookmark resolves to its expected target, and the graph has the expected parent edge. Use a template rather than relying on the current default display.

Lab 4: rebase source shapes

Build a forked graph and apply the same destination with `-r`, `-s`, and `-b` in separate disposable copies. With `-r`, only the selected commits are explicitly sourced; with `-s`, the source's descendants are included; with `-b`, the branch-shaped selection follows the command's topology definition. Compare the parent arrays after each operation.

Then introduce a merge and repeat. The visual graph can make the three results look similar while their selected sets differ. A proper lab report contains the source expression, the expanded selected IDs, destination IDs, and resulting parent edges. This is the evidence needed to choose a rebase mode for a real stack.

Lab 5: insertion after and before

Start with `A-B-C` and a separate `D`. Use the insert-after mode to place `D` after `B` and the insert-before mode to place `D` before `C`. The final graph can be linear in both cases, but the operation has different destination semantics when the named commit has multiple children or when the destination is a merge. Add a second child to `B` and verify whether the command rewrites one path or creates a graph with multiple parents/children according to the documented selection.

Use a parent template in the assertion. “It appears between `B` and `C`” is not enough if `D` gained a second parent or if `C` was rebased to a new successor.

Lab 6: partial squash

Create one change touching `src/api.rs`, `tests/api.rs`, and `docs/api.md`. Squash only the tests into the parent. Use a path listing before and after, then compare the diff of the source and destination. The source tree may still include unchanged paths; what matters is the diff relative to its parent. Repeat using an unqualified `src` path from a subdirectory and then using `root:src`. The differing result demonstrates why portable scripts should use root-qualified paths.

Lab 7: split with a descendant

Split a mixed change that has a child. Record the original child commit ID and parent. After the split, locate the child's successor through its change ID and inspect the new two-edge path. Confirm that the first and second split changes contain disjoint diffs whose combination equals the original diff. Then modify the description of the first part and show that the second part's commit ID can change again because its parent changed.

This lab makes two consequences visible: a split is not only a file-selection operation, and descendant rebasing is not an optional follow-up step. The child is maintained as a graph successor even when the user never selected it directly.

Lab 8: absorb ownership

Make an API change and a later test change, then edit one line in the API and another line in the test while at the tip. Run `absorb` and inspect which diff hunks moved into which ancestors. Use a deliberately ambiguous diff hunk to see how the tool handles uncertainty under the target configuration. The expected assertion is about diff ownership, not merely that the working-copy diff became empty.

Keep the original diff in a file before `absorb` and compare the union of resulting diffs with it. If the union is not equivalent, investigate line-range handling, ignored files, or a tool/configuration problem before accepting the operation.

Lab 9: duplicate versus revert

Create a feature change and a later change that modifies the same area. Duplicate the feature onto a separate base, then in another copy revert it at the later base. The duplicate carries the positive diff and gets a new change identity. The revert carries the inverse and gets a new change identity. If later context conflicts, the revert can contain a stored conflict; resolve it without deleting the fact that the original feature occurred.

Use this lab when a reviewer says “bring that diff over” or “undo that diff”. Those sentences do not specify whether the desired operation is copy or inverse.

Lab 10: abandon and recovery

Make a stack with a bookmark at its tip and abandon its middle change. Inspect the resulting descendants, bookmark state, evolution log, and operation log. Do not run storage GC. Recover the old change by its explicit identity, then either duplicate it onto a new base or restore the prior repository view in a copy of the fixture.

Repeat after making an unrelated description edit. Compare immediate `undo` with historical operation inspection. The lab's conclusion is that recovery is easier when the operator records operation IDs and uses an explicit recovery policy before

performing further work.

Lab 11: non-distributive ranges

Use a merge graph and compare `(C|D) ..` with `C .. | D ..`. Render both sets as symbolic descriptions. Then compute `C .. & D ..` and demonstrate the equality with `(C|D) ..`. Add a hidden predecessor explicitly and repeat; observe how the search domain changes. This lab should be a permanent regression test because range intuition is one of the most persistent sources of Git-to-jj mistakes.

Lab 12: symbol collision

Create a bookmark whose name is also a unique prefix of a commit ID, then resolve the bare symbol, `commit_id()`, and `bookmarks(exact:"name")` forms. Add a tag with the same name and compare `tags(exact:"name")`; observe the documented priority of the bare symbol. Repeat with a non-unique prefix and with a divergent change ID. The expected results belong in the query-lab documentation so later aliases do not accidentally depend on resolution priority.

Lab 13: fileset and shell parsing

Create files whose names contain spaces, parentheses, wildcard characters, and a colon. Run a simple path without inner syntax, then a complement and a function expression. Capture the shell's arguments with a harmless wrapper if necessary. The wrapper shows whether the shell expanded a glob before jj received it. Quote the complete expression and use explicit `root-file:` or `root-glob:` forms in the corrected command.

Repeat from a nested directory. A fileset can be syntactically correct yet select the wrong directory because its base coordinate changed. Root-qualified expressions are the portable answer.

Lab 14: one fileset, four consumers

Define `source_paths = root:src ~ root-prefix-glob:src/**/generated/**`. Use it with `file list`, `diff`, `restore`, and `squash` in copies of the same fixture. The set of matched paths should be stable, but the command effect differs: `list` displays, `diff` filters output, `restore` copies selected file contents, and `squash` changes diff ownership. Record the source/destination commits for the two mutating commands.

Lab 15: template optional values

Render author and committer signatures for a commit with distinct identities. Render a root with no parents, an unsigned commit with no signature, and a commit with no bookmark. Build a template that handles all three without error using conditionals and fallbacks. Then deliberately remove the guard and keep the failing command in the lab notes as a type/error example.

The assertion is that the template has a defined output contract for absence. A blank field is not the same as the literal string `(none)`; choose deliberately.

Lab 16: JSON output

Make a description containing quotes, a newline, a tab, and Unicode. Render a JSON object with full commit/change IDs and parent arrays, then parse it with a JSON parser. Compare with a hand-written tab-separated template. The JSON record survives the description content; the hand-written format needs escaping rules. This is the minimum experiment before using `jj` output as CI input.

Lab 17: conflict propagation

Create two branches that edit the same line, merge or rebase them, and inspect the resulting conflict value. Add a descendant without resolving the parent. Query `conflicts()` over the stack and materialise the conflict in the workspace. Resolve only the intended file, snapshot, and query again. Inspect the descendant successor and run the test at the tip.

Repeat with a multi-sided conflict and write down each parent. The purpose is to prevent “ours/theirs” from becoming an unexplained policy when the graph has more than two meaningful sides.

Lab 18: operation forensics

Perform a description edit, a bookmark move, an ancestor rewrite, and a snapshot. Use `op log` to identify them and `op diff` to distinguish their repository-view effects. Read the repository at an earlier operation with `--at-operation`. Verify that this historical read does not import current filesystem edits. Then use `op restore` only in a disposable copy and compare the current operation graph before and after.

Lab 19: workspace staleness

Add a build workspace at a stack tip. Rewrite the ancestor from the development workspace and list workspaces. Inspect the build workspace's filesystem before updating it. If it contains no valuable edits, update-stale and assert that its `@` is current. If it contains an unrecorded build configuration, preserve that file first and repeat. This demonstrates why stale repair is a stateful operation rather than a screen refresh.

Lab 20: cross-clone handoff

Create a bare remote and two clones. Make an unfinished change in clone A, publish a temporary bookmark, fetch in clone B, and create a new working-copy child there. Check that the commit is available in B but the operation IDs from A are not. Rewrite in B and push a deliberately different bookmark. Fetch in A and inspect both remote observations. Clean the temporary name only after confirming which machine owns the work.

This is the final lab because it combines the book's layers: a bookmark is the transport interface, the commit graph is portable through Git, the working-copy symbol is local, and the operation graph is clone-local.

Hosting protocol fieldbook — workflows

jj does not know whether a Git ref will be shown as a GitHub branch, a pull request head, a Gerrit change, or an internal deployment ref. It exports Git-compatible objects and names; the host interprets the ref and commit message according to its own protocol. A disciplined workflow therefore has a local half and a server half. The local half can be validated with a bare Git remote. The server half must be labelled as documented or server-dependent.

GitHub pull requests as a ref state machine

Phase	Local jj state	Git ref state	GitHub state
Unpublished	Mutable unnamed stack below fetched trunk	No feature ref	No PR or an old PR unrelated to current stack
Boundary chosen	Bookmark at intended review tip	Local branch/ref candidate	Not yet changed
First push	Bookmark tracked or explicitly related to remote	Remote feature ref created	PR can be opened against selected base
Review iteration	One or more changes rewritten; successors and IDs change	Same ref updated to a new tip	New commit set/iteration displayed by the host
Stack update	Early change rewritten; descendants automatically rebased	Several publication refs may move	Dependent PR bases and diffs may need UI updates
Merge	Fetch host result; rebase/clean remaining local work	Trunk ref advances; feature ref may remain	Host applies chosen merge strategy
Cleanup	Forget/delete local publication names intentionally	Delete feature ref only when intended	PR remains an audit/UI object even if its head ref is deleted

GitHub's pull request is not a jj object. The local bookmark is the bridge because the host requires a Git ref. A PR can remain open while the local bookmark is deleted if the remote ref remains; conversely, deleting a local bookmark cannot close a PR. Decide explicitly whether cleanup is local, remote, or UI-level.

For a single-change PR, the local stack may be `main@origin-A-B-@` with a bookmark at `@`. For a stack, create a bookmark at each review boundary and ensure each dependent ref has the intended base. When an early change is amended, the descendant ref must move as well. Push order matters for a human review experience, even if the Git transport can send all reachable objects in one negotiation.

Rejected updates and remote movement

A rejected push is an information event. Fetch the remote and render the local and remote targets. The remote may have moved because another machine pushed, because a reviewer or automation updated the branch, because the bookmark was renamed, or because the server refuses the selected update. Do not immediately rebase onto whatever appears first in the default log.

```
jj git fetch --remote origin
jj bookmark list --remote origin --tracked
jj log -r 'review | review@origin' -T '
commit_id.normal_hex() ++ " " ++ change_id.normal_hex() ++ " " ++
description.first_line() ++ "\n"
```

If the remote target contains work that must be preserved, rebase or merge onto it according to the review policy. If the remote target is an obsolete version of the same review and the server permits an intentional replacement, document the old target and perform that replacement through the hosting system or a deliberately verified native-Git operation; `jj 0.45.1's jj git push` has no backwards-update flag. Tracking helps compare the states; it does not make the decision for you.

Fieldbook: fork/upstream topology

Keep the fork remote and upstream remote distinct. Fetch upstream trunk under its own remote name, publish PR refs to the fork, and avoid aliases that treat `origin/main` as universally authoritative. The graph can contain both:

```
main@upstream -- A -- B
                \
                 C -- D
                   ↑
                pr/42@fork
```

For a contributor, `main@upstream` is the update base while `pr/42@fork` is the publication observation. A local bookmark named `pr/42` can track the fork ref; it should not be confused with a local `main` that tracks upstream. If the fork ref is deleted after merge, forget or delete only the local state whose removal is intended.

Gerrit changes as commit-message identity

Gerrit commonly associates patch sets by a `Change-Id: I...` footer in the commit message. That footer is not jj's change ID. The local jj change ID may stay constant across a rewrite; the Gerrit footer may stay constant so the server associates a new commit with the same review; and the commit ID changes every time the relevant object content, metadata, parent, or footer changes.

Local event	jj change ID	Commit ID	Gerrit interpretation
Initial description and upload	Q	q_0	Change I123, patch set 1
Edit subject/body but preserve footer	Q	q_1	Usually another patch set of I123, subject changed
Rewrite parent through descendant rebase	Q	q_2	New patch set if uploaded with same footer
Remove or replace footer	Q	q_3	May create a new change or be rejected by server policy
Duplicate the change	D, not Q	d_0	Footer determines whether it is associated with I123 or another review
Abandon locally	Q may remain in evolution	Old objects may remain locally	Server review is unaffected unless a protocol command changes it

Never use a jj change ID where a Gerrit footer is required. Conversely, do not replace a jj change ID with a Gerrit footer in local revsets. Keep both in the commit description/log record and explain which identity a review comment refers to.

Three-change upload sequence

```
main@origin -- T -- Q -- R -- S
                Iq  Ir  Is
                ↑   ↑
            review Q review S
```

1. Fetch the target branch and create the stack in local `jj` without publication bookmarks.
2. Verify each description has the intended Gerrit footer and that the changes are in dependency order.
3. Render the stack with full commit IDs and `jj` change IDs; record this as upload iteration zero.
4. Run the available `jj 0.45.1 jj germit upload --dry-run` with explicit remote, branch, and commit selection.
5. Upload the stack and record the server URLs/identities returned by the Gerrit instance.

The upload command's review options can set reviewers, CCs, labels, topics, hashtags, WIP/ready state, privacy, notifications, and other server push options. Their spelling and server acceptance are version/instance-sensitive. In a shared script, pass the remote and branch explicitly and keep server options in a reviewed configuration file. Test the local selection and footer before relying on the server response.

Early review feedback

Suppose the reviewer comments on `Q`. Run `jj edit Q`, make the smallest change that addresses the comment, and snapshot. The local graph becomes `Q'`, `R'`, `S'`. The `Q` change ID normally remains `Q`; all three commit IDs can change because `R` and `S` inherit a new parent. The Gerrit footer should remain `lq`. Before upload, compare the interdiff of `Q` and `Q'`, and separately compare `R/R'` and `S/S'` to identify changes caused only by rebasing.

```
jj evolog -r 'Q'
jj log -r 'Q::S' -T '
change_id.short() ++ " " ++ commit_id.short() ++ " " ++
parents.map(|p| p.commit_id().short()).join(",") ++ " " ++
description.first_line() ++ "\n"
'
jj germit upload --dry-run --remote germit --remote-branch main -r 'Q::S'
```

Upload the intended commits with the same footer. The server-side patch-set association and displayed diff are documented by Gerrit and can vary with submit rules. If a footer was accidentally changed, stop and decide whether to restore it or intentionally create a new review; do not guess from a short commit ID.

Partial landing and trunk movement

If Q lands but R and S remain under review, the remote main ref may point to a server-created commit rather than q'. Fetch it. The remaining local stack is no longer necessarily based on the object that was uploaded for Q. Rebase R/S onto the fetched trunk and inspect their diff ranges against the new base. Preserve their Gerrit footers and upload new patch sets.

```
jj git fetch --remote gerrit
jj log -r 'main@gerrit'
jj rebase -s 'roots(main@gerrit..S)::' -d main@gerrit
jj log -r 'main@gerrit..@' -T 'commit_id.short() ++ " " ++ description.first_line() ++ "\n"
jj gerrit upload --remote gerrit --remote-branch main -r 'main@gerrit..@'
```

The exact source expression must be adjusted for the actual graph. The point is to avoid keeping an obsolete local Q as the base merely because it shares a description with the landed review. If a server performs a squash or merge, the fetched ref is the authority for the new base.

Topics, reviewers, and push options

Topics and reviewers are Gerrit metadata, not jj bookmarks. A topic can group several Gerrit changes while the local graph supplies their dependency edges. Reviewers and labels can be supplied at upload time or through server UI/API. Push options can request draft/WIP, ready, private, notifications, validation, attention-set changes, or topic updates where the server allows them. Treat these options as side effects and record them in the upload command log.

Do not make a local bookmark called `topic` carry the meaning of a Gerrit topic. A bookmark names a commit; a Gerrit topic groups server-side changes. They can share a string for human convenience, but they are different namespaces and can

drift.

Conflict, abandonment, and replacement

If the stack conflicts while rebasing onto new Gerrit trunk, resolve the stored conflict locally, test, and upload. If a review is abandoned, decide whether the local change is also abandoned, duplicated for another target, or retained as a historical change. If a replacement review is created, use a new Gerrit footer when the server expects a distinct change and a new publication boundary. A `jj` operation `restore` can recover local graph state; it cannot resurrect a review the server has permanently deleted.

One repository, two publication protocols

A single local DAG can feed GitHub and Gerrit independently:

```
+- github/pr-42 ---> GitHub PR
main@upstream ---+
                  \- gerrit stack ----> refs/for/main
```

The graph is shared; the names, remotes, review identities, and server policies are not. Put host-specific publication names in separate namespaces, such as `github/pr-42` and `gerrit/topic-42`. Fetch both remotes but qualify their trunk symbols. After rewriting a shared ancestor, inspect both publication boundaries before pushing either. It may be correct to update one review and leave the other at the previous object.

Keep a local publication ledger with columns for local bookmark, remote, remote bookmark, host review ID, `jj` change IDs, commit IDs, base ref, and last fetch. This is especially valuable when the same code is under review on two hosts with different policies. The ledger is not `jj` state; it is a human review aid that explains why a local graph has multiple outward interfaces.

Appendix directory and quick lookups

Use this directory to choose the right appendix. The short entries below are deliberately labelled as directory cards where a fuller reference appears later in the same book. The links are the authoritative route: they prevent a reader from mistaking a one-page orientation card for the complete fileset, template, configuration, or hosting reference.

Appendix A — Complete command tree

```
abandon absorb arrange bisect [run]
bookmark [advance create delete forget list move rename set track untrack]
commit config [edit gc get list path set unset] describe diff diffedit
duplicate edit evolog file [annotate chmod list search show track untrack]
fix gerrit [upload]
git [clone colocation [disable enable status] export fetch import init push
    remote [add list remove rename set-url] root]
help interdiff log metaedit new next operation
    [abandon diff integrate log restore revert show]
parallelize prev rebase redo resolve restore revert root run
show sign simplify-parents sparse [edit list reset set] split squash status
tag [delete list set track untrack] undo unsign
util [backend [name] completion config-schema exec gc install-man-pages
    markdown-help snapshot]
version workspace [add forget list rename root update-stale]
```

Role: quick lookup. This is generated from `jj 0.45.1 help`. The command paths are expanded in the reader-facing entries in Part XV. Use the tree to find a family, then use the command entry for arguments, options, cardinality, mutation, and examples.

Appendix B — Option index

Flag	Where/meaning
<code>-R, --repository</code>	Operate on a repository path.
<code>--ignore-working-copy</code>	Do not snapshot/update filesystem.
<code>--no-integrate-operation</code>	Create but do not integrate operation.
<code>--ignore-immutable</code>	Bypass rewrite protection.
<code>--at-operation</code>	Load historical repository view.
<code>-r</code>	Commits/revset; meaning is command-specific.
<code>-s, -b</code>	Rebase source/branch selection.
<code>-o/-d, -A, -B</code>	Rebase onto/destination, insert-after, insert-before.
<code>--from, --to</code>	Tree/diff/content endpoints on commands whose entries list them.
<code>--bookmark, --remote</code>	Publication/ref selection.
<code>--all, --tracked</code>	Command-specific set selection.
<code>--color, --no-pager, --quiet</code>	Output controls.

Do not transfer a short flag between commands by analogy. The expanded index below records ownership and interactions for the options most likely to affect graph scope or publication. The command contracts in Part XV explain the complete command-specific effects; use installed help for any release-specific spelling or default not shown here.

B.1 Options by contract

Option	Owner	Argument	Effect and interaction
<code>-R, --repository</code>	Global	PATH	Selects the repository. It does not select a workspace; the repository still resolves the active workspace from the current directory unless another workspace option applies.
<code>--ignore-working-copy</code>	Global	none	Reads the recorded repository view without snapshotting the filesystem. Pair with historical operation queries and machine output when a stable view matters.
<code>--at-operation,</code> <code>--at-op</code>	Global	OPERATION	Views an earlier operation. It implies a historical repository view; do not treat today's filesystem as if it were part of that view.
<code>--no-integrate-operation</code>	Global	none	Leaves a newly created operation unintegrated for explicit concurrency handling. It is an operation-DAG control, not a commit-DAG selection.
<code>--ignore-immutable</code>	Global	none	Bypasses the immutable rewrite guard. It does not make a commit mutable in policy or make a remote update safe.
<code>--color</code>	Global	auto/always/never	Controls terminal colour. Use <code>never</code> for parsers, fixtures, and evidence captures; labels and hyperlinks are presentation, not data.

Option	Owner	Argument	Effect and interaction
<code>--no-pager</code>	Global	none	Prevents a pager process. It should be paired with explicit colour suppression in automation.
<code>--config</code>	Global	KEY=VALUE	Applies an ephemeral configuration override. It is useful for testing an alias or safety policy without mutating files.
<code>--config-file</code>	Global	PATH	Adds a configuration file. Layer order matters; inspect <code>config list</code> before relying on a repository policy.
<code>-r, --revisions</code>	Many commands	REVSET	Selects commits. Cardinality is command-specific: <code>log</code> accepts a set, while <code>edit</code> normally requires one commit.
<code>-s, --source</code>	<code>rebase</code>	REVSET	Rebases a source set and its descendants as defined by the command. It is not a synonym for “the current branch”.
<code>-b, --branch</code>	<code>rebase</code>	REVSET	Selects a branch-shaped source domain. Inspect the resulting set before using it on a forked graph.
<code>-d, --destination</code>	<code>rebase</code>	REVSET	Moves selected source roots onto destinations. Multiple destinations can produce merges or duplicated placement; use a bounded source set.

Option	Owner	Argument	Effect and interaction
<code>-A, --insert-after</code>	rebase	REVSET	Inserts source commits after a destination. Parent-edge changes can rebase descendants even when diff contents are unchanged.
<code>-B, --insert-before</code>	rebase	REVSET	Inserts source commits before a destination. It has different graph semantics from <code>--insert-after</code> ; do not infer one from the other.
<code>--skip-emptied</code>	rebase	none	Controls handling of commits whose diffs become empty during the rewrite. Inspect successors and bookmark targets afterwards.
<code>--into</code>	squash	REVSET	Chooses the destination change. It is distinct from the source commit selected by <code>-r</code> and from a fileset restricting moved paths.
<code>--from, --to</code>	squash, diff, file commands	REVSET	Names tree/diff endpoints on the listed commands. Read the command entry: the same words do not imply the same direction in every consumer.
<code>--keep-emptied</code>	squash	none	Preserves a source commit after its selected diff is moved. It changes graph shape, not merely display.

Option	Owner	Argument	Effect and interaction
<code>--insert-before</code> , <code>--insert-after</code>	split	REVSET	Places the split result relative to a chosen commit when the command supports explicit placement. Check the resulting parent edges.
<code>--message, -m</code>	describe, split, commit, new	TEXT	Sets a description. A description rewrite changes the commit ID and can rebase descendants; it does not inherently change the jj change ID.
<code>--bookmark, -b</code>	git push, bookmark commands	NAME	Names a publication ref. In git push, specify the remote separately when multiple remotes exist.
<code>--remote</code>	Git/bookmark/Gerrit commands	NAME	Selects a remote. Tracking relationships do not replace explicit remote selection for a mixed GitHub/Gerrit repository.
<code>--tracked</code>	git push, bookmark commands	none	Selects tracking state. It does not mean “all local bookmarks” and does not move a bookmark to @.
<code>--all</code>	Git transport commands	none	Broadens publication or ref selection according to the command. Treat it as a review-required option.
<code>--deleted</code>	git push	none	Publishes deletion of eligible remote refs. Verify the local/remote bookmark relationship before using it.
<code>--allow-private</code>	git push	none	Allows private commits into the push selection in jj 0.45.1. Privacy policy and server access remain separate concerns.

Option	Owner	Argument	Effect and interaction
<code>--allow-conflicts</code>	git push	none	Allows conflicted commits to be pushed in jj 0.45.1. It does not resolve the conflict or guarantee host acceptance.
<code>--named NAME=REVISION</code>	git push	assignment	Creates an explicit publication mapping for a push. Use it when a temporary publication name is safer than moving a durable local bookmark.
<code>-c, --change</code>	git push	CHANGE	Publishes by change identity in jj 0.45.1. Verify the selected evolution and resulting remote ref instead of assuming commit-ID selection.
<code>--dry-run</code>	Git/Gerrit push commands	none	Shows a prospective publication. It is a precondition check, not a lock against another process or contributor changing the remote.
<code>--revision</code>	gerrit upload	REVSET	Restricts Gerrit upload selection. Make the base and range visible before upload; server patch-set association is a separate identity layer.
<code>--remote-branch</code>	gerrit upload	NAME	Names the Gerrit destination branch. Pass it explicitly on 0.45.1 in repositories with multiple publication protocols.
<code>--reviewer, --cc</code>	gerrit upload	ADDRESS	Adds review metadata to the upload request. It affects the server request, not the local commit graph.

Option	Owner	Argument	Effect and interaction
<code>--topic</code> , <code>--hashtag</code>	<code>gerrit upload</code>	TEXT	Supplies Gerrit review grouping metadata. Server plugins and policy determine its later interpretation.
<code>--wip</code> , <code>--ready</code> , <code>--private</code>	<code>gerrit upload</code>	none	Controls review state in the upload request. Keep these options explicit in publication aliases.
<code>--operation</code>	<code>op</code> commands	OPERATION	Selects a repository operation for inspection or recovery. It is not a commit and cannot be substituted with a commit ID.
<code>--tool</code>	<code>resolve</code> , <code>diff/merge</code> commands	NAME	Selects an external or builtin tool. Validate executable paths and tool exit status before automation.
<code>--workspace</code>	commands whose syntax lists it	NAME	Names a workspace in commands that accept it. The active <code>@</code> still belongs to one workspace at a time.
<code>--clear</code> , <code>--add</code>	<code>sparse set</code>	PATTERN	Replaces or extends sparse patterns. <code>sparse edit</code> opens the configuration editor and does not consume a fileset argument in 0.45.1.

Three rules prevent most option errors. First, classify the option as global view control, commit selection, fileset selection, graph transformation, publication, or output. Secondly, inspect whether its argument is a single commit, a set, a fileset, a name, or an external program. Finally, make the postcondition explicit: selected IDs, successor relation, bookmark target, remote ref, filesystem paths, or operation ID. A syntactically valid option with the wrong domain is still a semantic failure.

Directory entry — Revset quick reference

`::x` ancestors; `x::` descendants; `x..y` ancestors of `y` excluding ancestors of `x`; `x::y` descendants of `x` that are ancestors of `y`; `x-` parents; `x+` children; `&` intersection; `|` union; `~` difference/complement depending on position; `root()`, `none()`, `all()`, `heads()`, `roots()`, `visible_heads()`, and typed predicates. Patterns use explicit kinds such as `exact:`, `substring:`, `glob:`, and `regex:`. Parameterised aliases are TOML functions.

Directory entry — Fileset reference

Directory only — continue to the complete reference. Use quoted literal paths and explicit `glob:` patterns; combine sets with the installed fileset operators/functions. The complete grammar, scope prefixes, quoting rules, and consumer matrix are in Appendix D.1.

Directory entry — Template reference

Directory only — continue to the complete reference. Templates evaluate typed commit/operation objects. The complete type/method and formatting maps are in Appendix E.1 and Appendix E.2.

Directory entry — Configuration reference

Directory only — continue to the complete reference. The human configuration reference is in Appendix F.1 and Appendix F.2. The schema captures machine-detectable release changes; it is not a substitute for the explanations and interaction rules in those sections.

Directory entry — GitHub recipes

Directory only — continue to the complete fieldbook. Fetch `main@origin`; build an unnamed mutable stack; create one bookmark per PR boundary; push explicit bookmarks; rebase the stack onto updated main; and clean publication names. The complete lifecycle fieldbook is Appendix H fieldbook.

Directory entry — Gerrit recipes

Directory only — continue to the complete fieldbook. Preserve Gerrit Change-Id footers; use `jj change` IDs for local evolution; upload to the configured Gerrit remote/branch; revise any stack member; re-upload; fetch landed changes; and rebase the remaining stack. The complete lifecycle fieldbook is Appendix I fieldbook.

Directory entry — Recovery recipes

Directory only — continue to the complete fieldbook. Record `jj op log`; inspect `jj --at-op OP log`; use `undo` for the last operation, `op restore` for a known state, `evolog` for a rewritten change, and a temporary bookmark for cross-machine recovery. The complete incident fieldbook is Appendix J fieldbook.

Directory entry — Linux shell integration

Directory only — continue to the complete fieldbook. Bash and Zsh use the output of `jj util completion bash` or `zsh`; Fish can use the generated completion supplied by `jj 0.45.1`. The complete prompt, completion, pager, editor, and automation procedures are in Appendix K fieldbook.

Reference appendices: dense lookup material

The earlier appendices are quick navigation aids. The following reference pages are intentionally denser: they are designed to be useful at the terminal after the teaching chapters have established the mental model. They are derived from the 0.45.1 local help captures and carry target-release notes where the surface differs.

Appendix C — Complete revset operator/function reference

Expression/function	Result and signature	Operational note
<code>all()</code> , <code>none()</code>	Universe or empty set.	Use deliberately; <code>all()</code> can include more history than a normal visible log.
<code>root()</code>	Root commit singleton.	Graph anchor; not a normal mutable working change.
<code>parents(X, [depth])</code>	Parents to optional depth.	Without depth, the documented traversal form can walk ancestry; use postfix <code>X-</code> for immediate parents.
<code>children(X, [depth])</code>	Children to optional depth.	Forks can produce several results; commands requiring one need a boundary.
<code>ancestors(X, [depth])</code>	Ancestor closure.	Includes selected X according to the function's closure semantics.
<code>descendants(X, [depth])</code>	Descendant closure.	Can include multiple branches and hidden/visible members depending on domain.
<code>first_parent(X, [depth])</code>	First-parent traversal.	Merge parent order is a policy choice; do not use for generic reachability.
<code>first_ancestors(X, [depth])</code>	First-parent ancestry.	Useful for release-line reports, not for all changes reachable from a merge.
<code>reachable(S, D)</code>	Members of domain D reachable from sources S.	Source and domain are separate; this is safer than a broad closure in a multi-root repo.
<code>connected(X)</code>	Connected portion of a set.	Useful before commands whose diff operation rejects gaps.
<code>heads(X)</code> , <code>roots(X)</code>	Maximal/minimal members of X.	Reduce a graph set to publication tips or stack roots.
<code>latest(X, [count])</code>	Most recent selected commits.	Ordering is jj's graph/date policy; inspect result before destructive use.
<code>fork_point(X)</code>	Graph-derived fork point.	May be non-singleton or absent in complex/disconnected inputs; test cardinality.

Expression/function	Result and signature	Operational note
<code>merge_point(X)</code>	Graph-derived merge point.	Use to reason about common integration rather than a bookmark name.
<code>bisect(X)</code>	Graph bisection point.	Complements <code>jj bisect run</code> ; test semantics against a known range.
<code>exactly(X,N)</code>	Filters/validates X to exact cardinality.	Useful in aliases that must fail closed before <code>edit</code> or <code>describe</code> .
<code>coalesce(X,...)</code>	First non-empty commit set.	Useful for fallback trunk names; document precedence.
<code>merges()</code> , <code>forks()</code>	Merge/fork predicates.	Graph shape, not ref state.
<code>bookmarks([PATTERN])</code>	Commits targeted by local bookmarks.	Pattern filters names; use remote form for imported refs.
<code>remote_bookmarks([NAME],[REMOTE])</code>	Commits targeted by remote bookmarks.	Remote qualification is essential with GitHub/Gerrit together.
<code>tracked_remote_bookmarks(...)</code> , <code>untracked_remote_bookmarks(...)</code>	Remote refs grouped by tracking relation.	Tracking is local relationship metadata, not automatic <code>@</code> following.
<code>tags([PATTERN])</code> , <code>remote_tags(...)</code>	Tag targets.	<code>jj 0.45.1</code> provides tracked/untracked remote-tag predicates.
<code>visible_heads()</code>	Visible graph heads.	Repository-level display boundary.
<code>change_id(PREFIX)</code> , <code>commit_id(PREFIX)</code>	Identity-based selection.	Ambiguous prefixes require disambiguation; commit and change identity differ.
<code>description(PATTERN)</code> , <code>subject(PATTERN)</code>	Description/first-line predicates.	Use explicit <code>exact</code> , <code>substring</code> , <code>glob</code> , or <code>regex</code> .
<code>author(PATTERN)</code> , name/email forms	Author metadata predicates.	Author is not committer; email matching is usually the robust policy.
<code>committer(PATTERN)</code> , name/email forms	Committer metadata predicates.	Useful for imported/re-written provenance reports.
<code>author_date(PATTERN)</code> , <code>committer_date(PATTERN)</code>	Date predicates.	Use explicit timezone/cutoff policy and test boundaries.
<code>mine()</code>	Author email matches current user.	Not a server identity or committer predicate.

Expression/function	Result and signature	Operational note
<code>signed()</code>	Signed commits.	Presence differs from cryptographic status verification.
<code>files(EXPR)</code>	Commits whose trees match a fileset expression.	Nested fileset language is independent of revset syntax.
<code>diff_lines(TEXT, [FILES])</code>	Commits whose diff contains matching lines.	Added/removed variants distinguish direction; quote patterns.
<code>conflicts()</code> , <code>empty()</code>	Stored conflict/empty predicates.	<code>conflicts()</code> is the revset name; template uses <code>conflict()</code> .
<code>divergent()</code> , <code>present(X)</code>	Divergent changes; present commit resolution.	Useful for forensic checks and ambiguous identities.
<code>mutable()</code> , <code>immutable()</code>	Mutability predicates.	Computed relative to immutable-head configuration, not simply local/remote naming.
<code>visible()</code> , <code>hidden()</code>	Visibility predicates.	Hidden is not erased; avoid GC during recovery.
<code>working_copies()</code> , <code>at_operation(OP, X)</code>	Workspace and historical operation predicates.	Local workspace/operation state; not transferable through Git.
<code>trunk()</code>	Configured trunk commit/set.	Configuration-dependent; inspect effective config before automation.
<code>builtin_log()</code> , <code>builtin_immutable_heads()</code>	Built-in configured expressions.	Useful as a base when customising defaults; release-sensitive.

Operator precedence card

```

f(x)
x- x+
p:x
x:: ::x x::y x.. ..x x..y
~x
x & y x ~ y
x | y

```

Function calls bind strongest and union weakest. `x..y` means `::y ~ ::x`; `x::y` means `x:: & ::y`. Parenthesise any expression whose intended domain is not obvious. A revset is a set value: use `log --count` to inspect cardinality and `roots/heads` to make graph boundaries explicit.

Revset functions by family

This table is a reader-facing semantic index for the functions extracted from the 0.45.1 help. Optional arguments and pattern forms are abbreviated where the command's help gives several overloads; the teaching sections provide evaluated examples. The target-release-specific tag predicates are explicitly marked.

Family	Function/signature	Result and useful composition
Universe	<code>all()</code> , <code>none()</code>	All commits in the current view, or the empty set. Use as algebraic identities; broad <code>all()</code> may include more history than an ordinary log.
Anchor	<code>root()</code>	The repository root singleton. Use as a graph anchor, not as a working change.
Boundary	<code>heads(X)</code> , <code>roots(X)</code>	Reduce X to nodes with no selected children or parents. Applying them before/after an intersection changes the result.
Visibility	<code>visible_heads()</code> , <code>visible()</code> , <code>hidden()</code>	Visibility/presentation boundaries. Hidden does not mean unreachable or deleted.
Immutability	<code>immutable_heads()</code> , <code>builtin_immutable_heads()</code>	Configured or builtin rewrite-protection boundaries. Compare them when diagnosing a policy change.
Traversal	<code>ancestors(X)</code> , <code>descendants(X)</code>	Transitive parent/child closure including the input. Intersect to form corridors.
Traversal	<code>parents(X)</code> , <code>children(X)</code>	Parent/child relationships, with optional depth forms where documented. Use for explicit edge inspection.
Traversal	<code>first_parent(X)</code> , <code>first_ancestors(X)</code>	Follow first-parent integration history rather than every reachable parent.
Traversal	<code>reachable(X,Y)</code> , <code>connected(X)</code>	Reachability/connectivity predicates for non-linear graphs. Test direction; reachability is not symmetric.

Family	Function/signature	Result and useful composition
Topology	<code>merges()</code> , <code>forks()</code>	Select merge/fork topology, then intersect with a domain. A merge is not merely a commit with two reviews.
Topology	<code>latest(X, N)</code>	Choose latest commits by timestamp/ordering from X. A policy choice, not a cardinality assertion.
Topology	<code>fork_point(X)</code> , <code>merge_point(X)</code>	Compute the documented graph point for one input set. Ambiguity and multiple common ancestors require inspection.
State	<code>mutable()</code> , <code>immutable()</code>	Configured mutability predicates. Intersect with reachability for safe work selections.
State	<code>empty()</code>	Commits whose tree has no change relative to its parents. Description alone does not make a tree non-empty.
State	<code>conflicts()</code>	Commits with files in a conflicted state. This is distinct from a merge commit and from conflict markers currently visible on disk.
State	<code>divergent()</code>	Multiple visible evolutions of one change ID. Treat as a logical-identity problem, not just a graph fork.
Identity	<code>commit_id(PATTERN)</code>	Select commit IDs matching a pattern. Exact immutable-object selection.
Identity	<code>change_id(PATTERN)</code>	Select logical change IDs/evolutions. More stable across ordinary rewrite than commit ID.
Identity	<code>present(X)</code>	Resolve only commits present in the current view; useful for optional symbols in aliases.
Authorship	<code>author(PATTERN)</code> , <code>author_name(PATTERN)</code> , <code>author_email(PATTERN)</code>	Match author signature fields. Combine with date/topology and explicit pattern kinds.

Family	Function/signature	Result and useful composition
Authorship	<code>author_date(PATTERN)</code>	Match author timestamps. Author time can differ from committer time after imports/rebases.
Committer	<code>committer(PATTERN)</code> , <code>committer_name(PATTERN)</code> , <code>committer_email(PATTERN)</code>	Match committer metadata. Useful for identifying the process that rewrote a commit.
Committer	<code>committer_date(PATTERN)</code>	Match commit/rewrite time. Do not substitute it for author date in provenance queries.
Description	<code>description(PATTERN)</code> , <code>subject(PATTERN)</code>	Match full description or first-line subject. Newlines, trailers, and case matter.
Refs	<code>bookmarks()</code> , <code>remote_bookmarks()</code>	Commits carrying local or remote bookmark refs. Use unions for “published anywhere”.
Refs	<code>tracked_remote_bookmarks()</code> , <code>untracked_remote_bookmarks()</code>	Partition remote bookmarks by tracking relationship; useful for publication audits.
Refs	<code>tags()</code> , <code>remote_tags()</code>	Tag-bearing commits, local/remote. Tags are release names, not review branches.
Refs	<code>tracked_remote_tags()</code> , <code>untracked_remote_tags()</code>	Available in jj 0.45.1; untracked remote tags participate in default immutable heads.
Workspace	<code>working_copies()</code>	Commits currently serving as working copies. Workspace records and names are managed by <code>jj workspace</code> ; <code>@</code> remains <code>workspace-local</code> .
History	<code>at_operation(OP, X)</code>	Evaluate X in a historical operation view. Do not mix its graph with current filesystem state casually.
History	<code>jj evolog -r X</code>	Evolution is inspected by command, not selected by a revset function. Use <code>evolog</code> for predecessor/successor forensic inspection.

Family	Function/signature	Result and useful composition
Files	<code>files(FILESET)</code>	Match commits by changed paths. Pattern scope and diff direction are significant.
Files	<code>diff_lines(TEXT, [FILESET])</code>	Match changes whose diff lines contain the text, optionally limited by a fileset. Beware generated/binary content.
Files	<code>diff_lines_added(TEXT, [FILESET]),</code> <code>diff_lines_removed(TEXT, [FILESET])</code>	Match added/removed diff lines. Use bounded ranges and inspect false positives.
Review	<code>mine()</code>	Author matches configured user identity. A convenient predicate, not proof of ownership.
Review	<code>signed()</code>	Select signed commits in jj 0.45.1. Cryptographic validity and server trust remain separate checks.
Query control	<code>coalesce(X, Y, ...)</code>	Return the first present/non-empty set/value. Useful in aliases with optional refs.
Query control	<code>exactly(X, N)</code>	Evaluate X and error unless it has exactly N members; useful before singleton commands.
Refs	<code>trunk()</code>	Resolve the configured default trunk bookmark. Prefer an explicit remote bookmark when a script must be unambiguous.
Query control	<code>builtin_log()</code>	The builtin log revset/default. Use as a base when preserving release defaults.
Bisect	<code>bisect()</code>	Expose bisection state/candidate selection. Candidate semantics belong to <code>jj bisect run</code> .
Aliases	configured aliases	Revset aliases are functions in the same language. They can take parameters and can shadow names; version them and test them.

The table intentionally says “as named by target help” for the conflict spelling because aliases and version-specific terms can differ between releases. When porting a query, run it with the target binary and do not silently turn an unavailable predicate into a prose synonym.

Appendix D — Complete fileset reference

Form	Meaning	Example
literal/prefix	cwd-relative path prefix	'src'
cwd:"path"	explicit cwd-relative prefix	'cwd:"src/lib"'
file:"path"	exact cwd-relative file	'file:"Cargo.toml"'
glob:"pattern"	cwd-relative wildcard	'glob:"*.rs"'
prefix-glob:"pattern"	wildcard plus recursive prefix	'prefix-glob:"src/*.rs"'
root:"path"	root-relative prefix	'root:"src"'
root-file:"path"	exact root-relative file	'root-file:"Cargo.toml"'
root-glob:"pattern"	root-relative wildcard	'root-glob:"**/*.rs"'
root-prefix-glob:"pattern"	root-relative wildcard plus prefix	'root-prefix-glob:"src/**"'
glob-i:, cwd-glob-i:	case-insensitive cwd-relative glob variants	'cwd-glob-i:"*.TXT"'
root-glob-i:	case-insensitive root-relative glob variant	'root-glob-i:"docs/*.MD"'
all()	all paths	'all() ~ glob:"**/*.lock"'
none()	no paths	Useful as an alias fallback.

Fileset precedence is function, pattern, complement, intersection/difference, union. The language supports `&`, `~`, `|`, and parentheses. Shell quoting is mandatory around operators and recommended around every complex expression.

Appendix E — Complete template types and methods

Type	Important methods	Notes
Boolean	none	Literal <code>true</code> / <code>false</code> ; serialisable and conditional.
Integer	none	Arithmetic/comparison; use <code>+</code> , not <code>++</code> , for addition.
String	<code>len</code> <code>contains</code> <code>match</code> <code>starts_with</code> <code>ends_with</code> <code>remove_prefix</code> <code>remove_suffix</code> <code>trim</code> <code>trim_start</code> <code>trim_end</code> <code>substr</code> <code>first_line</code> <code>lines</code> <code>split</code> <code>replace</code> <code>upper</code> <code>lower</code> <code>escape_json</code>	UTF-8 text; substring offsets are bytes.
ByteString	String methods except JSON-specific distinctions	Preserves arbitrary bytes; use careful path/output handling.
List	<code>len</code> <code>join</code> <code>filter</code> <code>map</code> <code>any</code> <code>all</code> <code>first</code> <code>last</code> <code>get</code> <code>reverse</code> <code>skip</code> <code>take</code>	Empty/index errors are runtime errors.
AnyList	<code>join</code>	Usually produced by <code>map</code> .
Option	Boolean test; contained methods when set	Unset comparisons are ordered below set values; guard dereference.
CommitId	<code>short</code> <code>shortest</code>	<code>Shortest</code> returns a <code>ShortestIdPrefix</code> .
ChangeId	<code>normal_hex</code> <code>short</code> <code>shortest</code>	Change identity is not commit identity.
ShortestIdPrefix	<code>prefix</code> <code>rest</code> <code>upper</code> <code>lower</code>	Useful for styling the unique portion.
Commit	<code>description</code> <code>trailers</code> <code>change_id</code> <code>commit_id</code> <code>parents</code> <code>author</code> <code>committer</code> <code>signature</code> <code>mine</code> <code>working_copies</code> <code>current_working_copy</code> <code>bookmarks</code> <code>local_bookmarks</code> <code>remote_bookmarks</code> <code>tags</code> <code>local_tags</code> <code>remote_tags</code> <code>divergent</code> <code>hidden</code> <code>change_offset</code> <code>immutable</code> <code>contained_in</code> <code>conflict</code> <code>empty</code> <code>diff</code> <code>files</code> <code>conflicted_files</code> <code>root</code>	Cannot print directly; compose methods.

Type	Important methods	Notes
CommitRef	name remote present conflict normal_target removed_targets added_targets tracked tracking_present tracking_ahead_count tracking_behind_count synced	Describes local/remote bookmark or tag targets.
Signature	name email timestamp	Author and committer have independent signatures.
Email	local domain	Splits at first @; malformed addresses remain values.
Timestamp	ago format utc local after before since	Use explicit UTC/strftime for reproducible output.
TimestampRange	start end duration	Useful for age/duration reports.
TreeDiff	files color_words git stat summary	Render a method; the object itself is not printable.
TreeDiffEntry	path display_diff_path status status_char source target	Statuses include modified/added/removed/copied/renamed.
DiffStats	files total_added total_removed	Entries provide bytes/lines/path/status methods.
TreeEntry	path conflict conflict_side_count file_type executable	Tree metadata, including conflict and executable state.
RepoPath	absolute display parent	Repository-relative path with optional parent.
FsPath	absolute relative	Filesystem path; non-UTF-8 handling matters.
Trailer	key value	Descriptions may contain duplicate keys.
Operation	current_operation description id attributes time user snapshot workspace_name root parents	Operation context for <code>op log</code> .
OperationId	short	Local operation identity.
WorkspaceRef	name target root	Root is optional/stale-able.

Type	Important methods	Notes
AnnotationLine	commit content line_number original_line_number first_line_in_hunk	Context for file annotate.
CryptographicSignature	status key display	Calls can be slow because verification may execute external tools.
ConfigValue	as_boolean as_integer as_string as_string_list	Extract the expected TOML type.
SizeHint	lower upper exact zero	Used for tracking counts; upper/exact may be unknown.
RegexCaptures	len get name	Only available in a template replace lambda.

Global template function signatures

```

fill(Integer, Template) -> Template
indent(Template, Template) -> Template
pad_start(Integer, Template, [Template]) -> Template
pad_end(Integer, Template, [Template]) -> Template
pad_centered(Integer, Template, [Template]) -> Template
truncate_start(Integer, Template, [Template]) -> Template
truncate_end(Integer, Template, [Template]) -> Template
hash(Stringify) -> String
label(Stringify, Template) -> Template
hyperlink(Stringify, Template, [Template]) -> Template
raw_escape_sequence(Template) -> Template
replace(StringPattern, Template, |RegexCaptures| -> Template) -> Template
stringify(Stringify) -> String
json(Serialize) -> String
if(Boolean, Any, [Any]) -> Any
try(Any, Any...) -> Any
coalesce(Template...) -> Template
concat(Template...) -> Template
join(Template, Template...) -> Template
separate(Template, Template...) -> Template
surround(Template, Template, Template) -> Template
config(Stringify) -> Option<ConfigValue>
git_web_url([String]) -> String

```

The function call and method names above are the jj 0.45.1 target inventory. An unavailable method after a future upgrade is a release difference, not a reason to silently change a template's meaning; record the fallback or feature-test it.

Standalone reference cards

D.1 — Fileset grammar and consumer matrix

The target fileset language has a small grammar with explicit scope prefixes and set operators. The shell passes one quoted expression to `jj`; `jj` then parses the expression. The conceptual grammar is:

```
fileset := atom | fileset " | " fileset | fileset " & " fileset
         | fileset " ~ " fileset | "~" fileset | "(" fileset ")"
atom    := "all()" | "none()" | prefix ":" pattern
prefix  := cwd | file | glob | prefix-glob | root | root-file
         | root-glob | root-prefix-glob
         | glob-i | cwd-glob-i | root-glob-i | ...
```

Case-insensitivity is expressed by appending `-i` to a glob-pattern prefix. Thus `cwd-glob-i:` is the case-insensitive cwd-relative form and `root-glob-i:` is the case-insensitive root-relative form. The important design rule is not to infer a prefix's scope from the command's current directory. Use `root-file:` and `root-glob:` in repository scripts; use `cwd-file:` when a command is intentionally relative to the caller.

Form	Base directory	Matching kind	Use when
<code>file:P</code>	Command/working-copy path context	One literal path	You want exactly one named file.
<code>glob:P</code>	Command/working-copy path context	Glob pattern	You need a pattern and have controlled cwd.
<code>prefix-glob:P</code>	Command/working-copy path context	Prefix-oriented glob	You want recursive matching below a named prefix.
<code>root-file:P</code>	Repository root	One literal path	Portable scripts and docs.
<code>root-glob:P</code>	Repository root	Glob pattern	Portable recursive selection.
<code>root-prefix-glob:P</code>	Repository root	Prefix-oriented glob	Portable subtree selection.
<code>none()</code>	—	Empty set	Disable a configured selection in an alias.

Form	Base directory	Matching kind	Use when
<code>all()</code>	—	All paths	Explicit whole-tree operation after inspection.

When a literal path contains a colon, use the appropriate quoting/escaping form from the installed help; a colon at the start of an atom is a language delimiter. When a shell variable supplies the expression, quote its expansion:

```
paths='root-glob:src/**/* .rs ~ root-glob:src/**/generated/**'
jj file list "$paths"
jj diff -r @ -- "$paths"
```

The second command illustrates a subtle boundary: a positional argument after `--` is passed as one fileset to `jj`, but the command still decides whether that position means a fileset. Do not assume every command accepts the same separator convention; check its entry in Part XV.

Consumer	Default tree/diff	Partial selection effect	Post-operation check
<code>jj diff</code>	Current commit or endpoints	Limits display, not graph.	Compare with an unfiltered summary.
<code>jj file list</code>	Current working tree/commit	Limits listed paths.	Run from root and subdirectory to test scope.
<code>jj restore</code>	From source to into destination	Copies selected paths.	Diff the destination and inspect residual files.
<code>jj squash</code>	From source change into destination	Moves selected diff paths.	Diff both source and destination.
<code>jj split</code>	Selected commit	Assigns selected paths/diff hunks to one side.	Inspect both resulting commits.
<code>jj fix</code>	Selected source commits	Restricts fixer input with the 0.45.1 <code>-s/--source</code> option.	Query successors and run tests at the stack tip.
<code>jj file chmod</code>	Working-copy tree	Limits executable-bit changes.	Show file modes through Git/object inspection if relevant.

Consumer	Default tree/diff	Partial selection effect	Post-operation check
<code>jj file track/untrack</code>	Filesystem tracking state	Changes what snapshot considers.	Run status and inspect the next snapshot.

E.1 — Template type and method map

Templates are easiest to look up by the value you currently have, not alphabetically by function name.

Type	Important members	Typical rendering	Failure to test
<code>Commit</code>	<code>commit_id</code> , <code>change_id</code> , <code>description</code> , <code>parents</code> , <code>author</code> , <code>committer</code> , <code>state</code> , <code>predicates</code> , <code>refs</code> , <code>diff</code>	Log/show records.	Root, merge, conflict, empty, hidden, and divergent commits.
<code>CommitId</code>	<code>short</code> , <code>shortest</code> , <code>normal_hex</code>	Human ID and exact object evidence.	Ambiguous prefixes and new objects after rewrite.
<code>ChangeId</code>	<code>short</code> , <code>shortest</code> , <code>normal_hex</code>	Logical evolution identity.	Divergent evolutions and changed IDs after explicit update.
<code>Signature</code>	<code>name</code> , <code>email</code> , <code>timestamp</code>	Author/committer fields.	Different author/committer and timezone conversion.
<code>Timestamp</code>	<code>ago</code> , <code>format</code> , <code>utc</code> , <code>local</code> , <code>after</code> , <code>before</code> , <code>since</code>	Review age and reproducible dates.	Locale, clock skew, and author versus committer time.
<code>CommitRef</code>	<code>name</code> , <code>remote</code> , <code>tracking/conflict/sync</code> methods, <code>targets</code>	Bookmark/tag/ref labels.	Conflicted refs and removed/added targets.
<code>RepoPath/FsPath</code>	<code>absolute</code> , <code>relative</code> , <code>display</code> , <code>parent</code>	Portable path output.	Running from a subdirectory and sparse paths.
<code>TreeDiff</code>	<code>summary</code> , <code>git</code> , <code>color_words</code> , <code>stat</code> , <code>files</code>	Diff/stat output.	Large diffs, binary files, conflict values.
<code>TreeEntry</code>	<code>path</code> , <code>status</code> , <code>source</code> , <code>target</code> , and mode-related fields	File-level reports.	Renames, deletes, copies, and executable bits.

Type	Important members	Typical rendering	Failure to test
Operation	<code>id</code> , <code>description</code> , <code>parents</code> , <code>time</code> , <code>user</code> , <code>snapshot/workspace</code> fields	Operation/recovery log.	Unintegrated/concurrent operations.
WorkspaceRef	<code>name</code> , <code>target</code> , <code>root</code>	Workspace dashboards.	Stale or moved workspace paths.
List/Option	<code>map</code> , <code>filter</code> , <code>join</code> , <code>first</code> , <code>last</code> , <code>try</code> , <code>coalesce</code>	Conditional collections.	Empty lists/options and wrong element type.

The same expression can have different meaning in different contexts. `id` in an operation template is an operation ID; `commit_id` in a commit template is a commit object ID. Prefer explicit field names in mixed documentation.

E.2 — Operators and formatting rules

Operator/function	Operands	Notes
<code>++</code>	Template/string-like	Concatenation; use <code>concat</code> for long fragments.
<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>	Integer	Arithmetic; parenthesise before concatenation.
<code>==</code> , <code>!=</code>	Comparable values	Use for flags and field tests; do not compare formatted colour output.
<code>&&</code> , <code> </code> , <code>!</code>	Boolean	Short-circuiting conditions.
<code>if</code>	Boolean, then, optional else	Guard optional values and state fields.
<code>label</code>	Label name, content	Semantic style; keep machine output unlabelled.
<code>json</code>	Serializable value	Escape at the boundary; test control characters.
<code>replace</code>	Pattern, content, lambda	Lambda receives captures; regex and string patterns differ.

F.1 — Configuration reference by task

Task	Keys to inspect	Safe diagnostic	Version note
Identity	<code>user.name</code> , <code>user.email</code> , <code>operation.username</code> , <code>operation.hostname</code>	<code>jj config get user.email</code>	Commit and operation identity are different.
Interactive editing	<code>ui.editor</code> , <code>ui.diff-editor</code> , <code>ui.merge-editor</code> , <code>merge-tools.*</code>	<code>jj config list</code>	External tools require local executable validation.
Output	<code>ui.pager</code> , <code>ui.paginate</code> , <code>ui.color</code> , <code>ui.graph</code> , templates	<code>jj --no-pager --color never log</code>	jj 0.45.1 documents less -FRXK as the non-Windows default.
History selection	<code>revsets.log</code> , <code>revsets.fix</code> , <code>revsets.run</code> , aliases	<code>jj log -r DEFAULT</code>	Defaults can change release-to-release.
Rewrite safety	<code>immutable-head</code> alias, <code>git.private-commits</code> , snapshot policy	<code>jj log -r 'immutable_heads()'</code>	Untracked remote tags participate in the jj 0.45.1 default immutable-head policy.
Git transport	<code>git.fetch</code> , <code>git.push</code> , <code>git.private-commits</code> , remotes	<code>jj git remote list</code>	Credentials remain Git/host configuration.
Gerrit	<code>gerrit.default-remote</code> , <code>gerrit.default-remote-browse</code> , <code>gerrit.review-url</code>	<code>jj config get gerrit.default-remote</code>	Dedicated defaults are documented in jj 0.45.1; pass flags explicitly in mixed-remote repositories.
Filesystem snapshots	<code>snapshot.auto-track</code> , <code>snapshot.auto-update-stale</code> , <code>snapshot.max-new-file-size</code> , working-copy EOL/exec-bit keys	<code>jj status</code>	Changing snapshot policy can alter what a future operation records.
Fixers	<code>fix.tools.*</code> , fix defaults	Use a narrow <code>jj fix -s REVSET -- FILESET</code> trial, then inspect <code>jj op show -p</code> and use <code>jj undo</code> if needed.	jj 0.45.1 has no <code>--dry-run</code> option for <code>fix</code> ; tool support and scope remain policy-dependent.

Task	Keys to inspect	Safe diagnostic	Version note
Signing	<code>signing.backend</code> , <code>signing.key</code> , <code>signing.behavior</code> , <code>git.sign-on-push</code>	<code>jj config list</code> with secrets redacted	Key agents and server trust are external.
Conditionals	<code>-when.repositories</code> , hostnames, workspaces, commands, platforms, environments	<code>jj config path</code>	Use the installed schema for the exact table shape.

This task-oriented appendix is intentionally more useful than an alphabetic dump. After a future upgrade, generate the new schema and review every changed key's scope and default against the `jj 0.45.1` baseline. Preserve a copy of the old configuration while testing.

Appendix F — Complete configuration families

Namespace/family	Important members and design role	Scope/edge case
<code>user.*</code>	name, email; signing identity inputs.	User scope normally; author can be overridden explicitly.
<code>ui.*</code>	pager, colour, editor, diff/merge editor, default command, movement, timestamps, conflict markers, output style.	Workspace/repo overrides can surprise prompts and automation.
<code>git.*</code>	backend, push/fetch defaults, private commits, remote/ref behaviour.	Push remote defaults must not silently choose GitHub versus Gerrit.
<code>gerrit.*</code>	<code>default-remote</code> , <code>default-remote-branch</code> , <code>review-url</code> in the target surface.	jj 0.45.1 documents these keys; explicit upload flags remain safer in mixed-remote repositories.
<code>revsets.*</code>	log, status, fix, sign, arrange, simplify-parents, converge/relevant defaults.	Default selection is policy; inspect before mutation.
<code>revset-aliases</code>	Named and parameterised commit sets.	Aliases are parsed expressions and can shadow builtins.
<code>fileset-aliases</code>	Named path sets and path-pattern functions.	Separate namespace from revset aliases.
<code>templates.*</code>	log, show, evolog, file, operation, workspace defaults; builtin templates.	Human output defaults should not be assumed in scripts.
<code>template-aliases</code>	Keywords/functions/patterns for rendering.	Builtin shadowing and optional values are common traps.
<code>merge-tools.*</code>	External merge program, arguments, capabilities, conflict handling.	External executable and exit status are security/automation concerns.
<code>diff-editor/diff-tools.*</code>	Diff editing and rendering programs.	Different from a merge editor; fileset and endpoint still come from command.
<code>fix.tools.*</code>	Fixer command, file patterns, range support, environment.	Can rewrite many commits; test line-range behaviour.
<code>signing.*</code>	Backend, key, command/agent and verification policy.	Sign/unsign rewrite commits; hardware signing can be slow.

Namespace/family	Important members and design role	Scope/edge case
immutable-head policy	Revset defining rewrite protection.	Changing config changes safety boundary; <code>--ignore-immutable</code> remains exceptional.
snapshot/workspace	Snapshot intervals, file tracking/ignore/size, sparse/workspace behaviour.	Filesystem state and commit trees remain distinct.
command aliases	Safe shortcuts and publication commands.	Document mutation, remote, and argument forwarding.

For an exhaustive machine check, compare `jj util config-schema` output between binaries. For a useful human configuration, explain why a key exists, where it is safe, and which other keys it interacts with; a list of key names alone is not a configuration reference.

Appendix G — Git-to-jj lookup

This is an operation-oriented lookup, not a claim that the two systems have identical primitives. Start with the `jj` state transition in the middle column; use the Git wording only to identify the intention.

Git intention	jj operation	Important semantic difference
<code>git commit</code>	<code>jj describe</code> , then <code>jj new</code> ; or let the working-copy snapshot record the edit	The working-copy commit already exists and is evolved; <code>new</code> creates the next change boundary.
<code>git commit --amend</code>	Edit the target with <code>jj edit</code> , change files, then snapshot; or use <code>jj describe</code> for metadata	Descendants are automatically rebased and can receive new commit IDs.
<code>git rebase -i</code>	<code>jj rebase</code> , <code>jj squash</code> , <code>jj split</code> , <code>jj edit</code> , and <code>jj describe</code>	History editing is decomposed into graph and diff transformations; there is no single todo-list mode required for ordinary surgery.

Git intention	jj operation	Important semantic difference
<code>git reset</code>	<code>jj edit</code> , <code>jj restore</code> , <code>jj abandon</code> , or <code>jj op restore</code> , depending on whether the intent is workspace movement, content undo, commit removal, or repository-state recovery	These operations affect different state layers and are not interchangeable.
<code>git checkout COMMIT</code>	<code>jj edit COMMIT</code>	Moves the workspace to a commit in the DAG and updates its working-copy materialisation; it is not a detached Git HEAD mode.
<code>git checkout -b NAME</code>	<code>jj bookmark create NAME -r REV</code>	A bookmark is a publication/name ref, not the container for local work.
<code>git branch -f NAME REV</code>	<code>jj bookmark set NAME -r REV</code> or <code>jj bookmark move NAME --to REV</code>	Review whether the movement is forward, backward, local, remote, and tracked before publishing it.
<code>git cherry-pick COMMIT</code>	<code>jj duplicate -r COMMIT</code> , or rebase a selected source onto a destination	Duplicate creates a new logical change; it does not preserve the source change identity.
<code>git revert COMMIT</code>	<code>jj revert -r COMMIT</code>	Creates a new change that undoes the selected commit's tree effect; it does not erase the original evolution.
<code>git add -p</code>	<code>jj split</code> , <code>jj diffedit</code> , or a fileset-limited <code>jj squash</code>	Diff ownership is edited in the commit graph; there is no persistent index staging area.
<code>git restore PATH</code>	<code>jj restore --from SOURCE --to DEST -- FILESET</code>	Restore is content undo/copy between trees, not an operation-log undo and not a rebase.
<code>git stash</code>	Create a new <code>jj</code> change, or use a temporary bookmark/workspace when a named hand-off is needed	Unfinished work is normally already represented by the working-copy commit; there is no hidden stash stack with separate semantics.
<code>git worktree</code>	<code>jj workspace add</code> , <code>jj workspace list</code> , and <code>jj workspace forget</code>	Workspaces share the <code>jj</code> store and have workspace-local @; inspect stale working-copy state before rewriting shared history.

Git intention	jj operation	Important semantic difference
<code>git reflog</code>	<code>jj op log</code> , <code>jj evo log</code> , <code>jj undo</code> , and explicit commit lookup	The operation log records repository-state transitions, while evolution records a change's rewritten commit successors; neither is merely a branch-tip log.
<code>git merge</code>	<code>jj new</code> <code>--insert-before/--insert-after</code> or an explicit multi-parent rebase/merge operation where appropriate	Conflicts can be stored as commit values, and a merge is a graph object rather than only a paused working-tree state.
<code>git bisect</code>	<code>jj bisect</code> and <code>jj bisect run</code> in <code>jj 0.45.1</code>	The tested domain and command exit status must be explicit; inspect the resulting commit selection before acting on it.
<code>git push --force-with-lease</code>	Move the local publication bookmark and run <code>jj git push</code> with an explicit remote/bookmark	jj's local rewrite safety and Git's remote lease are separate protections; inspect the fetched remote bookmark and push result.

When no row fits exactly, describe the desired postcondition in `jj` terms: which commits should exist, which parent edges should change, which content should move, which name should point where, and whether the operation should be recoverable through the operation log. That formulation is more reliable than inventing a one-to-one translation.

Appendices H–K — operational recipes and Linux integration

The preceding reference cards describe the language and command surfaces. These appendices turn that information into short, repeatable operating procedures. Each recipe starts by naming the state that must be true before the command is run, and ends with an inspection step. That discipline matters in `jj` because the most dangerous errors are usually selection errors: the command is valid, but its commit set, destination, remote, or workspace is not the one the operator intended.

Appendix H — GitHub recipes fieldbook

GitHub consumes Git refs. `jj` supplies those refs through bookmarks and the Git transport, but the local graph does not need to be organised around them. In the recipes below, the local change IDs are the working identities and the bookmarks are publication handles. Replace `origin` explicitly when the repository has more than one remote.

H.1 Publish one mutable change as a PR

1. Fetch the base and inspect it: `jj git fetch --remote origin`, then `jj log -r 'trunk() | @ | ancestors(@)'`.
2. Put the working-copy commit on top of the fetched base if necessary. A publication bookmark is not required while the change is still private.
3. Give the change a reviewable description and run tests at the intended commit. Record the exact commit with `jj log -r @ -T 'change_id ++ " " ++ commit_id ++ "\n"'`.
4. Create a publication handle only when the hosting service needs a branch: `jj bookmark create feature/allocator -r @`.
5. Push that handle to the intended remote: `jj git push --remote origin --bookmark feature/allocator`. The push result, not the mere existence of a local bookmark, is the publication event.
6. Create or update the PR with the host tool. Verify the remote bookmark and local target before every update.

```
jj log -r 'feature/allocator | feature/allocator@origin'
jj bookmark list feature/allocator
jj git push --remote origin --bookmark feature/allocator
```

After review, edit or squash the local change. The resulting commit ID may change while the `jj` change ID remains the same. Move the bookmark to the successor only after checking `jj diff -r 'feature/allocator@origin..@'` and the test result. A rejected non-fast-forward push is a state disagreement to investigate, not a prompt to use an unqualified force operation.

H.2 Publish a stack of dependent PRs

Suppose the local graph is:

```
main@origin
|
A (base)
|
B feature/api
|
C feature/cli
|
@ (scratch edits)
```

Make the publication boundary explicit. The first PR points at B and the second at C. Use distinct bookmarks, for example `pr/api` and `pr/cli`. Keep the lower bookmark at B while C is being reviewed; changing C must not silently publish B's private edits.

```
jj bookmark create pr/api -r B
jj bookmark create pr/cli -r C
jj git push --remote origin --bookmark pr/api --bookmark pr/cli
```

When B changes, jj rebases its descendants as part of the rewrite. Inspect the successor graph before pushing both handles:

```
jj log -r 'ancestors(C) | descendants(B)'
jj diff -r 'pr/api@origin..B'
jj diff -r 'pr/cli@origin..C'
jj git push --remote origin --bookmark pr/api --bookmark pr/cli
```

The host may display a stack as separate PRs, a single stacked series, or a sequence of branches. That presentation is host policy. The local invariant is simpler: each publication bookmark names one graph node, and the upper node depends on the lower node.

H.3 Rebase a stack after main moves

Fetch first, then identify the first local commit that is not part of the new trunk. Do not blindly rebase every visible head: that may include unrelated workspaces or another feature.

```
jj git fetch --remote origin
jj log -r 'trunk()..pr/cli'
jj rebase -s 'pr/api' -d 'main@origin'
jj log -r 'main@origin..pr/cli'
```

If the stack has a lower public bookmark and an upper public bookmark, a source selection rooted at the lower change moves its descendants together. If you intentionally want only one change, select that commit and inspect the resulting descendants. Conflicts remain values in the graph; they do not imply a Git-style continuation state. Resolve them, test, and then push each affected bookmark.

H.4 Diagnose a rejected update

Classify the rejection before changing anything. Compare the local bookmark, the remote bookmark, and the graph between them:

```
jj bookmark list pr/api
jj log -r 'pr/api | pr/api@origin | ancestors(pr/api@origin)'
jj diff -r 'pr/api@origin..pr/api'
jj git push --remote origin --bookmark pr/api --dry-run
```

Common causes are a stale fetch, another contributor moving the branch, a host-side merge, or a bookmark whose target is no longer the intended successor. Fetch again and decide whether to integrate the remote tip, replace the remote branch through the project's accepted force-update policy, or create a new publication bookmark. Keep the old remote target in the operation record and review notes. Never encode a force-push policy in a general-purpose alias without an explicit remote and bookmark.

H.5 Clean up after merge or closure

After GitHub merges a PR, fetch the remote and verify that the remote bookmark has been deleted or moved according to the repository policy. Delete or forget the local publication handle only after the remote state is understood:

```
jj git fetch --remote origin
jj bookmark list --remote origin
jj bookmark delete pr/api
jj log -r 'immutable_heads() | trunk()'
```

Deleting a local bookmark is not the same operation as deleting its remote counterpart. In `jj 0.45.1`, remote deletion is requested with `jj git push --deleted`, subject to project policy; use it only after confirming the target. The commit remains reachable through other descendants, the operation log, or the remote object store until normal garbage collection makes it unreachable.

H.6 Use an upstream repository and a fork

Give the two remotes different roles. Fetch both, but publish only to the fork unless project policy explicitly permits publishing upstream. A local bookmark can be tracked against one remote bookmark at a time; the names can be identical while the remote relationship is different.

```
jj git remote add upstream https://github.com/example/project.git
jj git fetch --remote upstream
jj bookmark create fork/allocator -r @
jj git push --remote origin --bookmark fork/allocator
```

Use fully qualified remote bookmarks in revsets when the same branch name exists on both remotes. The name `main@origin` and `main@upstream` are different graph references even if they currently identify the same commit.

H.7 Combine `jj` with `gh` in a colocated repository

First make `jj` import any Git-side ref changes, then use `gh` for host operations that are not version-control graph operations. A PR merge performed by `gh` changes the remote; it does not directly update the local `jj` view until a fetch/import operation.

```
jj git fetch --remote origin
gh pr view --web
gh pr checks
jj git fetch --remote origin
jj log -r 'main@origin'
```

When invoking native Git plumbing, use `GIT_DIR=$(jj git root)` only after checking whether the repository is colocated and what the command will modify. Prefer read-only commands for diagnosis. If Git moves a ref, follow it with `jj git fetch` or the documented import operation before reasoning from `jj`'s remote-bookmark symbols.

Appendix I — Gerrit recipes fieldbook

Gerrit adds a server-side review identity to the Git transport. A `jj` change ID is generated by `jj` and follows an evolution in the local commit graph. Gerrit's `Change-Id` is a commit-message footer interpreted by Gerrit to associate patch sets with a review. They are independent identifiers and should be displayed separately in review tooling.

I.1 Create and upload a patch stack

```
jj git fetch --remote origin
jj new main@origin
jj describe -m 'parser: accept quoted allocator names'
# edit files and let jj snapshot them
jj new
jj describe -m 'cli: expose allocator diagnostics'
jj log -r 'main@origin..@'
jj gerrit upload --remote origin --remote-branch main -r 'main@origin..@'
```

The exact upload selection and server defaults are release-sensitive. On `jj 0.45.1`, prefer explicit `--remote` and `--remote-branch` when a repository has GitHub and Gerrit remotes. Inspect the Gerrit defaults before relying on them. Before upload, inspect each commit message for an existing Gerrit footer and decide whether the project's hook or server tooling should add it. Do not assume `jj`'s change ID will satisfy Gerrit's footer requirement.

I.2 Revise the bottom change after review

```
jj edit 'description(exact:"parser: accept quoted allocator names")'  
# edit the parser and run focused tests  
jj diff  
jj log -r 'ancestors(@) | descendants(@)'  
jj Gerrit upload --remote origin --remote-branch main -r 'main@origin..@'
```

Editing an ancestor changes its commit object and normally rebases its descendants. The `jj` change ID of the edited change is the useful local continuity signal; the Gerrit footer is the server continuity signal. The descendants receive new commit IDs because their parent changed, even where their patches are otherwise unchanged. If the server rejects a patch set, preserve the local graph and inspect the generated commit messages rather than manually copying identifiers between unrelated changes.

I.3 Resolve a conflict in the middle of a stack

```
jj log -r 'main@origin..@'  
jj resolve --list  
jj resolve --tool :builtin -r @  
# inspect each side, edit the materialised conflict, then snapshot  
jj diff  
jj status  
jj Gerrit upload --remote origin --remote-branch main -r 'main@origin..@'
```

A conflict in a lower stack member can make later members conflicted through rebasing. Resolve at the earliest useful node, then inspect descendants. A clean lower node does not prove that all descendants are clean: their diffs may conflict independently. Upload only after querying `conflicts()` over the exact review range.

I.4 Continue after one change lands

Once Gerrit lands the lower review, fetch the new target and rebase the remaining local stack onto it. The old remote review bookmark may disappear or move according to Gerrit's ref policy; use the fetched refs and commit messages to identify the remaining work.

```
jj git fetch --remote origin
jj log -r 'main@origin..@'
jj rebase -s 'description(glob:"cli: expose allocator diagnostics")' -d main@origin
jj log -r 'main@origin..@'
jj gerrit upload --remote origin --remote-branch main -r 'main@origin..@'
```

Do not re-upload a landed change merely because its old local bookmark remains. Decide whether the local commit is now an ancestor of the fetched trunk; if it is, delete or move the local publication handle. If it is not, investigate whether the server rebased, squashed, or rejected a related change.

I.5 Use upload options without hiding the publication boundary

Options such as reviewers, CCs, topic, hashtags, WIP/ready state, comments, and submit behaviour belong to the Gerrit upload operation. Keep them explicit in a reviewed command or a narrowly scoped alias. A command alias that silently supplies `--submit`, an alternate remote, or a broad commit set makes recovery harder and can publish unrelated work.

```
jj gerrit upload --remote origin --remote-branch main \
  --reviewer reviewer@example.org --topic allocator-stack \
  -r 'main@origin..@'
```

Because Gerrit server configuration and plugins vary, treat server-side topic interpretation, automatic footer insertion, submit queues, and policy checks as externally validated behaviour. The local `jj` invariants—commit selection, commit rewriting, conflicts, bookmarks, and operation recovery—can still be tested in a local Git repository.

I.6 Abandon or replace a review

Abandoning a Gerrit review does not abandon the local `jj` change. Keep the local node if its diff may be useful; create a new change description or a replacement branch only when the server workflow requires a new review identity. Record the old Gerrit footer in the review note and avoid carrying it to a deliberately unrelated change. If the local change was accidentally abandoned, recover it from `jj op log`, an evolution query, or another reachable descendant before publishing again.

Appendix J — Recovery recipes fieldbook

Recovery starts with observation. Stop making ordinary operations once the mistake is recognised, save the current operation ID, and inspect both graphs:

```
jj op log --limit 12
jj op show --operation @
jj log -r 'all()' --limit 80
jj evolog -r @
```

The commit DAG answers “what commits exist and how are they related?” The operation DAG answers “which repository view produced the current state, and what earlier views are available?” A bookmark may be absent from the current view while its target still exists in the commit store or an older operation view.

Symptom	First inspection	Preferred recovery direction	Do not assume
Wrong change edited	<code>jj op log</code> , current/previous <code>jj op show</code>	<code>jj undo</code> if the last operation is isolated; otherwise restore a named prior view.	Commit ID alone identifies the intended operation.
Useful work abandoned	<code>jj evolog</code> , <code>jj log -r all()</code> , op history	Find the abandoned successor and create a new visible/bookmarked name.	Abandon immediately deletes the object.
Bookmark moved to wrong node	<code>jj bookmark list</code> , op diff	Move it back explicitly after checking the intended target.	Moving a bookmark rewrites commits.
Several operations occurred after mistake	Operation IDs and operation parents	Restore or inspect a prior operation; copy useful work forward deliberately.	Repeated undo is always equivalent to restoring one known view.
Remote was changed incorrectly	Local/remote bookmark targets plus server audit	Fetch, preserve evidence, then perform an approved ref correction.	The local operation log can undo a server-side push.
Concurrent operation integrated unexpectedly	Operation log and operation descriptions	Inspect the integrated view; recover a prior local view if needed.	Operation IDs are Git reflog entries.

J.1 Recover the last mistaken rewrite

```
jj op log --limit 5
jj op show --operation @
jj undo
jj status
jj log -r 'heads(all())'
```

Use `undo` when the last operation is the mistake and no valuable operation should be preserved from that view. Confirm the resulting working-copy snapshot and bookmark targets. If the mistaken operation was followed by useful work, do not blindly undo a sequence; inspect the operation DAG and restore or reproduce the useful work in a controlled view.

J.2 Recover one known prior repository view

```
jj op log
jj op show --operation OPERATION_ID
jj op diff --from OPERATION_ID --to @
jj op restore --operation OPERATION_ID
```

Substitute an actual operation ID, not a commit ID. Operation restore changes the repository view, including refs and workspace state represented by that operation. It is therefore more powerful than moving a bookmark back. Immediately make a safety bookmark or export any work that should survive before continuing ordinary development.

J.3 Recover after a bad push

Local recovery and remote recovery are separate. First preserve the local state and identify the old remote target from logs, host audit, or another clone. Then contact the project owner if a protected ref or review ref was affected. A safe local sequence is:

```
jj bookmark list
jj op log
jj git fetch --remote origin
jj log -r 'all() & remote_bookmarks(remote=origin)'
# create a temporary local name before any correction
jj bookmark create recovery/bad-push -r 'pr/api@origin'
```

Only after the target is preserved should an approved push repair the remote bookmark. The operation log records the local push command, but it cannot rewind a remote server. This distinction is the operational boundary between jj recovery and Git-host administration.

Appendix K — Linux shell integration fieldbook

Shell integration should expose state without turning every prompt into a repository scan. Keep prompt commands read-only, cheap, and tolerant of directories outside a repository. Put mutation in explicit aliases or functions that print their selected commit and remote before acting.

K.1 A conservative Bash/Zsh prompt component

```
jj_prompt() {  
  command -v jj >/dev/null 2>&1 || return 0  
  jj root >/dev/null 2>&1 || return 0  
  local state  
  state=$(jj log --no-graph -r @ -T 'change_id.short() ++ " " ++ bookmarks' 2>/dev/null) || return  
  printf '[jj:%s]' "$state"  
}  
PS1='\u@\h:\w $(jj_prompt)\$ '
```

Use command substitution carefully: a prompt must not invoke a command that snapshots, edits, pages, or contacts a remote. If the repository is large, replace the log query with a cached or less frequent hook. The prompt output is advisory; before publication, run an explicit status/log query in the terminal.

K.2 Safe inspection and publication functions

```
jj_head() {  
  jj log --no-graph -r "$1" -T 'change_id ++ " " ++ commit_id ++ " " ++ description.first_line() +  
}  
  
jj_publish() {  
  test "$#" -eq 2 || { echo 'usage: jj_publish REMOTE BOOKMARK' >&2; return 2; }  
  local remote=$1 bookmark=$2  
  jj bookmark list "$bookmark" || return  
  jj log -r "$bookmark" -T 'bookmark ++ " " ==> " ++ commit_id ++ "\n' || return  
  jj git push --remote "$remote" --bookmark "$bookmark"  
}
```


These functions deliberately take the remote and bookmark as arguments. A shell alias such as `jj git push` with no remote is convenient for a single-remote repository, but is a poor default for a working copy that can publish to both GitHub and Gerrit.

K.3 Pagers, editors, and external tools

Integration	Diagnostic	Design rule
Pager	<code>jj --no-pager log</code>	Make scripts independent of paging; configure human paging separately.
Colour	<code>jj --color never log</code>	Machine captures should disable colour and labels that are not data.
Editor	<code>jj config get ui.editor</code>	Test the executable and exit status before using it in a mutation alias.
Diff editor	<code>jj config get ui.diff-editor</code>	Confirm whether the tool edits a diff or only renders one.
Merge editor	<code>jj config get ui.merge-editor</code>	Test two-sided and multi-sided conflicts; do not assume Git's marker layout.
SSH/GPG signing	<code>jj config list</code> with secrets redacted	Validate keys, agents, and verifier policy outside the repository fixture.
Completion	<code>jj util completion --help</code>	Install generated completion for the pinned binary; regenerate on upgrade.

K.4 Install completion without coupling scripts to a release

Generate completion from the binary that the shell will actually execute, then install it through the distribution's supported mechanism. Keep the generated file outside the book repository unless it is intentionally versioned:

```
jj util completion bash > /tmp/jj-completion.bash
# inspect it before installing through the shell's completion directory
jj util completion zsh > /tmp/_jj
jj util completion fish > /tmp/jj.fish
```

Completion is a convenience layer, not a compatibility contract. A completion file from another jj release can expose commands and options absent from 0.45.1. Pin the binary and completion together when reproducing a build or documenting a deployment image.

K.5 Automation boundary checklist

1. Set `JJ_BIN` or call the pinned binary explicitly in CI.
2. Use `--no-pager` and `--color never` for captured output.
3. Pass commit and remote arguments explicitly; do not depend on a developer's aliases.
4. Use a machine-readable template only after testing empty, root, merge, conflict, and divergent cases.
5. Capture the operation ID and command output for mutating jobs.
6. Run the command in a disposable clone or workspace when the selection could rewrite multiple commits.
7. After the command, query the intended invariant—bookmark target, conflict count, test result, or exact object ID.

This checklist is the Linux equivalent of a transaction boundary. jj makes rewriting routine, which is powerful for interactive work; automation should make the chosen scope and resulting graph equally explicit.